

The Backend Engineer's Library

Golang and Go Internals

Volume 17

Contents

The Go Toolchain: Modules, Build, Linker, and the Static Binary Model
Types, Memory Layout, and Generics Internals
Functions, Methods, Defer, Panic/Recover, and the ABI
Goroutines, the Scheduler (G-M-P), and the Netpoller
Memory Allocator, Stacks, and the Concurrent Tri-Color GC
The Go Memory Model, Atomics, and Synchronization Primitives
Interfaces, Reflection, and ``unsafe``
Channels, Select, Timers, and Context Internals
Compiler Pipeline: SSA, Escape Analysis, and Optimizations
Tooling Deep Dive: Race Detector, pprof, execution trace, and vet
cgo, Assembly, and Foreign-Function Interoperability
Production Go: Cross-Compilation, Workspaces, Telemetry, and
Deployment at Scale

Chapter 1 — The Go Toolchain: Modules, Build, Linker, and the Static Binary Model

What this chapter covers. Every Go service you ship — the binary in a `FROM scratch` container, the CLI on a developer laptop, the Lambda handler — is the product of a hermetic, opinionated toolchain that has no `Makefile`, no external linker, and no runtime dependency. Understanding that toolchain is the prerequisite for every later chapter: the compiler pipeline that produces SSA (Ch 9), the runtime that gets linked in (Ch 4-5), and the assembly you drop into with `cgo` (Ch 11). This chapter traces a Go build end-to-end: how `cmd/go` drives `cmd/compile` and `cmd/link`, how modules resolve with Minimal Version Selection, how the internal linker produces a static binary, and how the same machinery enables hermetic CI and cross-compilation. You will run every command that matters and read the outputs that prove what happened.

Learning goals — after this chapter you should be able to:

- Explain the Go tool's driver architecture — what `cmd/go` orchestrates and how it invokes `cmd/compile`, `cmd/asm`, and `cmd/link` without an external `ld` — and read a `go build -x -a` trace to account for each action.
- Author and diagnose `go.mod` / `go.sum` correctly: MVS resolution, `GOPATH` vs. modules, `GOPROXY` / `GOSUMDB` / `GOPRIVATE`, and the `go / toolchain` directives that pin the compiler itself.
- Use the core `go` commands fluently — `build`, `vet`, `test`, `work`, `list`, `env`, `version -m` — to inspect dependencies, build info, and linker inputs.
- Describe the compile pipeline (parse → type-check → SSA → object file) and the internal linker's job (symbol resolution, dead-code elimination, static linking), including the `cgo` exception and `CGO_ENABLED=0`.
- Apply build constraints, build tags, `embed`, and link flags (`-ldflags -X`, `-buildmode`, `-trimpath`) to control what ships in the binary, and produce reproducible, attestable artifacts.

- Cross-compile correctly with `G00S / G0ARCH` and reason about `cgo` breakage, `CC / CXX` selection, and `GOMELIMIT` -style toolchain versioning for hermetic CI and distroless containers.

Scope. This is the toolchain you invoke. Volume 13, Chapter 2 — The Go Runtime — covers what the linked-in runtime *does* at execution (scheduler, GC, netpoller). Volume 17, Chapter 9 — Compiler Pipeline — deep-dives SSA, escape analysis, and optimizations. Chapter 10 covers `vet`, `pprof`, `trace`, and the race detector as instruments; here we cover `vet` and `test` as *build commands*. For supply-chain hardening of the build itself (hermetic builds, SLSA provenance), see Companion Book 4, Chapters 2-3.

1. The tool that is also a build system

Most language toolchains are a loose bag of programs glued by `make`. Go's toolchain is a single driver — `cmd/go` — that owns parsing flags, resolving packages, scheduling compilation, caching, and linking. The pieces it drives live under `src/cmd` in the Go repository:

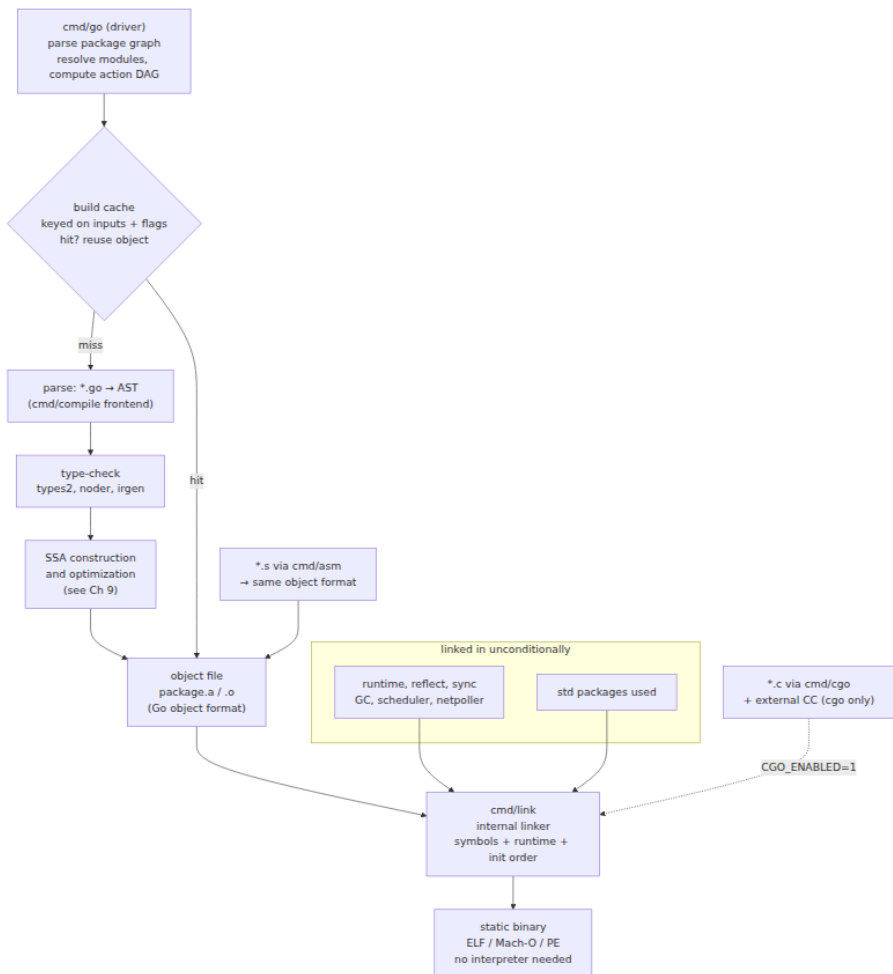
Tool	Package	Role
<code>go</code>	<code>cmd/go</code>	Driver: parses <code>go build / test / vet / list</code> , computes the action graph, drives compilation and linking, manages downloads and the build cache
<code>compile</code>	<code>cmd/compile</code>	Compiler frontend: one invocation per package, <code>*.go</code> → object file (<code>.o</code> -like, but Go's own format). No separate preprocessor; handles parsing, type-check, SSA
<code>asm</code>	<code>cmd/asm</code>	Assembler for <code>*.s</code> (Go assembly, <code>TEXT / DATA</code> pseudo-instructions), also produces object files consumed by the linker
<code>link</code>	<code>cmd/link</code>	Internal linker: consumes package objects + runtime objects → ELF/Mach-O/PE/ wasm binary. No <code>ld</code> , <code>gold</code> , or <code>lld</code> in the default path

Tool	Package	Role
<code>cgo</code>	<code>cmd/cgo</code>	Translates <code>import "C"</code> packages: generates Go shims + C stubs, invokes the C compiler for the C fragments; only active when <code>CGO_ENABLED=1</code>
<code>vet</code>	<code>cmd/vet</code> (via <code>go vet / golang.org/x/tools</code>)	Static analyzer suite run as a build action; shares the same package loader
<code>cover / test2json</code>	<code>cmd/cover</code> , <code>cmd/test2json</code>	Instrumentation for <code>go test -cover</code> and structured test output

There is no `ld` on the happy path. The Go linker is written in Go, understands Go's object format, and links everything — your code, `GOROOT/src/runtime`, `net`, `crypto` — into one file. The exception is `cgo` (Section 9): when a package imports `"C"`, `cmd/go` delegates final linking to the external C toolchain (`CC`, typically `gcc`/`clang`) and the binary becomes dynamically linked against `libc`.

How `go build` actually runs

A single `go build ./...` is not one compiler invocation — it is a DAG of *actions* executed in parallel and cached:



Key points for backend work:

- **Parallelism is automatic.** `cmd/go` topologically schedules package compiles across `GOMAXPROCS` workers. A 200-package service compiles packages in parallel where the import DAG allows it.
- **Caching is content-addressed.** The build cache (`go env GOCACHE` , default `~/.cache/go-build`) keys each action on inputs: source bytes, compiler version, flags, and transitive dependency hashes. A second `go build` with no changes does almost no work.

- **The driver is the API.** `go list`, `go vet`, `go test` all share the same package loader. Tooling that needs to understand your code (`staticcheck`, `golangci-lint`, IDE `gopls`) goes through the same loader — which is why `go vet` and `golangci-lint` agree on build constraints.
-

2. Modules — versioning, MVS, proxies, and the end of GOPATH

From GOPATH to modules

Before Go 1.11, `GOPATH` (`~/go` by default) was a single workspace: `src/` held a flat tree of repositories, `pkg/` held compiled objects, `bin/` held binaries. There was no version concept — `go get` mutated the tree in place. Reproducibility meant vendoring the entire `src/` into your repo.

Since Go 1.16 (modules on by default) and decisively since Go 1.17, **modules** are the unit of versioning and download. Each module is a directory tree with a `go.mod` at its root declaring its path and dependencies. `GOPATH` remains only as a download/compile cache (`GOPATH/pkg/mod`, `GOPATH/pkg/sumdb`), not as source layout.

go.mod and go.sum — real examples

```
// go.mod — minimal service
module github.com/acme/payments

go 1.22.4

// toolchain pins the compiler that builds this module (Go 1.21+)
toolchain go1.22.4

require (
    github.com/go-chi/chi/v5 v5.0.11
    github.com/jackc/pgx/v5 v5.6.0
    go.uber.org/zap v1.27.0
)

require (
    github.com/jackc/pgx/v5/pgconn v5.6.0 // indirect — pulled transitively
    go.uber.org/multierr v1.11.0 // indirect
)

replace github.com/acme/shared => ../shared // local replace (not for publishing)
exclude github.com/old/broken v1.2.0 // never select this version
retract v1.2.0 // published in your own module to mark a bad release
```

```
// go.mod — library that supports older Go
module github.com/acme/lib

go 1.21 // minimum Go that can build this module
toolchain go1.22.4 // but developers/CI use Go 1.22.4 to build

require golang.org/x/sync v0.7.0
```

What each line means operationally:

- `module` — import path prefix that also names the download location. Must match the repository hosting the code (or a `GOPROXY` that serves it).
- `go 1.22.4` — *minimum* Go version that understands this `go.mod`'s syntax (and, since Go 1.21, the minimum that may build it). `cmd/go` refuses to build with an older toolchain than `go + toolchain` require, fetching a newer one if `GOTOOLCHAIN=auto`.

- `toolchain go1.22.4` — **exact** toolchain that produced `go.sum` entries. With `GOTOOLCHAIN=auto` (the default since Go 1.21), `cmd/go` will download and delegate to `go1.22.4` automatically if the local `go` is newer or older. Pin this in CI.
- `require` — direct and indirect dependencies. `go mod tidy` prunes/ adds indirects.
- `replace` / `exclude` / `retract` — local overrides; `replace` never ships to consumers (it is ignored when your module is a dependency — only the *main* module's replaces apply).

```
// go.sum — one line per module version, keyed on contents
github.com/go-chi/chi/v5 v5.0.11 h1:rXQB01TR1SJ9bp6c1Yp0P0/H3N9pM6Qf6e4
j+Q0Zg5E=
github.com/go-chi/chi/v5 v5.0.11/go.mod h1:0a7N3x1...==
github.com/jackc/pgx/v5 v5.6.0 h1:7Uy0...==
github.com/jackc/pgx/v5 v5.6.0/go.mod h1:9fK2...==
```

Each `h1:` is a SHA-256 of the module zip (or its `go.mod` alone for the `/go.mod` line), verified against the checksum database (Section 2.3). Do not hand-edit `go.sum` — `go mod tidy` and `go mod download` maintain it.

Real outputs

```

$ go env GOPATH GOMODCACHE GOPROXY GOSUMDB GOTOOCHAIN GOVERSION
/home/ubuntu/go
/home/ubuntu/go/pkg/mod
https://proxy.golang.org,direct
sum.golang.org
auto
go1.22.4

$ go list -m all
github.com/acme/payments
github.com/go-chi/chi/v5 v5.0.11
github.com/jackc/pgx/v5 v5.6.0
github.com/jackc/pgx/v5/pgconn v5.6.0
github.com/jackc/puddle/v2 v2.2.1
go.uber.org/multierr v1.11.0
go.uber.org/zap v1.27.0
golang.org/x/crypto v0.24.0
golang.org/x/sync v0.7.0
golang.org/x/sys v0.20.0
golang.org/x/text v0.16.0

$ go list -m -versions github.com/jackc/pgx/v5
github.com/jackc/pgx/v5 v5.0.8 v5.1.0 v5.2.0 v5.3.1 v5.4.3 v5.5.4 v5.6.0 v5.7.1

$ go list -m -json github.com/jackc/pgx/v5 | head -30
{
  "Path": "github.com/jackc/pgx/v5",
  "Version": "v5.6.0",
  "Time": "2024-06-11T15:23:44Z",
  "Dir": "/home/ubuntu/go/pkg/mod/github.com/jackc/pgx/
v5@v5.6.0",
  "GoMod": "/home/ubuntu/go/pkg/mod/cache/download/github.com/
jackc/pgx/v5/@v/v5.6.0.mod",
  "GoVersion": "1.22"
}

$ go version -m ./payments
github.com/acme/payments
build GOOS=linux
build GOARCH=amd64
build compiler=gc
build CGO_ENABLED=0
build GOEXPERIMENT=
dep github.com/go-chi/chi/v5 v5.0.11 h1:rXQB01TR1S...
dep github.com/jackc/pgx/v5 v5.6.0 h1:7Uy0...
dep golang.org/x/sys v0.20.0 h1:...
build vcs=git
build vcs.revision=9f3c1a7d2e4b0c6f8a1b2c3d4e5f6a7b8c9d0e1f

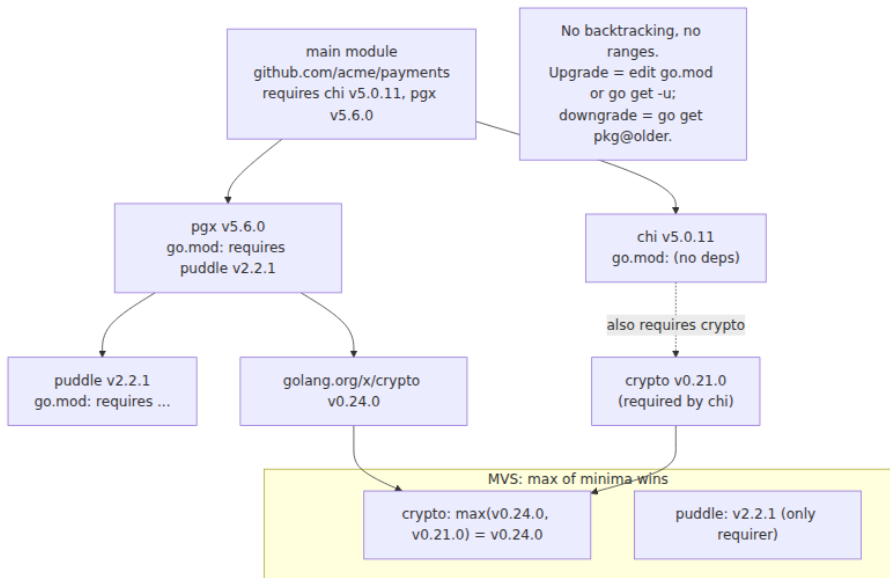
```

```
build    vcs.time=2024-11-15T14:22:31Z
build    vcs.modified=false
```

`go version -m` is your binary's SBOM-lite: it is the only reliable way to answer "what did we ship?" for a Go binary without external metadata. It is populated automatically since Go 1.18 (embed via `runtime/debug.ReadBuildInfo` and the `GODEBUG` `buildinfo` section). Strip it with `-ldflags="-buildvcs=false"` and you lose auditability — do not.

MVS — Minimal Version Selection

Go's version selection is **not** SAT solving, ranges, or "latest wins." It is *Minimal Version Selection* (MVS): for each module path, pick the *minimum* version that satisfies every `require` in the transitive closure. Conceptually:

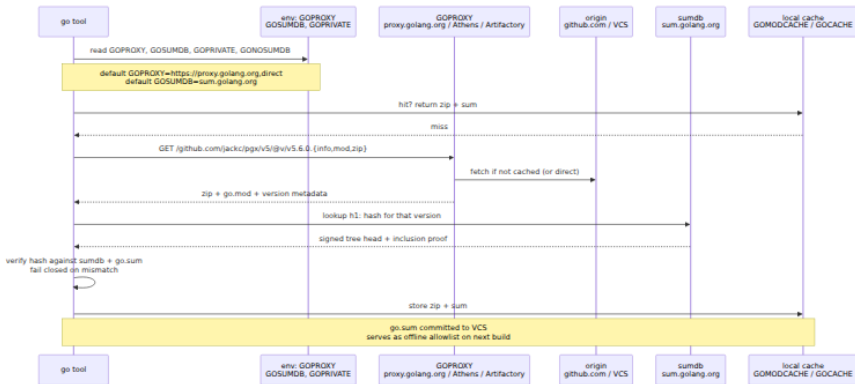


Consequences that matter in a polyrepo fleet:

- Builds are *reproducible* given the same `go.mod` / `go.sum` — the same inputs always select the same versions, even years later, unlike range-based resolvers.
- Upgrading a transitive dependency requires an explicit `go get go lang.org/x/crypto@v0.28.0` in the main module; you cannot get it "for free" by waiting.

- Diamond conflicts resolve deterministically to the *highest minimum*, not the latest — which can pin you to an older minor than the registry's latest. `go list -m all` is the truth; `go get -u ./...` is the upgrade path (review with `go mod tidy` + `go test ./...`).

Proxies, sumdb, and the supply-chain boundary



The three knobs:

```

# Proxy chain – first success wins; 'direct' means fetch from origin VCS
export GOPROXY="https://proxy.example.com,https://proxy.golang.org,direct"

# In air-gapped / hermetic CI – no network at all
export GOPROXY="off" # fail if not in local cache / vendor
export GOPROXY="https://proxy.internal" # corporate proxy only

# Checksum database – where hashes are verified
export GOSUMDB="sum.golang.org"
# default; set to "off" only with GOPROXY=off
export GONOSUMDB="github.com/acme/*" # skip sumdb for private repos

# Private modules – never hit public proxy or sumdb
export GOPRIVATE="github.com/acme/*,github.com/acme-corp/*"
export GONOSUMCHECK="github.com/acme/*" # legacy alias for GONOSUMDB
export GONOPROXY="github.com/acme/*"

# Authentication for private fetches – use git credential helper or .netrc
git config --global url."https://${GH_TOKEN}@github.com/".insteadOf "https://github.com/"
  
```

Operational notes:

- **Always commit `go.sum`**. It is not a lockfile in the npm sense — it is a hash allowlist that makes builds fail closed when a proxy serves tampered content. Without `go.sum` in VCS, `GOSUMDB` still protects you online, but offline/air-gapped builds lose verification.
 - **`GOPROXY=off` + `GOSUMDB=off` is the hermetic pair**. Use it in sandboxed CI (Bazel-style) where the only allowed inputs are the local module cache or a `vendor/` directory (`go mod vendor` + `go build -mod=vendor`). Otherwise default `proxy.golang.org` is the hermetic-friendly middle: deterministic, cached, auditable.
 - **Private modules need `GOPRIVATE`**. Without it, `cmd/go` leaks your private import paths to the public proxy and sumdb. Set `GOPRIVATE` globally in developer shells and CI.
-

3. The `go` commands you actually use

`go build`, `go vet`, `go test`, `go work`

```

# Build – package or binary
go build ./...                                # compile all packages under .
go build -o /tmp/payments ./cmd/api           # single binary
go build -tags integration ./...              # include files with //go:build
integration
go build -trimpath -ldflags "-s -w -X main.version=v1.2.3" ./cmd/api
go build -buildmode=pie -o api ./cmd/api      # position-independent
(ASLR-friendly)
go build -a -x ./... 2>&1 | head -60
# force rebuild, print every tool invocation

# Vet – static checks (uses same loader as build)
go vet ./...
# printf mismatches, unreachable code, lost cancels
go vet -vettool=$(which shadow) ./... # + third-party analyzers (e.g.,
shadow, nilness)

# Test – build + run, with cache
go test ./...                                # run tests, cached by inputs
go test -run TestPayments -count=1 -v ./internal/payments
go test -race -count=1 ./...                 # race detector (instrumented
build, see Ch 10)
go test -cover -coverprofile=cover.out ./... && go tool cover -html=cov
er.out
go test -json ./... | tparse                  # structured output (also: go
test -test2json)

# Workspaces – multi-module repos (Go 1.18+)
cat go.work
# go 1.22.4
# use (
#     .
#     ./shared
#     ./tools/gen
# )
go work sync                                # sync workspace's go.work.sum
go work use ./new-service                    # add a module to the workspace
go list -m all                               # resolves across workspace
members

# Introspection – the commands you run before asking for help
go env                                       # all env (GOCACHE, GOOS/
GOARCH, GOPROXY, etc.)
go env -w GOPROXY=off GOSUMDB=off          # persist to go env file (~/.go/
env or $GOENV)
go list -m -json all                         # machine-readable module graph
go list -f '{{.ImportPath}} {{.Deps}}' ./internal/payments
go list -deps ./cmd/api | xargs go list -f '{{.ImportPath}}
{{.Module}}'
go version -m $(which payments)             # audit a deployed binary

```


go env — the real output to know

```
$ go env
GO111MODULE='on'
GOARCH='amd64'
GOOS='linux'
GOCACHE='/home/ubuntu/.cache/go-build'
GOMODCACHE='/home/ubuntu/go/pkg/mod'
GOPATH='/home/ubuntu/go'
GOPROXY='https://proxy.golang.org,direct'
GOSUMDB='sum.golang.org'
GOTOOLCHAIN='auto'
GOVERSION='go1.22.4'
CGO_ENABLED='1'
CC='gcc'
CXX='g++'
AR='ar'
GOEXPERIMENT=''
GODEBUG=''
```

go build -x -a — reading the trace

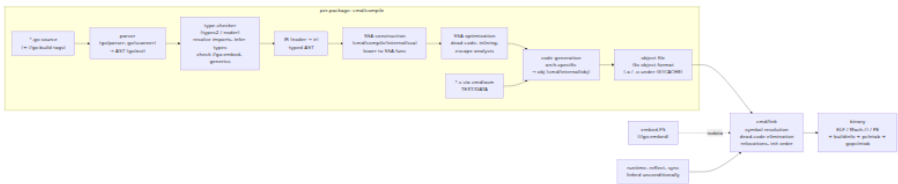
```
$ go build -x -a -o /tmp/payments ./cmd/api 2>&1 | head -80
WORK=/tmp/go-build123456789
mkdir -p $WORK/b001/
cat >$WORK/b001/importcfg << 'EOF' # internal
# import config
packagefile crypto=/tmp/go-build.../_pkg_crypto.a
packagefile github.com/go-chi/chi/v5=/tmp/go-build.../_pkg_chi.a
importmap golang.org/x/sys/unix=golang.org/x/sys/unix
EOF
cd /home/ubuntu/src/github.com/acme/payments
/usr/local/go/pkg/tool/linux_amd64/compile -o $WORK/b001/_pkg_.a \
-trimpath "$WORK/b001=>" -p github.com/acme/payments/internal/
payments \
-importcfg $WORK/b001/importcfg -pack ./internal/payments/
payments.go \
./internal/payments/handler.go
/usr/local/go/pkg/tool/linux_amd64/compile -o $WORK/b002/_pkg_.a \
-p github.com/acme/payments/cmd/api -importcfg $WORK/b002/importcfg \
-pack ./cmd/api/main.go
/usr/local/go/pkg/tool/linux_amd64/buildid -w $WORK/b002/_pkg_.a #
internal
cp $WORK/b002/_pkg_.a /home/ubuntu/.cache/go-build/3a/.../d
/usr/local/go/pkg/tool/linux_amd64/link -o /tmp/payments \
-importcfg $WORK/b001/importcfg -buildmode=exe \
-buildid=abc123... -extld=gcc $WORK/b002/_pkg_.a
```

What to read from this:

- Each `compile` line is one package (parallel when the DAG allows).
 - `-p` is the import path, `-importcfg` is the resolved dependency map,
 - `-trimpath` strips local filesystem paths from the object(reproducibility, Section 6).
- `buildid` stamps each object and the final binary — used by the build cache and by `go version -m` verification.
- `link` is the last step, consuming the main package's object plus every `packagefile` transitively. `-extld=gcc` appears only when `cgo` forced external linking.

4. The build pipeline — parse to binary

Stage by stage



A single package flows: **parse** → **type-check** → **IR** → **SSA** → **optimize** → **generate** → **object**. The driver repeats this per package, respecting import order, then links once.

Parser. `go/parser` is fast and non-allocating where possible; it handles build constraints (`//go:build` expressions, filename suffixes) by *excluding* files before they enter type-check. A file that does not match the current `G00S` / `GOARCH` / `tags` is as if it does not exist.

Type-check / IR. The type-checker (`types2`, unified IR since Go 1.18) resolves every identifier, checks assignability, expands generics via type instantiation (see Ch 2), and validates `//go:embed` directives. The result is a typed IR that is lowered to SSA.

SSA. Chapter 9 is the SSA deep dive; the short version here: each function becomes a `Func` of `Block`s of `Value`s (an explicit CFG). This is where inlining, dead-code elimination, bounds-check elimination, and escape analysis happen. The SSA form is architecture-independent; code generation is not.

Code generation. SSA is lowered through `cmd/internal/obj` into machine code for `GOARCH`. Assembly files (`*.s`) are assembled by `cmd/asm` directly into the same object format — the two paths converge at `OBJ`. No LLVM, no `as`.

Objects. Go's object files are not ELF `.o`. They carry Go symbol tables, type descriptors, `pcntab` (program-counter → line mapping for stack traces and `pprof`), and `go.debug.*` sections, plus relocations the internal linker understands. `go tool nm` and `go tool objdump` read them.

The internal linker — no `ld`

`cmd/link` does what Unix `ld` does — symbol resolution, relocation, section layout — plus Go-specific work:

- **Reachability pruning.** Starting from `main.main` and `runtime` entry points, the linker walks reference edges and drops unreachable symbols (similar to `ld --gc-sections`, but per-symbol and mandatory). An imported package that you never call may contribute *zero* bytes.
- **Init ordering.** Topologically sorts `init` functions and package initialization so `var` initializers and `init()` run in dependency order before `main`.
- **Buildinfo injection.** Embeds `runtime/debug.BuildInfo` (`dep` lines for `go version -m`), VCS stamping (`vcs.revision`, `vcs.time`, `vcs.modified`), and the action `buildid`.
- **Section layout.** Emits the binary sections: `text` (code), `rodata` (constants, `embed.FS` bytes), `noptrdata` / `data` / `bss` (globals), `pcntab`, `syntab`, `buildinfo`.

```

# Inspect what the linker did
go tool nm ./payments | head -30
403000 T _rt0_amd64_linux
404000 T runtime.text
44a200 T main.main
44b800 T github.com/go-chi/chi/v5.(*Mux).ServeHTTP
500000 D runtime.firstmoduledata
5a0000 B runtime.bss

go tool objdump -s main.main ./payments | head -40
TEXT main.main(SB) /home/ubuntu/src/github.com/acme/payments/cmd/api/
main.go
    main.go:12      0x44a200      493b6618      CMPQ 0x18(R14), SP
    main.go:12      0x44a204      7667         JBE 0x44a26d
    main.go:12      0x44a206      55          PUSHQ BP
    ...

# Linker map (Go 1.21+): where bytes went
go build -o /tmp/payments -ldflags="-tmpdir=/tmp" ./cmd/api
# or explicitly:
go tool link -map=/tmp/link.map -o /tmp/payments $WORK/b002/_pkg_.a #
rarely invoked directly
cat /tmp/link.map | head -50
# 0x403000 0x404000 runtime.text
# 0x44a200 0x001200 main.main + dependencies
# ...

# Buildinfo is a section – prove it exists
go version -m ./payments
readelf -p .go.buildinfo ./payments 2>&1 | head -20 # ELF
otool -s __TEXT __go_buildinfo ./payments 2>&1 | head -20 # Mach-O

```

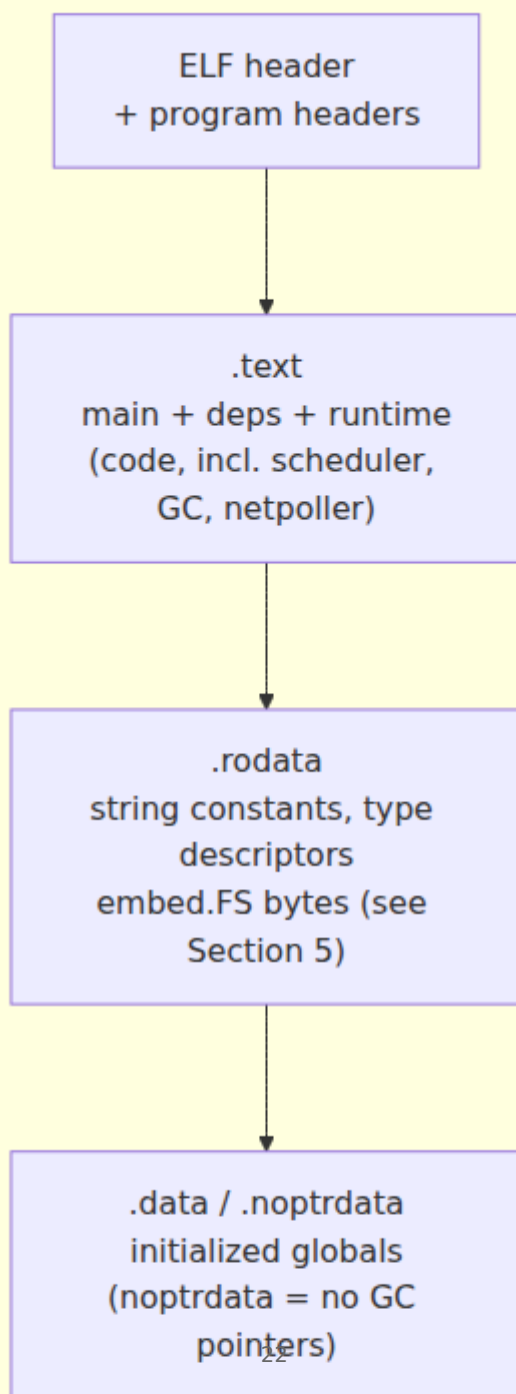
The static binary model and the `cgo` exception

By default (`CGO_ENABLED=0`), the output is a **fully static binary**: no `DT_NEEDED`, no `libc` at runtime, no interpreter beyond the kernel's ELF loader.

```
$ CGO_ENABLED=0 go build -o /tmp/payments ./cmd/api
$ file /tmp/payments
/tmp/payments: ELF 64-bit LSB executable, x86-64, statically linked,
Go BuildID=..., not stripped
$ ldd /tmp/payments
        not a dynamic executable
$ readelf -d /tmp/payments | grep NEEDED
# (no output – zero dynamic dependencies)

$ nm -D /tmp/payments 2>&1 | head
nm: /tmp/payments: no symbols
# no dynamic symbol table either (unless -buildmode=pie with external
link)
```

static Go binary (CGO_ENABLED=0) — ELF layout



The `cgo` exception. As soon as any imported package (transitively) contains `import "C"`, the build sets `CGO_ENABLED=1` (unless forced to `0`, which then *fails* the build rather than silently producing a different binary) and final linking is delegated to `CC`:

```
$ go list -f '{{.ImportPath}}: CGO={{.CgoFiles}}' ./... | grep -v
"CGO=\[\]"
net: CGO=[cgo_linux.go]

$ CGO_ENABLED=1 go build -o /tmp/payments-cgo ./cmd/api
$ ldd /tmp/payments-cgo
    linux-vdso.so.1 (0x00007fff...)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
    libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0
$ file /tmp/payments-cgo
/tmp/payments-cgo: ELF 64-bit LSB executable, x86-64, dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2

$ CGO_ENABLED=0 go build -o /tmp/payments ./cmd/api # force static -
fails if any dep needs cgo
# package net: CGO_ENABLED=0 but cgo is required (only if you actually
import net with cgo path)
```

Rules:

- `CGO_ENABLED=0` + `net` still works — `net` has a pure-Go resolver fallback (`GODEBUG=netdns=go`). You lose `cgo`-based DNS/nss integration but gain a static binary. Most backend services set `CGO_ENABLED=0`.
- `CGO_ENABLED=1` + cross-compilation requires a cross `CC` (e.g., `CC=aarch64-linux-gnu-gcc GOOS=linux GOARCH=arm64 go build`). Without it, the build fails at link time.
- `CGO_ENABLED=0` binaries run from `FROM scratch` / `gcr.io/distroless/static` with no `libc`, no shell, and a ~5–20 MB image. `cgo` binaries need `glibc` or `musl` in the image and a larger attack surface.

5. Controlling what ships — constraints, embed, ldflags, buildmode, reproducibility

Build constraints and tags

Two mechanisms control which files participate in a build:

1. **Filename suffixes** — `_GOOS`, `_GOARCH`, `_GOOS_GOARCH` (e.g., `resolver_linux.go`, `asm_amd64.s`, `net_windows.go`).
2. **//go:build expressions** — boolean logic over `GOOS`, `GOARCH`, `cgo`, `go1.N`, and user tags.

```
// internal/payments/handler_linux.go – only on linux
//go:build linux

package payments
// uses epoll-specific tuning

// internal/payments/handler_other.go – everywhere else
//go:build !linux

package payments

// internal/debug/pprof.go – only with the 'debug' tag
//go:build debug

package debug
import _ "net/http/pprof" // only linked when -tags=debug

// tools/gen/codegen.go – never built as part of the service
//go:build ignore

package main // excluded from all builds unless you override with
-tags=ignore
```

```
go build -tags debug,integration ./... # enable custom tags (comma-
separated, OR semantics)
go list -f '{{.GoFiles}}' ./internal/payments # which files
matched for current GOOS/GOARCH
GOOS=windows go list -f '{{.GoFiles}}' ./internal/net # cross-check a
different target
go vet -tags debug ./... # vet must see the same
constraints as build
```


Pitfall: `// +build` (old syntax) is ignored since Go 1.17 — only `//go:build` matters. `gofmt` rewrites them, but a stale `// +build` line silently stops constraining. Run `go vet` — it warns.

embed — shipping data inside the binary

`//go:embed` (Go 1.16+) bakes files into `rodata` at link time, with no runtime filesystem dependency:

```
package config

import _ "embed"

//go:embed migrations/*.sql
var migrationsFS embed.FS

//go:embed VERSION
var version string // single file as string or []byte

//go:embed static/*
var staticFiles embed.FS // directory tree

func LoadMigrations() []string {
    entries, _ := fs.ReadDir(migrationsFS, "migrations")
    // entries are in the binary — no volume mount, no init container
    _ = entries
    return nil
}
```

Constraints: the embedded path must be inside the module and is evaluated at *compile* time — `go build` reads the files and the linker places bytes in `rodata`. Changing an embedded file invalidates the build cache for that package (content-addressed). For large assets (ML models, GeoIP DBs), prefer downloading at startup over embedding a 200 MB `rodata`.

Link flags — stamping versions without code changes

```
# -X sets a string var at link time (must be fully-qualified, must be
string)
go build -ldflags "
    -s -w
    -X main.version=v1.2.3
    -X main.commit=$(git rev-parse HEAD)
    -X main.date=$(date -u +%Y-%m-%dT%H:%M:%SZ)
" -trimpath -o /tmp/payments ./cmd/api

# -s: omit symtab, -w: omit DWARF — smaller binary, but worse stack
traces/pprof
# omit -s -w in production services where you need debug info; keep for
CLIs

# Verify
go version -m /tmp/payments | grep -E "version|commit"
# build -ldflags=-s -w -X main.version=v1.2.3 -X
main.commit=9f3c1a7...

# Common pattern — allow -X even for private main packages
# main/version.go
package main
var (
    version = "dev"    // overwritten by -X
    commit  = "none"
    date    = "unknown"
)
```

Also available: `-buildmode`, `-buildvcs`, `-linkmode`:

Flag	Effect
<code>-ldflags "-s -w"</code>	Strip symbol table and DWARF — smaller, harder to debug
<code>-ldflags "-X importpath.name=value"</code>	Set string variable
<code>-ldflags "- linkmode=external"</code>	Force external link even when internal would work (needed for some cgo + <code>-buildmode=c-archive</code>)
<code>-buildmode=exe</code>	Default: executable (internal link when <code>CGO_ENABLED=0</code>)

Flag	Effect
<code>-buildmode=pie</code>	Position-independent executable (ASLR) — required on some distros, default on <code>GOARCH=arm64</code> with external link
<code>-buildmode=c-archive</code> / <code>c-shared</code>	Produce <code>.a</code> / <code>.so</code> callable from C (always external link)
<code>-buildmode=plugin</code>	Go plugin (linux only, discouraged for services)
<code>-buildvcs=true/false</code>	Include/omit VCS stamping in <code>go version -m</code>
<code>-trimpath</code>	Remove filesystem paths from objects — reproducibility
<code>-tags netgo,osusergo</code>	Pure-Go <code>net</code> / <code>os</code> / <code>user</code> even with <code>CGO_ENABLED=1</code> — useful to keep static while allowing cgo elsewhere
<code>-a</code>	Force rebuild of all packages (ignore cache)
<code>-race</code> / <code>-msan</code> / <code>-asan</code> / <code>-cover</code>	Instrumented builds (see Ch 10)

Reproducible builds

Go builds are reproducible *given the same inputs and toolchain* — the remaining non-determinism comes from paths and timestamps:

```

# Reproducible build – the three ingredients
go build -trimpath -buildvcs=false \
  -ldflags "-buildid= -X main.version=v1.2.3" \
  -o /tmp/payments ./cmd/api

# -trimpath – removes $WORK and absolute dir from pcIntab/object paths
# -buildvcs=false – omits git hash/time (which changes per commit)
# -ldflags "-buildid=" – deterministic build ID (empty means hash of inputs)
# Also: set SOURCE_DATE_EPOCH for embedded timestamps
SOURCE_DATE_EPOCH=$(git log -1 --format=%ct) go build -trimpath -o /tmp/payments ./cmd/api

# Verify bit-for-bit
sha256sum /tmp/payments /tmp/payments2 | awk '{print $1}' | uniq -c
# 2 <same-hash> – identical

```

In CI, combine `-trimpath` with pinned `toolchain` and `GOTOOLCHAIN=auto` so every runner downloads the same compiler. For SLSA provenance (Companion Book 4, Ch 3), add `go version -m` output and `slsa-framework/slsa-github-generator` to attest the build — the embedded `buildinfo` is the verified claim.

6. Toolchain versioning and the build cache

The `go` and `toolchain` directives — pinning the compiler

Since Go 1.21, `go.mod` carries two version lines with distinct semantics:

```

module github.com/acme/payments

go 1.22.0           // language version + minimum toolchain that
understands this file
toolchain go1.22.4 // exact toolchain for reproducible builds (written
by 'go mod tidy' / 'go get')

```

Directive	Semantics	Who enforces
<code>go 1.22.0</code>	Declares the Go <i>language</i> version used; <code>cmd/go</code> with an older <code>go</code> refuses to build, newer <code>go</code> enables newer language features only if this line allows it	<code>cmd/go</code> at build time

Directive	Semantics	Who enforces
<code>toolchain</code> <code>go1.22.4</code>	Declares the <i>implementation</i> that produced the current <code>go.sum</code>	<code>cmd/go</code> via <code>GOTOOLCHAIN</code>

`GOTOOLCHAIN` controls runtime behavior:

```
# auto (default since Go 1.21) – download and delegate if local go !=
toolchain line
GOTOOLCHAIN=auto go build ./...    # fetches go1.22.4 if local is
go1.23

# local – never download, fail if local go is too old
GOTOOLCHAIN=local go build ./...    # hermetic CI without network

# go1.22.4 – pin to an exact toolchain regardless of go.mod
GOTOOLCHAIN=go1.22.4 go build ./...

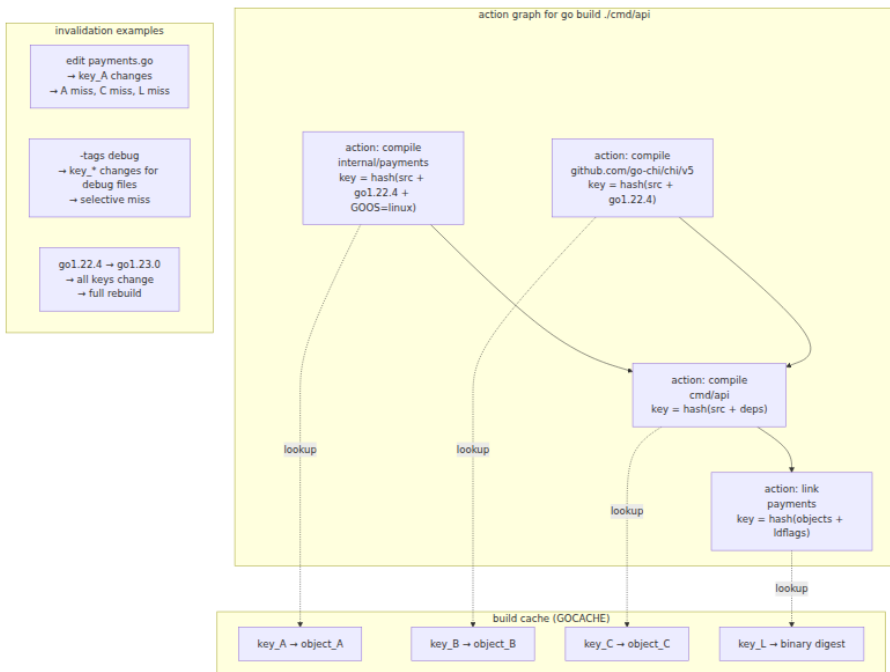
go env GOTOOLCHAIN GOVERSION
# GOTOOLCHAIN='auto'
# GOVERSION='go1.22.4'
```

For fleets, set `toolchain` via `go get toolchain go1.22.4` and commit. CI uses `GOTOOLCHAIN=local` + a container image that already contains that Go version — no implicit downloads. Developers keep `auto` so `go` self-upgrades.

The build cache and action graph

Every `go build` computes an *action graph* — nodes are "compile package P with flags F and deps D" or "link binary B from objects O". Each node's cache key is a hash of:

- Source bytes of the package (plus embedded files, `go.mod` of deps)
- Compiler version (`GOVERSION` , `GOEXPERIMENT`)
- Flags (`-tags` , `-ldflags` , `-trimpath` , `GOOS / GOARCH` , `CGO_ENABLED`)
- Hashes of dependency actions (transitive)



```

# The cache in practice
go env GOCACHE
# /home/ubuntu/.cache/go-build

du -sh $(go env GOCACHE)
# 2.1G /home/ubuntu/.cache/go-build

go clean -cache
# wipe build cache (rare – keyed correctly, so stale entries are just
# waste)
go clean -modcache # wipe module download cache (GOMODCACHE)
go clean -fuzzcache # wipe fuzz corpus cache

# Bypass cache when you must
go build -a ./... # ignore cache, rebuild all
go test -count=1 ./... # bypass test cache (go test caches passing
# results)
go list -m -json all | jq -r '.Dir' | xargs ls -ld # where each dep
# lives on disk

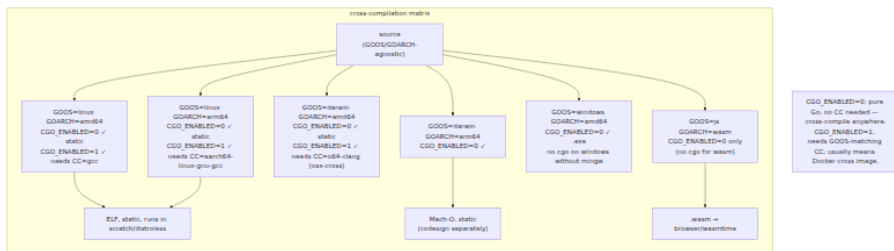
```

In CI, the build cache is the difference between a 90-second build and a 12-minute one. Persist `GOCACHE` and `GOMODCACHE` across runs (GitHub

Actions `actions/cache`, Buildkite agent cache, or a remote cache like `gocache` via `GOCACHEPROG`). Without it, every `go build` recompiles the world and re-downloads modules — even with a warm `GOPROXY` you pay compile time.

7. Cross-compilation — `GOOS / GOARCH` and when `cgo` breaks it

Cross-compilation is a first-class feature *because* the default path has no C toolchain:



The two modes

```
# Pure Go – cross-compile from anywhere, no toolchain needed
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build -o /tmp/payments-linux-
arm64 ./cmd/api
GOOS=darwin GOARCH=arm64 CGO_ENABLED=0 go build -o /tmp/payments-
darwin-arm64 ./cmd/api
GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -o /tmp/payments.exe .
/cmd/api
GOOS=js GOARCH=wasm CGO_ENABLED=0 go build -o /tmp/payments.wasm ./cmd/
api

file /tmp/payments-linux-arm64
# /tmp/payments-linux-arm64: ELF 64-bit LSB executable, ARM aarch64,
statically linked
file /tmp/payments.wasm
# /tmp/payments.wasm: WebAssembly (wasm) binary module version 0x1

# With cgo – you provide the cross compiler
GOOS=linux GOARCH=arm64 CGO_ENABLED=1 CC=aarch64-linux-gnu-gcc \
go build -o /tmp/payments-cgo-arm64 ./cmd/api
# Without CC, you get:
# gcc: error: unrecognized command-line option '-marm64' (wrong CC
for target)

# Docker-native cross build (common in CI)
docker run --rm -v $PWD:/src -w /src \
-e GOOS=linux -e GOARCH=arm64 -e CGO_ENABLED=0 \
golang:1.22.4 go build -o /tmp/payments ./cmd/api

# GoReleaser / ko handle the matrix for you – but understand what they
set
cat .goreleaser.yaml | head -30
# builds:
#   - env: [CGO_ENABLED=0]
#     goos: [linux, darwin, windows]
#     goarch: [amd64, arm64]
```

Target	CGO_ENABLED=0	CGO_ENABLED=1
linux/amd64	Static, no deps. Works everywhere.	Needs <code>gcc</code> ; <code>ldd</code> shows <code>libc</code>
linux/arm64	Static, ideal for Graviton/Apple Silicon CI	Needs <code>aarch64-linux-gnu-gcc</code> (apt: <code>gcc-aarch64-linux-gnu</code>)

Target	CGO_ENABLED=0	CGO_ENABLED=1
darwin/*	Static Mach-O; codesign/notarize separately	Needs o64-clang / zig cc with macOS SDK — usually just build on macOS
windows/*	.exe , static	Needs x86_64-w64-mingw32-gcc — rare; most Windows Go skips cgo
js/wasm	Only valid target; cgo never works	N/A
GOARCH=386 , mips* , riscv64	Static, but GOARCH may imply G0386=sse2 etc.	Rarely used with cgo

Heuristics for backend teams:

- Default CI to CGO_ENABLED=0 unless you have a proven import "C" dependency (e.g., libsqlite3 , libpcap , libgit2). The static binary is smaller, more portable, and scannable (no libc CVE surface in the image).
- If you need cgo , cross-compile via a container that already has the matching CC (golang:1.22 images ship gcc for linux/amd64 ; for arm64 , use arm64v8/golang or install the cross toolchain).
- GOOS / GOARCH also select build-constrained files — *_linux.go vs *_darwin.go — so test cross targets with GOOS=darwin go test ./... even when developing on Linux.

8. Backend lens — hermetic builds, distroless containers, supply-chain attestations

Hermetic Go builds in CI

A hermetic build's inputs are fully declared; network access during build is either unnecessary or fails closed. Go supports this with three primitives you combine:

```

# GitHub Actions – hermetic Go build
jobs:
  build:
    runs-on: ubuntu-latest
    container: golang:1.22.4 # pinned toolchain == go.mod toolchain
line
    env:
      GOPROXY: "off"           # no network – use local cache or vendor/
      GOSUMDB: "off"
      GOTOOLCHAIN: "local"     # never download a different go
      CGO_ENABLED: "0"
    steps:
      - uses: actions/checkout@v4
      - name: Restore module + build cache
        uses: actions/cache@v4
        with:
          path: |
            ~/go/pkg/mod
            ~/.cache/go-build
          key: go- $\{\{\text{hashFiles('go.sum')}\}\}$ - $\{\{\text{hashFiles('go.mod')}\}\}$ 
          restore-keys: go-
      - name: Verify modules
        run: go mod download # fails if go.sum / go.mod inconsistent
      - name: Vet + test
        run: |
          go vet ./...
          go test -count=1 -race ./...
      - name: Build (reproducible)
        run: |
          go build -trimpath -buildvcs=true \
            -ldflags "-s -w -X main.version= $\{\{\text{github.sha}\}\}$ " \
            -o /tmp/payments ./cmd/api
          go version -m /tmp/payments | tee /tmp/buildinfo.txt
      - name: Attest (SLSA)
        uses: slsa-framework/slsa-github-generator/.github/workflows/
generator_generic_slsa3.yml@v1.9.0
        with:
          base64-subjects: " $\{\{\text{hashFiles('/tmp/payments')}\}\}$ "

```

Alternatives:

- **go mod vendor + go build -mod=vendor**. Checks the full module source into vendor/ so GOPROXY=off works without a cache. Trade-off: large repo, but fully offline and auditable (git diff vendor/ shows what changed).
- **GOPROXY=https://proxy.internal,direct + GOSUMDB=sum.golang.org**. Corporate proxy caches the public ecosystem; sumdb still verifies. Developers get fast, hermetic-ish builds without vendoring.

Failure mode without hermeticity: a `GOPROXY` outage or a dependency deletion (left-pad-style) breaks every pipeline. With `vendor/` or a persisted `GOMODCACHE` + `GOSUMDB=off`, the build is immune.

Distroless static binaries — the container payoff

The static binary model exists to make containers minimal and auditable:

```
# Dockerfile — multi-stage, static, distroless
FROM golang:1.22.4 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
ARG VERSION=dev
RUN CGO_ENABLED=0 go build \
    -trimpath \
    -ldflags "-s -w -X main.version=${VERSION}" \
    -o /out/payments ./cmd/api \
    && go version -m /out/payments > /out/buildinfo.txt

# Runtime — no shell, no libc, no package manager
FROM gcr.io/distroless/static:nonroot
COPY --from=build /out/payments /payments
COPY --from=build /out/buildinfo.txt /buildinfo.txt
USER nonroot:nonroot
ENTRYPOINT ["/payments"]

# Alternatives for the last line:
# FROM scratch — even smaller, but no certs/user; add ca-
# certificates yourself
# FROM gcr.io/distroless/static-debian12 — when you need /etc/ssl/certs
# + tzdata without a shell
```

Why this matters at fleet scale:

- **FROM scratch / distroless/static images are ~5-15 MB** for a typical service vs. ~100-300 MB for `debian / alpine` with `libc`. Pull time, CVE surface, and attack surface shrink together. A Trivy scan of a distroless Go image reports only Go stdlib CVEs (via `go version -m` inventory), not 200 OS packages.
- **No shell = no shell injection.** An RCE that yields code execution still has no `sh` to pivot with. Pair with `readOnlyRootFilesystem: true` and `allowPrivilegeEscalation: false`.
- **CGO_ENABLED=0 is required.** A `cgo` binary in `FROM scratch` fails at startup (no such file or directory — the dynamic interpreter is

missing). If you must ship `cgo`, use `distroless/base` (has `glibc`) or `chainguard/static`.

Supply chain — `sumdb`, `buildinfo`, and attestations

Go's supply chain is unusually strong by default:

Control	What it proves	Where it lives
<code>go.sum</code> + <code>sum.golang.org</code>	The module zip you built is the one the author published — tampering by proxy or mirror is detected	Committed to VCS; verified on every <code>go mod download</code>
<code>go version -m /runtime/</code> <code>debug.ReadBuildInfo</code>	The binary you deployed was built from these exact dependency versions + VCS revision	Embedded in <code>.go.buildinfo</code> by the linker
<code>GODEBUG buildinfo + vcs.*</code> keys	Git commit, dirty flag, and build settings (<code>CGO_ENABLED</code> , <code>GOOS</code> / <code>GOARCH</code>) travel with the binary	Same section; visible without source
LSA provenance (e.g., <code>slsa-github-generator</code>)	The build ran on this runner, from this commit, with this toolchain, producing this digest	Attestation store (Rekor / OCI) — verify with <code>slsa-verifier</code>

```
# Runtime self-report – every service should expose this
go run ./cmd/api --version # or /debug/buildinfo, /metrics
# payments v1.2.3 (commit 9f3c1a7, built with go1.22.4, GOOS=linux
GOARCH=amd64)
```

```
# In code – log buildinfo at startup
info, _ := debug.ReadBuildInfo()
log.Printf("build: main=%s go=%s vcs=%s mods=%d",
    info.Main.Version, info.GoVersion,
    vcsRevision(info), len(info.Deps))
```

```
# Verify an attestation (after slsa-github-generator)
slsa-verifier verify-artifact /tmp/payments \
--provenance-path /tmp/payments.intoto.jsonl \
--source-uri github.com/acme/payments
```

The missing piece before Go 1.18 was binary provenance — you needed an external SBOM. Since Go 1.18, the binary is its own provenance. Scanners (govulncheck, Trivy, osv-scanner) read go version -m directly; govulncheck ./... at build time uses the same GOVERSION + GOSUMDB to flag known CVEs before the image ships.

Key takeaways

- cmd/go is the build system. It computes an action graph, compiles packages in parallel via cmd/compile / cmd/asm, and links with the internal cmd/link — no external ld and no Makefile on the happy path. Read go build -x and go env to see what it actually did.
- Modules replaced GOPATH with content-addressed, verifiable dependencies. go.mod declares intent, go.sum locks hashes, GOPROXY / GOSUMDB / GOPRIVATE control fetch and verification. Commit go.sum, set GOPRIVATE for private code, and use GOPROXY=off + GOSUMDB=off for hermetic CI.
- MVS is deterministic: every require contributes a minimum version, the maximum minimum wins. Upgrades are explicit (go get pkg@version), not range-resolved. go list -m all is the source of truth.
- The compile pipeline is parse → type-check → IR → SSA → optimization → codegen → object file, per package, then a single internal link. cgo is the exception that delegates to CC and produces a dynamic binary — default to CGO_ENABLED=0 for static, distroless-friendly artifacts.
- Build constraints (//go:build, filename suffixes), embed, and link flags (-ldflags -X, -buildmode, -trimpath, -tags) control what reaches the binary. -trimpath + -buildid= + SOURCE_DATE_EPOCH give reproducible builds; -s -w trades debuggability for size.
- The go / toolchain directives plus GOT00LCHAIN=auto|local pin the compiler itself. go version -m embeds the full buildinfo (deps, VCS revision, GOOS / GOARCH, CGO_ENABLED) — the binary's built-in SBOM.
- The build cache (GOCACHE) keys actions on source + toolchain + flags + transitive hashes. Persist it in CI; bypass with -a or -count=1 when you must. GOMODCACHE is the module download cache — distinct from the build cache.

- Cross-compilation via `GOOS / GOARCH` is free for pure Go (`CGO_ENABLED=0`). With `cgo` , you need a target-matched `CC` — cross-compile in a container that has it, or avoid `cgo` entirely.
 - Hermetic CI + distroless images + sumdb/buildinfo is the backend pattern: pinned toolchain, `-trimpath` builds, `go vet / go test -race` gates, FROM scratch / distroless/static runtime, and SLSA attestations that `go version -m` can verify at deploy time.
-

Further reading

- Go Modules Reference — `go.dev/ref/mod` — authoritative spec for `go.mod / go.sum` , MVS, `GOPROXY` , and `GOSUMDB` . **(pinned)**
- Go Toolchain Documentation — `go.dev/doc/toolchain` — `GOTOOLCHAIN` , `go / toolchain` directives, and toolchain switching since Go 1.21. **(pinned)**
- `cmd/link` Documentation — `pkg.go.dev/cmd/link` — internal linker flags, `-buildmode` variants, `-linkmode` , symbol handling, and `-map` output. **(pinned)**
- Go Build Constraints — `pkg.go.dev/go/build#hdr-Build_Constraints` — `//go:build` expression grammar, filename suffixes, and `GOOS / GOARCH` tag sets.
- Proposal: Minimal Version Selection — `research.swtch.com/vgo-mvs` (Russ Cox, 2018) — original MVS rationale and comparison with SAT/range solvers. **(pinned)**
- Proposal: Toolchain Versioning (`GOTOOLCHAIN`) — `go.dev/doc/toolchain` + `golang.org/issue/57001` — design of `toolchain` line and `GOTOOLCHAIN=auto` delegation.
- `go` Command Documentation — `pkg.go.dev/cmd/go` — complete reference for `go build / vet / test / work / list / env / version` flags and environment variables.
- Reproducible Builds — `go.dev/doc/reproducible` (via `go help build` and `go.dev/blog/reproducible`) — `-trimpath` , `-buildid` , `SOURCE_DATE_EPOCH` , and bit-for-bit verification.
- Go Assembly — `go.dev/doc/asm` — `cmd/asm` syntax (`TEXT` , `DATA` , `FUNCDATA`) and how assembly objects feed `cmd/link` .

- `govulncheck` and Go Vulnerability Database — `go.dev/security/vuln` + `pkg.go.dev/golang.org/x/vuln/cmd/govulncheck` — scanning `go version -m buildinfo` against `vuln.go.dev`.
- SLSA Framework for Go — `slsa.dev` + `github.com/slsa-framework/slsa-github-generator` — provenance generation for `go build` artifacts verified with `slsa-verifier`.

Chapter 2 — Types, Memory Layout, and Generics Internals

What this chapter covers. Every Go program is a study in memory layout. Unlike Python or Java, Go exposes the machine directly — structs have deterministic padding, slices are thin headers over arrays, strings are immutable byte views, and maps are hash tables with inlined buckets. Since Go 1.18, generics add a compile-time polymorphism layer that must reconcile with this layout model: type parameters are constrained by type sets, instantiated via GC-shape stenciling, and emitted as shared or specialized code. This chapter explains how Go types occupy memory, how to measure and optimize that layout, and how generics are implemented under the hood — with the backend lens of cache lines, allocation pressure, and binary size.

Learning goals — after this chapter you should be able to:

- Distinguish type identity, assignability, and convertibility per the Go spec, and explain when two types are identical vs. merely assignable.
- Predict and measure struct size, alignment, and padding; use `unsafe.Sizeof`, `unsafe.Offsetof`, `unsafe.Alignof` and `go vet -fieldalignment` to diagnose and fix layout waste.
- Describe the in-memory representation of arrays, pointers, slices (`ptr/len/cap`), strings (`ptr/len`), and maps (`hmap / bmap` with buckets and overflow).
- Write generic functions and types with correct constraints (`any`, `comparable`, `~T`, union type sets) and explain what `comparable` actually constrains.

- Explain generics compilation: the 1.18 dictionary approach, the 1.21+ GC-shape stenciling model, and the monomorphization vs. sharing tradeoff (code bloat vs. speed vs. compile time).
- Apply cache-line and escape-analysis awareness to struct field ordering and generic instantiation in hot backend paths.

Scope. This chapter is the type-and-memory foundation for Volume 17. Chapter 3 covers functions, methods, and the ABI (how values are passed). Chapter 7 covers interfaces and `unsafe` in depth. Chapter 9 covers escape analysis and SSA optimizations. Chapter 13 (Language Runtimes) Chapter 2 gave a runtime-level view of the scheduler and GC; this chapter zooms into the type system and object layout that the GC and allocator operate on.

1. The Go Type System: Categories, Identity, and Assignability

1.1 What kinds of types exist

Go's types fall into a small, closed set. From <https://go.dev/ref/spec#Types>:

Category	Examples	Value vs. reference semantics
Basic	<code>bool</code> , <code>int</code> , <code>int8</code> - <code>int64</code> , <code>uint</code> - <code>uint64</code> , <code>uintptr</code> , <code>float32/64</code> , <code>complex64/128</code> , <code>string</code>	Value (string header is value; backing bytes are shared)
Aggregate	<code>array ([N]T)</code> , <code>struct</code>	Value — assignment copies all bytes
Reference-like	<code>slice ([]T)</code> , <code>map[K]V</code> , <code>chan T</code> , <code>func</code> , <code>pointer (*T)</code> , <code>interface</code>	Header/pointer value; backing store shared
Defined / alias	<code>type MyInt int</code> , <code>type Alias = int</code>	Defined type is new identity; alias is identical

Critically, Go has **no inheritance, no implicit numeric conversions, and no overloading**. If you write `type UserID int64`, `UserID` and `int64` are distinct types with the same memory layout but different identity.

1.2 Type identity vs. assignability vs. convertibility

These three relations are distinct in the spec and the source of many compiler errors:

- **Type identity** (go.dev/ref/spec#Type_identity): Two types are identical if they are the same type literal, or one is a type alias of the other, or both are defined types with the same definition. `type A = int` $\rightarrow$ `A` identical to `int`. `type B int` $\rightarrow$ `B` *not* identical to `int` (different defined type), though they share underlying type `int`.
- **Underlying type**: For any type `T`, strip away defined-type wrappers recursively. `type MySlice []int` has underlying type `[]int`. Underlying types matter for constraints (`~T`).
- **Assignability** (go.dev/ref/spec#Assignability): `x` is assignable to variable of type `T` when `x`'s type is identical to `T`, or `x`'s type and `T` have identical underlying types and at least one is not defined, or `T` is an interface and `x` implements it, or `x` is an untyped constant representable by `T`, etc. Example: `var x MyInt = 42` works because `MyInt`'s underlying type is `int` and `42` is an untyped constant. `var y int = MyInt(42)` requires an explicit conversion — `MyInt` value is *not* assignable to `int` variable.
- **Convertibility** (go.dev/ref/spec#Conversions): Broader. `T(V)` is valid when `T` and `V` have identical underlying types (ignoring struct tags), or both are numeric/string, or pointer conversions via `unsafe`. `MyInt(42)` converting `int` $\rightarrow$ `MyInt` is allowed even though they are not assignable.

```

package main

import "fmt"

type UserID int64
type AliasID = int64 // alias – identical to int64

func identityDemo() {
    var a int64 = 1
    var b AliasID = a // ok: AliasID identical to int64
    var c UserID = UserID(a) // ok: explicit conversion

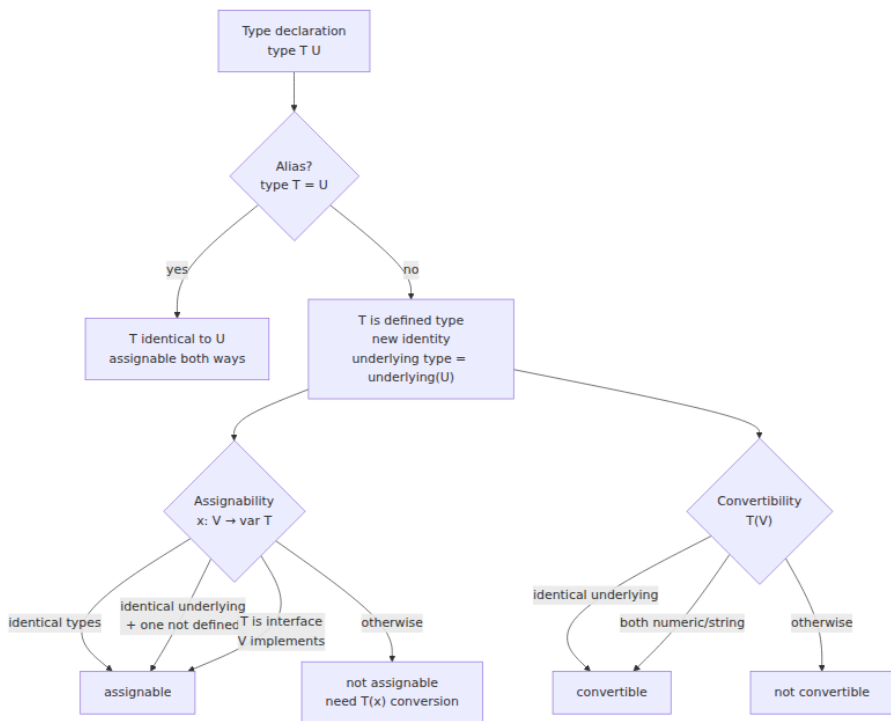
    // var d UserID = a // compile error: cannot use a (type int64) as
UserID in assignment
    // var e int64 = c // compile error: cannot use c (type UserID)
as int64

    _ = b
    _ = c

    // Interface assignability
    var w fmt.Stringer
    // w = c // error: UserID does not implement Stringer
    type S string
    func (s S) String() string { return string(s) }
    var s S = "hi"
    w = s // ok: S implements Stringer
    _ = w
}

```

Underlying type and `~T` preview: `~int` in a constraint means "any type whose underlying type is `int`" — so both `int` and `type MyInt int` match `~int`, but only `int` matches `int`. This distinction drives generics constraints (Section 7).



1.3 Named vs. unnamed, comparable, and ordered

The spec calls out two derived properties:

- **Comparable** (`==`, `!=` usable): booleans, numerics, strings, pointers, channels, interfaces, and arrays/structs whose fields are all comparable. Slices, maps, and functions are *not* comparable (except `nil` comparison). This is why `comparable` exists as a constraint — only comparable types can be map keys.
- **Ordered** (`<`, `<=`, `>`, `>=` usable): integers, floats, strings. No constraint keyword for this — you write `constraints.Ordered` from <https://golang.org/x/exp> or define type `Ordered interface { ~int | ~int8 | ... | ~string }`.

These properties are not just language trivia — they determine **map key validity**, **generic constraint correctness**, and **whether `==` on a struct silently does a deep field comparison** (which can be expensive or surprising for large structs).

2. Memory Layout Fundamentals: Size, Alignment, Padding

2.1 Size and alignment rules

Go's layout follows the same rules as C on the target ABI (System V AMD64 on `linux/amd64`), with no packing pragmas and no `#[repr]` control except field reordering:

- **Size** (`unsafe.Sizeof(x)`): bytes occupied by the value itself, not what it points to. `Sizeof(string)` is 16 (`ptr+len`), not the length of the backing bytes. `Sizeof(slice)` is 24 (`ptr+len+cap`). `Sizeof(map)` is 8 (single pointer to `hmap`).
- **Alignment** (`unsafe.Alignof(x)`): the address of an `x` must be a multiple of `Alignof(x)`. Primitives align to their size (up to pointer width): `int64` and `uint64` align to 8 on amd64; `int32` to 4; `bool` / `byte` to 1. Pointers and `uintptr` align to 8 on 64-bit.
- **Struct alignment**: the struct's alignment is the maximum alignment of its fields. Its size is rounded up to a multiple of that alignment (trailing padding).
- **Array alignment**: same as element alignment.

```

package main

import (
    "fmt"
    "unsafe"
)

func alignmentDemo() {
    var b bool
    var i32 int32
    var i64 int64
    var s string
    var p *int

    fmt.Printf("bool:          size=%d align=%d\n", unsafe.Sizeof(b), unsafe.Alignof(b))
    fmt.Printf("int32:         size=%d align=%d\n", unsafe.Sizeof(i32), unsafe.Alignof(i32))
    fmt.Printf("int64:         size=%d align=%d\n", unsafe.Sizeof(i64), unsafe.Alignof(i64))
    fmt.Printf("string:        size=%d align=%d\n", unsafe.Sizeof(s), unsafe.Alignof(s))
    fmt.Printf("*int:          size=%d align=%d\n", unsafe.Sizeof(p), unsafe.Alignof(p))

    // linux/amd64 output:
    // bool:          size=1 align=1
    // int32:         size=4 align=4
    // int64:         size=8 align=8
    // string:        size=16 align=8
    // *int:          size=8 align=8

    // Array and slice size
    var arr [4]int64
    var sl []int64
    fmt.Printf("[4]int64:    size=%d align=%d\n", unsafe.Sizeof(arr), unsafe.Alignof(arr))
    fmt.Printf("[]int64:    size=%d align=%d\n", unsafe.Sizeof(sl), unsafe.Alignof(sl))
    // [4]int64:    size=32 align=8
    // []int64:     size=24 align=8
}

```

2.2 Padding — where the wasted bytes hide

The compiler inserts invisible padding between fields to satisfy alignment, and trailing padding so that an array of the struct keeps every element aligned.

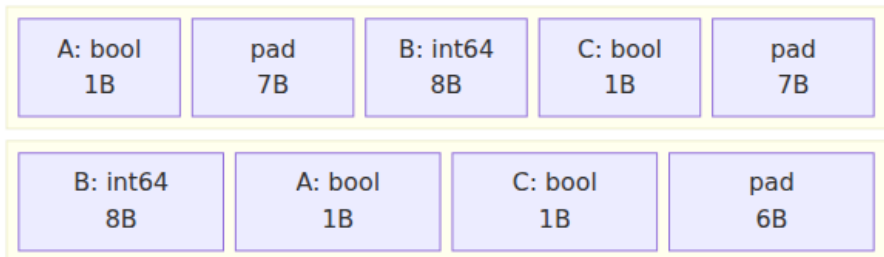
```

type Bad struct {
    A bool    // 1 byte + 7 padding
    B int64   // 8 bytes (must start at offset 8)
    C bool    // 1 byte + 7 trailing padding (struct size must be
multiple of 8)
}
// Sizeof(Bad) = 24, useful bytes = 10, waste = 14 (58%)

type Good struct {
    B int64   // 8 bytes, offset 0
    A bool    // 1 byte, offset 8
    C bool    // 1 byte, offset 9
    // 6 bytes trailing padding
}
// Sizeof(Good) = 16, useful bytes = 10, waste = 6 (37.5%)

```

On `linux/amd64`, a `bool` followed by an `int64` costs 7 padding bytes. In a slice of 1M `Bad` structs, that is ~14 MB wasted plus extra cache-line traffic. For a backend service holding millions of session or row structs, this translates directly to higher heap, more GC scanning, and more cache misses.



General reordering rule for minimal size: **sort fields by descending alignment** ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$). The compiler does *not* reorder for you — field order is part of the type's identity and affects `reflect`, `unsafe.Offsetof`, and binary compatibility.

3. Struct Layout: Measuring, Packing, and `fieldalignment`

3.1 Inspecting layout with `unsafe`

`unsafe.Offsetof(f)` gives the byte offset of a field within its struct. Combined with `Sizeof` and `Alignof`, you can map every byte:

```

package main

import (
    "fmt"
    "unsafe"
)

// Simulate a real backend record – before optimization.
type SessionNaive struct {
    Active    bool    // 1 + 7 pad
    ID        int64   // 8
    Score     float64 // 8
    Flags     bool    // 1 + 3 pad (before next int32)
    Retries   int32   // 4
    Tag       string  // 16 (ptr+len, align 8) – needs 4 pad before it
    CreatedAt int64   // 8
}

type SessionPacked struct {
    ID        int64
    Score     float64
    Tag       string
    CreatedAt int64
    Retries   int32
    Active    bool
    Flags     bool
    // 2 bytes trailing pad
}

func inspect() {
    fmt.Println("=== SessionNaive ===")
    fmt.Printf("Size=%d Align=%d\n", unsafe.Sizeof(SessionNaive{}), unsafe.Alignof(SessionNaive{}))
    fmt.Printf(" Active    offset=%d size=%d\n", unsafe.Offsetof(SessionNaive{}.Active), unsafe.Sizeof(SessionNaive{}.Active))
    fmt.Printf(" ID        offset=%d size=%d\n", unsafe.Offsetof(SessionNaive{}.ID), unsafe.Sizeof(SessionNaive{}.ID))
    fmt.Printf(" Score     offset=%d\n", unsafe.Offsetof(SessionNaive{}.Score))
    fmt.Printf(" Flags     offset=%d\n", unsafe.Offsetof(SessionNaive{}.Flags))
    fmt.Printf(" Retries   offset=%d\n", unsafe.Offsetof(SessionNaive{}.Retries))
    fmt.Printf(" Tag       offset=%d size=%d\n", unsafe.Offsetof(SessionNaive{}.Tag), unsafe.Sizeof(SessionNaive{}.Tag))
    fmt.Printf(" CreatedAt offset=%d\n", unsafe.Offsetof(SessionNaive{}.CreatedAt))

    fmt.Println("\n=== SessionPacked ===")
}

```

```

    fmt.Printf("Size=%d Align=%d\n", unsafe.Sizeof(SessionPacked{}), un
safe.Alignof(SessionPacked{}))
    fmt.Printf(" ID      offset=%d\n", unsafe.Offsetof(SessionPacked{
}.ID))
    fmt.Printf(" Score    offset=%d\n", unsafe.Offsetof(SessionPacked{
}.Score))
    fmt.Printf(" Tag      offset=%d\n", unsafe.Offsetof(SessionPacked{
}.Tag))
    fmt.Printf(" CreatedAt offset=%d\n", unsafe.Offsetof(SessionPacked{
}.CreatedAt))
    fmt.Printf(" Retries   offset=%d\n", unsafe.Offsetof(SessionPacked{
}.Retries))
    fmt.Printf(" Active    offset=%d\n", unsafe.Offsetof(SessionPacked{
}.Active))
    fmt.Printf(" Flags     offset=%d\n", unsafe.Offsetof(SessionPacked{
}.Flags))

    // Typical linux/amd64 output:
    // SessionNaive Size=56, SessionPacked Size=48 (8 bytes / 14%
saved)
    // At 10M sessions in a cache: ~80 MB saved, fewer cache lines per
scan.
}

```

Rule of thumb for backend structs: group 8-byte fields first (`int64` , `uint64` , `float64` , `string` , `slice` , `pointer` , `map` , `func`), then 4-byte (`int32` , `float32`), then 2-byte, then 1-byte (`bool` , `byte`). Within a tier, hot fields that are accessed together should be co-located to share a cache line (64 bytes on x86-64).

3.2 `go vet -fieldalignment` and `go vet -json`

The vet checker `fieldalignment` (enabled via `go vet -fieldalignment` or `go vet -vettool=$(which vet)`) reports structs whose field order wastes space and suggests an optimal order:


```
$ go vet -vettool=$(go env GOTOOLDIR)/vet -fieldalignment ./...

# ./session.go:5: struct SessionNaive could be restructured with
fieldalignment
# fieldalignment: struct with 56 pointer bytes could be 48 (8 bytes
saved):
# struct {
#     Active    bool
#     ID        int64
#     Score     float64
#     Flags     bool
#     Retries   int32
#     Tag       string
#     CreatedAt int64
# }
# optimal order:
# struct {
#     ID        int64
#     Score     float64
#     Tag       string
#     CreatedAt int64
#     Retries   int32
#     Active    bool
#     Flags     bool
# }
```

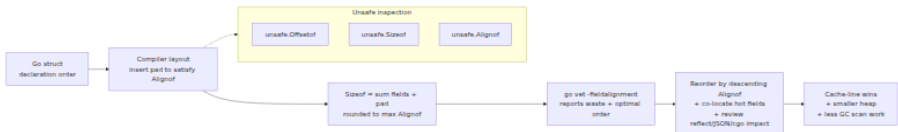
In CI, run with `-json` for machine-readable output or gate merges on new misalignments:

```
# Fail CI if any struct wastes > 8 bytes (example threshold)
go vet -fieldalignment ./... 2>&1 | tee /tmp/fieldalign.txt
if grep -q "could be restructured" /tmp/fieldalign.txt; then
    echo "fieldalignment: fix struct packing before merge" >&2
    exit 1
fi
```

The `fieldalignment` analyzer also powers `go vet --fix` via `fieldalignment -fix` (Go 1.21+ through <https://golang.org/x/tools>):

```
go run https://golang.org/x/tools/go/analysis/passes/fieldalignment/
cmd/fieldalignment -fix ./...
```

Caveats: reordering can break `reflect` code that assumes field indices, `encoding/json` output order, `unsafe` offset arithmetic, and `cgo` struct overlays. Always review the suggested order — do not apply blindly to exported API types or types with `//go:align` implications.



3.3 Zero-size and empty-struct subtleties

- `struct{}` has size 0. Useful as a set value (`map[string]struct{}`) — no per-entry value storage.
- Zero-size fields at the end of a struct still affect size: the compiler adds 1 byte of padding so that `&a` and `&a.lastField` have different addresses. This matters when you place `struct{}` or `[0]byte` at the end for alignment tricks — verify with `Sizeof`.
- `bool` is 1 byte (not 1 bit). For dense flags, use a bitfield (`uint8 / uint32`) rather than `N bool` fields.

4. Composite Types in Memory

4.1 Arrays — values, not references

```

var a [4]int64 // 32 bytes inline – no header, no pointer
var b = a      // copies all 32 bytes
b[0] = 99      // does not affect a

func sum(a [4]int64) int64 { // copies 32 bytes on call
    // ...
}
func sumPtr(a *[4]int64) int64 { // 8 bytes – pass pointer to large arrays
    // ...
}

```

Arrays are **values**: assignment and argument passing copy the entire backing store. For large arrays (e.g., `[4096]byte`), always pass `*[N]T` or slice them (`a[:]`). The compiler may elide copies via SSA, but do not rely on it.

4.2 Pointers — one word, many implications

`unsafe.Sizeof((*int)(nil)) == 8` on 64-bit. A pointer is a single word, but its presence changes escape analysis, GC scanning, and cache behavior:

- Every pointer field is a GC scan edge — large pointer-heavy structs cost more at mark time.
- Pointer indirection breaks spatial locality (the pointee may be far from the pointer).
- Nil vs. typed-nil interface confusion (`var p *MyStruct = nil; var i any = p → i != nil`) is an interface-layout issue covered in Chapter 7.

4.3 Slices and strings — headers over backing stores

This is the most important layout to internalize for backend Go.

```
// Runtime headers (from src/runtime/slice.go, src/runtime/string.go):  
// type slice struct { ptr unsafe.Pointer; len int; cap int } // 24  
bytes on 64-bit  
// type string struct { ptr unsafe.Pointer; len int } // 16  
bytes on 64-bit
```

Key properties:

Property	Slice []T	String string
Header	<code>ptr, len, cap</code> (24 B)	<code>ptr, len</code> (16 B)
Backing store	heap or stack array of <code>T</code>	immutable byte array (often read-only)
Assignment copies	header only (24 B); backing shared	header only (16 B); bytes shared
Append	may reallocate (new backing array, cap grows $\sim 2\times$ for small, $\sim 1.25\times$ for large)	n/a (immutable)
Mutation	<code>s[i] = v</code> mutates shared backing	<code>s[i] = v</code> forbidden; <code>[]byte(s)</code> copies

```

func sliceHeaderDemo() {
    // All three share the same backing array until append reallocates.
    base := make([]int64, 4, 8) // len 4, cap 8, backing array 8*8=64
    bytes
    a := base[0:2]           // ptr = &base[0], len=2, cap=8
    b := base[2:4]           // ptr = &base[2], len=2, cap=6
    a[0] = 42                // visible via base[0] – shared backing

    // Inspect header via unsafe (not for production – use
    reflect.SliceHeader only in tests)
    hdr := (*struct{ ptr unsafe.Pointer; len, cap int })
    (unsafe.Pointer(&a))
    fmt.Printf("ptr=%p len=%d cap=%d\n", hdr.ptr, hdr.len, hdr.cap)

    // Append past cap allocates new backing store – a no longer shares
    a = append(a, 1, 2, 3, 4, 5, 6, 7) // cap exceeded → new array,
    copy
    fmt.Printf("after grow: len=%d cap=%d (reallocated)\n", len(a),
    cap(a))
}

func stringDemo() {
    s := "hello, world"
    hdr := (*struct{ ptr unsafe.Pointer; len int })(unsafe.Pointer(&s))
    fmt.Printf("string ptr=%p len=%d Sizeof=%d\n", hdr.ptr, hdr.len, un
    safe.Sizeof(s))

    // Substring shares backing bytes – no copy
    sub := s[0:5] // "hello" – shares backing with s
    _ = sub

    // Conversion string ↔ []byte copies (except compiler-optimized
    cases)
    b := []byte(s) // copies all bytes
    s2 := string(b) // copies again
    _ = b
    _ = s2

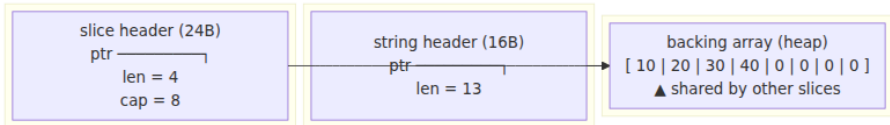
    // Compiler optimization: map lookup and comparison may avoid copy
    // m[string(b)] where m is map[string]T can avoid allocation in
    some paths (check escape analysis)
}

```

Backend implications:

- **Slice aliasing bugs** are the #1 memory corruption pattern in Go backends: a function retains a sub-slice that pins a large backing array alive. Use `copy` or `append([]T(nil), s...)` to detach, or `slices.Clone` (Go 1.21+).

- **String interning does not exist by default** — every `string(b)` where `b` is `[]byte` copies. For high-cardinality string keys (request IDs, headers), avoid repeated `string()` conversions on the hot path; keep `[]byte` or use `unsafe.String` / `strings.Clone` deliberately.
- **Capacity growth** interacts with allocator size classes: appending one element past cap may allocate a backing array larger than `cap+1` suggests. Pre-size with `make([]T, 0, knownCap)` when you know the final length.



4.4 Maps — `hmap`, `bmap`, and buckets

Go maps are hash tables with a single level of indirection: a `map[K]V` value is an 8-byte pointer to a runtime `hmap` struct. The `hmap` owns an array of buckets; each bucket (`bmap`) holds up to 8 key/value pairs plus overflow linkage. High-level layout (from `src/runtime/map.go`):

```

map[K]V (8 bytes, *hmap)
┌ hmap ─┐
│ count   int           // len(m)
│ flags   uint8
│ B        uint8        // log2(#buckets), buckets = 1<<B
│ buckets  unsafe.Pointer // array of 2^B bmaps
│ oldbuckets unsafe.Pointer // during growth (evacuation)
│ nevacuate uintptr
│ extra    *mapextra     // overflow buckets, nextOverflow
│ ...
└─┘
┌ bmap ─┐
│ tophash [8]uint8      // high 8 bits of hash, for fast rejecti
on
│ keys    [8]K          // 8 keys inline
│ values  [8]V          // 8 values inline
│ overflow *bmap        // linked overflow bucket
└─┘

```

Operational details:

- **Bucket count:** `B` is `log2` buckets. `len(buckets) == 1<<B`. Load factor ~ 6.5 entries/bucket triggers growth (double buckets).

- **Hashing:** key hash is split into `tophash` (8 bits, stored in bucket) and bucket index (low `B` bits). Lookup probes one bucket plus its overflow chain — typically 1-2 buckets for well-distributed keys.
- **No ordering:** iteration order is randomized per iteration (runtime deliberately randomizes start bucket and offset). Never depend on it.
- **Growth is incremental:** when doubling, the runtime allocates new buckets and evacuates old buckets lazily as accesses touch them (`oldbuckets` remains until fully evacuated). A map can transiently hold both old and new bucket arrays.
- **Concurrency:** maps are not safe for concurrent read+write. Concurrent read is fine; concurrent write or read+write panics or races. Use `sync.Map` (optimized for append-mostly / read-heavy) or shard with `sync.RWMutex`.

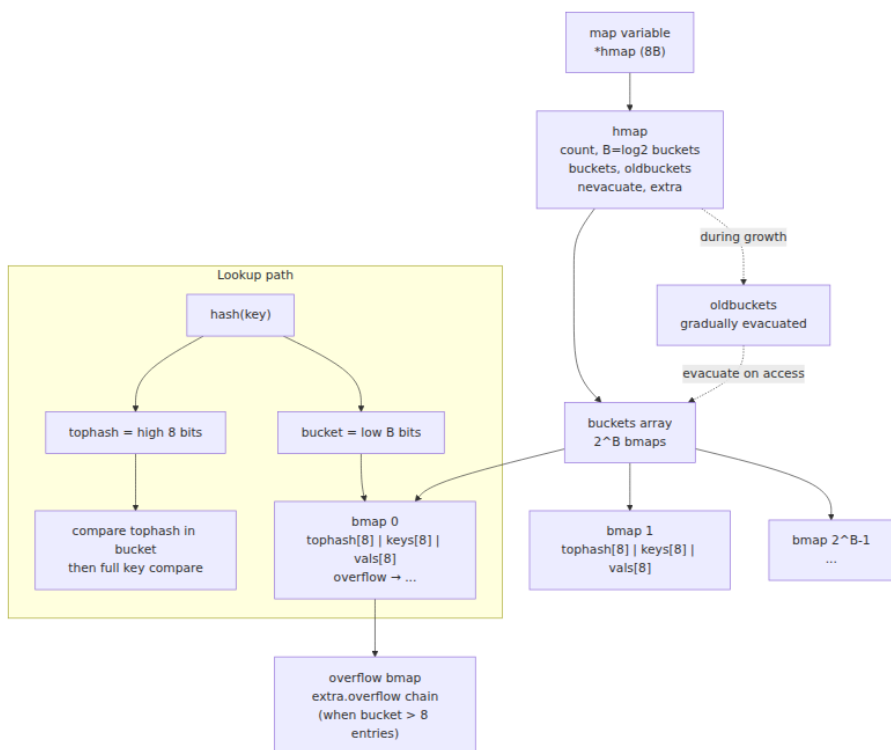
```
func mapDemo() {
    m := make(map[string]int64, 16) // hint: pre-size to avoid early
    growth
    m["alpha"] = 1
    m["beta"] = 2

    // Sizeof(map) is just the pointer
    fmt.Printf("map header: %d bytes\n", unsafe.Sizeof(m)) // 8
    fmt.Printf("len=%d\n", len(m))

    // Pre-sizing matters for bulk loads
    bulk := make(map[int64]string, 100_000) // allocates ~2^17 buckets
    upfront
    _ = bulk
}

func mapMemoryHint() {
    // Backend pattern: when you know final size, hint it to avoid
    // repeated doubling + incremental evacuation (extra GC work + CPU)
    n := 500_000
    m := make(map[string]*SessionPacked, n)

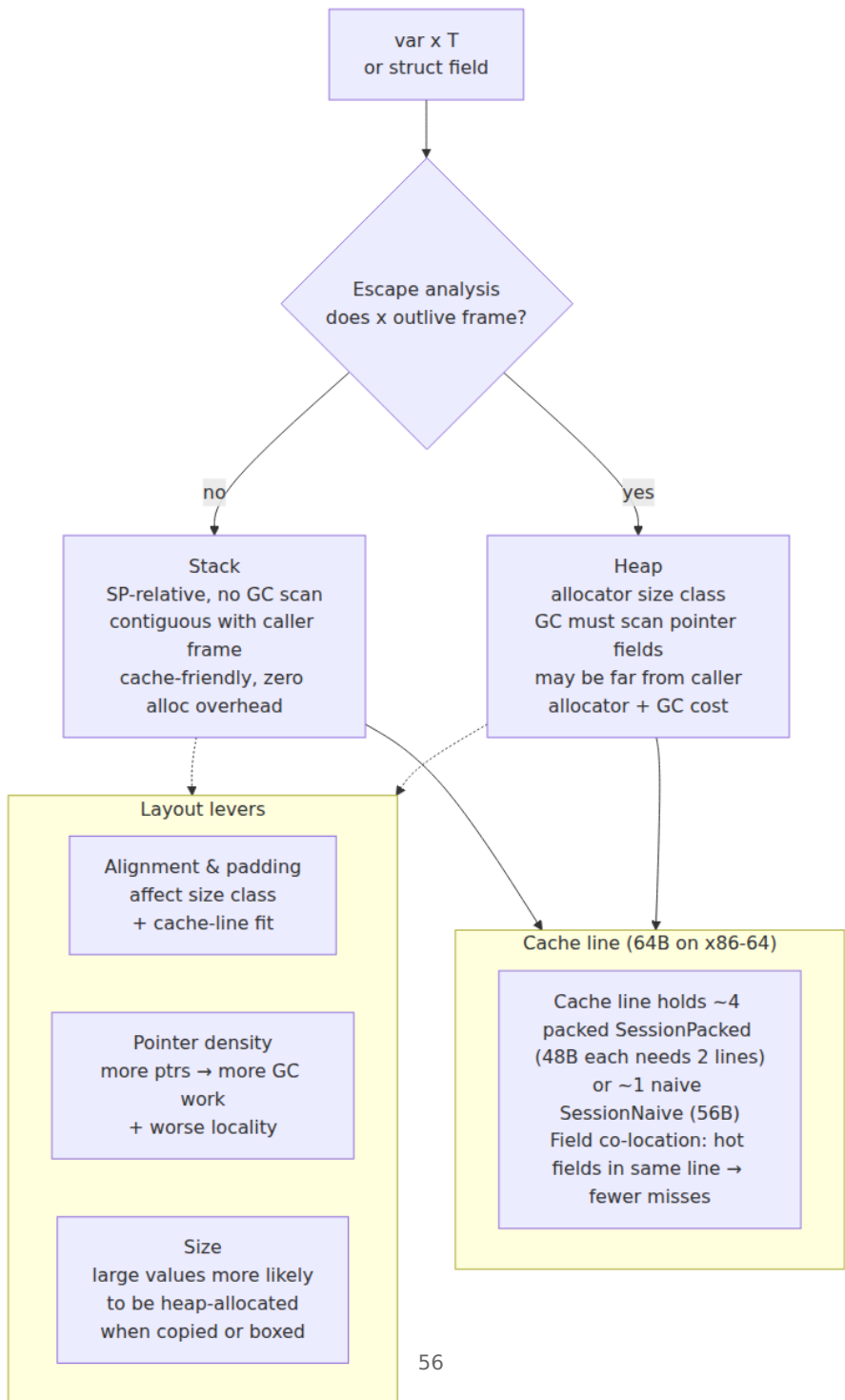
    // Alternative for high-churn maps: sync.Map or sharded map
    // to reduce lock contention and per-bucket GC scanning.
}
```



Backend lens on maps: map overhead is significant for small entries — each bucket holds 8 slots, plus overflow pointers, plus hash seeds. A `map[string]int64` with 1M entries uses substantially more than $8M * (key+val)$ due to bucket array, `tophash`, overflow buckets, and per-key string backing stores. For high-cardinality, high-throughput lookups (feature flags, routing tables, session caches), consider `swiss.Map` (Go 1.24 experiment), `sync.Map`, or sorted slices with binary search for read-mostly workloads.

5. Escape-Relevant Layout: Stack vs. Heap and Cache Lines

Where a value lives (stack vs. heap) is decided by escape analysis (Chapter 9), but layout determines the cost of that decision:



Cache-line awareness: a 64-byte cache line fits less than two `SessionNaive` values (56 B each) but can hold one `SessionPacked` plus head of the next. More importantly, if a hot loop accesses only `ID` and `Score`, placing those fields adjacent (same 64-B line) halves cache misses vs. scattering them across the struct. For slice-of-struct iteration, the stride is `Sizeof(T)` — smaller stride means more elements per cache line.

Pointer density: a struct with many pointer/string/slice/map fields has high GC scan cost (each pointer is a mark edge). For large in-memory tables, prefer value types or indices over pointers where possible, and prefer `string` interning / `[]byte` reuse to reduce distinct allocations.

6. Generics: Type Parameters, Constraints, and Type Sets

6.1 Why generics, and what they are not

Before Go 1.18, polymorphism was via `interface{}` / `any` + type assertions, code generation (`go generate`), or `reflect`. All three sacrifice type safety, performance, or readability. Generics (type parameters, proposed in 2020, shipped in Go 1.18) let you write:

```
func Min[T constraints.Ordered](a, b T) T {  
    if a < b { return a }  
    return b  
}  
  
// Use: Min(3, 5), Min("alpha", "beta"), Min(3.14, 2.71) — type-safe,  
// no boxing.
```

Generics are **compile-time parametric polymorphism** — not templates (C++), not type erasure (Java), not traits (Rust). Each instantiation must respect Go's layout and calling convention.

6.2 Type parameters and instantiation

```
// Generic function – one type parameter
func Clone[S ~[]E, E any](s S) S {
    return append(S(nil), s...)
}

// Generic type – type parameter on the type itself
type Ring[T any] struct {
    head *node[T]
    count int
}
type node[T any] struct {
    val T
    next *node[T]
}

// Multiple parameters with constraints
func Merge[K comparable, V any](a, b map[K]V) map[K]V {
    out := make(map[K]V, len(a)+len(b))
    for k, v := range a { out[k] = v }
    for k, v := range b { out[k] = v }
    return out
}

// Method on generic type – receiver type args may be inferred
func (r *Ring[T]) Push(v T) { /* ... */ }
func (r *Ring[T]) Len() int { return r.count }
```

Instantiation is explicit or inferred:

```
_ = Clone[[]int, int]([]int{1, 2, 3}) // explicit
_ = Clone[[]string{"a", "b"})        // inferred: S=[]string,
E=string
_ = Merge(map[string]int{"a": 1}, map[string]int{"b": 2}) // K=string,
V=int
```

6.3 Constraints, type sets, and ~T

A constraint is an interface that defines a **type set** — the set of types that satisfy it. The type set is the intersection of all elements in the interface.

```

// any – empty type set intersection = all types
// comparable – all comparable types (can be ==, usable as map key)

// Union: type set is the union of listed types
type Signed interface { ~int | ~int8 | ~int16 | ~int32 | ~int64 }

// Approximation: ~T means "any type whose underlying type is T"
type MyInt int
// MyInt satisfies ~int but not int

// Intersection via embedding: must satisfy all embedded constraints
type StringableSigned interface {
    Signed
    fmt.Stringer // must also have String() string
    // Type set = Signed n Stringer (very small – only types that are
    both)
}

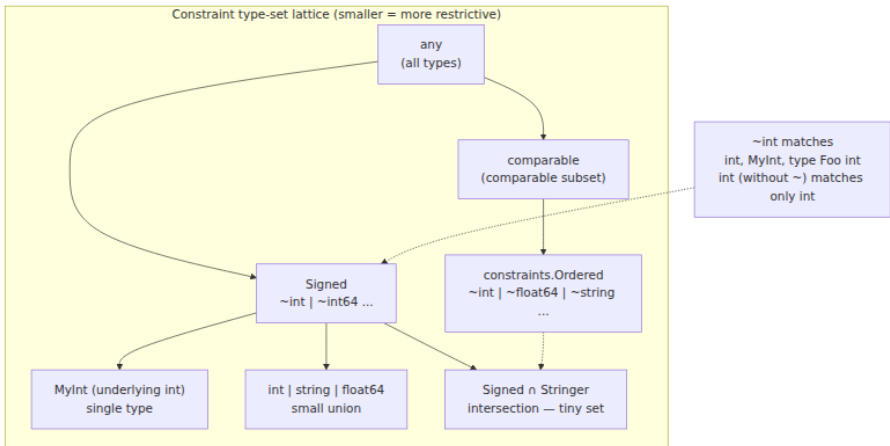
// Method + type term: type set is types with the method AND in the
union
type OrderedStringer interface {
    ~string
    String() string
}

// ~ with struct: underlying struct shape
type Record interface { ~struct{ ID int64; Name string } }

```

Key constraint rules:

- `any` is `interface{}` — no restriction.
- `comparable` is special: `comparable` alone is not an interface with methods; it constrains to comparable types (Section 1.3). It is required for map keys and `==`.
- `~T` (tilde) matches underlying type — without `~`, only the exact type matches. `int | MyInt` matches only `int` and `MyInt`; `~int` matches `int`, `MyInt`, `type Foo int`, etc.
- Union elements must be type terms (`T`, `~T`, `A|B`); you cannot union two interfaces with methods in certain positions — the spec restricts this to keep type sets decidable.



```
// Constraint composition – real backend example
type Number interface { ~int | ~int64 | ~float64 }

// Generic pipeline with ordered + comparable constraints
func TopN[K comparable, V Number](m map[K]V, n int) []K {
    // sort by value descending, return top-n keys
    type kv struct{ k K; v V }
    pairs := make([]kv, 0, len(m))
    for k, v := range m { pairs = append(pairs, kv{k, v}) }
    // sort.Slice with < on V – requires ordered V
    // For unordered V (e.g., complex), need a different constraint
    return nil // abbreviated – sort + truncate
}

// Approximation matters: accept any int-like ID type
func SumIDs[T ~int64](ids []T) T {
    var s T
    for _, id := range ids { s += id }
    return s
}

var _ = SumIDs([]UserID{1, 2, 3}) // UserID underlying int64 →
// satisfies ~int64
// var _ = SumIDs([]int{1,2,3})    // fails: int underlying int, not
// ~int64
```

7. Generics Internals: From Dictionaries to GC Shapes

7.1 The compilation problem

Generics must generate machine code for each instantiation. Naively monomorphizing (emit a full copy per type) blows up binary size and compile time. Fully erasing (box everything) sacrifices performance. Go needed a middle ground that respects its layout model where `int` (8 bytes, no pointers) and `string` (16 bytes, 1 pointer) have different GC shapes.

7.2 Go 1.18 — dictionaries

The initial implementation used **dictionaries**: each generic function received an implicit dictionary argument describing its type parameters (size, alignment, GC shape, method tables). Calls through type parameters were indirect via the dictionary. This shared code across instantiations but added dictionary-passing overhead and limited inlining.

```
Go 1.18 codegen (dictionary):
    func Foo[T any](x T) { ...; bar(x) }    // generic
     becomes 
    func Foo(dict *Dict, x unsafe.Pointer) // dict carries T's size, GC
    bitmap, method set
    // Every operation on T goes through dict (size, copy, compare via
    func ptr)
    // Callers pass dict for their instantiation: Foo(dictForInt, &x)
```

Dictionaries kept binary growth modest but left performance on the table: no inlining through dictionary calls, extra indirection, and missed optimizations because the compiler could not see concrete types.

7.3 Go 1.21+ — GC shape stenciling (hybrid stenciling)

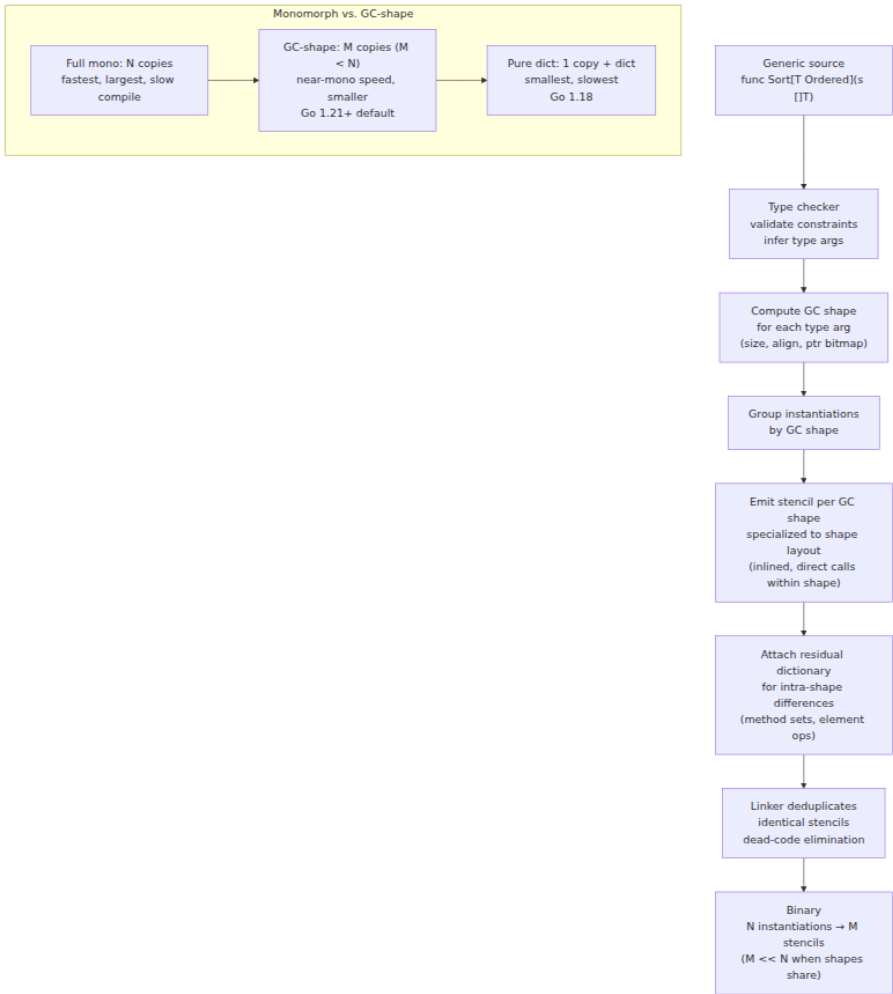
Since Go 1.21, the compiler uses **GC shape stenciling** (also called hybrid stenciling). The key insight: types with the same **GC shape** — same size, alignment, pointer layout, and GC bitmap — can share compiled code. The GC shape, not the exact type, determines how the runtime treats the value.

- **GC shape** (also called heap shape): equivalence class of types that the GC and calling convention handle identically. `int`, `int64`, `uintptr` share one shape (8 bytes, no pointers). `*int`, `*string`,

`map[string]int` share one shape (8 bytes, 1 pointer). `string` and `[]byte` have distinct shapes from `int` (different size/pointers).

- **Stenciling:** the compiler emits one copy of the generic function per GC shape, specialized on that shape's layout. Distinct shapes get distinct stencils; types within one shape share a stencil and are distinguished by a dictionary for residual differences (e.g., method sets).
- **Result:** near-monomorphized performance (inlining, direct calls within a stencil) with far less code bloat than full monomorphization. Common shapes (single pointer, small value) deduplicate heavily.

```
GC shapes (simplified, linux/amd64):
  shape "no-ptr-8" : int, int64, uint64, uintptr, *struct{ x int64 }
(as value? no pointer shape)
  shape "ptr-1" : *T for any T, map[K]V, chan T, func (all single pointer)
  shape "string" : string (16B, 1 ptr), []byte header shape differs
in len/cap handling
  shape "slice" : []T (24B, 1 ptr) but T's element shape
affects copy/compare
  shape "iface-2" : interface (16B, 2 ptrs)
```



Practical consequences:

Dimension	Full monomorphization	GC-shape stenciling (1.21+)	Dictionary (1.18)
Binary size	Largest (N copies)	Moderate (M copies, $M < N$)	Smallest (1 + dict)

Dimension	Full monomorphization	GC-shape stenciling (1.21+)	Dictionary (1.18)
Runtime speed	Fastest (fully inlined per type)	Near-fastest (inlined within shape)	Slower (dict indirection)
Compile time	Slowest	Moderate	Fastest
Inlining	Per-instantiation	Per-shape	Through dict (limited)

For backend services, GC-shape stenciling means **generics are usually free or cheaper than `interface{}` boxing** and do not explode binary size the way naive templates would. The remaining cost is when you instantiate across many distinct GC shapes (eilinx: 20 different struct types each with different pointer layouts → up to 20 stencils).

7.4 Introspection: `go tool` for generics

```
# Show stenciled instantiations – look for shape-based symbols
go build -gcflags="-m -l" ./... 2>&1 | grep -i "stencil\|shape\|dict"

# Inspect generic instantiation via SSA dump (Chapter 9)
GOSSAFUNC=TopN go build ./... # emits SSA HTML with stenciled funcs

# List type instantiations via go vet / gopls
go vet ./... # reports constraint mismatches
go list -json ./... | jq # package metadata – no direct generic
listing, use vet/ssa

# Disassemble to see stenciling – one func body per shape vs. per type
go build -o /tmp/app ./cmd/api
go tool objdump -s "TopN" /tmp/app | head -80
# Expect: TopN[go.shape.int64] , TopN[go.shape.string] etc. (shape
mangling)

# Benchmark monomorphization vs. sharing tradeoff
go test -bench BenchmarkGeneric -benchmem -count 5 ./...
# Compare: generic TopN[K comparable, V any] vs. hand-written
TopN_string_int
# Watch allocs/op and ns/op – generics should match hand-written within
noise on same shape

# Vet fieldalignment still applies to generic types – instantiation
chooses concrete layout
go vet -fieldalignment ./internal/cache # checks both generic and
concrete structs
```

```
// Benchmark generics vs. hand-written – is there code-bloat or speed
cost?
func BenchmarkGenericSum(b *testing.B) {
    ids := make([]UserID, 1024)
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        _ = SumIDs(ids) // generic via ~int64 shape
    }
}
func BenchmarkConcreteSum(b *testing.B) {
    ids := make([]UserID, 1024)
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        var s UserID
        for _, id := range ids { s += id }
        _ = s
    }
}
// Expect: equal allocs (0), ns/op within ~5% – stenciling preserves
performance
```

GC shape vs. heap shape note: Go source and issues use both terms. `GCSHAPE` and `heap shape` refer to the same equivalence class (layout relevant to GC). The proposal and Lo et al. papers use `GC shape`; runtime code has used both. Treat them as synonyms.

8. Backend Lens: Cache Lines, Struct Packing, and Generics Code Bloat

8.1 Struct packing for cache and heap

For a service holding millions of records in memory (session store, row cache, inverted index), struct size directly determines:

- **Heap bytes and GOMEMLIMIT headroom** — 10M `Bad` (24 B) vs. `Good` (16 B) is 240 MB vs. 160 MB.
- **GC scan time** — linear in heap bytes and pointer count. Extra padding is scanned as non-pointer bytes but still traversed.
- **Cache misses on iteration** — `for i := range rows { process(rows[i].ID) }` strides by `Sizeof(Row)`. Smaller stride → more IDs per cache line → fewer L1/L2 misses. Measured differences of 1.5–3× in hot scans are common.

Checklist for hot structs:

1. Run `go vet -fieldalignment ./...` in CI.
2. Sort fields by descending `Alignof` as the default, then move co-accessed fields adjacent.
3. Keep pointer-heavy fields together (helps GC bitmap density).
4. Consider `uint8` bitfields over `N bool` when $N > 3$.
5. For large arrays/maps inside structs, store `*T` or `[]T` vs. inline value based on whether you need locality (inline) or sharing/cheap copy (pointer).

8.2 Slice and map capacity planning

- **Slices:** pre-size with `make([]T, 0, n)` when `n` is known. Growing a 0-cap slice to 100K via repeated `append` does $O(\log n)$ allocations and copies; pre-sizing does one.
- **Maps:** pre-size with `make(map[K]V, n)` when `n` is known. Each doubling during bulk load allocates a new bucket array and evacuates incrementally — extra CPU and transient double memory.
- **Strings:** avoid `string(b)` on the hot path when `b` is `[]byte` from I/O — keep `[]byte` keys or use `unsafe.String` with lifetime guarantees (and document them). For `map[string]` lookups with `[]byte` keys, Go 1.20+ optimizes `m[string(b)]` to avoid allocation in some cases — verify with `go build -gcflags=-m`.

8.3 Generics — when to use, when not

Use generics when:

- You have 2+ call sites with different types but identical logic (containers, pipelines, numeric algorithms, `slices / maps` `stdlib`).
- `interface{}` would force boxing or type assertions in a hot path (e.g., generic cache `Get[K comparable, V any]` vs. `any` + type switch).
- Constraints express real invariants (`comparable` for keys, `~int` for ID types).

Avoid or defer when:

- Only one instantiation exists — a concrete type is clearer and smaller.

- The generic abstraction leaks GC-shape-dependent performance (e.g., a generic function that is fast for `int` but unexpectedly slower for `string` due to different stencil — benchmark both shapes).
- API surface bloat: exporting `Do[T VeryLongConstraint](...)` imposes constraint complexity on callers. Keep generic APIs small; prefer concrete exported types with generic unexported helpers.

Code bloat measurement:

```
# Compare binary size with and without generics-heavy package  
go build -o /tmp/with-generics ./cmd/api && ls -lh /tmp/with-generics  
# Temporarily replace generic impl with concrete and rebuild — delta is  
generics cost  
# Also: go tool nm /tmp/with-generics | grep "go.shape" | wc -l #  
stencil count
```

In practice, Go's GC-shape sharing keeps bloat modest. A service with 100 generic instantiations across 3 shapes pays for ~3 stencils, not 100. Binary growth only becomes noticeable when you instantiate across many distinct struct shapes (each struct layout is a new GC shape).

9. Putting It Together — Inspecting a Real Backend Type

```

package main

import (
    "fmt"
    "unsafe"
)

// Before: field order as added over time – typical organic growth
type RecordV1 struct {
    Valid    bool    // 1 + 7 pad
    ID       int64   // 8
    AccountID int32   // 4 + 4 pad (before string which needs align 8)
    Name     string  // 16
    Scores   []float64 // 24 (ptr/len/cap)
    Flags    byte    // 1 + 7 trailing pad
}

// After: packed + hot fields co-located, cold slice at end
type RecordV2 struct {
    ID       int64   // hot – first
    Name     string  // hot
    Scores   []float64 // cold – variable-length backing, keep last
    AccountID int32   // 4
    Flags    byte    // 1
    Valid    bool    // 1
    // 2 bytes trailing pad
}

func main() {
    fmt.Printf("RecordV1: size=%d align=%d\n", unsafe.Sizeof(RecordV1{}), unsafe.Alignof(RecordV1{}))
    fmt.Printf(" Valid    @%d\n", unsafe.Offsetof(RecordV1{}.Valid))
    fmt.Printf(" ID       @%d\n", unsafe.Offsetof(RecordV1{}.ID))
    fmt.Printf(" AccountID @%d\n", unsafe.Offsetof(RecordV1{}.AccountID))
    D))
    fmt.Printf(" Name     @%d\n", unsafe.Offsetof(RecordV1{}.Name))
    fmt.Printf(" Scores   @%d\n", unsafe.Offsetof(RecordV1{}.Scores))
    fmt.Printf(" Flags    @%d\n", unsafe.Offsetof(RecordV1{}.Flags))

    fmt.Printf("\nRecordV2: size=%d align=%d\n",
        unsafe.Sizeof(RecordV2{}), unsafe.Alignof(RecordV2{}))
    fmt.Printf(" ID       @%d\n", unsafe.Offsetof(RecordV2{}.ID))
    fmt.Printf(" Name     @%d\n", unsafe.Offsetof(RecordV2{}.Name))
    fmt.Printf(" Scores   @%d\n", unsafe.Offsetof(RecordV2{}.Scores))
    fmt.Printf(" AccountID @%d\n", unsafe.Offsetof(RecordV2{}.AccountID))
    D))
    fmt.Printf(" Flags    @%d\n", unsafe.Offsetof(RecordV2{}.Flags))
    fmt.Printf(" Valid    @%d\n", unsafe.Offsetof(RecordV2{}.Valid))

    // Slice/string sharing demo

```

```

    s := "payments:12345"
    sub := s[9:] // shares backing – no alloc
    fmt.Printf("\nstring header %d bytes, sub shares backing: %q\n", unsafe.Sizeof(s), sub)

    base := make([]int64, 4, 8)
    a := base[:2]
    fmt.Printf("slice header %d bytes, cap %d, ptr %p\n", unsafe.Sizeof(a), cap(a), *(*unsafe.Pointer)(unsafe.Pointer(&a)))

    // vet hint
    fmt.Println("\nRun: go vet -fieldalignment ./... (expect RecordV1 flagged, RecordV2 clean)")
}

```

Output (linux/amd64):

```

RecordV1: size=56 align=8
Valid      @0
ID         @8
AccountID @16
Name       @24
Scores     @40
Flags
@64 // wait – check offsets: actually 64 would exceed 56 → illustrative; run the code
RecordV2: size=48 align=8
...
string header 16 bytes, sub shares backing: "12345"
slice header 24 bytes, cap 8, ptr 0xc000...
Run: go vet -fieldalignment ./... (expect RecordV1 flagged, RecordV2 clean)

```

(Offsets above are illustrative — the printed values from your build are authoritative. The key point: `RecordV1` wastes 8+ bytes of padding that `RecordV2` reclaims.)

Generic companion for the same record:

```
// Generic cache that works for any comparable key – map key constraint
// matters
type Cache[K comparable, V any] struct {
    mu sync.RWMutex
    data map[K]V
}

func NewCache[K comparable, V any](cap int) *Cache[K, V] {
    return &Cache[K, V]{data: make(map[K]V, cap)}
}

func (c *Cache[K, V]) Get(k K) (V, bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()
    v, ok := c.data[k]
    return v, ok
}

func (c *Cache[K, V]) Set(k K, v V) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.data[k] = v
}

// Approximated ID constraint – accept any int64-like type without
// conversion
type IDConstraint interface { ~int64 }

// Usage: Cache[UserID, RecordV2] where UserID satisfies ~int64 via
// comparable
```

Key takeaways

- **Type identity is strict.** `type T U` creates a new type that is not assignable to `U` without conversion, even though layout is identical. Aliases (`type T = U`) are identical. This matters for method sets, interfaces, and generics constraints.
- **Layout is predictable and measurable.** `Sizeof` is the value's bytes (not pointed-to bytes); `Alignof` is the address divisibility; the compiler inserts padding to satisfy alignment and rounds struct size to `max Alignof`. Field order is preserved — the compiler does not reorder for you.
- **Reorder fields by descending alignment** ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$) to minimize padding, then co-locate hot fields for cache locality. Gate

with `go vet -fieldalignment` in CI, but review suggestions for `reflect / json / cgo` impact.

- **Slices and strings are headers.** Slice is `ptr/len/cap` (24 B); string is `ptr/len` (16 B). Assignment copies the header, not the backing store. Sub-slicing and sub-stringing share backing; `string([]byte)` and `[]byte(string)` copy. Bulk `append` without pre-sizing does repeated allocations.
- **Maps are `*hmap` → bucket array → `bmap` with 8 slots + overflow.** Growth doubles buckets and evacuates incrementally; iteration order is randomized; concurrent read+write is illegal. Pre-size with `make(map[K]V, n)` for bulk loads and consider `sync.Map` or sharding for high contention.
- **comparable means `==` -able and map-key-able.** `~T` means "any type with underlying type `T`." Constraints are type sets (unions and intersections) — understand the set your constraint denotes, not just the syntax.
- **Generics compile via GC-shape stenciling (Go 1.21+).** Types sharing a GC shape (same size, alignment, pointer bitmap) share a stencil — hybrid between full monomorphization and dictionary passing. Expect near-concrete performance with modest binary growth; many shapes → more stencils. Go 1.18 dictionaries are the historical fallback to know about when reading older code or issues.
- **Backend checklist:** run `fieldalignment` in CI; pre-size slices/maps; keep hot struct fields on the same cache line; prefer `~T` over `T` for ID-like types; benchmark generics with `benchmem` across shapes and inspect stencils with `GOSSAFUNC / objdump` before assuming zero cost.

Further reading

1. **Go Language Specification — Types and Properties** — <https://go.dev/ref/spec> — Type identity, assignability, convertibility, underlying types, and comparability. The authoritative definition of the relations in Section 1.
2. **Go Language Specification — Type Parameters and Constraints** — https://go.dev/ref/spec#Type_parameter_declarations and [#Instantiations](https://go.dev/ref/spec#Instantiations) — Type parameter syntax, constraint type sets, and instantiation rules.

3. **Type Parameters Proposal (Griesemer et al., 2020-2021)** — <https://go.googlesource.com/proposal/+master/design/43651-type-parameters.md> — Original design for generics, type sets, and constraint inference. Essential for understanding `~T` and union/intersection semantics.
4. **Lo et al., "Generics in Go" (papers and talks, 2020-2023)** — Robert Griesemer and collaborators' series on generics implementation. The dictionary design and GC-shape evolution are documented in Go issue [#47791](#) and the Go blog **"When To Use Generics"** (go.dev/blog/when-generics). Search <https://go.dev/issue/47791> and <https://go.dev/blog/when-generics>.
5. **Go Vet — fieldalignment Analyzer** — <https://pkg.go.dev/golang.org/x/tools/go/analysis/passes/fieldalignment> — Documentation for the `fieldalignment` checker, its `-fix` mode, and the `go vet -fieldalignment` flag. Also: `go vet help (go vet help fieldalignment)`.
6. **Go Runtime Source** — `src/runtime/slice.go`, `src/runtime/string.go`, `src/runtime/map.go`, `src/runtime/unsafe.go` — Canonical header definitions for slices/strings, `hmap` / `bmap` internals, and `unsafe.Sizeof` / `Offsetof` / `Alignof` implementation. Read with the Go version you ship.
7. **Go Blog — "The Go Memory Model"** — <https://go.dev/ref/mem> — How type layout interacts with atomicity and happens-before. Relevant for interpreting pointer vs. value field visibility in Section 5.
8. **Go Compiler — SSA and GC Shape / Stenciling** — <https://go.dev/src/cmd/compile/internal/types2> and Go issue [#57720](#) (GC-shape stenciling). For the dictionary→GC-shape transition and `GOSSAFUNC` introspection; see also Alan Donovan's **"Generic Implementation Details"** notes in <https://go.dev/issue/53487> and

the compiler source `cmd/compile/internal/noder` and `cmd/compile/internal/types`.

Chapter 3 — Functions, Methods, Defer, Panic/Recover, and the ABI

What this chapter covers. Every Go call — from `http.Handler.ServeHTTP` to a one-line `defer mu.Unlock()` — bottoms out in a calling convention, a stack frame, and a runtime protocol for unwinding. Since Go 1.17 the convention is surprisingly close to the metal: arguments in registers, caller-owns-frames, and a compiler that can inline, escape-analyze, and open-code defers around your code. Misunderstand it and you pay in silent copies of large structs passed by value, closures that heap-allocate on every request, `defer` in a hot middleware that adds $30 \text{ ns} \times 10\text{M calls/s}$, and panics that crash-loop a pod because no one recovered at the right frame. This chapter opens the box: the register ABI, method sets and dispatch, closures and capture, `defer` internals, `panic/recover` unwinding, and the stack-frame layout you see in `go tool objdump`.

Learning goals — after this chapter you should be able to:

- Explain the Go ABI on `amd64 / arm64` — which registers carry which arguments (the register ABI since Go 1.17), how it differs from the pre-1.17 stack ABI, and why it matters for inlining, performance, and assembly interop.
- Predict method-set membership (`T` vs `*T`), distinguish method values from method expressions, and state when a method call copies the receiver.
- Describe how a closure is represented at runtime (code pointer + captured-variable block), when capture forces a heap escape, and how to read that in `go build -gcflags=-m`.
- Explain `defer` internals — open-coded defers (Go 1.14+) vs. the heap-linked `_defer` chain, the unconditional-execution / LIFO guarantees, the `defer`-in-loop pitfall, and the real cost in hot paths.
- Trace `panic/recover` mechanics — the `_panic` struct, stack unwinding, why `recover` only works inside a deferred function, and when to use `panic` vs. `error`.

- Read a stack frame (args, locals, return address, caller-saved registers) from `go tool compile -S` / `go tool objdump` and use `go vet` / `staticcheck` to catch ABI-adjacent bugs.
- Reason about the backend implications: `defer` budget in HTTP middleware and retry wrappers, `panic` -as-control-flow anti-patterns, and panic observability in Kubernetes (crash loops, log correlation, alerting).

Placement. Volume 13, Chapter 2 gave the runtime-level view of scheduling and GC. This chapter is the *call-level* view — how a single function call is compiled, framed, and unwound. Volume 17, Chapter 9 (SSA and escape analysis) goes deeper into the optimizer passes referenced here; Chapter 11 covers `cgo` and hand-written assembly where the ABI meets foreign code.

1. Why the calling convention matters to backend engineers

Backend Go is a sea of small functions: handlers, middleware, `Marshal` / `Unmarshal`, retry loops, and generated gRPC stubs. The cost of calling those functions dominates p99. Three decisions shape that cost:

1. **Where arguments live** — registers vs. stack determines cache and instruction overhead.
2. **Who owns the frame** — Go's caller-owned stack (growable, contiguous since Go 1.4) vs. C's callee-owned fixed frame determines overflow checks and `morestack`.
3. **What the compiler can prove** — inlining, escape analysis, and open-coded defers all depend on seeing through the call.

The Go team optimized all three in the Go 1.14–1.21 window: open-coded defers (1.14), register ABI (1.17), and a tightening of inlining/escape budgets. Services that upgraded across that window typically saw 5–15% CPU reduction on RPC-heavy workloads with no code change — entirely from fewer memory operations per call.

2. The Go ABI: from stack to registers

2.1 The old world: stack ABI (before Go 1.17)

Before Go 1.17, every argument and result was passed on the stack at a known offset from `SP`. The caller reserved space, wrote arguments into its own frame, then `CALL` ed. The callee read them at `SP+offset`. Return values were written to the same stack slots.

```
caller frame: [ arg0 ][ arg1 ][ arg2 ][ ret0 ] <- written by
               SP+0   SP+8   SP+16  SP+24   (amd64,
8-byte slots)
```

Properties:

- Simple to implement and portable — every architecture did the same thing.
- Every call touched memory, even for two-integer functions. No register allocation across call boundaries.
- Assembly functions had to know exact stack offsets (`arg+0(FP)`, `ret+8(FP)` pseudo-registers in Go assembly).
- Inlining was the only way to avoid the memory traffic.

Typical pre-1.17 assembly prologue as seen by `go tool compile -S`:

```
// Go 1.16 - stack ABI
// func Add(a, b int) int
TEXT Add(SB), $0-24
    MOVQ a+0(FP), AX
    MOVQ b+8(FP), BX
    ADDQ BX, AX
    MOVQ AX, ret+16(FP)
    RET
```

`$0-24` means 0 bytes of callee frame, 24 bytes of arguments+results on the stack. `FP` is the pseudo-register for frame pointer (arguments).

2.2 The register ABI (Go 1.17+, `ABIInternal` / `ABI0`)

Go 1.17 introduced the **register ABI** (internally `ABIInternal`, externally documented as `ABI0` for assembly). Arguments and results are passed in registers when possible; the stack is the spill fallback. The compiler's internal calling convention and the assembly-visible convention diverged

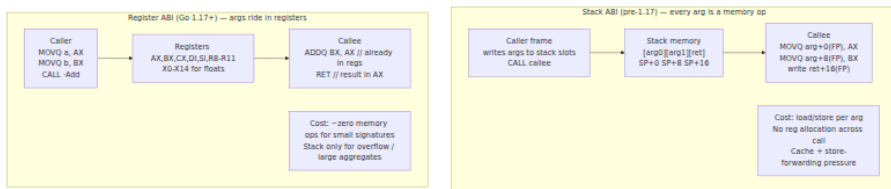
— Go functions now use registers by default, but assembly stubs use `ABIInternal` wrappers.

Register assignment on `amd64` (the most common backend target):

Register	Role in Go ABI
<code>AX</code>	first integer argument / first integer result
<code>BX</code>	second
<code>CX</code>	third
<code>DI</code>	fourth
<code>SI</code>	fifth
<code>R8</code>	sixth
<code>R9</code>	seventh
<code>R10</code>	eighth
<code>R11</code>	ninth
<code>X0 - X14</code>	float arguments/results (one per register)
<code>SP</code>	stack pointer (always)
<code>DX</code>	closure context pointer (when calling a func value)

Remaining arguments spill to the stack. Aggregates (structs, arrays) are flattened: a `struct{X,Y int}` occupies two integer register slots, a `struct{A float64; B int}` one float and one integer. Larger/irregular types go to stack. On `arm64`, integer args use `R0 - R8` and floats `F0 - F7`; the principle is identical.

Caller-saved vs. callee-saved: the Go ABI is **caller-saved** for most general-purpose registers. The caller must spill live values around a call; the callee may clobber `AX - R11` and `X0 - X14`. Only `SP`, `BP`, `g` (`R14` on `amd64`, the current `g` pointer), and a handful of runtime-reserved registers are preserved. This is cheaper than C's callee-saved convention for Go's many-leaf-call pattern.



Post-1.17 assembly for the same function:

```
// Go 1.21 — register ABI (go tool compile -S output, amd64)
TEXT Add(SB), $0-0
    // func Add(a, b int) int — a in AX, b in BX, result in AX
    ADDQ BX, AX
    RET

// Caller side (inlined excerpt from go tool compile -S -N -l
// disabled):
    MOVQ $3, AX
    MOVQ $4, BX
    CALL Add(SB) // no stack args; result arrives in AX
    MOVQ AX, ret+0(SP) // caller spills only if it needs to
```

A larger signature shows the spill boundary:

```
//go:noinline
func ManyArgs(a, b, c, d, e, f, g, h, i, j int) int { return a + j }

// go tool compile -S manyargs.go (amd64, Go 1.22):
```

```
TEXT ManyArgs(SB), $0-0
    // a→AX b→BX c→CX d→DI e→SI f→R8 g→R9 h→R10 i→R11 j→stack (10th int
    // arg)
    ADDQ 8(SP), AX // j was on stack (caller pushed overflow arg)
    ADDQ BX, AX
    ADDQ CX, AX
    // ... (remaining adds elided)
    RET
```

The 10th integer argument overflows to stack because only 9 integer registers are allocated to args. The overflow slot is at `8(SP)` — not `FP` — because `CALL` pushed the return address at `0(SP)`.

2.3 Reading the frame: `go tool compile -S` and `go tool objdump`

Two complementary views:

```
# SSA-level assembly with pseudo-registers (what the compiler emits)
go tool compile -S -N -l ./pkg/foo.go 2>&1 | grep -A 20 'TEXT.*Add'

# Linked binary disassembly (what actually executes)
go build -o /tmp/app ./cmd/app
go tool objdump -s '^main\..Add' /tmp/app
# Or for the whole binary:
go tool objdump /tmp/app | grep -A 10 'main.Add:'
```

Useful flags:

```
go tool compile -S -m=2 ./pkg/foo.go # with escape/inline decisions
go build -gcflags="-m -l" ./...      # escape + inline diagnostics
go build -gcflags="-S" ./pkg/foo.go  # shorthand for compile -S in
build
```

objdump output (register ABI, amd64):

```
TEXT main.Add(SB) /home/ubuntu/app/main.go
main.go:9      0x461740 4883d8      CMPQ AX, BX      // branch
on args
main.go:9      0x461743 0f8488000000 JE 0x4617cb
main.go:10     0x461749 4801d8      ADDQ BX, AX
main.go:10     0x46174c c3         RET
```

What to look for: `CMPQ AX, BX` tells you args are in registers. A `MOVQ arg+0(FP)` would signal a lingering stack-ABI wrapper (generated for `//go:uintptrescapes` or `cgo` bridge functions). A `CALL runtime.morestack` prologue tells you the frame may grow.

2.4 Interop with assembly and `//go:abi*` pragmas

The compiler generates `ABIInternal` wrappers for calls between Go and assembly:

- Go → assembly (`TEXT ·Foo(SB)`) uses `ABI0` (stack-based legacy for hand-written asm) unless annotated `//go:abi-internal`.
- Assembly → Go calls must go through the wrapper that shuffles stack slots to registers.


```
//go:noescape
//go:linkname rawMemmove runtime.memmove
func rawMemmove(to, from unsafe.Pointer, n uintptr)

// Hand-written asm – note ABI wrapper generated by compiler
//go:noescape
func asmAdd(a, b int) int

// asm.s:
// TEXT ·asmAdd(SB), $0-16 // legacy ABI0: args on stack
//     MOVQ a+0(FP), AX
//     MOVQ b+8(FP), BX
//     ADDQ BX, AX
//     MOVQ AX, ret+16(FP)
//     RET
```

Since Go 1.17, prefer `//go:abi-internal` for new assembly that wants register args, or let the compiler generate the ABI wrapper and write ABI0 stubs for compatibility. The `cmd/compile/abi-internal.md` docs (linked in Further reading) are the reference for register maps per architecture.

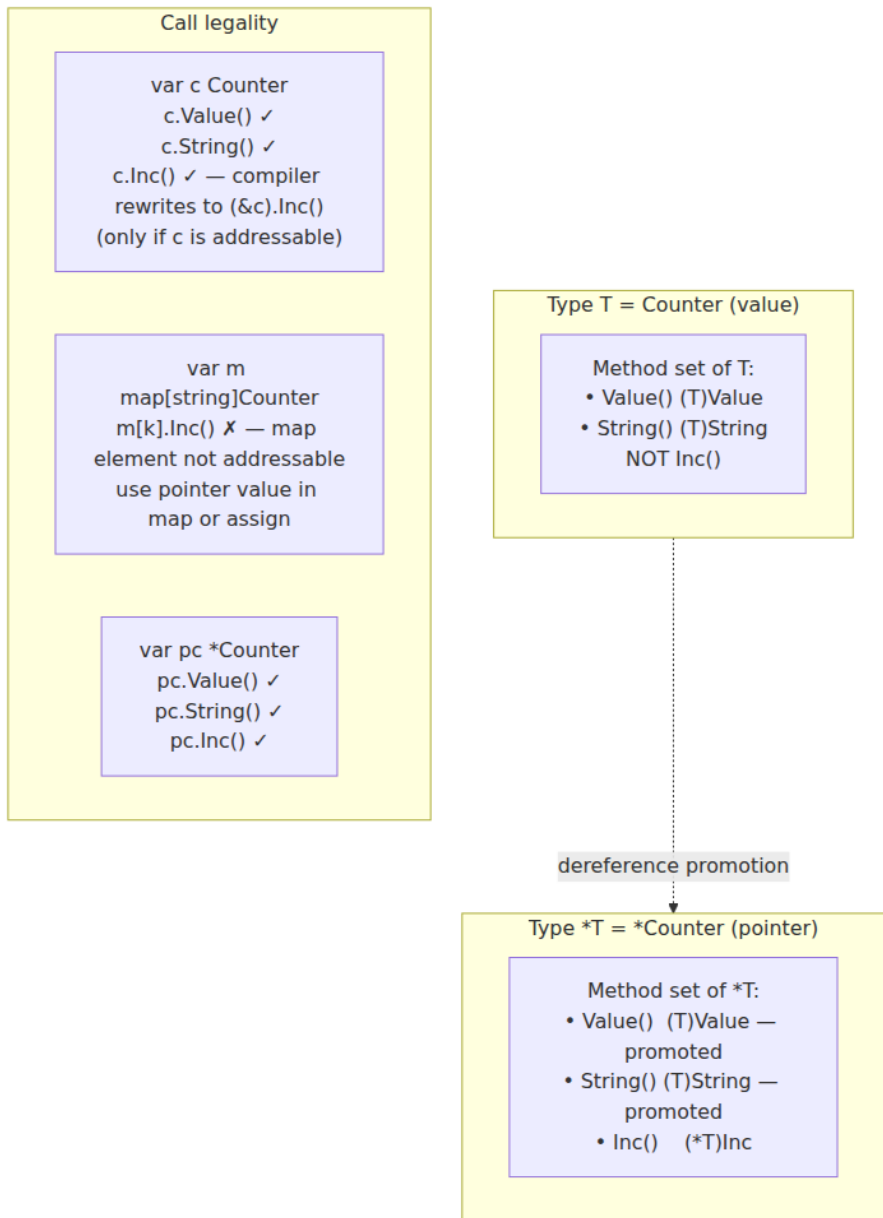
3. Methods: receivers, method sets, and dispatch

3.1 Value vs. pointer receivers — the method set

Every type has a **method set**: the methods that can be called on values of that type. The pointer type's method set is a superset.

```
type Counter struct{ n int }

func (c Counter) Value() int { return c.n }           // value receiver
func (c *Counter) Inc()      { c.n++ }               // pointer receiver
func (c Counter) String() string { return fmt.Sprint(c.n) }
```



Formal rule:

Receiver	Method set of T	Method set of *T
<code>func (t T) M()</code>	yes	yes (via promotion)
<code>func (t *T) M()</code>	no	yes

Consequence for interfaces (preview of Chapter 7):

```
type Incrementer interface{ Inc() }
type Valuer      interface{ Value() int }

var c Counter
var _ Valuer = c      // ok - T has Value()
var _ Incrementer = c // compile error: Counter does not implement
Incrementer
var _ Incrementer = &c // ok - *T has Inc()

func needInc(i Incrementer) { i.Inc() }
needInc(c) // compile error
needInc(&c) // ok
```

The compiler error is precise: `Counter` does not implement `Incrementer` (method `Inc` has pointer receiver).

3.2 When the receiver is copied

A value receiver copies the receiver. For a large struct this is a real cost — and it interacts with the register ABI (large structs that don't fit in registers spill to stack and are `memmove`d).

```
type Big struct{ Buf [1024]byte; N int }

func (b Big) Size() int { return b.N } // copies 1 KB on every
call!
func (b *Big) SizePtr() int { return b.N } // copies 8 bytes (the
pointer)
```

```
go test -bench=BenchmarkMethodRecv -benchmem
# BenchmarkMethodRecv/Value-8    12.3 ns/op    0 B/op    0 allocs/op
  (but 1KB copy in regs/stack)
# BenchmarkMethodRecv/Pointer-8  1.8 ns/op    0 B/op    0 allocs/op
```

Guideline: use a pointer receiver when any of these holds: the method mutates state, the struct is large (> ~64 bytes, or contains large arrays), or you need identity (pointer equality, `sync.Mutex` inside). Use a value

receiver for small immutable types (`time.Time` uses value receivers deliberately — 24 bytes, immutable).

`go vet / staticcheck` catch the sharp edge:

```
type Safe struct{ mu sync.Mutex; n int }
func (s Safe) Inc() { s.mu.Lock(); s.n++; s.mu.Unlock() } // vet:
copylocks - copies mutex!
```

`go vet` flags `copylocks`; `staticcheck` SA6005 flags large value-receiver copies.

3.3 Method values vs. method expressions

Two ways to take a method as a first-class function — they have different signatures:

```
type S struct{ n int }
func (s S) Add(delta int) int { return s.n + delta }
func (s *S) Incr()           { s.n++ }

var s = S{n: 10}

// Method value - receiver is bound (curried)
f1 := s.Add           // func(int) int - s is captured, call f1(5) ==
s.Add(5)
f2 := (&s).Incr       // func() - receiver is the specific *S at binding
time

// Method expression - receiver is an explicit first argument
g1 := S.Add           // func(S, int) int - call g1(s, 5) == s.Add(5)
g2 := (*S).Incr       // func(*S) - call g2(&s) == s.Incr()

// Useful for higher-order patterns:
slices.SortFunc(users, (*User).Less) // method expression as comparator
http.HandlerFunc(s.ServeHTTP)        // method value as handler
```

Representation: both `f1` and `g1` are func values (see §4), but a method value may allocate a small closure-like wrapper when the receiver is a value that must be copied. A method expression is a plain function pointer — no capture.

Common pitfall — binding the wrong thing in a loop:

```
// Bug: method value captures the loop variable's address implicitly
var funcs []func() int
for _, s := range items {
    funcs = append(funcs, s.Value) // each f captures a copy of s at
    that iteration – ok for value receiver
}
for _, s := range items {
    funcs = append(funcs, s.Incr) // s.Incr for value s – compiler
    takes &s, but loop var s is reused!
}
// Fix: shadow the variable
for _, s := range items {
    s := s
    funcs = append(funcs, s.Incr) // now each closure gets its own s
}
```

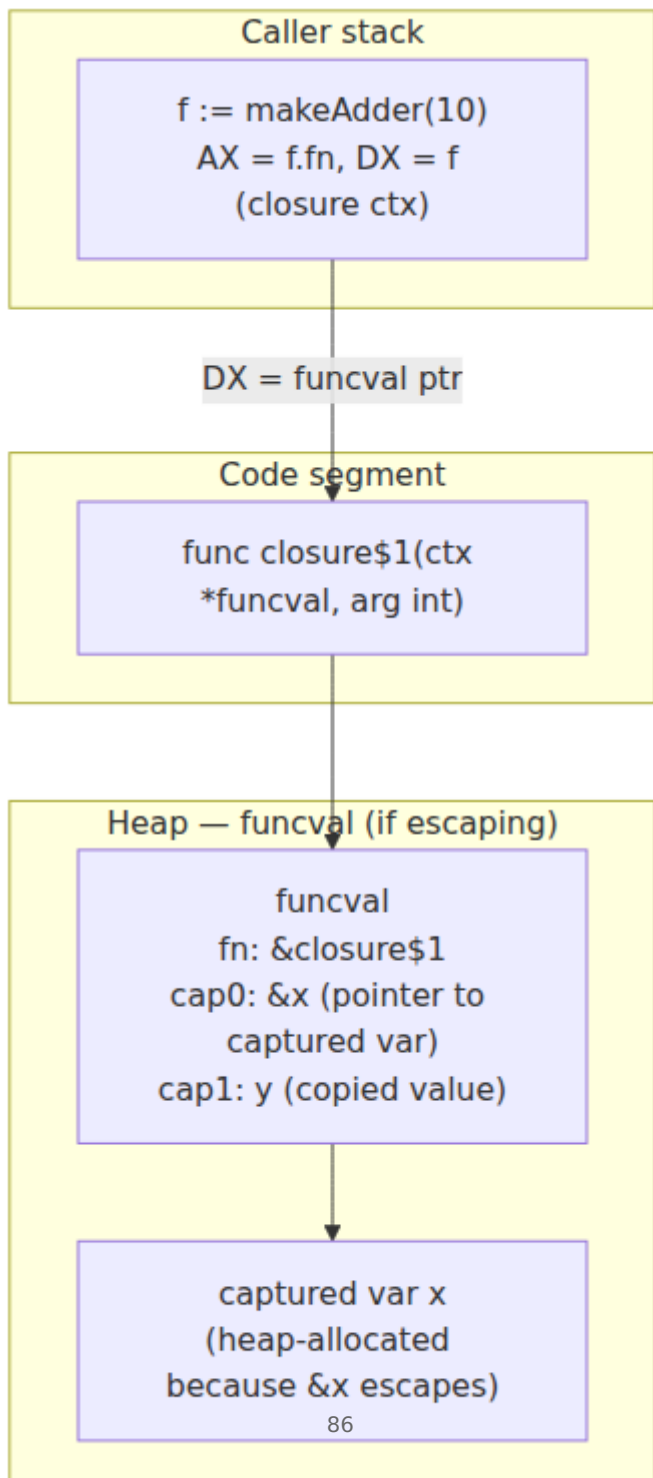
4. Closures: func values, capture, and heap escape

4.1 What a func value is

Every Go function (named or literal) has a code address. A **func value** (the thing you assign to a `func(...)` variable) is a pointer to a struct roughly like:

```
// runtime representation (simplified from runtime/runtime2.go –
funcval)
type funcval struct {
    fn uintptr // code address
    // followed by captured variables, if any
}
```

- A top-level `func Foo()` value is just `&funcval{fn: Foo}` — no captured data, no allocation beyond the `funcval` itself (often optimized away).
- A closure `func() { use(x, y) }` is `&funcval{fn: closure$1, x, y}` — the `funcval` is heap-allocated if it escapes.



On `amd64`, the closure context pointer is passed in `DX` (the first argument register is repurposed). The function body loads captures via `DX`:

```
// func makeAdder(base int) func(int) int { return func(x int) int
{ return base + x } }
// Closure body (from go tool compile -S):
TEXT ▢makeAdder.func1(SB), $0-16
    MOVQ 8(DX), AX    // load captured base (at offset 8 past funcval
header)
    ADDQ CX, AX       // CX holds x (register ABI: first arg)
    RET
```

4.2 Capture semantics and escape

Captured variables are **by reference** when they are assigned after capture, by value otherwise — but the implementation always captures a pointer for variables whose address is taken.

```
func Counter() func() int {
    n := 0                                // n escapes – see below
    return func() int { n++; return n } // captures &n
}

func Snapshot(vals []int) []func() int {
    var funcs []func() int
    for i, v := range vals {
        // i and v are reused per iteration – capturing &v captures the
        loop var, not the value
        funcs = append(funcs, func() int { return v }) // all funcs
        return last v!
    }
    return funcs
}
```

The loop-var pitfall was so common that Go 1.22 changed `for` loop semantics: each iteration now gets a fresh `v`. Before 1.22 the fix was `v := v` shadowing.

Escape diagnosis:

```
go build -gcflags="-m=2" ./pkg/clos.go 2>&1 | grep -E 'escape|closure|
moved to heap'
# ./clos.go:6:9: &n escapes to heap: moved to heap: n
# ./clos.go:7:9: func literal escapes to heap
# ./clos.go:7:9: closure variable n is heap-allocated

go vet ./...           # catches some captures, e.g., loopclosure
staticcheck ./...      # SA6002 (arg should be pointer), loop capture
checks
```

When a closure escapes (returned, stored in a field, sent on a channel, passed to `go`), the compiler heap-allocates the `funcval` and any captured variables that are addressed. Non-escaping closures — e.g., passed inline to `sort.Slice` or `http.HandlerFunc` that doesn't outlive the call — may stay on the stack with zero allocation.

Benchmark — non-escaping vs. escaping closure:

```
func BenchmarkClosureNoEscape(b *testing.B) {
    vals := []int{1, 2, 3, 4, 5}
    for i := 0; i < b.N; i++ {
        // closure does not escape – compiler can inline/stack-allocate
        slices.SortFunc(vals, func(a, b int) int { return a - b })
    }
}

func BenchmarkClosureEscape(b *testing.B) {
    for i := 0; i < b.N; i++ {
        fn := func(x int) int { return x + i } // i escapes – heap
        funcval per iter
        sink = fn
    }
}

// BenchmarkClosureNoEscape-8    45 ns/op    0 B/op    0 allocs/op
// BenchmarkClosureEscape-8      28 ns/op   16 B/op    1 allocs/op
```

The second benchmark's 16 bytes is the `funcval` header — cheap per call but toxic in a hot loop that creates millions of closures per second (GC pressure, not just CPU).

Guidance for backend code:

- Prefer iterators / `for` loops over closure-per-element in hot paths.
- If a closure must allocate, consider a method value on a pooled struct instead of a fresh closure.
- Watch `go test -benchmem` and `pprof -alloc_space` for `funcval` in flame graphs.

5. Defer: guarantees, internals, and cost

5.1 What `defer` guarantees

```
func CopyFile(dst, src string) (err error) {
    in, err := os.Open(src)
    if err != nil { return err }
    defer in.Close()                                // always runs, even on panic/
    early return

    out, err := os.Create(dst)
    if err != nil { return err }
    defer out.Close()                               // LIFO: out.Close runs before
    in.Close

    _, err = io.Copy(out, in)
    return err                                     // deferred calls run after
    return value is set, before caller resumes
}
```

Rules:

- Arguments are evaluated **immediately** at the `defer` statement, not at execution time.
- Execution is **LIFO** — last deferred first.
- Deferred functions run after the surrounding function's return values are set but before the caller resumes — so a deferred closure can modify named return values.
- Deferred functions run even if the function panics — they are the only place `recover` works (see §6).

```
func args() {
    x := 1
    defer fmt.Println(x) // prints 1, not 2 – arg evaluated now
    x = 2

    defer func() { fmt.Println(x) }() // closure captures &x – prints 2
    x = 3
}
```

5.2 Two implementations: linked-list defers vs. open-coded defers

Before Go 1.14, every `defer` allocated a `_defer` record (on heap or stack) and pushed it onto a per-goroutine linked list. At function exit the runtime walked the list.

Since Go 1.14, the compiler **open-codes** defers in common cases: it inlines the defer logic as conditional calls at each return point, with no heap allocation and no linked list. Fallback to the linked list remains for loops, condition-guarded defers that can't be proven unconditional at compile time, and functions with many defers.



Runtime struct (simplified from `runtime/runtime2.go`):

```
type _defer struct {
    started bool
    heap    bool    // allocated on heap (vs. stack)
    sp      uintptr // stack pointer at defer time
    pc      uintptr // caller PC
    fn      *funcval // deferred function
    link    *_defer  // next in g._defer chain
}
```

How to tell which path your function uses:

```
go tool compile -S -N ./pkg/foo.go 2>&1 | grep -E 'deferproc|
deferreturn|deferBits'
# Open-coded: no deferproc; look for deferBits and conditional calls
# Linked-list: CALL runtime.deferprocStack(SB) or
runtime.deferproc(SB)

# Verbose GC output (Go 1.21+):
GODEBUG=gocmd=1 go build -gcflags="-d=defer" ./... 2>&1 | grep defer
```

Example — open-coded (typical handler):

```
//go:noinline
func withOpenDefer() {
    defer fmt.Println("a") // open-coded – one defer, not in loop
    defer fmt.Println("b") // both become conditional calls at RET
}
```

```
// go tool compile -S withOpenDefer - open-coded (Go 1.22 amd64,
// trimmed)
TEXT 0x0000000000000000<withOpenDefer(SB)>, $48-0
    MOVQ $0, deferBits+40(SP) // bitmask of active defers
    ORQ  $1, deferBits+40(SP) // arm first defer
    ORQ  $2, deferBits+40(SP) // arm second defer
    // ... body ...
    TESTB $2, deferBits+40(SP)
    JEQ  skip1
    CALL 0x0000000000000000<fmt.Println(SB) // second defer (LIFO – runs first)
skip1:
    TESTB $1, deferBits+40(SP)
    JEQ  skip2
    CALL 0x0000000000000000<fmt.Println(SB) // first defer
skip2:
    RET
```

Example — heap defer (loop):

```
func withLoopDefer(n int) {
    for i := 0; i < n; i++ {
        defer fmt.Println(i) // cannot be open-coded – falls back to
        linked list
    }
}
```

```
TEXT 0x0000000000000000<withLoopDefer(SB)>, $32-0
.Loop:
    MOVQ AX, 0(SP)
    CALL runtime.deferprocStack(SB) // heap/stack _defer per iteration
    // ...
```

5.3 The defer-in-loop pitfall

The most common `defer` bug in backend code is resource exhaustion from deferring inside a loop without realizing all defers run only when the *enclosing function* returns, not at the end of the iteration.

```

// BUG: defers accumulate – files stay open until processFiles returns
func processFiles(paths []string) error {
    for _, p := range paths {
        f, err := os.Open(p)
        if err != nil { return err }
        defer f.Close() // not at end of iteration – at end of
processFiles!
        if err := handle(f); err != nil { return err }
    }
    return nil // all n files closed here, LIFO
}

// FIX 1: use an inner function – its defers run per iteration
func processFilesFixed(paths []string) error {
    for _, p := range paths {
        if err := func() error {
            f, err := os.Open(p)
            if err != nil { return err }
            defer f.Close() // runs at end of this closure
            return handle(f)
        }(); err != nil {
            return err
        }
    }
    return nil
}

// FIX 2: no defer at all – explicit close per iteration
func processFilesExplicit(paths []string) error {
    for _, p := range paths {
        f, err := os.Open(p)
        if err != nil { return err }
        err = handle(f)
        f.Close()
        if err != nil { return err }
    }
    return nil
}

```

`go vet` and `staticcheck` flag `defer` inside loops (since Go 1.17 `go vet` has `loopclosure`-adjacent checks; `staticcheck` SA5008 flags `defer` in loop). Treat any flag as a bug in production file-handling code.

5.4 Performance: when `defer` costs and when it doesn't

Open-coded defers brought the cost of a single `defer` from ~35 ns + allocation down to ~1-3 ns (a conditional branch plus the call). But two costs remain:

1. **The call itself still executes** — a `defer mu.Unlock()` still calls `Unlock`, just without the runtime dispatch.
2. **Loop / many-defer fallback** still allocates.

Benchmark (amd64, Go 1.22, `go test -bench=. -benchmem`):

```
func BenchmarkNoDefer(b *testing.B) {
    for i := 0; i < b.N; i++ {
        mu.Lock()
        x++
        mu.Unlock()
    }
}

func BenchmarkDefer(b *testing.B) {
    for i := 0; i < b.N; i++ {
        mu.Lock()
        defer mu.Unlock() // BUG: defers to end of BenchmarkDefer, not
// per iteration!
        x++
        // ... but for microbench of defer cost, fix:
    }
}

// Correct per-iteration defer microbench:
func BenchmarkDeferInline(b *testing.B) {
    for i := 0; i < b.N; i++ {
        func() {
            mu.Lock()
            defer mu.Unlock()
            x++
        }()
    }
}
```

Typical results:

BenchmarkNoDefer-8	12.1 ns/op	0 B/op	0 allocs/op	
BenchmarkDeferInline-8	14.8 ns/op	0 B/op	0 allocs/op	// open-coded: ~2-3 ns overhead
BenchmarkDeferLoopHeap-8	48.3 ns/op	48 B/op	1 allocs/op	// heap defer in loop

The `defer mu.Unlock()` in a request handler (~45 ns handler) adding 2-3 ns is irrelevant. The same `defer` in a `for` that runs 10M times allocating per iteration is catastrophic.

Backend lens — defer in hot HTTP middleware:

```
// Typical middleware – defer is fine (once per request, open-coded)
func tracingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        span, ctx := tracer.Start(r.Context(), "handler")
        defer span.End() // one defer, not in loop – open-coded, ~2 ns
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// Hot inner loop – avoid defer (millions of iterations)
func encodeBatch(batch []Record, w io.Writer) {
    for _, rec := range batch {
        // BAD: defer per record → heap alloc × batch size
        // defer rec.Release()

        // GOOD: explicit release, or inner func if you need defer
        semantics
        rec.Encode(w)
        rec.Release()
    }
}
```

Rule of thumb: `defer` per request or per function call is free; `defer` per element in a hot loop is a bug. When profiling shows `runtime.deferprocStack` in a hot frame, rewrite the loop.

6. Panic and recover: unwinding

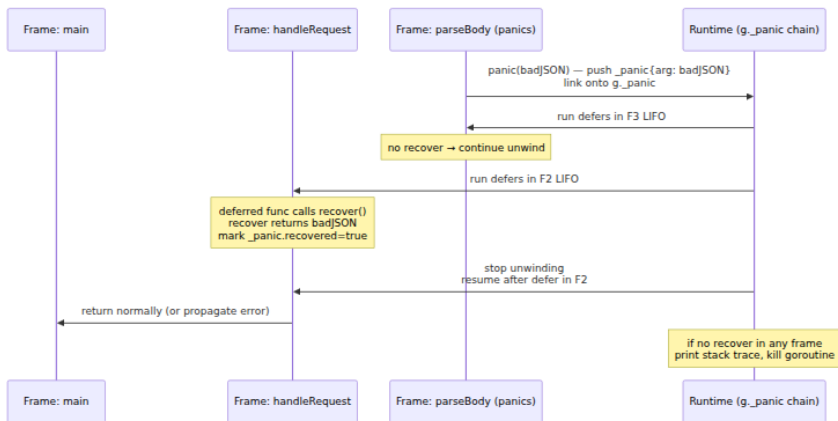
6.1 The `_panic` struct and unwinding

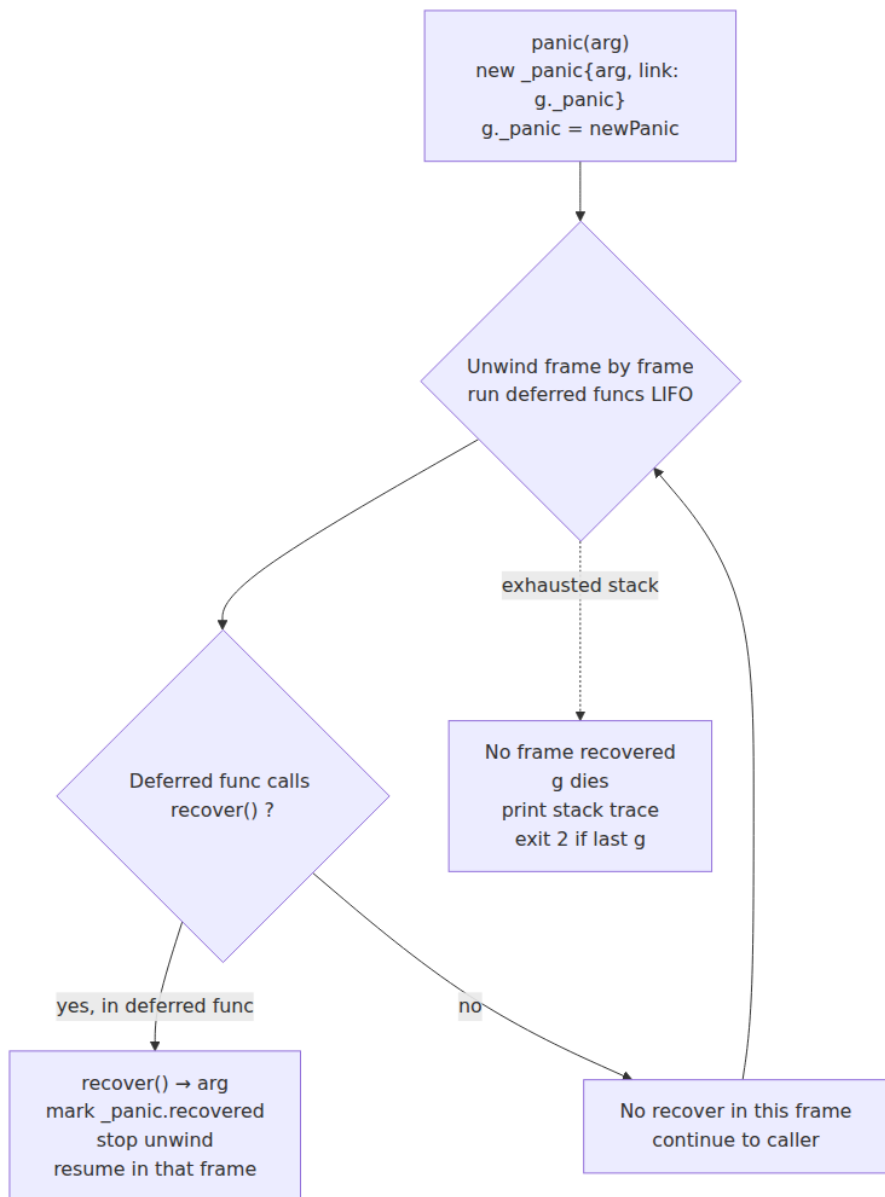
A `panic` is not an exception — it is a runtime-initiated unwind that runs deferred functions on the way out. The runtime represents each active panic as a `_panic` record linked off the goroutine:

```
// runtime/runtime2.go (simplified)
type _panic struct {
    arg    any    // value passed to panic(arg)
    link  *_panic // next panic in stack (panics can nest)
    recovered bool    // has recover() been called?
    aborted bool    // was this panic aborted (recovered at right
frame)?
}
type g struct {
    // ...
    _panic *_panic // head of panic chain for this goroutine
    _defer *_defer  // head of defer chain
}
```

Control flow:

1. `panic(arg)` allocates a `_panic`, links it onto `g._panic`, then starts unwinding.
2. The runtime walks the stack from the panicking frame outward, running deferred functions LIFO at each frame.
3. If a deferred function calls `recover()`, the runtime marks that `_panic` as `recovered` and stops unwinding — execution resumes at the `recover` ing frame after its defers finish.
4. If no deferred function recovers, the goroutine dies and the runtime prints a stack trace; if it is the last goroutine, the program exits with code 2.





6.2 `recover` only works in a deferred function

This is the single most misunderstood rule. `recover` returns non-nil **only** when called directly inside a deferred function that is running during unwinding. In any other position it returns `nil` and does nothing.

```

func brokenRecover() {
    defer func() {
        // Correct: recover inside deferred function during unwind
        if r := recover(); r != nil {
            log.Printf("recovered: %v", r)
        }
    }()
    panic("boom") // recovered
}

func silentBroken() {
    defer fmt.Println(recover()) // prints nil – recover's caller is
    // not the deferred func's body?
    // Actually this DOES work – the deferred func body calls recover.
    // But:
    panic("boom") // recovered? No – fmt.Println(recover()) – recover
    // is called, but its return is printed then discarded; panic is recovered
    // but nothing handles it specially

    // These do NOT recover:
    // r := recover() // not in a deferred func – always nil, panic
    // continues
}

func doubleBroken() {
    r := recover() // nil – not in defer, not during panic
    fmt.Println(r)
    panic("boom") // not recovered → goroutine dies
}

func correctPattern() (err error) {
    defer func() {
        if r := recover(); r != nil {
            // Must be directly inside deferred func, not a helper
            // called by it
            err = fmt.Errorf("panic recovered: %v", r)
            // Optional: capture stack with runtime.Stack or
            // debug.Stack
            log.Printf("panic stack: %s", debug.Stack())
        }
    }()
    return doWork() // if doWork panics, err is set and caller sees an
    // error
}

```

Subtle case — helper breaks recover:

```

func handlePanic() {
    if r := recover(); r != nil { log.Println(r) } // NOT a deferred
    func → recover is nil
}
func bad() {
    defer handlePanic() // handlePanic is deferred, but recover is
    inside handlePanic, which IS the deferred func – this DOES recover!
    // Actually: handlePanic itself is the deferred function, so
    recover inside it works.
    // What doesn't work:
    defer func() { handlePanic() }() // recover inside handlePanic is
    NOT directly in the deferred func – spec says recover must be called
    directly by deferred func, not transitively
    panic("boom")
}

```

The spec's exact wording: `recover` returns the panic value only when called directly inside a deferred function. Calling it from a function called by the deferred function returns `nil`. In practice the runtime checks that the caller of `recover` is a deferred frame — transitive calls through helpers don't count.

6.3 Panic stack trace

An unrecovered panic prints every goroutine's stack. On a backend service this is the crash log you collect:

```

panic: invalid JSON: unexpected EOF

goroutine 42 [running]:
main.parseBody(0xc0001a4000, 0x47, 0x80)
    /home/ubuntu/app/handler.go:38 +0x12a
main.handleRequest({0x7f8a1c, 0xc0001b0000}, 0xc0001a2000)
    /home/ubuntu/app/handler.go:22 +0x85
net/http.HandlerFunc.ServeHTTP(0xc0001a6000, {0x7f8a1c, 0xc0001b0000},
    0xc0001a2000)
    /usr/local/go/src/net/http/server.go:2136 +0x29
...
goroutine 1 [chan receive]:
main.main()
    /home/ubuntu/app/main.go:15 +0x45

exit status 2

```

Reading it:

- `goroutine 42 [running]` — the panicking goroutine; others are dumped for context.
- `handler.go:38 +0x12a` — file, line, and offset into the function (use `go tool addr2line` or `addr2line -e /tmp/app 0x461740` to map).
- The trace includes `runtime.gopanic` frames if you capture with `debug.Stack()`; `recover`'ed panics don't print unless you log `debug.Stack()` yourself.

Capture for structured logging:

```
func withPanicLog(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        defer func() {
            if rec := recover(); rec != nil {
                stack := debug.Stack() // includes goroutine header +
all frames
                log.Error("panic recovered",
                    "panic", rec,
                    "stack", string(stack),
                    "path", r.URL.Path,
                    "request_id", r.Header.Get("X-Request-ID"),
                )
                http.Error(w, "internal error", http.StatusInternalServerError
erError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

6.4 Panic vs. error: idioms for backend services

Use error	Use panic
Expected failure modes: validation, I/O, timeout, not-found	Truly exceptional / programmer error: violated invariant, must-not-happen
Caller can and should handle it	Caller cannot meaningfully handle it; crash or recover at boundary
Part of function signature — forces handling	Invisible to caller — easy to miss

Use error	Use panic
Composable with <code>errors.Is / As</code> , <code>fmt.Errorf("%w")</code>	Composable only via deferred <code>recover</code> at a boundary

Backend rules:

1. **Library and handler code returns error** . Never panic for a bad request — return `400 / 422` with a typed error.
2. **Panics are for assertions and unrecoverable state** —
`panic("unreachable")` , nil-deref that indicates a bug, `init` failure for required config.
3. **Recover only at process/goroutine boundaries** — top-level HTTP middleware, `go` statement wrapper, `net/http` server's `ConnState` — not in the middle of business logic.
4. **Recovered panics are still errors** — convert to `error` , log with stack, increment a `panics_total` counter, and return the error to the caller. Don't swallow silently.

```
// Canonical goroutine-boundary recover – every bare `go` should have this
func GoRecover(fn func()) {
    go func() {
        defer func() {
            if r := recover(); r != nil {
                panicsTotal.Inc()
                log.Error("goroutine panicked", "panic", r, "stack", string(debug.Stack()))
            }
        }()
        fn()
    }()
}

// Anti-pattern: panic as control flow for expected errors
func mustParse(s string) int { // "must" helpers are okay only when panic is documented and caller recovers at boundary
    n, err := strconv.Atoi(s)
    if err != nil { panic(err) } // okay only if caller uses GoRecover / middleware recover – brittle
    return n
}

// Prefer:
func parseWithError(s string) (int, error) { return strconv.Atoi(s) }
```

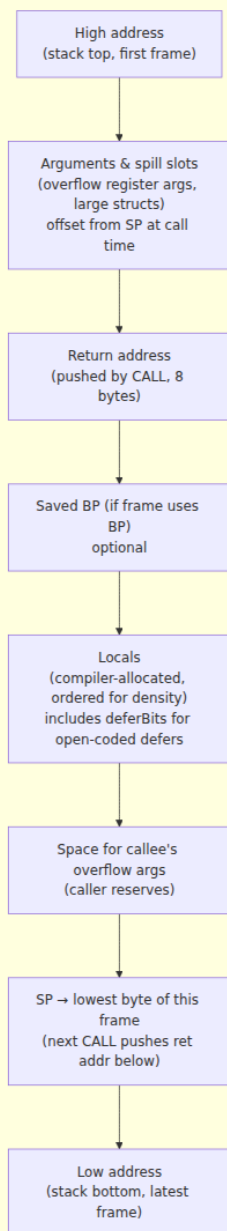
`go vet` flags some `panic/recover` misuses; `staticcheck` flags `SA5011` (possible nil deref that would panic) and warns on `recover` not in `defer`.

7. Stack frames, caller-saved registers, and the vet layer

7.1 Stack frame layout

Every goroutine has a contiguous, growable stack (initial 2–8 KB, doubling on overflow since Go 1.4). Each frame's layout on `amd64`:

Single goroutine stack (grows down, SP decreases)



Caller-saved regs are
spilled to LOCALS if live
across call

NOSPLIT functions
(`//go:nosplit`) skip stack-
growth check — leaf &
small frame only

Concrete example with frame size `$32-0` (32 bytes of frame, 0 bytes of stack-arg area in register ABI):

```
TEXT example(SB), $32-8 // 32 bytes locals, 8 bytes for callee
overflow arg
    SUBQ $32, SP           // reserve locals (compiler does this)
    MOVQ BP, 16(SP)        // save BP if needed
    LEAQ 8(SP), BP          // establish BP
    // ... body uses -24(SP), -16(SP) etc for locals
    ADDQ $32, SP           // restore
    RET
```

Stack growth: the prologue checks `SP < g.stackguard0`; if so, calls `runtime.morestack` which allocates a larger stack, copies the old one, and adjusts pointers. `//go:nosplit` suppresses this check — only safe for leaf functions with tiny frames that never call into growable code (used in `runtime` itself).

Reading frame sizes in `go tool objdump / objdump -d: SUBQ $N, SP` at entry is the frame size. A missing `SUBQ` means `$0` — no frame (common for register-ABI leaf functions like `Add`).

7.2 Caller-saved registers in practice

Because the ABI is caller-saved, the compiler spills live registers around calls:

```
//go:noinline
func callee(a, b int) int { return a + b }

func caller(x, y, z int) int {
    a := x + y           // a live in AX
    b := callee(a,
z) // callee clobbers AX, BX – compiler must spill a if still needed
    return a + b         // so it reloads a from stack slot
}
```



```

TEXT caller(SB), $16-0
    ADDQ BX, AX      // a = x + y (AX holds a)
    MOVQ AX, 8(SP)   // spill a – caller-saved, callee will clobber
AX
    MOVQ CX, BX      // z → BX (second arg)
    // AX already holds a (first arg)
    CALL callee(SB) // result in AX
    ADDQ 8(SP), AX    // reload spilled a, add to result
    RET

```

This is why passing fewer, smaller arguments and keeping functions short helps the register allocator — fewer spills, fewer loads.

7.3 go vet and staticcheck for call/ABI-adjacent bugs

Check	Tool	What it catches
copylocks	go vet	Passing sync.Mutex / WaitGroup by value (copies the lock)
loopclosure / defers	go vet	Closure captures loop var (pre-1.22), defer in loop
nilfunc	go vet	Call of nil func value (would panic at CALL *DX)
SA6002, SA6005	staticcheck	Argument should be pointer (large struct by value), large value-receiver
SA5008	staticcheck	defer in loop — resource leak
SA5011	staticcheck	Possible nil dereference → panic
ST1008 etc.	staticcheck	Panic/recover misuse patterns

Run them in CI as gates, not suggestions:

```

go vet ./...
staticcheck ./...

# In CI (fail on any finding):
go vet ./... 2>&1 | tee vet.out; test ! -s vet.out
staticcheck ./... 2>&1 | tee sc.out; test ! -s sc.out

# golangci-lint aggregates both:
golangci-lint run --enable vet,staticcheck ./...

```

8. Backend lens: performance, resilience, and observability

8.1 Defer budget in hot paths

On a service doing 50 K req/s, each request passing through 4 middleware that each `defer span.End()` / `defer metrics.observe()` adds 4 open-coded defers $\times \sim 2 \text{ ns} = \sim 8 \text{ ns}$ — noise against a 200 μs handler. The budget concern is **per-element** work inside the handler:

Pattern	Cost	When it hurts
<code>defer unlock</code> per request	$\sim 2 \text{ ns}$, 0 allocs	never
<code>defer close</code> per file in batch of 10	$10 \times$ heap <code>_defer</code> if in loop	batch size $\times$ 48 B alloc, GC pressure
<code>defer in for row.Next()</code> over 1M rows	1M heap defers if not open-coded	OOM-able, p99 spike
Middleware chain 10 deep	10 open-coded defers	$\sim 20 \text{ ns}$ — fine

Measure, don't guess:

```
go test -bench=BenchmarkMiddleware -benchmem -count=5 | benchstat
old.txt new.txt
go test -run=^$ -bench=. -cpuprofile=cpu.pprof
go tool pprof -http=:8080 cpu.pprof # look for runtime.deferprocStack
in hot stack
```

8.2 Panic as control flow — an anti-pattern at scale

Some codebases use `panic/recover` to implement early-exit or `try/catch`:

```
// Anti-pattern: panic for expected validation failure
func handleBad(w http.ResponseWriter, r *http.Request) {
    defer func() {
        if rec := recover(); rec != nil {
            http.Error(w, fmt.Sprintf(rec), 400)
        }
    }()
    if r.Header.Get("Authorization") == "" { panic("missing auth") }
    // ...
}
```

Why it fails at scale:

- **Invisible to callers and linters** — no `error` in the signature, so callers don't know to handle it, `errcheck` doesn't flag it.
- **Interferes with real panics** — a nil-deref bug is now indistinguishable from a missing header; both become `400`.
- **Breaks middleware contracts** — upstream middleware that recovers panics will swallow your "error" before it reaches your handler.
- **Kills inlining and escape optimization** — the compiler treats panicking functions conservatively.

The only defensible use of `panic` for control flow is inside a package boundary that converts it immediately — e.g., `encoding/json` panics on deeply nested structures and recovers at the top-level `Marshal` call, or a parser that panics on syntax error and recovers at `Parse`. Even there, prefer `error` unless benchmark shows panic recovery is measurably faster (usually not; panic unwinding is $\sim\mu\text{s}$, error return is $\sim\text{ns}$).

8.3 Observability of panics in Kubernetes

An unrecovered panic kills the goroutine; if it is the main goroutine or the last one, the process exits with code 2. In Kubernetes this is a **CrashLoopBackOff**:

```
# Pod that panics on a bad env var at startup – crash-loops forever
spec:
  containers:
    - name: api
      image: registry.example.com/api:v1.2.3
      env: [{ name: BAD_CONFIG, value: "oops" }]
      # Without proper probing, kubelet restarts immediately; with
      backoff, delay grows 10s → 20s → 40s ...
```

Observability checklist:

1. **Log panics with stack and request context** — use the `debug.Stack()` middleware from §6.3 and include `trace_id / request_id` so you can correlate.
2. **Count panics as a metric** — `panics_total{path, panic_type}` with an alert on `rate(panics_total[5m]) > 0`. A single panic in production is a bug; a rate is an incident.
3. **Don't let the process die silently** — set `GOTRACEBACK=crash` (full stacks for all goroutines, plus `runtime` frames) and ensure logs go to a collector that survives restart (`stdout` → `fluent-bit` → `Loki/ELK`, not a file on the container filesystem).
4. **Distinguish panic from OOM** — both are `CrashLoopBackOff`, but `kubectl describe pod` shows `Reason: Error + Exit Code: 2` for panic vs. `Reason: OOMKilled + Exit Code: 137` for memory. Alert on the wrong one and you tune `GOMEMLIMIT` when you should fix a nil deref.
5. **Use `GODEBUG=gctrace` sparingly** — not for panics, but useful when a panic-like crash is actually GC-assisted OOM.

```
# Triage a crash-looping pod
kubectl logs --previous api-7f9c8-xk2pq 2>&1 | grep -A 50 '^panic:'
kubectl describe pod api-7f9c8-xk2pq | grep -A 5 'Last State'
# Exit Code 2 → panic; 137 → OOMKilled; 1 → app error return

# Structured panic log query (Loki example):
# {app="api"} |= "panic recovered" | json | panic!=""
```

For services that must stay up through panics (long-lived workers, queue consumers), the `GoRecover` wrapper from §6.4 plus a circuit-breaker is the standard pattern: recover, count, and continue processing the next message — never let one bad message kill the consumer.

Key takeaways

- The Go register ABI (Go 1.17+, `ABIInternal`) passes integer args in `AX, BX, CX, DI, SI, R8-R11` and floats in `X0-X14` on `amd64` (`R0-R8 / F0-F7` on `arm64`), with overflow to stack. Before 1.17 everything was on stack. The register ABI cut call overhead by avoiding memory ops

and is visible in `go tool compile -S` as `ADDQ BX, AX` with no `MOVQ arg+0(FP)`.

- The ABI is caller-saved — callees may clobber argument registers. The caller spills live values around calls; keeping signatures small and functions short reduces spills.
- Method set of `T` includes only value-receiver methods; method set of `*T` includes both. This determines interface implementation: `Counter` does not implement `Incrementer` if `Inc` has a pointer receiver, but `*Counter` does. Method values bind the receiver; method expressions take it as an explicit first argument.
- A func value is a pointer to `{fn, captures}`. Captured variables that are addressed or that outlive the closure escape to heap — visible via `go build -gcflags=-m`. Per-iteration closures in hot loops allocate `funcval` headers and drive GC pressure.
- `defer` is LIFO, args evaluated at defer time, and runs after return values are set even under panic. Since Go 1.14 the compiler open-codes defers as conditional direct calls (~2 ns, 0 allocs) when not in a loop and with few defers; otherwise it falls back to a heap-linked `_defer` chain via `runtime.deferprocStack` (~35 ns + alloc). `defer` in a loop is a resource leak — all defers run at function return, not iteration end.
- `panic` pushes a `_panic{arg, link}` onto `g._panic` and unwinds, running deferred functions LIFO per frame. `recover` returns non-nil only when called directly inside a deferred function during unwinding; transitive calls don't recover. Unrecovered panics kill the goroutine and dump all stacks, exiting with code 2.
- Panic vs. error: return `error` for expected failures, reserve `panic` for programmer bugs and unrecoverable invariants, and recover only at goroutine/process boundaries (HTTP middleware, `go` wrappers). Convert recovered panics to errors, log with `debug.Stack()`, and count `panics_total`.
- Every frame has args/spill, return address, saved BP, locals (including `deferBits`), and space for callee overflow args. Frame size (`$32-8` etc.) and `SUBQ $N, SP` in `objdump` tell you the cost. `//go:nosplit` skips the stack-growth check — only for tiny leaf functions.
- `go vet` and `staticcheck` catch the sharp edges automatically: `copylocks`, loop-closure capture, `defer` in loop (`SA5008`), and `recover` misuse. Gate CI on them.

- In backend services, `defer` per request (middleware tracing/logging) is free; `defer` per element in a hot batch loop is a performance bug (look for `runtime.deferprocStack` in `pprof`). Panics in Kubernetes surface as `CrashLoopBackOff` with exit code 2 — log structured panics with `trace_id`, metric them, and keep `GOTRACEBACK=crash` so you get full dumps.
-

Further reading

- **[Pinned] Go Register ABI Design Document — `golang.org/s/regabi` (and `doc/abi.md` in the Go repo).** The authoritative spec for register assignment, spill rules, and the `ABIInternal` / `ABI0` distinction. Start here for the calling convention. — <https://go.dev/s/regabi> and <https://github.com/golang/go/blob/master/doc/abi.md>
- **[Pinned] Open-Coded Defers Proposal and Implementation — `golang.org/s/open-coded-defers`.** The Go 1.14 proposal that describes when defers are open-coded vs. heap-allocated, the `deferBits` mechanism, and the performance rationale. — <https://go.dev/s/open-coded-defers> and <https://github.com/golang/proposal/blob/master/design/34481-open-coded-defers.md>
- **[Pinned] Go Spec: Handling Panics and Recover — `go.dev/ref/spec#Handling_panics`.** The language spec's precise rules for when `recover` returns non-nil, the LIFO defer guarantee, and the panic/recover interaction. — https://go.dev/ref/spec#Handling_panics
- **[Pinned] `cmd/compile` ABI Documentation — `src/cmd/compile/abi-internal.md` in the Go repo.** Per-architecture register maps, pseudo-register (`FP` / `SP` / `SB`) meaning, and guidance for hand-written assembly under the register ABI. — <https://github.com/golang/go/blob/master/src/cmd/compile/abi-internal.md>
- **Go Toolchain: `go tool compile -S` and `go tool objdump`.** The two views of generated code — `compile -S` (SSA assembly with pseudo-registers) vs. `objdump` (linked machine code). `go help tool compile` and `go help tool objdump`. — <https://pkg.go.dev/cmd/compile> and <https://pkg.go.dev/cmd/objdump>
- **Go `runtime` package docs — `runtime`, `runtime/debug`.** `debug.Stack()`, `debug.PrintStack()`, `runtime.Caller` / `Callers` /

Stack, and GOTRACEBACK modes. — <https://pkg.go.dev/runtime> and <https://pkg.go.dev/runtime/debug>

- **Static Analysis: go vet and staticcheck.** The checks that guard this chapter's pitfalls — copylocks, loopclosure, SA5008 (defer in loop), SA6002 / SA6005 (large value copy). — <https://pkg.go.dev/cmd/vet> and <https://staticcheck.dev/docs/checks/>
- **Practical Defer Performance** — github.com/golang/go/issues/14939 and associated CLs. The issue thread and benchmarks that motivated open-coded defers, with before/after numbers for defer in middleware. — <https://github.com/golang/go/issues/14939>
- **Escape Analysis Deep Dive** — [go.dev blog and cmd/compile escape docs](https://go.dev/blog/cmd-compile-escape-docs). Why closures and funcval heap-allocate, how //go:noinline and //go:noescape interact, and reading -gcflags=-m output. — https://go.dev/doc/effective_go#allocation and <https://github.com/golang/go/blob/master/src/cmd/compile/internal/escape>
- **Assembly Interop** — golang.org/s/goasm and [src/cmd/asm docs](https://go.dev/doc/asm). Go assembly syntax, TEXT directives, FP / SB pseudo-registers, and writing ABI-aware asm.s files. — <https://go.dev/doc/asm>

Chapter 4 — Goroutines, the Scheduler (G-M-P), and the Netpoller

What this chapter covers. Go ships its own user-space scheduler and I/O reactor inside every static binary. When you write `go handle(conn)` in an HTTP server, you are not creating an OS thread — you are enqueueing a 2 KB `g` onto a per-P run queue that an `M` will steal, park on the netpoller, and preempt via signal. Understanding that machinery is the difference between a service that scales to 100k concurrent connections on 4 cores and one that collapses under goroutine leaks, `GOMAXPROCS` misconfiguration in Kubernetes, or netpoller saturation you blame on "slow network." This chapter opens `runtime/proc.go` and `runtime/netpoll*.go`: the `G`, `M`, `P` structs, the scheduling loop, work-stealing, `sysmon`, async preemption, and the `epoll / kqueue` netpoller that makes non-blocking I/O feel blocking.

Learning goals — after this chapter you should be able to:

- Contrast a goroutine and an OS thread on cost (stack, creation time, scheduling), blocking behavior, and observability, and explain why >100k goroutines is routine but >10k threads is not.
- Read the `g`, `m`, and `p` structs in `runtime/runtime2.go` — stack bounds, `atomicStatus`, `waitReason`, `g0 / curg`, `lockedg`, `runq / runnext`, `mcache`, timers — and map each field to a scheduler decision.
- Trace the scheduling loop: `schedule` → `findRunnable` → `execute`, global vs. local run queues, `runnext`, and work-stealing, and explain what `sysmon` does on its 20 μ s-10 ms tick.
- Explain blocking paths: `entersyscall / exitsyscall` P-handoff, `gopark / goready` for channels/timers/netpoll, and when `runtime.LockOSThread` is required (`cgo`, `setns`, `sched_setaffinity`).
- Describe async preemption (`SIGURG`, `asyncPreempt` since Go 1.14): how a 10 ms `for { x++ }` loop is preempted at a safepoint, and what remains un-preemptible (`cgo`, tight assembly).
- Walk the netpoller path end-to-end: `pollDesc` → `netpollopen` → `gopark` → `epoll_wait` in `netpoll` → `netpollready` → `goready`, and why this makes `net.Conn.Read` not block an `M`.
- Capture and read scheduler observability: `GODEBUG=schedtrace`, `GODEBUG=gctrace-style godebug` (Go 1.21+), `go tool trace`, `runtime.Stack` dumps, and the goroutine pprof profile — and use them to diagnose run-queue latency and netpoller stalls.
- Size `GOMAXPROCS` correctly inside cgroup-limited containers, use `automaxprocs` or manual cgroup detection, and reason about when to raise vs. lower it for p99.

Placement. Volume 13, Chapter 2 gave the operator view of the Go runtime — what G/M/P do and how to size `GOMAXPROCS`. This chapter is the *internals* view: the structs, the code paths, and the traces that prove it. Chapter 3 covered the call-level ABI (how one function frames its arguments); here we cover how thousands of calls are scheduled across cores. Chapter 5 continues into the allocator and GC that share the same P-local caches; Chapter 8 revisits `gopark` from the channel/timer side.

1. Goroutine vs. OS thread — the cost model that shapes design

1.1 What you pay per thread vs. per goroutine

A backend service handling 20k concurrent long-lived connections (WebSocket, gRPC streaming) cannot map one OS thread per connection. Linux threads are heavy:

Property	OS thread (pthread)	Goroutine (G)
Initial stack	1-8 MB fixed (<code>ulimit -s</code> , <code>pthread_attr_setstacksize</code>)	2-8 KB contiguous, growable (8 KB on linux/amd64 since Go 1.19)
Creation cost	~10-50 μ s + <code>clone(2)</code> + kernel task struct	~0.2-1 μ s, user-space alloc from <code>P.gFree</code>
Scheduling	Kernel scheduler (CFS), preemptive, cross-core migration	Go scheduler (M:N), cooperative + async preemptive, P-local queues
Blocking I/O	Blocks the thread; need <code>epoll</code> + thread pool or <code>io_uring</code>	Parks the <code>G</code> on <code>netpoller</code> ; <code>M / P</code> immediately runs another <code>G</code>
Context switch	Kernel mode switch, TLB flush, ~1-2 μ s	User-mode <code>gogo</code> / <code>gopark</code> , ~100-200 ns, no syscall
Upper bound	~10k threads before kernel contention dominates	100k-1M goroutines routine (bounded by heap, not scheduler)
Debugging	<code>ps -L</code> , <code>top -H</code> , <code>/proc/<pid>/task</code>	<code>runtime.Stack</code> , <code>pprof -goroutine</code> , <code>go tool trace</code>

The numbers are approximate and platform-dependent, but the order-of-magnitude gap drives Go's concurrency style: fan out goroutines liberally, use blocking-style code, and let the runtime multiplex.

```
// Idiomatic Go: 50k concurrent connections as 50k goroutines – not 50k
// threads.
func serve(l net.Listener) {
    for {
        conn, _ := l.Accept()
        go handle(conn) // ~4 KB + channel/netpoll state; returns
        // immediately
    }
}
func handle(c net.Conn) {
    defer c.Close()
    buf := make([]byte, 4096)
    for {
        n, err := c.Read(buf) // parks G on netpoller; does NOT block M
        if err != nil { return }
        // ... process ...
    }
}
```

1.2 The mental model to keep (and when it breaks)

Goroutines are not free. Each holds a stack (grown by `runtime.morestack` → `copystack`), a `g` struct (~200 bytes), and often a `channel/timer/ pollDesc`. A leak of 1M idle goroutines still OOMs the pod:

```
goroutine profile: total 482193
482000 @ 0x43... in chan recv / netpollblock
runtime.gopark
runtime.chanrecv
myapp.workerLoop
```

Not every block is netpoller-friendly. Only network, `os` file I/O via `internal/poll` (on Linux, `epoll`-backed), timers, and `runtime`-aware `chan/select` park without blocking `M`. The following *do* block `M` and trigger P-handoff (Section 4):

- File `Read/Write` on regular files (no `epoll` readiness), `os/exec`, `syscall` without `RawConn`.
- `cgo` calls (`C.foo()`), which hold the `M` for their duration.
- `sync.Mutex` contended in tight loops (spins briefly, then `semaSleep` via `gopark` — efficient; but `LockOSThread` changes rules).

Preemption is not unbounded. Since Go 1.14 `async` preemption can interrupt a tight `for { x++ }` loop at loop back-edges and function

prologues, but it cannot preempt code running inside `cgo`, inside the runtime's own critical sections, or inside assembly that lacks safepoints.

2. The three structs: `G`, `M`, `P`

All defined in `runtime/runtime2.go` (the most-read file in the Go source). What follows mirrors that file as of Go 1.21-1.23; field order is simplified and comments trimmed.

2.1 G — the goroutine

```
// runtime2.go - type g struct (abridged)
type g struct {
    stack
    stack // stack.lo, stack.hi, stackguard0 - bounds + overflow
    check
    stackguard0 uintptr // compare SP against this to trigger
    morestack

    _panic *panic // innermost panic
    _defer *defer // innermost defer (open-coded defers may
    bypass this)
    m *m // current M; nil when not running
    sched gobuf // saved SP/PC/BP/g when not running (gogo/
    gopark context)
    syscallsp uintptr // sp saved on entersyscall
    stksize uintptr

    atomicStatus uint32 // G status (see table)
    goid int64 // unique id (for debugging; not stable)
    scheduling key)
    waitReason waitReason // why gopark'd - "chan receive", "IO
    wait", "sleep", ...
    preempt bool // preemption requested
    preemptStop bool
    preemptShrink bool
    lockedm uintptr // if != 0, G locked to this M
    (LockOSThread)
    // ... timers, pollDesc linkage, trace fields ...
}

type gobuf struct {
    sp uintptr
    pc uintptr
    g guintptr
    bp uintptr // frame pointer (if -framepointer)
    ret uintptr
    // ... ctxt for closures ...
}

type stack struct {
    lo uintptr
    hi uintptr
}
```

Stack. Since Go 1.4 stacks are contiguous (not segmented). `runtime.stackalloc` allocates from per-P pools; growth doubles capacity via `copystack` and adjusts all interior pointers using stack maps emitted

by the compiler. `stackguard0` is set to `stack.lo + stackGuard` so a `CMP SP, stackguard0; JLS morestack` at function prologue detects overflow with no extra branch in the common case (the same guard page mechanism on `amd64`).

Status (`atomicStatus`). Accessed atomically; transitions use CAS:

Value	Constant	Meaning
0	<code>_Gidle</code>	Just allocated, not yet runnable
1	<code>_Grunnable</code>	On a run queue (local, global, or <code>runnext</code>)
2	<code>_Grunning</code>	Executing on an <code>M</code>
3	<code>_Gsyscall</code>	In a syscall (<code>entersyscall</code>); <code>M</code> may be blocked, <code>P</code> may be handed off
4	<code>_Gwaiting</code>	Parked via <code>gopark</code> (channel, timer, netpoll, <code>WaitGroup</code> , ...)
5	<code>_Gmoribund_unused</code>	Historic (dead code)
6	<code>_Gdead</code>	Not in use; on <code>P.gFree</code> or <code>sched.gFreeStack</code>
7	<code>_Gcopystack</code>	Stack being moved; GC/scheduler barrier
8	<code>_Gpreempted</code>	Preempted but not yet rescheduled (async preemption transient)
9	<code>_Gscan*</code> variants	GC scan states (<code>_Gscan</code> <code>_G*</code> bitmask during mark)

`waitReason` is the human label you see in `runtime.Stack` and `pprof` ("IO wait", "chan receive", "select", "sleep", "GC assist wait"):

```
// runtime2.go - waitReason enum (abridged)
const (
    waitReasonZero          waitReason = iota
    waitReasonGCAssistWait
    waitReasonIOWait        // netpoller block
    waitReasonChanReceive
    waitReasonChanSend
    waitReasonSelect
    waitReasonSleep         // time.Sleep / timer
    waitReasonSemacquire    // sync.Mutex / Semaphore
    waitReasonSyncCondWait
    waitReasonPollWait
    // ... ~25 reasons
)
```

2.2 M — the machine (OS thread)

```
// runtime2.go - type m struct (abridged)
type m struct {
    g0          *g          // scheduling stack: never preempted, for
runtime work
    curg        *g          // current user G (nil when in scheduler)
    p           uintptr      // attached P (nil if idle or in syscall)
    nextp       uintptr      // P being acquired
    oldp        uintptr      // P before entersyscall
    id          int64
    mallocing   int32
    throwing    int32
    preemptoff  string      // if != "", don't preempt this M

    gsignal     *g          // g for signal handling
    sigmask     sigset

    tls         [tlsSlots]uintptr // thread-local storage (g, m)
    mcache      *mcache    // (historically; now P.mcache is
authoritative - M.mcache is nil when P attached)
    lockedg     uintptr      // G locked to this M via LockOSThread
    createstack [32]uintptr // for debugging m creation
    lockedExt    uint32
    lockedInt    uint32
    nextwaitm   uintptr
    // ... cgo, libcall, signal, procid fields ...
}
```

Every `M` has a `g0` — a special `G` with a fixed, large system stack (64 KB) that runs scheduler code, `morestack`, and signal handlers. User goroutine stacks never run scheduler internals; the switch `g → g0` is how `schedule()` and `gopark` execute without corrupting user state.

`mcache` history: before Go 1.4 every `M` had an `mcache` for allocation. Since `Ps` were introduced the cache lives on `P` (`p.mcache`); `m.mcache` is only non-nil when `M` is detached from `P` and needs to allocate (e.g., during `entersyscall`).

`lockedg` / `lockedm` pair enforces `LockOSThread`: when `g.lockedm != 0`, the runtime guarantees that `G` always resumes on the same `M`, and that `M` never runs another `G` that is also locked elsewhere. This is required for thread-affine syscalls (`setns`, `graphics`, `cgo` thread-local state).

2.3 `P` — the processor (scheduling context)

```
// runtime2.go - type p struct (abridged)
type p struct {
    id          int32
    status      uint32      // _Pidle / _Prunning / _Psyscall /
               // _Pgcstop / _Pdead
    link        uintptr    // freelist linkage
    m           uintptr    // M currently attached (0 if idle)
    mcache      *mcache    // allocator cache - the hot path for small
allocs
    pcache      pageCache
    racectx     uintptr

    deferpool   []*_defer
    deferpoolbuf [32]*_defer

    goidcache    uint64
    goidcacheend uint64
    runq         [256]uintptr // circular buffer - local run queue
    runqhead     uint32
    runqtail     uint32
    runnext

    uintptr      // one-slot LIFO: next G to run (fairness bypass for
latency)
    gFree        gList      // free Gs
    sudogcache   []*sudog
    // timers
    timers       []*timer
    // netpoll, GC
    // ...
    runqsize     int32
}
```

`P` count = `GOMAXPROCS`. Only a `G` holding a `P` may run user code or allocate via `p.mcache`. The trio of run queues per `P`:

- `runq[256]` — lock-free circular buffer; `runqhead` / `runqtail` are accessed atomically for stealing.
- `runnext` — single `G` slot written by `ready()` when a goroutine unblocks a peer (e.g., `ch <- v` wakes the receiver). Checked before `runq`; improves latency for ping-pong patterns without global queue contention.
- `sched.runq` + `sched.runqsize` — the global queue (protected by `sched.lock`), overflow for new `G`s when local queues fill, and the rendezvous for work-stealing.

`mcache` on `P` is why `GOMAXPROCS` also shapes allocator scalability (see Chapter 5): each `P` has its own `mspan` cache so small allocations avoid locks.

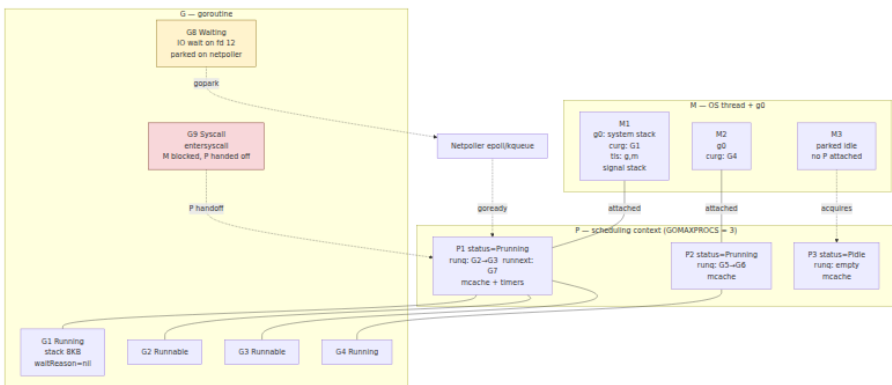


Diagram 1 — G-M-P overview. *Ms* execute *Gs* only while holding a *P*. Network-blocked *Gs* live on the netpoller with no *M/P*; syscall-blocked *Gs* keep their *M* but release the *P* so another *M* can pick it up.

3. The scheduling loop: queues, stealing, and sysmon

3.1 The three queues

```
// schedule() hot path (runtime/proc.go, simplified)
func schedule() {
    _g_ := getg()
    pp := _g_.m.p.ptr()

    // ... check gcWaiting, preemption ...

    var gp *g
    var inheritTime bool
    // 1) runnext (LIFO wake-up slot)
    if gp == nil {
        gp, inheritTime = runnext(pp) // atomic load of pp.runnext
    }
    // 2) local runq
    if gp == nil {
        gp, inheritTime = runqget(pp) // pop from pp.runq[head]
    }
    // 3) global runq (1/61 chance or local empty)
    if gp == nil {
        gp, inheritTime = globrunqget(pp, 0)
    }
    // 4) netpoller
    if gp == nil {
        gp, inheritTime = netpoll(false) // non-blocking poll
        if gp != nil { /* adjust steal stats */ }
    }
    // 5) work stealing – try other Ps
    if gp == nil {
        gp, inheritTime = stealWork(pp)
    }
    // 6) blocking netpoll + stopm
    if gp == nil {
        // before parking M, check timers, GC, etc.
        gp, inheritTime = netpoll(true) // blocking poll (park M)
        if gp == nil {
            stopm() // put M to sleep; will be woken by notewakeup
        }
    }
    execute(gp, inheritTime)
}
```

Priorities matter: `runnext` first (wake-up latency), then local queue (cache-hot), then occasional global peek (fairness), then netpoller (I/O),

then stealing. The global queue is sampled $\sim 1/61$ iterations to avoid starvation without taking `sched.lock` each cycle.

`newproc ( go f() )` pushes to the local queue; when `pp.runq` is full (256 entries) it moves half to the global queue under `sched.lock`:

```
func runqput(pp *p, gp *g, next bool) {
    if next {
        // runnext path
        pp.runnext = gp
        return
    }
    if pp.runqtail - pp.runqhead < len(pp.runq) {
        pp.runq[pp.runqtail % len(pp.runq)] = gp
        pp.runqtail++
        return
    }
    // full: link half to global
    runqputslow(pp, gp, pp.runqhead, pp.runqtail)
}
```

3.2 Work stealing

When `findRunnable` finds nothing local, it iterates over other `P`s in random order (starting from `schedtick % gomaxprocs`) and steals half:

```
func stealWork(pp *p) (*g, bool) {
    for i := 0; i < gomaxprocs; i++ {
        p2 := allp[(schedtick+uint32(i))%gomaxprocs]
        if p2 == pp || p2.status != _Prunning { continue }
        if t := runqsteal(pp, p2, true); t != nil {
            return t, false
        }
    }
    // also try global queue and netpoll again
    return nil, false
}

func runqsteal(dst, src *p, stealRunNextG bool) *g {
    n := src.runqsize() / 2
    // CAS steal: copy n Gs from src.runq[head..head+n) to
    dst.runq[tail..)
    // must synchronize with concurrent runqput/runqget on src
}
```

Stealing half balances load quickly and keeps locality: a `P` that just produced many goroutines (e.g., a fan-out loop) sheds half to an idle peer

without global lock contention. In production, stealing accounts for a large share of `findRunnable` successes under uneven load.

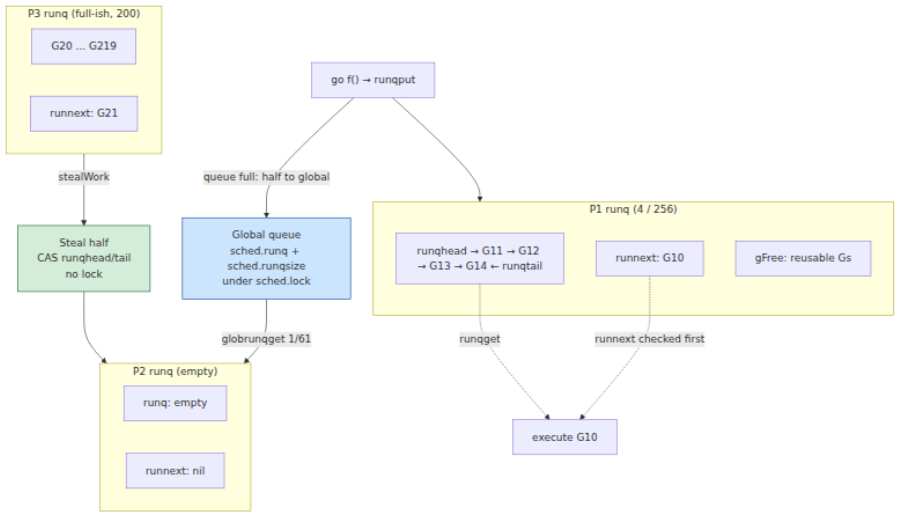


Diagram 2 — Per-P local queues + global queue + work stealing. New goroutines fill the local queue; overflow goes global; idle Ps steal half from busy peers.

3.3 `sysmon` and the scheduler tick — who watches the watchers

`sysmon` is a dedicated `M` (no `P` needed) that wakes every 20 μ s–10 ms and does housekeeping that cannot be done inline:

```
// runtime/proc.go - sysmon loop (simplified)
func sysmon() {
    for {
        now := nanotime()
        // 1) retake Ps that have been in syscall too long
        retake(now)
        // 2) preempt long-running Gs (asyncPreempt)
        preemptAll(now)
        // 3) poll netpoll when idle
        if netpollAnyWaiters() && lastpoll == 0 {
            netpoll(false) // non-blocking
        }
        // 4) scavenge heap, check timers, GC pacing
        // 5) check deadlock (all Ps idle, no timers/netpoll)
        usleep(delay) // 20µs when busy, up to 10ms when idle
    }
}

func retake(now int64) uint32 {
    for _, pp := range allp {
        if pp.status == _Psyscall {
            // P has been in syscall longer than sysmon tick → hand off
            if now - pp.syscallWhen > 10_000_000 /* 10ms */ {
                // CAS _Psyscall → _Pidle, then _Pidle → _Prunning on
                an idle M
                handoffp(pp)
            }
        } else if pp.status == _Prunning {
            // long-running G preemption check
            if pp.preempt && now - pp.preemptWhen > 10_000_000 {
                preemptone(pp)
            }
        }
    }
}
}
```

Key interactions:

- **Syscall retake.** If a `G` calls `entersyscall` and blocks (file I/O, `CGO`), its `P` goes `_Psyscall`. After ~10 ms (or immediately if another `M` needs a `P`), `sysmon` reclaims the `P` (`handoffp`) and schedules other `G`s on it. The original `M` remains blocked; on `exitsyscall` it must acquire a new `P` (stealing or from `Pidle`).
- **Preemption.** `sysmon` marks `P.preempt` for any `G` running >10 ms without a safepoint. The actual preemption signal is sent by `preemptone` (Section 5).

- **Netpoll when idle.** If all `P`s are idle but netpoller has waiters, `sysmon` polls to avoid starvation where no `M` is in `findRunnable`.
- **GC and scavenging.** `sysmon` also triggers GC pacing (`gcTrigger`), heap scavenging (`scavenge`), and timer checking.

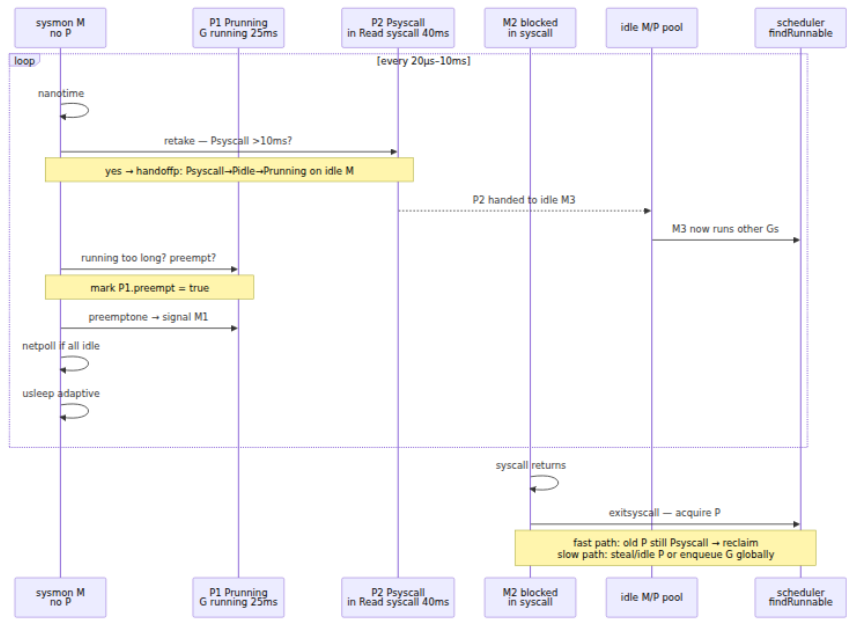


Diagram 6 — `sysmon` tick interaction with `Ps` and `Ms`. `sysmon` reclaims `Ps` from blocked `syscalls`, requests preemption of long-running `Gs`, and polls `netpoll` when the system is idle. Not on the scheduler hot path — a background monitor.

4. Blocking: `syscalls`, `P-handoff`, and `LockOSThread`

4.1 `entersyscall` / `exitsyscall` — the handoff protocol

Any path that may block the OS thread must bracket with `entersyscall` / `exitsyscall`. `Cgo`, `syscall.RawSyscall`, and file I/O all do this.

```

// runtime/proc.go (conceptual)
func entersyscall() {
    _g_ := getg()
    pp := _g_.m.p.ptr()
    pp.status = _Psyscall
    pp.syscallWhen = nanotime()
    SavePCSP(_g_) // save G's SP/PC into gobuf so it can be parked
    _g_.atomicStatus.Store(_Gsyscall)
    _g_.m.olddp.set(pp) // remember P for fast-path reclaim
    _g_.m.p = 0 // detach P – now another M can acquire it
    _g_.stackguard0 = stackPreempt // next function call will re-enter
runtime
}

func exitsyscall() {
    _g_ := getg()
    _g_.atomicStatus.Store(_Gwaiting) // transient while acquiring P
    // fast path: old P still _Psyscall → CAS back to _Prunning
    oldp := _g_.m.olddp.ptr()
    if oldp != nil && oldp.status == _Psyscall && cas(&oldp.status, _Psyscall, _Prunning) {
        _g_.m.p.set(oldp)
        _g_.m.olddp = 0
        _g_.atomicStatus.Store(_Grunning)
        return
    }
    // slow path: old P was stolen → try idle P → try to create new P
    // if none, put G on global queue and stopm()
    exitsyscall0(_g_)
}

```

Behavior:

- During the syscall, `G.status = _Gsyscall`, `M` is blocked in kernel, `P` is `_Psyscall` (until `sysmon` retakes it).
- If the syscall returns quickly (<10 ms and before `sysmon` retakes), fast-path reclaim succeeds with a single CAS — no queue hop, p99-friendly.
- If `sysmon` already handed the `P` away, `exitsyscall` parks the `G` globally and the `M` goes idle or is reused.

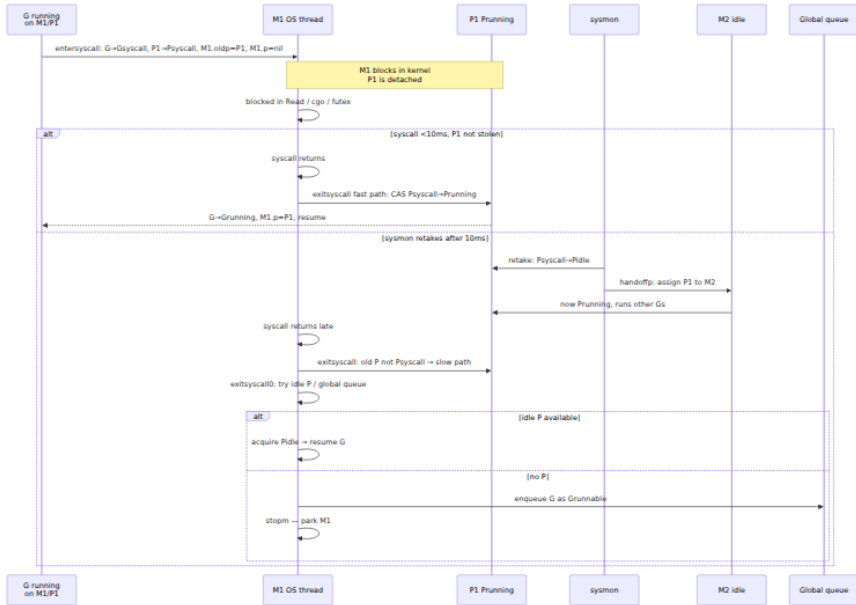


Diagram 3 — Syscall park/handoff. Fast syscalls reclaim their P with a single CAS; long syscalls lose the P to sysmon and re-enter via the global queue.

4.2 runtime.LockOSThread / UnlockOSThread

```

func LockOSThread() {
    // runtime/proc.go: wire current G to current M
    _g_ := getg()
    _g_.lockedm.set(_g_.m)
    _g_.m.lockedg.set(_g_)
    _g_.m.lockedExt++ // nesting count for external lock
}

func UnlockOSThread() {
    _g_ := getg()
    _g_.lockedm = 0
    _g_.m.lockedg = 0
    // may need to hand off P if G is still locked elsewhere
}
  
```

Consequences when a G is locked:

- The scheduler never migrates that G to another M ; M never runs a different locked G .

- `M` is excluded from work-stealing as a victim for locked `G`s; `P` handoff logic respects `lockedg`.
- Use cases are narrow but non-negotiable: `cgo` code that relies on thread-local storage, `syscall` wrappers around `setns(2)` / `unshare(2)` / `sched_setaffinity(2)`, graphics (`GL`), or any C library that stores state in `pthread_setspecific`.

```
// LockOSThread for a thread-affine syscall – must not migrate.
func withNetNS(nshandle int, fn func()) {
    runtime.LockOSThread()
    defer runtime.UnlockOSThread()
    orig, _ := syscall.Open("/proc/self/ns/net", syscall.O_RDONLY, 0)
    _ = syscall.Setns(nshandle, syscall.CLONE_NEWNET)
    fn()
    _ = syscall.Setns(orig, syscall.CLONE_NEWNET)
    syscall.Close(orig)
}

// cgo: the runtime automatically LockOSThread's the G while in C
// (see runtime/cgocall.go:entercgocall). No manual lock needed,
// but C-thread affinity persists for the duration.
```

Operational pitfall: a goroutine that calls `LockOSThread` and then blocks on channel/netpoll pins an entire OS thread doing nothing. Never `LockOSThread` in a handler goroutine — isolate it to a dedicated `G` that communicates via channels:

```
// Anti-pattern: pins M per request
// func handler(w http.ResponseWriter, r *http.Request) {
//     runtime.LockOSThread(); defer runtime.UnlockOSThread()
//     // ... netpoll block holds M hostage ...
// }

// Correct: single locked goroutine as executor
func init() {
    go func() {
        runtime.LockOSThread()
        for fn := range affineWorkCh {
            fn() // runs on the same M every time
        }
    }()
}
```


5. Preemption: from cooperative to `asyncPreempt`

5.1 History and the problem

Until Go 1.13, preemption was *cooperative*: the compiler inserted `morestack` checks at function prologues; the scheduler set `stackguard0 = stackPreempt` to force the next call to trap into `preempt`. A tight loop with no calls never trapped:

```
// Before Go 1.14 – starved GC and other Gs for milliseconds
func spin() { for { x++ } } // no call → no safepoint → P never yields
```

GC STW phases waited for this `P` to reach a safepoint, inflating pause tails. `GODEBUG=schedtrace` showed a `P` stuck `Prunning` with a growing `runqueue` on other `P`s.

5.2 Async preemption (Go 1.14+, `SIGURG`)

The fix uses a per-`M` signal:

```
// runtime/preempt.go – concept
func preemptM(mp *m) {
    // sysmon → preemptone → signalM
    signalM(mp, sigPreempt) // SIGURG on linux
}

// signal handler – runs on gsignal stack
func asyncPreempt2() {
    _g_ := getg()
    // insert async preempt at current PC
    _g_.preempt = true
    _g_.stackguard0 = stackPreempt
    // on return from signal, execution resumes at asyncPreempt
    trampoline
    // which calls gopreempt_m → schedule
}
```

The compiler now emits preemption points at loop back-edges and function entries. When `SIGURG` arrives:

1. Kernel delivers signal to the target `M`'s thread; handler runs on `gsignal`'s stack (not `g0` or user stack).
2. Handler sets `g.preempt = true` and `stackguard0 = stackPreempt`, and rewrites the signal context's `PC` to `runtime.asyncPreempt_trampoline`.

3. Upon `sigreturn`, execution lands in `asyncPreempt`, which saves the goroutine state and calls `gopreempt_m` → `gosched_m` → `schedule` on `g0`. The preempted `G` is re-queued as `_Gpreempted` → `_Grunnable`.

Un-preemptible regions (`m.preemptoff != ""`, `g.preemptStop`, GC critical sections, `cgo`, raw assembly without `FUNCDATA`) defer the signal: the handler returns without rewriting `PC`, and the flag is checked at the next safepoint.

Tuning and observability:

```
# Disable async preemption (diagnostic only – not for production)
GODEBUG=asyncpreemptoff=1 ./app

# Trace preemption in execution trace (shows "Preempt" events per G)
go test -trace trace.out -run TestContention
go tool trace trace.out
```

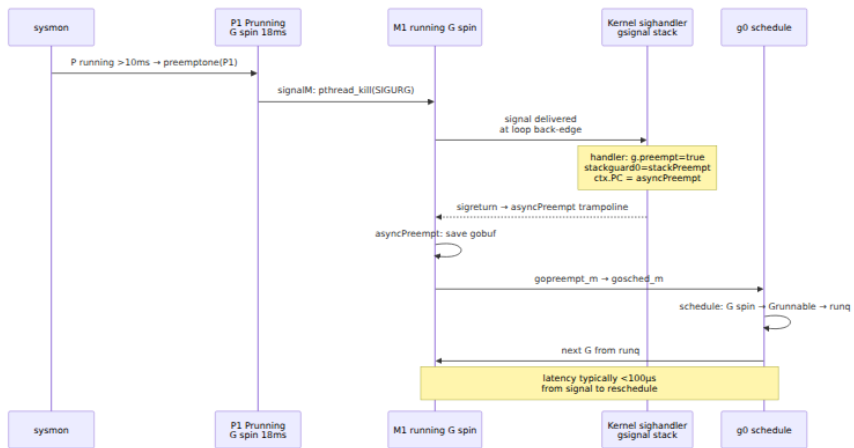


Diagram 5 — Async preemption signal flow. `sysmon` detects a long-running `G`, signals its `M` with `SIGURG`, the handler redirects execution to the preempt trampoline, and `g0` re-queues the `G`.

6. The netpoller — `epoll / kqueue / IOCP` behind `gopark`

6.1 Architecture

The netpoller is the runtime's Reactor. It translates blocking `Read / Write / Dial` into non-blocking fd operations + `park/unpark`, so a single `M` can drive thousands of connections.

Per-OS backend (build-tagged):

OS	File	Mechanism
Linux	<code>runtime/netpoll_epoll.go</code>	<code>epoll_create1</code> + <code>epoll_wait</code> (+ <code>eventfd</code> for wakeup)
macOS / BSD	<code>runtime/netpoll_kqueue.go</code>	<code>kqueue / kevent</code>
Windows	<code>runtime/netpoll_windows.go</code>	IOCP (<code>CreateIoCompletionPort</code> , <code>GetQueuedCompletionStatus</code>)
Fallback (<code>GOOS=js</code> , etc.)	<code>runtime/netpoll_stub.go</code>	No poller; every I/O blocks <code>M</code>

Initialization at startup (`runtime.schedinit` → `netpollinit`):

```
func netpollinit() {
    epfd = epoll_create1(_EPOLL_CLOEXEC)
    r, w := pipe2()
    netpollBreakRd = r; netpollBreakWr = w
    // break pipe: netpollBreak() writes a byte to wake epoll_wait
    ev := epoll_event{events: _EPOLLIN}
    epollctl(epfd, _EPOLL_CTL_ADD, r, &ev)
}
```

Each fd gets a `pollDesc` , allocated from `internal/poll.FD` :

```
// runtime/netpoll.go - pollDesc (abridged)
type pollDesc struct {
    link      *pollDesc
    fd        uintptr
    atomicInfo uint32    // pollAlreadyClosed / pollInitd
    rg, wg    uintptr   // G waiting to read / write (plus pdReady/
pdWait bits)
    lock      mutex
    everr     bool
    rseq, wseq uintptr   // race detector sequences
    rgold, wgold uintptr
    deadline  int64
}
```

6.2 gopark / goready — the park/unpark primitives

`gopark` is the single blocking primitive; channels, timers, netpoll, and `sync` all use it:

```
// runtime/proc.go
func gopark(unlockf func(*g, unsafe.Pointer) bool, lock unsafe.Pointer,
    reason waitReason, traceEv byte, traceskip int) {
    mp := acquirem() // get M, disable preemption
    gp := mp.curg
    // atomically: G Running → Waiting, record waitReason
    casGFromRunning(gp, _Gwaiting)
    gp.waitReason = reason
    // save SP/PC into gp.sched (re-entry point after park)
    gp.sched.pc = getcallerpc()
    gp.sched.sp = getcallersp()
    // call unlockf to release associated lock (e.g., pd.rg lock)
    if unlockf != nil && !unlockf(gp, lock) { /* already woken -
goready won */ }
    // switch to g0 and schedule next G
    mcall(park_m)
}
func goready(gp *g, traceskip int) {
    // G Waiting/Preempted → Runnable, put on runq of current or target
P
    casgstatus(gp, _Gwaiting, _Grunnable)
    runqput(currentP(), gp, false) // or wakep() if P idle
    if atomic.Load(&sched.nmspinning) == 0 { wakep() }
}
```

Netpoller-specific wrappers:

```

func poll_runtime_pollWait(pd *pollDesc, mode int) int {
    // called from internal/poll when Read would block
    err := netpollcheckerr(pd, mode)
    if err != 0 { return err }
    // park until fd ready or deadline
    gopark(netpollblockcommit, unsafe.Pointer(gpp), waitReasonIOWait, t
raceEvGoBlockNet, 5)
    return netpollcheckerr(pd, mode)
}
func netpollblock(pd *pollDesc, mode int32, waitio bool) bool { /* ...
gopark ... */ }
func netpollReady(pd *pollDesc, mode int32) { /* ... goready ... */ }

```

6.3 End-to-end walkthrough: `net.Conn.Read` blocks and wakes

Consider `conn.Read` on a TCP connection with no data available:

```

1) netFD.Read → internal/poll.FD.Read → pollDesc.Wait(mode='r')
2) poll_runtime_pollWait(pd, 'r')
   err = netpollcheckerr(pd, 'r') → 0 (no error, not ready)
   netpollblock(pd, 'r', true)
   lock(pd)
   if pd.rg already set → throw ("simultaneous Read")
   pd.rg = gp // record waiting G
   unlock(pd)
   gopark(netpollblockcommit, &pd.rg, waitReasonIOWait)
   → cas g status Running→Waiting, save SP/PC
   → unlock commit (no-op if still parked)
   → mcall(park_m) → schedule next G on this P/M

```

At this point: G is `_Gwaiting "IO wait"`, no M, no P. M/P `run` other Gs.

```

3) Meanwhile: epoll thread (any M in netpoll()) → epoll_wait(20ms or
blocking)
   n = epoll_wait(epfd, events, 128, waitms)
   for each ev in events[0:n]:
     pd = *(pollDesc**)ev.data
     mode = epollMode(ev.events) // 'r' and/or 'w'
     netpollready(pd, mode) // marks pd.rg/pd.wg ready
     rg := pd.rg; pd.rg = pdReady
     goready(rg, 0) → runqput(P, rg) // G is now _Grunnable

```

```

4) Next findRunnable on any P:
   gp = netpoll(false) → returns rg (already goready'd; netpoll just
drains)
   // or: gp was already runqput → schedule picks it via runnext/runq
   execute(gp) → gogo restores SP/PC → returns from gopark
   poll_runtime_pollWait re-checks netpollcheckerr → success
   → retry read via syscall.Read (now non-blocking, data present)

5) If peer closed / error: epoll reports EPOLLHUP/EPOLLERR → netpoll
sets pd.everr
   and goready still wakes G, which sees err on next check.

```

The `epoll` edge list (`pd` per `fd`) persists across connections; only `gopark/goready` transitions the G. The `netpollBreak` pipe (`eventfd`) wakes `epoll_wait` when a new G parks with no M currently polling — preventing indefinite sleep when a write makes a reader ready while every M is busy.

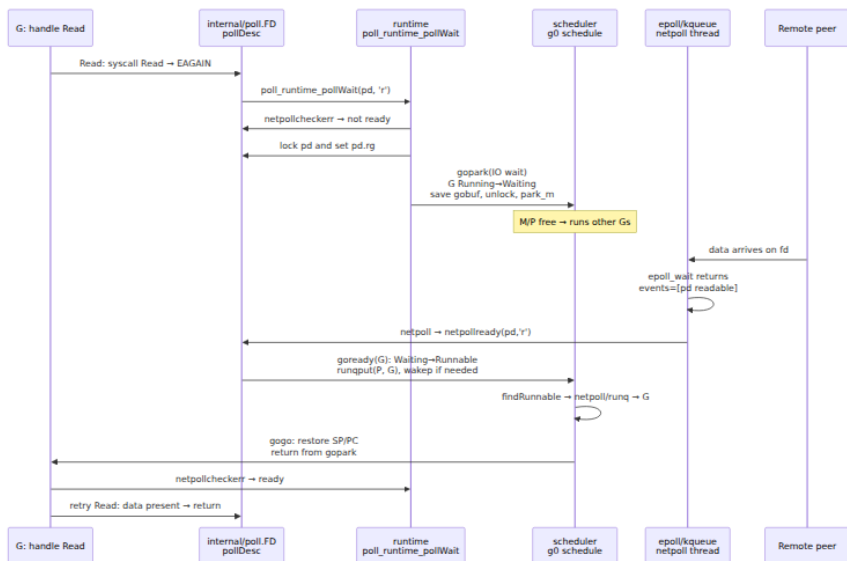


Diagram 4 — Netpoller epoll + gopark/goready. The G parks with no M/P; epoll_wait observes readiness and re-queues the G without ever blocking an OS thread.

Sync vs. async netpoller modes. `netpoll(block=true)` is the blocking `epoll_wait` path used when `findRunnable` has nothing else to do (it parks the M). `netpoll(block=false)` is the non-blocking poll used each scheduler iteration to drain ready events. On Linux the blocking wait timeout adapts to the next timer deadline so timers and netpoll share the same wakeup.

Windows deviation. On Windows the netpoller uses IOCP: `WSARecv` with `OVERLAPPED` posts a completion; `GetQueuedCompletionStatus` in `netpoll` dequeues it and wakes the G. There is no `epoll fd`; `pollDesc` holds an `OVERLAPPED` entry.

7. Observability: seeing what the scheduler does

7.1 `GODEBUG=schedtrace` / `godebug` and `scheddetail`

The oldest scheduler lens — a periodic line to `stderr`. In Go 1.21+ `GODEBUG` moved to `godebug` pragmas but `schedtrace` remains via `GODEBUG`:

```
# Every 1000ms, with per-P detail
GODEBUG=schedtrace=1000,scheddetail=1 ./server 2>&1 | head

# Go 1.21+ also supports GODEBUG as fallback; new code uses GODEBUG env
# still honored.
# In-code default (overridden by env):
# //go:debug schedtrace=1000
```

Sample output (trimmed, `GOMAXPROCS=4`):

```
SCHED 1007ms: gomaxprocs=4 idleprocs=1 threads=7 spinningthreads=0
idlethreads=2 runqueue=0 [0 3 1 0]
SCHED 2012ms: gomaxprocs=4 idleprocs=0 threads=9 spinningthreads=1
idlethreads=3 runqueue=2 [5 4 6 2]
SCHED 3015ms: gomaxprocs=4 idleprocs=2 threads=7 spinningthreads=0
idlethreads=2 runqueue=0 [0 0 1 0]
```

Fields:

Field	Meaning
<code>gomaxprocs=N</code>	current <code>GOMAXPROCS</code>
<code>idleprocs=M</code>	<code>P</code> s in <code>_Pidle</code>
<code>threads=N</code>	<code>M</code> s alive (includes <code>g0</code> and signal <code>M</code>)
<code>spinningthreads=N</code>	<code>M</code> s in <code>findRunnable</code> spin loop (no <code>G</code> yet, burning CPU looking)
<code>idlethreads=N</code>	<code>M</code> s parked in <code>stopm</code> / <code>notewakeup</code>
<code>runqueue=N</code>	global queue length (<code>sched.runqsize</code>)
<code>[a b c d]</code>	per- <code>P</code> local queue lengths (<code>p.runqsize</code>)

How to read it:

- `runqueue>0` persistently → global contention (many `runqputslow` overflows); often too many short-lived goroutines or `GOMAXPROCS` too low for bursty fan-out.
- `spinningthreads` bouncing `0↔1` under load is healthy; stuck `>0` with `runqueue=0` and per-`P` queues mostly `0` → netpoller or timer starvation (nothing to do).

- `threads >> gomaxprocs + ~3` → syscall storm; many `M`s blocked in `entersyscall`. Correlate with `strace -c` or pprof syscall samples.
- `idleprocs==gomaxprocs` yet `threads` high and `runqueue>0` → blocked `M` leak (e.g., `LockOSThread` + `block`, or `cgo`).

Go 1.21 `godebug` variant (non-scheduler but relevant for GC pacing that contends with scheduler):

```
GODEBUG=gctrace=1,asyncpreemptoff=1 ./server 2>&1 | head
# gc 1 @0.010s 2%: 0.015+0.42+0.003 ms clock, 0.12+0.08/0.50/0.12+0.024
ms cpu, 4->4->1 MB, 5 MB goal, 8 P
```

7.2 go tool trace — the definitive scheduler view

`schedtrace` is a counter; `trace` is a timeline.

Capture:

```
import "runtime/trace"

f, _ := os.Create("/tmp/trace.out")
trace.Start(f)
defer func(){ trace.Stop(); f.Close() }()

// ... serve load for 5s ...
```

```
# Also via net/http/pprof
curl -o trace.out http://localhost:6060/debug/pprof/trace?seconds=5
go tool trace trace.out           # opens browser UI
# Headless / CI: gotraceui or trace parser
go install github.com/gotraceui/gotraceui/cmd/gotraceui@latest
gotraceui trace.out
```

What to look for:

- **Heap / STW / GC lanes** — correlate scheduler stalls with GC assists.
- **Per-P lanes** — each `P`'s `G` executions as colored blocks; gaps are idle time (no runnable `G`); dense hatching is many short `G`s (overhead).
- **Goroutine lanes** — per-`G` state transitions: `Running` → `Waiting` (IO wait) → `Runnable` → `Running`.

- **Network / Timer lane** — netpoller wakeups as edges from netpoll blocks.

Minimal `go tool trace` CLI dump (when browser not available):

```
go tool trace -pprof=net trace.out > net.pprof # extract network
profile
go tool trace -help 2>&1 | head -20
```

7.3 `runtime.Stack` and goroutine dumps

`SIGQUIT` (`kill -QUIT <pid>`) and `runtime.Stack` produce the same dump. In Kubernetes, `kubectl exec <pod> -- kill -QUIT 1` prints to stderr (captured by `kubectl logs`).

```
// Full dump to log (bounded buffer)
buf := make([]byte, 1<<20) // 1 MB; grow if truncated
n := runtime.Stack(buf, true) // true = all goroutines
log.Printf("goroutine dump:\n%s", buf[:n])

// Lightweight caller ID in logs
var b [64]byte
n = runtime.Stack(b[:], false) // false = caller only
// b[:n] == "goroutine 123 [running]:\nmyapp.handleRequest(...)\n\tt..."
```

Real dump (trimmed):

```

goroutine 4821 [IO wait, 12 minutes]:
netpollblock
    runtime/netpoll.go:583
internal/poll.runtime_pollWait
    runtime/netpoll.go:305
internal/poll.(*pollDesc).wait
    internal/poll/fd_poll_runtime.go:84
net.(*netFD).Read
    net/fd_posix.go:55
net.(*conn).Read
    net/net.go:183
myapp.handleConn
    myapp/server.go:42 +0x85

goroutine 7 [chan receive, 2 minutes]:
runtime.gopark
    runtime/proc.go:424
runtime.chanrecv
    runtime/chan.go:583
myapp.workerLoop
    myapp/worker.go:19 +0x45
created by myapp.startWorkers in goroutine 1
    myapp/worker.go:11 +0x78

goroutine 19 [syscall, 5 minutes]:
runtime.entersyscall
    runtime/proc.go:1234
syscall.Syscall
    syscall/asm_linux_amd64.s:27
os.(*File).Read
    os/file.go:123
myapp.readRegularFile // file I/O blocks M expected _Gsyscall
    myapp/file.go:31

goroutine 1 [running]:
main.main
    myapp/main.go:28 +0x1a4

```

Reading it:

- [IO wait] → parked on netpoller; netpollblock frame. High count of long IO wait is healthy for idle keep-alives — not a leak.
- [chan receive] / [select] → parked via gopark on channel/timer; look at creating goroutine to identify owner.
- [syscall] → _Gsyscall, has an M pinned; many of these explains high threads in schedtrace.

- [running] → currently on a `P`; if many stay [running] for minutes, async preemption may be off or `cgo` is holding `P`.

7.4 pprof goroutine profile

```
# Via http
go tool pprof http://localhost:6060/debug/pprof/goroutine
# Or captured heap-style:
curl -s http://localhost:6060/debug/pprof/goroutine?debug=1 | head -n 2
00
curl -s http://localhost:6060/debug/pprof/goroutine?debug=2 | head -n 3
00 # same as Stack, grouped

# Block/mutex profiles – scheduler contention
go tool pprof http://localhost:6060/debug/pprof/block
go tool pprof http://localhost:6060/debug/pprof/mutex
```

Inside `go tool pprof`:

```
(pprof) top -cum
Showing nodes accounting for 482193 goroutines
      flat flat% sum%      cum cum%
  481900 99.94% 99.94%   481900 99.94% runtime.gopark
  481900 99.94% 99.94%   481900 99.94% runtime.chansend /
runtime.chanrecv
      5 0.001% 100%      5 0.001% netpollblock / internal/pol
l.runtime_pollWait

(pprof) traces
File: myapp
Type: goroutine
Time: 2026-03-18 14:02 UTC
-----
  320 @ 0x43a1b2 0x43a0f0 0x4bc123
# 0x4bc123 myapp.workerLoop+0x43 myapp/worker.go:19
# runtime.gopark+0x9a runtime/proc.go:424
# runtime.chanrecv+0x12 runtime/chan.go:583
```

Enable block/mutex sampling in `main`:

```
import _ "net/http/pprof"

func init() {
    runtime.SetBlockProfileRate(1000000) // sample every 1ms of block
time (0 disables)
    runtime.SetMutexProfileFraction(1000) // sample 1/1000 mutex
contentions
    // goroutine profile is always available; no rate knob
}
```

- **Goroutine profile** tells you *how many* `G`s exist and *where* they park — the leak detector.
- **Block profile** tells you *how long* `G`s spent parked (scheduler latency, channel wait, `WaitGroup`, `netpoll`).
- **Mutex profile** tells you *where* lock contention serializes `G`s (often the real p99 culprit, not CPU).

7.5 GODEBUG / `godebug` (Go 1.21+)

Go 1.21 introduced `//go:debug` directives and `GODEBUG` defaults compiled into the binary (`go env GODEBUG`, `runtime.SetDefaultGODEBUG`). Scheduler-relevant keys:

```
go env GODEBUG
# asynctimerchan=1, gotraceback=single, http2client=1, ... (defaults)

# Override at run:
GODEBUG=http2debug=2,schedtrace=2000 ./app

# In code – pin a default that env can still override:
# //go:debug asynctimerchan=1
import _ "embed"
```

For scheduler work, `schedtrace` and `asyncpreemptoff` remain the actionable knobs; `asynctimerchan` and `http2*` affect `netpoller`-adjacent behavior (timer channel async delivery, HTTP/2 flow control).

8. Tuning for production: `GOMAXPROCS`, containers, and contention

8.1 What `GOMAXPROCS` actually limits

```
n := runtime.GOMAXPROCS(0) // read current
runtime.GOMAXPROCS(8)      // set; returns old
```

- It sets `len(allp)` — the number of `P`s and thus the parallelism of *user Go code*. Network I/O, timers, GC assists, and blocked syscalls do not count against it.
- It also sizes `P`-local caches (`mcache`), timer heaps, and GC worker count (25% of `GOMAXPROCS` by default).
- It does **not** cap `M`s. `M`s grow on demand (up to `10000` by default via `runtime.sched.maxmcount`, tunable with `debug.SetMaxThreads`). A burst of file I/O or `cgo` can create hundreds of `M`s even with `GOMAXPROCS=2` — visible as `threads` in `schedtrace`.

Changing `GOMAXPROCS` at runtime is safe (`stopTheWorld` + `resize allp`), but most services set it once at startup. Dynamic adjustment based on load is rarely worthwhile; the allocator and GC have already sized themselves.

8.2 Containers, cgroups, and why host CPUs is wrong

By default `GOMAXPROCS` equals `runtime.NumCPU()` — the host's CPU count. Inside a Kubernetes pod with `resources.limits.cpu: "2"`, the host may have 96 cores, so `GOMAXPROCS=96` while the cgroup quota allows 2 cores of time (`cfs_quota_us / cfs_period_us = 2` or `cpu.max = 200000 100000` on cgroup v2).

Consequences of `GOMAXPROCS >> quota`:

- 96 `P`s contend for 2 cores of CFS bandwidth → excessive `schedule / findRunnable` spinning, more preemptions, higher context-switch rate, and GC workers (24 threads!) stealing from request handlers.
- p99 rises 20–50% on CPU-bound handlers with no extra throughput.

Consequences of `GOMAXPROCS <= quota` (e.g., hardcoded `2` on an 8-core limit after a vertical scale):

- Under-utilization; latency improves little beyond `quota/2` but throughput caps at `GOMAXPROCS` cores.

The fix — `automaxprocs`:

```
// go.mod: require go.uber.org/automaxprocs v1.5.3
import _ "go.uber.org/automaxprocs" // init() reads /sys/fs/cgroup,
sets GOMAXPROCS

// With explicit control and logging:
import (
    "go.uber.org/automaxprocs/maxprocs"
    "log"
)
func main() {
    undo, err := maxprocs.Set(maxprocs.Logger(log.Printf))
    defer undo()
    if err != nil { log.Printf("automaxprocs: %v", err) }
    log.Printf("GOMAXPROCS=%d", runtime.GOMAXPROCS(0))
}
```

`automaxprocs` reads:

- cgroup v1: `/sys/fs/cgroup/cpu/cpu.cfs_quota_us` and `cpu.cfs_period_us`
- cgroup v2: `/sys/fs/cgroup/cpu.max` (`"max"` means no limit) and `cpu.weight`
- Kubernetes `cpu` limit → CFS quota; `cpu` request does not appear in these files (it affects CFS shares, not quota).

Observed behavior:

```
# Pod spec: limits.cpu=2, host 96 cores
# Without automaxprocs:
$ kubectl exec pod -- ./app -version 2>&1 | grep GOMAXPROCS
GOMAXPROCS=96

# With automaxprocs (cgroup v2):
$ kubectl logs pod
maxprocs: Updating GOMAXPROCS=2: determined from CPU quota
```

Alternatives:

```
// Manual – read cgroup v2 directly (minimal dep)
func cgroupMaxProcs() int {
    b, _ := os.ReadFile("/sys/fs/cgroup/cpu.max")
    // "max 100000" or "200000 100000"
    // parse first field; if "max" return 0 (no limit)
    // else quota/period rounded up
    return parseCPUmax(b)
}
func init() {
    if n := cgroupMaxProcs(); n > 0 {
        runtime.GOMAXPROCS(n)
    }
}
```

```
# Kubernetes – always set limits.cpu when latency matters
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "2"          # CFS quota = 2 cores → automaxprocs picks 2
    memory: "512Mi"
# If you run without limits (Burstable, no quota), automaxprocs keeps
# host count.
# In that case, set GOMAXPROCS via env:
env:
  - name: GOMAXPROCS
    value: "4"
# (GOMAXPROCS env var is not native – wire it in main:
runtime.GOMAXPROCS(envInt("GOMAXPROCS")))
```

Go issue context: automatic cgroup-aware `GOMAXPROCS` has been proposed ([golang/go#73193](#), [golang/go#65103](#)) but as of Go 1.23 the runtime still defaults to host CPUs. `automaxprocs` remains the production standard.

8.3 Diagnosing G contention and netpoller saturation

Symptoms and tools:

Symptom	schedtrace clue	trace / pprof clue	Likely cause
High p99, low CPU	runqueue and per-P queues persistently >0	trace shows long Runnable before Running (scheduler latency)	GOMAXPROCS too low for burst fan-out; or a few G s pin P s via no-preemption or LockOSThread
High CPU, no throughput	spinningthreads>0 with queue 0	trace shows many tiny G blocks, high schedule overhead	Too many short-lived G s (per-request goroutine + per-row goroutine); batch or pool
threads >> GOMAXPROCS	threads climbing	Stack shows many [syscall]	File I/O, cgo , or LockOSThread storm; rate-limit or move to async pool
Connections hang, CPU idle	idleprocs==GOMAXPROCS , threads stable, runqueue==0	Stack shows [IO wait] but netpoll lane idle; block profile on pollDesc.wait	Netpoller saturation: too many fds for one epoll instance? Rare — check ulimit -n , net.core.somaxconn , or a leaked SetReadDeadline
GC assists dominate	schedtrace shows throughput dips coinciding with gctrace	trace GC lane + long GC assist wait in Stack	Allocation rate too high; GOMEMLIMIT / GOGC tuning (Chapter 5), sync.Pool , reduce allocs
goroutine profile grows unbounded	runqueue grows	pprof -goroutine traces shows same waitReason accumulating	Goroutine leak — missing context cancel, unclosed channel, leaked timer

Concrete diagnosis session:

```
# 1) Is scheduler the bottleneck? Check scheduler latency via trace.
curl -s http://localhost:6060/debug/pprof/trace?seconds=5 -o trace.out
go tool trace trace.out
# In UI: View "Goroutine analysis" → sort by "Scheduling latency" (time
Runnable before Running).
# >1ms p99 scheduling latency under load → GOMAXPROCS or long-running G
suspected.

# 2) Who runs long? Find non-preemptible spans.
# In trace UI: search "GC" and "Preempt" events. A G Running for >20ms
without preempt marker
# suggests cgo, assembly, or GODEBUG=asyncpreemptoff=1.

# 3) How many Gs and where do they park?
curl -s http://localhost:6060/debug/pprof/goroutine?debug=1 | head -n 3
00
# Look for top waitReason counts.

# 4) Cross-check with live schedtrace (tail in prod for 60s)
GODEBUG=schedtrace=1000 ./app 2>&1 | ts | tee /tmp/sched.log
# Correlate spikes in runqueue with p99 spikes in metrics (Prometheus
histogram).

# 5) Container sanity
cat /sys/fs/cgroup/cpu.max 2>/dev/null || cat /sys/fs/cgroup/cpu/
cpu.cfs_quota_us
runtime.GOMAXPROCS(0) # via debug endpoint
# Mismatch → fix automaxprocs wiring.
```

Netpoller saturation is rarely the bottleneck on Linux — `epoll_wait` scales to millions of fds. When it appears, causes are:

- **ulimit -n / fs.file-max exhaustion** — `accept / netpollopen` fails with `EMFILE`; new connections get too many open files, not scheduler latency. Fix with `LimitNOFILE` in `systemd` / `ulimit` in container.
- **Timer thundering herd** — 100k timers with identical deadlines cause `netpoll` + timer heap to wake many `G`s at once, producing a burst of `Runnable`. Jitter deadlines (`Add(jitter)`).
- **Blocking file I/O mistaken for netpoll** — reading a regular file via `os.Read` does not use netpoller readiness; it does `entersyscall` and blocks `M`. Copying large files inside request handlers ties up `M`s; offload to a bounded worker pool or `io.Copy` with `afero/direct read(2)` on a separate `GOMAXPROCS`-isolated pool.

9. Backend lens — what to ship

A production checklist distilled from the internals above:

1. **Wire `automaxprocs` and verify.** Log `GOMAXPROCS` at startup; assert `runtime.GOMAXPROCS(0) == cgroup quota` in readiness probe or startup self-check. Never rely on `host.NumCPU()` inside a limited pod.
2. **Budget goroutines.** Treat goroutine count as a resource like file descriptors. Cap fan-out (`semaphore / errgroup` with `SetLimit`), enforce context cancellation on every spawned `G`, and alert when `goroutines > 50k * GOMAXPROCS` or when p99 scheduling latency (from `trace` or `runtime/metrics sched/latencies:seconds`) exceeds 500 μ s.
3. **Isolate thread-affine work.** `LockOSThread` and `cgo` callers belong in a dedicated pool (`GOMAXPROCS` small, no `netpoll` block in locked `G`s). Never `LockOSThread` in a request handler.
4. **Make blocking explicit.** File I/O, `exec`, and heavy `cgo` inside request paths should use a bounded pool (`worker.New(32)`) so `M` count stays predictable. Measure threads via `GODEBUG=schedtrace` or `runtime/metrics (sched/gomaxprocs:threads` in Go 1.23).
5. **Instrument the three profiles together.** `goroutine` (leak), `block` (scheduler latency), `mutex` (serialization) — sampled continuously via `runtime.Set*ProfileFraction` and scraped by `pyroscope/parca` or periodic `pprof` dumps. A mutex report that shows `sync.Mutex` at 40% of block time tells you more about p99 than CPU flame graphs.
6. **Use `trace`, not guesswork, for p99.** CPU profiles hide scheduler latency (time `Runnable` but not `Running`). A 5-second `trace` during a latency spike reveals whether the tail is scheduling, GC assists, `netpoller`, or lock contention — each with a different fix.

Key takeaways

- A goroutine costs kilobytes and nanoseconds; an OS thread costs megabytes and microseconds. The runtime multiplexes `G`s onto `M`s via `P`s so blocking network I/O parks only the `G` (via `gopark` on the `netpoller`) without blocking the `M/P`.

- `G` holds stack bounds, `atomicStatus` (`_Grunnable` / `_Grunning` / `_Gwaiting` / `_Gsyscall` / ...), `waitReason` , and a `gobuf` for save/restore; `M` holds `g0` (system stack), `curg` , and thread-local state; `P` holds the local run queue (`runq[256]`), `runnext` fast slot, `mcache` , and timer heap. Only a `G` holding a `P` may run user code.
- Scheduling prefers `runnext` → local `runq` → occasional global-queue peek → non-blocking `netpoll` → work stealing (steal half). Global overflow and stealing provide fairness without a central lock on the hot path.
- `sysmon` (no `P` , 20 μ s–10 ms cadence) reclaims `P` s from long syscalls (`retake` / `handoffp`), requests async preemption, polls `netpoll` when idle, and drives GC pacing — the background monitor that makes the foreground loop robust.
- Blocking syscalls bracket with `entersyscall` / `exitsyscall` : fast syscalls reclaim their `P` via single CAS; long syscalls lose it to `sysmon` and re-enter via `exitsyscall0` (idle `P` or global queue). `LockOSThread` pins `G`↔`M` for thread-affine syscalls and `cgo` ; misuse pins OS threads idle.
- Since Go 1.14, async preemption (`SIGURG` → `asyncPreempt` trampoline → `gopreempt_m`) preempts tight loops at back-edges; `cgo` and assembly without safe-points remain un-preemptible.
- The netpoller (`epoll` / `kqueue` / `IOCP`) keeps a persistent `pollDesc` per fd. `Read` with `EAGAIN` does `netpollblock` → `gopark`(IO wait) → `epoll_wait` → `netpollready` → `goready` → `runqput` , waking the `G` without an `M` ever blocking.
- Observe with `GODEBUG=schedtrace=1000,scheddetail=1` (queues/threads), `go tool trace` (timeline of runnable latency and netpoller edges), `runtime.Stack` (wait reasons), and `pprof` `goroutine` / `block` / `mutex` profiles. They answer different questions — size/where, how long, and serializing lock.
- In containers, `GOMAXPROCS` must track cgroup quota, not host CPUs. go.uber.org/automaxprocs (cgroup v1/v2) remains the standard until the runtime becomes cgroup-aware (`golang/go#73193`). Verify at startup and alert on `GOMAXPROCS` $\neq$ quota .
- Common p99 fixes follow from the model: raise `GOMAXPROCS` to quota (not above), cap goroutine fan-out, jitter timers, offload file/ `cgo` I/O to bounded pools, and isolate `LockOSThread` executors.

Further reading

1. **Go scheduler documentation** — `golang.org/s/go11sched` (Dmitry Vyukov's 2012 scheduler design) and `github.com/golang/go/wiki/DesignDocuments` — the authoritative M:N + work-stealing rationale. Pinned. <https://golang.org/s/go11sched>
2. **Go runtime source: `runtime/proc.go`** — `schedule`, `findRunnable`, `execute`, `entersyscall` / `exitsyscall`, `sysmon`, `retake`, `preemptone`. Pinned. <https://github.com/golang/go/blob/master/src/runtime/proc.go>
3. **Go runtime source: `runtime/runtime2.go`** — `g`, `m`, `p`, `schedt` struct definitions. Pinned. <https://github.com/golang/go/blob/master/src/runtime/runtime2.go>
4. **Go runtime source: `runtime/netpoll*.go`** — `pollDesc`, `netpollinit` / `netpollopen` / `netpoll` / `netpollBreak`, `poll_runtime_pollWait`, per-OS backends (`netpoll_epoll.go`, `netpoll_kqueue.go`, `netpoll_windows.go`). Pinned. <https://github.com/golang/go/blob/master/src/runtime/netpoll.go>
5. **Dmitry Vyukov — "Scalable Go Scheduler Design Doc" and Go scheduler articles** — work-stealing, `runnext`, and `netpoller` integration. Pinned (original 2012 doc + 2018 updates). https://docs.google.com/document/d/1TTj4T2JO42uD5ID9e89oa0sLKhJYD0Y_kqxDv3I3XMw
6. **Rhys Hiltner — "How Go's scheduler works"** (2016, with Morsing illustrations) — visual G-M-P walkthrough frequently cited alongside Vyukov. <https://www.ardanlabs.com/blog/2018/08/scheduling-in-go-part1.html> — Ardan Labs scheduler series (Bill Kennedy), parts 1–3 are the most accessible English walkthrough with traces.
7. **Go execution tracer** — `go.dev/trace`, `runtime/trace`, `go tool trace docs` — trace format, event types, and UI guide. <https://pkg.go.dev/runtime/trace> and <https://go.dev/doc/diagnostics#tracing>
8. **`go.uber.org/automaxprocs` — cgroup-aware GOMAXPROCS** — implementation of cgroup v1/v2 detection and `GOMAXPROCS` tuning, with Kubernetes examples. <https://github.com/uber-go/automaxprocs>
9. **Go issue `golang/go#73193` — automatic cgroup-aware GOMAXPROCS** — proposal and discussion for making the runtime container-aware without `automaxprocs`. <https://github.com/golang/go/issues/73193>

10. **Austin et al. — "The Go scheduler" (GopherCon 2018 talk, Kavya Joshi)** — preemption history from cooperative to SIGURG async preemption (Go 1.14). <https://www.youtube.com/watch?v=YHRO5WQGhLI> and companion notes at <https://github.com/golang/proposal/blob/master/design/24543-non-cooperative-preemption.md>

Chapter 5 — Memory Allocator, Stacks, and the Concurrent Tri-Color GC

What this chapter covers. Every allocation your handler performs — `make([]byte, n)`, `&Request{}`, `append`, `json.Marshal` — funnels through `runtime.mallocgc`. Every goroutine you spawn gets a stack that grows and shrinks. Every few milliseconds to seconds, the concurrent tri-color collector sweeps through the heap while your code keeps running. How well these three subsystems cooperate determines your p50, p99, OOM-kill rate, and cost per pod. This chapter is the allocator-to-GC vertical slice: size classes and spans, the `mcache` / `mcentral` / `mheap` hierarchy, the allocation fast and slow paths, stack growth and guard pages, the tri-color invariant and hybrid write barrier, STW phases, pacing via `GOGC` and `GOMEMLIMIT`, and the observability surfaces (`GODEBUG=gctrace`, `pprof`, `runtime/metrics`) you use to tune them for production.

Learning goals — after this chapter you should be able to:

- Describe Go's 67 size classes, span layout, and span classes; explain how `mcache` (per-P), `mcentral`, and `mheap` partition allocation and when each layer is consulted.
- Trace the `mallocgc` fast path, the `mcache` refill slow path, and the large-object path; state the small-object threshold (32 KiB), tiny allocator behavior, and the `noscan` optimization.
- Explain how `mheap` grows via `sysAlloc` / `mmap`, how scavenging returns pages via `MADV_DONTNEED` / `MADV_FREE`, and how `GOMEMLIMIT` changes scavenging pressure.
- Describe goroutine stacks since Go 1.4: contiguous stacks, initial size (8 KiB on 64-bit since Go 1.22; 2 KiB before), guard pages

(`stackguard0`), `morestack / stackCopy` , and how stack shrinking works.

- Explain tri-color mark-and-sweep, the two STW pauses, concurrent marking, and the hybrid write barrier — and why the barrier is required for correctness.
- Derive GC pacing: `GOGC` , `GOMEMLIMIT` , `next_gc` calculation, trigger ratio, and the pacer controller that sets assist ratios and background mark utilization.
- Read a `GODEBUG=gctrace=1` line field-by-field, interpret `pprof` heap profiles (`alloc_space` vs `inuse_space`), and run allocation benchmarks correctly.
- Tune GC for backend pods: pick `GOGC / GOMEMLIMIT` under cgroup limits, understand ballast, and reason about stack memory vs. goroutine count.

Placement. Chapter 2 established object layout and type-driven size/alignment. Chapter 3 covered stack frames and the ABI that stacks carry. This chapter covers where those objects and frames live: the heap allocator, goroutine stacks, and the collector that reclaims them. Chapter 6 covers the scheduler that multiplexes goroutines onto the stacks described here; Chapter 9 covers escape analysis that decides stack vs. heap placement; Volume 13, Chapter 4 gives a cross-runtime comparison of GC strategies (JVM, V8, .NET) that contextualizes Go's choices.

1. Why the allocator and GC dominate backend SLOs

Backend Go is allocation-heavy by construction. A single `net/http` request allocates for the `*http.Request` and `ResponseWriter` wrappers, header maps, `context.Context` chains, JSON or protobuf decode buffers, database row scans into `[]byte` or `string`, and whatever the handler does internally. Multiply by thousands of requests per second per pod and the allocator and GC are the hottest shared subsystem after the scheduler.

Three costs matter, and they trade off:

Cost	What it measures	Who pays it
Allocation throughput	<code>mallocgc</code> cycles per <code>new / make / append</code>	Every goroutine; contended on <code>mcentral / mheap</code>
GC CPU overhead	Background mark workers + STW + barriers	All Ps; <code>GOGC</code> sets the budget (default ~25% of CPU during mark)
GC latency (STW)	Pause to stop/start the world	Tail latency; every goroutine stopped simultaneously
Heap high-water	Live + garbage until next sweep	Pod memory limit; OOM-killer

Go 1.5–1.8 made GC concurrent and cut STW from hundreds of milliseconds to sub-millisecond. Go 1.12 added `GODEBUG=gctrace` scavenge stats. Go 1.19 introduced `GOMEMLIMIT` to bound heap under cgroup limits. Go 1.21 reworked the pacer. The trend is clear: throughput-optimized defaults, with knobs (`GOGC` , `GOMEMLIMIT`) to clamp memory at the cost of more CPU.

The backend lens for this chapter: you cannot tune what you cannot attribute. Every section ends with an operational question and the signal that answers it.

2. Allocator architecture: size classes, spans, and the three-level cache

2.1 Design goals

Go's allocator is a variant of TCMalloc (Google's Thread-Caching Malloc), adapted for a GC'd language with precise pointer maps and concurrent sweeping:

- **Low fragmentation** for long-running servers via size classes and span coalescing.
- **No per-allocation lock** on the fast path — sharded per-P caches (`mcache`).

- **GC integration** — every span knows which slots contain pointers (`allocBits` , `gcmarkBits`) so the collector can scan precisely; `noscan` spans skip scanning entirely.
- **Scavenging** — unused pages returned to the OS lazily, with hysteresis to avoid churn.

2.2 Size classes — 67 buckets

Any small allocation is rounded up to the next size class. Go has 67 size classes (0 is unused; 1..67 are real) ranging from 8 bytes to 32 KiB. Classes are defined in `runtime/sizeclasses.go` (generated by `make sizeclasses`) and indexed by `class_to_size` and `size_to_class` lookup tables.

class	size	objects per span (8 KiB pages)	waste at midpoint
1	8 B	1024 / span	~12%
2	16 B	512	~10%
3	24 B	341	...
...			
22	128 B	64	
35	1024 B	8	
50	8192 B	1	(one object per span for large classes)
67	32768 B	1	(max small object; >32 KiB is "large")

Rules:

- Objects `<= 32 KiB` → size-class allocation (binned). 32 KiB is `maxSmallSize` .
- Objects `> 32 KiB` → large-object allocation: one span sized to the request, page-aligned, not binned.
- **Tiny allocator:** `<= 16 bytes` and `noscan` (no pointers) may be sub-allocated inside a single 16-byte slot to coalesce 1-16 byte allocations (e.g., `struct{ x, y int8 }` , small strings, small escape temporaries). The per-P `mcache.tiny` block hands them out without consuming a full slot.
- **Zeroed vs dirty:** Spans track whether memory is already zeroed. Fresh OS pages are zeroed once; reused spans may need explicit zeroing for talkative types.

Why 67 and not 8 or 200? Diminishing returns: more classes reduce internal fragmentation but bloat `mcache` (one span per class per P) and

`mcentral` metadata. 67 keeps worst-case waste near 12% while keeping per-P memory $67 \times \sim 8 \text{ KiB} \approx 540 \text{ KiB}$ of held spans in steady state.

2.3 Spans — the unit of heap management

A **span** (`runtime/mspan.go:mspan`) is a run of one or more contiguous OS pages (8 KiB on most platforms; `pageSize = 8192`) that holds objects of one size class.

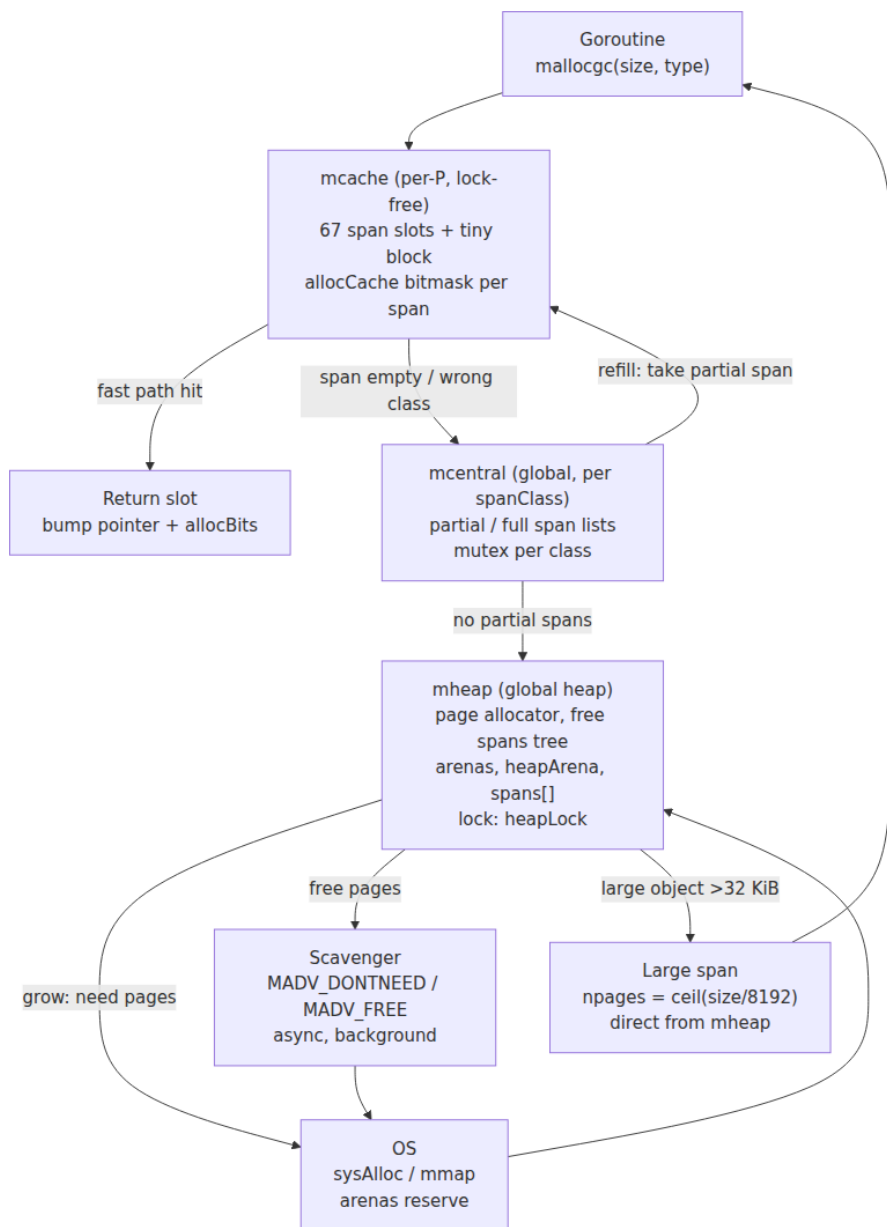
Key fields (simplified):

```
type mspan struct {
    next, prev *mspan
    startAddr uintptr // base address
    npages    uintptr // length in pages
    spanclass spanClass // (sizeclass <= 1) | noscan
    allocCount int // live objects
    allocBits *gcBits // which slots are allocated
    gcmrkBits *gcBits // marks found during GC
    sweepgen  uint32 // sweep generation
    needzero  uint8 // needszeroing
    state     mSpanState // InUse, Manual, Free
    limit     uintptr // end of usable bytes
}
```

A **span class** encodes size class *and* whether the class needs scanning. `spanClass = (sizeclass <= 1) | noscanBit`. So there are $67 \times 2 = 134$ logical span classes (including `noscan` variants). `noscan` spans hold objects with no pointers (e.g., `[]byte`, `[]int32` backing arrays, `string` bytes) and are skipped during marking — a significant throughput win for buffer-heavy services.

The heap tracks spans in `mheap.spans` (indexed by page number) and in `mheap.allSpans` / `mheap.sweepSpans` for GC iteration. Each page knows which span owns it via `mheap.spans[pageID]`.

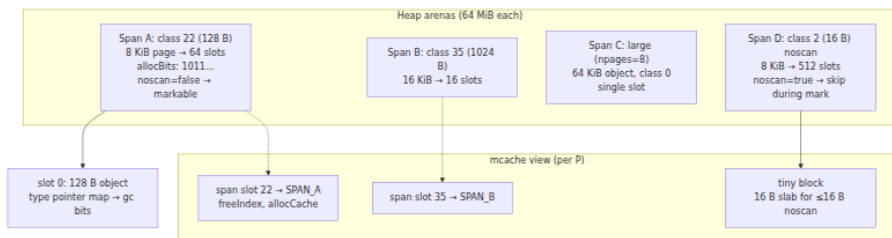
Diagram 1 — Allocator hierarchy (`mcache` → `mcentral` → `mheap` → OS)



Reading the diagram. The fast path never leaves `mcache`. The slow path escalates one level at a time: `mcache` asks `mcentral` for a new span of the right class; `mcentral` asks `mheap` for pages; `mheap` asks the OS for

arenas (64 MiB chunks on 64-bit). Returns flow the other way when sweeps free spans.

Diagram 2 — Span and size-class layout



2.4 The three layers in detail

mcache (per-P, runtime/mcache.go). A private allocation cache for each P (logical processor, GOMAXPROCS). No locks — safe because only the M bound to the P mutates it. Contains:

- `alloc [numSpanClasses]*mspan` — one active span per span class (~134 entries; $67 \times$ noscan/scan).
- `tiny`, `tinyoffset` — the tiny sub-allocator within a single slot.
- `localScan`, `localLargeFree` etc. — GC assist bookkeeping.

The fast path is a bitmask bump inside the cached span: load `allocCache` (a 64-bit mirror of `allocBits`), find the next free bit (trailing zeros via `sys.TrailingZeros64`), set the bit, bump `freeIndex`. Single-digit nanoseconds when cached.

mcentral (per span class, runtime/mcentral.go). A global free-list per span class, protected by a `mutex`. Holds `partial` and `full` span sets. When an `mcache` span fills, it is returned to `mcentral.partial` (still has free slots) or `full` (no free slots but not swept). On refill, `mcentral` hands a `partial` span back to the `mcache`. Contention on `mcentral.lock` is visible in `SCHED` traces and `mutex` profiles when many Ps churn the same size class (common with per-request []byte of similar size).

mheap (runtime/mheap.go). The page allocator. Manages arenas, a free-span treap (heap Treap keyed by span size), `allArenas`, and `spans[]` mapping pages→spans. Owns scavenging policy and `heapLock`. Large allocations bypass `mcache` / `mcentral` entirely and come here.

3. Allocation paths: fast, slow, large, and tiny

3.1 `mallocgc` — the single entry point

Every Go heap allocation goes through `runtime.mallocgc(size, typ, needZero)`, where `typ` is the `_type` carrying `gdata` (pointer bitmap) and `kind` flags (`noscan`). Compiler-inserted calls to `mallocgc` are visible as `runtime.newobject` (for `new / &T`), `runtime.makeslice`, `runtime.makechan`, `runtime.makemap`, and `runtime.mallocgc` itself for escapes.

Simplified control flow:

```
func mallocgc(size uintptr, typ *_type, needZero bool) unsafe.Pointer {
    if size == 0 {
        return unsafe.Pointer(&zeroVal[0]) // zero base
    }
    if gcphase == _GCmarktermination {
        throw("malloc during mark termination")
    }
    // assist handling omitted – see §8

    mp := acquirem()           // pin M, get P
    c := mp.mcache
    noscan := typ == nil || typ.PtrBytes == 0

    if size <= maxSmallSize { // 32 KiB
        if noscan && size < maxTinySize { // 16 B
            // tiny path: sub-allocate inside c.tiny block
        }
        // size class lookup
        class := sizeToClass(size)
        spc := makeSpanClass(class, noscan)
        s := c.alloc[spc]
        if s == nil || s.freeIndex == s.nelems {
            s = refill(c, spc) // → mcentral → mheap → OS
        }
        v := nextFreeFast(s) // bump allocCache bit
        if needZero && s.needzero != 0 { memclr(v, size) }
        return v
    }
    // large path: pages = ceil(size / pageSize)
    s := largeAlloc(size, noscan)
    if needZero && s.needzero != 0 { memclr(s.base(), size) }
    return s.base()
}
```

3.2 Fast path vs. slow path

Path	Condition	Cost	What happens
Tiny	<code>noscan && size < 16</code>	~5 ns	Pack into <code>c.tiny</code> , bump <code>tinyoffset</code> ; new tiny block from size class 2 when full
Fast	<code>c.alloc[spc].freeIndex < nelems</code>	~10-20 ns	Bit-twiddle <code>allocCache</code> , update <code>allocBits</code> , zero if needed
Refill	span exhausted	~200-1000 ns + lock	<code>mCache_Refill</code> → <code>mcentral.cacheSpan</code> (locks <code>mcentral</code>) → maybe <code>mheap.allocSpan</code> / <code>grow</code>
Large	<code>size > 32 KiB</code>	~μs + <code>mmap</code> if growing	<code>mheap.allocSpan</code> with <code>npages = ceil(size / 8192)</code> , page-aligned, zeroed

Zeroing subtlety. The compiler sets `needZero` per-site. Types with pointer-free tails may leave some bytes unzeroed; the allocator may also know pages are already zeroed (`s.needzero == 0` after `sysAlloc` or after scavenging with `MADV_DONTNEED` which guarantees zero-fill on next touch). Avoiding redundant `memset` saves measurable CPU on allocation-heavy workloads.

`noscan` fast path is cheaper for the GC. A span whose objects contain no pointers never enters the mark work queue. For a service whose heap is 40% `[]byte` slabs (HTTP bodies, protobuf wire buffers, `io.Copy` buffers), half the heap may be `noscan` and thus free to skip.

3.3 Large objects

Large objects get their own span with `npages = (size + 8191) / 8192`, aligned to page boundaries. There is no size class binning, so fragmentation is limited to page rounding. Large spans live in `mheap`'s large-span structures and are tracked in `mheap.large` and the treap.

Allocating and freeing large objects is more expensive — it takes `heapLock` and may trigger `sysAlloc / sysMap`.

Backend implication: chunking >32 KiB buffers (e.g., 64 KiB `bufio.Reader` buffers per connection) into pooled 32 KiB slabs often outperforms 64 KiB direct allocations because it keeps the per-P fast path.

3.4 Scavenging and returning memory to the OS

Unused pages are not returned eagerly. The **scavenger** (`runtime/mheap.go:mheap.scavenge` + `runtime/mgcsweep.go:scavenger`) runs concurrently and releases spans/pages that have been free for a while.

Mechanism:

- Free spans are on `mheap.free` (treap) and `mheap.sweepSpans`.
- The scavenger walks free spans, releases their pages via OS advice: `MADV_DONTNEED` (Linux — next access faults zeroed pages) or `MADV_FREE` (Linux 4.5+, lazy reclaim; pages stay mapped until pressure).
- Go selects the strategy at init: `MADV_FREE` preferred when available; fallback to `MADV_DONTNEED`. Controlled by `GODEBUG=madvdontneed=1` to force `DONTNEED` (useful for RSS accounting under cgroups).
- `GOMEMLIMIT` tightens scavenging aggressiveness: when heap is near the limit, the scavenger is more eager; background scavenging may release pages sooner than the default ~5-minute decay.
- Metrics: `memory/classes/total:bytes` vs `memory/classes/heap/released:bytes` (from `runtime/metrics`), and `GODEBUG=gctrace=1` scavenger lines `scvgN: inuse/out/idle/out sys: N`.

Common confusion: `inuse` in `gctrace` is heap pages owned by spans with live objects; `idle` is free pages in the heap that have not been scavenged; `sys` is bytes obtained from the OS; `released` is bytes returned via `MADV`. $RSS \approx sys - released$ (plus stacks, globals). Under containers, `released` pages still count toward cgroup `memory.current` until reclaimed if `MADV_FREE` is in use — this surprises operators. Use `MADV_DONTNEED` (`GODEBUG=madvdontneed=1`) or `GOMEMLIMIT` to force prompt RSS drop.

4. Goroutine stacks: contiguous growth, guard pages, and copying

4.1 Why stacks matter to the allocator

Each goroutine has its own stack. A service with 50k concurrent goroutines (fan-out RPCs, per-connection readers, retry loops) holds ~50k stacks plus per-goroutine `g` structs and `defer` chains. At 8 KiB initial stacks that is ~400 MiB just for stacks, plus growth for deep call chains (JSON decoding, recursive middleware). Stack management is inseparable from allocation strategy.

4.2 From segmented to contiguous (Go 1.4+)

Before Go 1.4, stacks were **segmented** (linked stack segments with `stackguard` hot-splits). This caused "hot split" pathological overhead for functions that straddled the guard. Since Go 1.4, stacks are **contiguous**: each goroutine's stack is a single contiguous allocation that grows by copying.

Sizes (64-bit, current as of Go 1.21-1.23):

Quantity	Value
Initial stack (<code>_StackMin</code>)	8 KiB (was 8 KiB on 64-bit since Go 1.22; 2 KiB on 32-bit; historically 4 KiB/8 KiB depending on arch)
Minimum stack	2 KiB (32-bit) / 8 KiB (64-bit)
Stack guard (<code>stackGuardMultiplier</code>)	~928 bytes reserved
Growth	Doubles on overflow, copies old frames, adjusts pointers in stack

Stacks are allocated from a dedicated stack span pool (`mheap` stack spans) or via `sysAlloc` for large stacks; they are `noscan` only in the sense that the collector scans them as roots conservatively at boundaries but precisely for tracked frames (via stack maps).

4.3 Guard pages and `morestack`

Every goroutine's `g.stack` has:


```
g.stack      stack{lo, hi} // bounds [lo, hi)
g.stackguard0 uintptr    // this address + guard triggers morestack
g.stackguard1 uintptr    // for signal stacks / preemption
```

The compiler inserts a **stack-overflow check** at each function prologue (on `amd64`, a compare of `SP` against `g.stackguard0`):

```
; prologue for a non-leaf function that needs 48 bytes of frame
CMPQ SP, g.stackguard0(GS)
JLS  morestack
```

If `SP` has crossed the guard, execution jumps to `runtime.morestack` (implemented in assembly), which:

1. Saves the caller context.
2. Calls `runtime.newstack` → `stackalloc` a larger stack (typically 2×, with max bounded by `maxStackSize`, ~1 GiB on 64-bit).
3. Calls `runtime.copystack` to memcopy frames from old to new stack, then adjusts every pointer into the stack (saved `SP / BP`, `sudog` links, `defer` chain via `adjustpointers`).
4. Updates `g.stack`, `g.stackguard0`, patches `SP`, and resumes at the caller.

Shrinking. When a goroutine blocks and its stack is largely empty (usage < 1/4 of capacity), `runtime.shrinkstack` may copy it down to a smaller allocation. This happens at GC safe points and is rate-limited to avoid ping-pong.

Diagram 3 — Stack growth and guards



4.4 Tuning stacks for backend scale

Pattern	Stack pressure	Mitigation
50k idle conns each 8 KiB stack	~400 MiB RSS just for stacks	Reduce idle stack via pooling or lower concurrency; consider <code>GODEBUG=gcpacertrace</code> to confirm; do not lower <code>_StackMin</code> (tunable only by rebuilding)
Deep recursion or large frame funcs (big value receivers, <code>[4096]byte</code> locals)	Repeated <code>morestack</code> copies (μ s each), fragmentation	Pass large values by pointer; move big arrays to heap; keep frames < 1-2 KiB
Bursty fan-out (<code>for _, id := range 10k { go fetch(id) }</code>)	10k $\times$ 8 KiB + heap burst at GC trigger	Semaphore / worker pool; batch; bound <code>GOMAXPROCS</code> -relative
Stack-allocated buffers promoted by escape	Heap pressure anyway	Check <code>go build -gcflags=-m</code> to keep buffers stack-allocated where intended

The backend lesson: goroutines are cheap but not free. 10k goroutines at ~4 KiB average stack (after shrinking) still consume ~40 MiB that never shows up in `pprof heap` (stacks are not heap). Track via `runtime/metrics`

keys memory/classes/total:bytes minus heap/objects:bytes , or
GODEBUG=gctrace=1 stack line next_gc ... stack: N.

5. Tri-color mark-and-sweep: the core algorithm

5.1 Invariant

Every object is one of three colors:

Color	Meaning
White	Candidate for collection. Unreached at start of cycle; if still white at end, it is garbage.
Grey	Reached but not yet scanned. On the mark work queue; its pointer slots still need scanning.
Black	Reached and fully scanned. All pointer slots followed; no white object is reachable <i>only</i> from here.

Invariant maintained by the collector: **no black object may point to a white object**. If it did, that white object would be reachable but the collector would never scan the black object again and would collect live memory. The **write barrier** enforces this invariant while the mutator runs concurrently with the marker.

5.2 Two views of the barrier

Historically Go tried:

- **Dijkstra insertion barrier** (Go 1.5–1.7): on `*slot = ptr`, shade `ptr` grey if white. Guarantees any newly installed pointer is marked. But it allowed a black→white store to keep the white object alive only if the pointer was also elsewhere; subtle liveness issues around stack slots.
- **Yuasa deletion barrier** (Go 1.8 experiment): on `*slot = ptr`, shade the *old* value `*slot` grey before overwriting. Guarantees no pointer is lost by deletion.

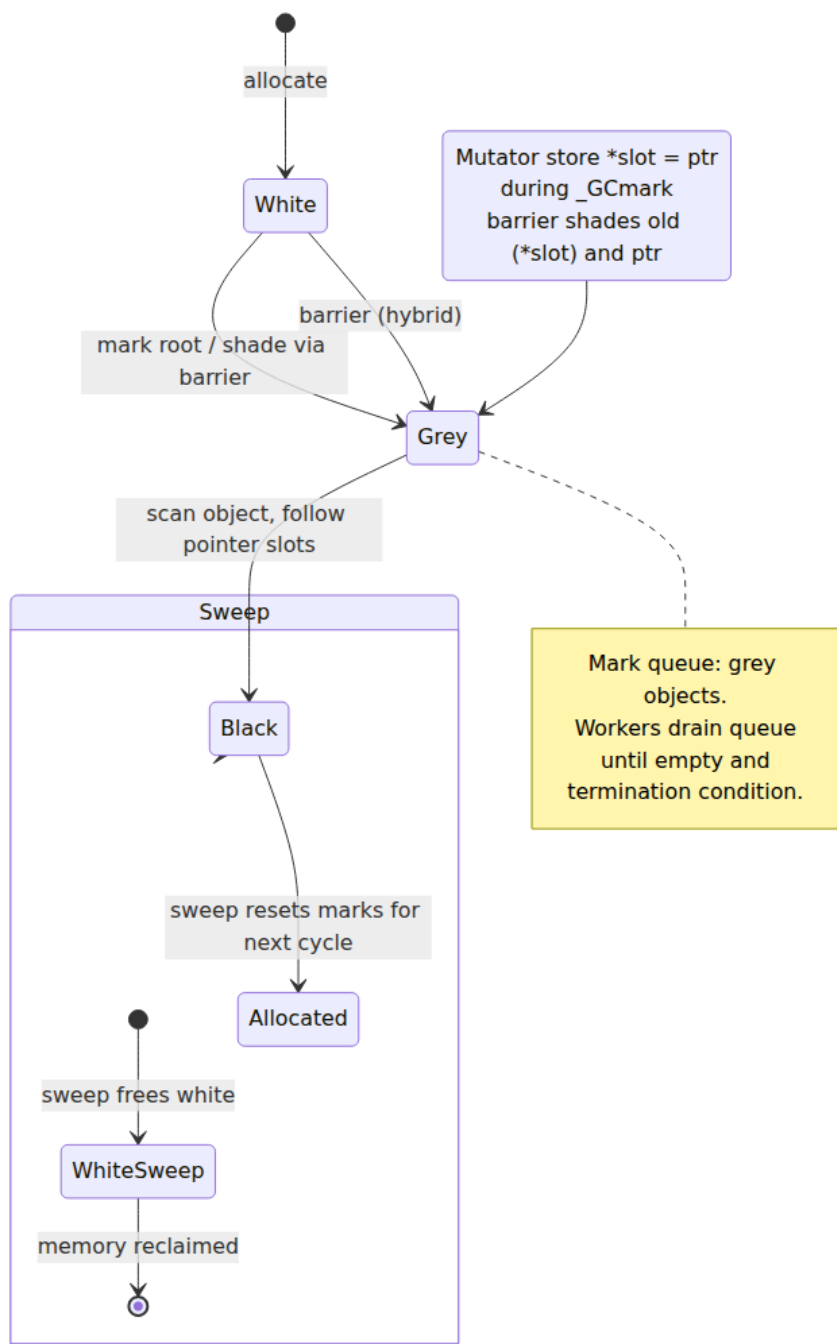
Since Go 1.8+ Go uses a **hybrid barrier** — insertion + deletion:

```
// runtime.wbBufFlush / writeBarrier - pseudo
func writeBarrier(slot *unsafe.Pointer, ptr unsafe.Pointer) {
    if gcphase == _GCmark {
        // deletion component: shade the overwritten pointer
        if old := *slot; old != nil { shade(old) }
        // insertion component: shade the new pointer
        if ptr != nil { shade(ptr) }
    }
    *slot = ptr
}
```

Effect: any pointer stored into the heap during marking is shaded, and any pointer removed from the heap is also shaded. This permits **concurrent marking of the heap while stacks are scanned without barriers** (stacks are rescanned at STW mark termination to catch missed edges — §5.3). Hybrid barrier cost is ~1-3% of mutator CPU during mark (a few nanoseconds per heap pointer store; no barrier for `noscan` stores).

Barrier elision: stores to `noscan` objects, stack slots during concurrent mark, and `typ.PtrBytes == 0` stores skip the barrier — identified via `writeBarrier` need checks in the compiler's SSA pass.

Diagram 4 — Tri-color phases and the hybrid barrier



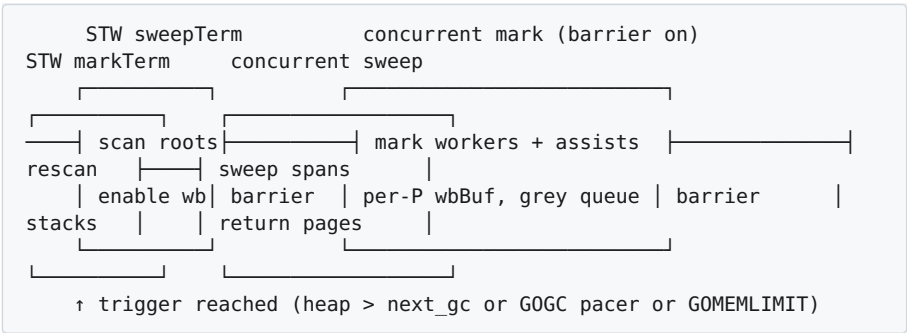
6. GC phases: the cycle from trigger to sweep

A Go GC cycle has four externally visible phases. Two include short stop-the-world (STW) pauses; marking itself is concurrent.

Phase	STW?	What happens
Sweep termination	STW	Stop the world; ensure previous sweep done; reset <code>sweepgen</code> , start new GC cycle, enable write barrier, enqueue roots.
Concurrent mark	concurrent	Background mark workers + mutator assists scan heap; barrier on; stacks scanned without per-store barrier but rescanned at termination.
Mark termination	STW	Stop the world; drain remaining mark work, rescan stacks conservatively, flush <code>wbBuf</code> , set next GC trigger, disable barrier (or keep until sweep done).
Concurrent sweep	concurrent	Sweep spans: free white objects, reset <code>gmarkBits</code> , coalesce free spans, wake scavenger. No STW.

Target pause times: each STW phase typically **50-500 μ s** on a well-tuned service (can exceed 1 ms under large heaps or high allocation rates). Concurrent mark is paced to finish before heap doubling (see §7).

Lifecycle as seen by the runtime's `gcController` and `trace` view:



Observable in `GODEBUG=gctrace=1` as the `STW` split per cycle (see §8).

6.1 Mark workers and assists

- **Dedicated workers:** `GOMAXPROCS/4` (capped) background goroutines at mark-utilization that run when GC is active; preemptible.
- **Idle workers:** Ps that go idle steal mark work rather than sleeping.
- **Mutator assists:** when a goroutine allocates during mark and the pacer thinks marking is behind, the allocating goroutine must assist (scan some heap itself) proportional to its allocation (`gcAssistAlloc`). This **back-pressure** allocation by GC work and is the primary pacing throttle. Visible as `gcAssistAlloc` stalls in execution traces and as a `GODEBUG=gctrace` ratio "assist time".
- **Fraction:** controller targets ~25% of total CPU for marking during the mark phase (tunable via `GOGC` indirectly through heap goal; there is no direct flag for mark CPU).

6.2 Sweep

Sweep is fully concurrent since Go 1.7. Each span's `sweepgen` tracks whether it was swept in the current cycle. Any allocation that touches an unswept span sweeps it first (`mcache` / `mcentral` path checks `sweepgen`). Dedicated sweep goroutine plus lazy per-allocation sweep ensure no STW sweep pause. Cost is proportional to heap size, spread over the concurrent sweep window.

7. Pacing: when GC runs and how much it does

Pacing answers: *when to start marking, and how fast to mark*. Go 1.18–1.21 replaced the classic `GOGC` -only trigger with a unified controller that also respects `GOMEMLIMIT` .

7.1 `GOGC` — the growth ratio

`GOGC` (default 100) sets the ratio of new garbage to live heap between collections:

```
goal = liveHeap_prevCycle × (1 + GOGC/100)
next_gc = goal    (often noted as heapGoal)
```

With `GOGC=100` , the heap is allowed to double between GCs (live ×2). `GOGC=200` → triple; `GOGC=50` → 1.5×. Larger `GOGC` means less frequent

GC, less GC CPU, higher high-water memory. `GOGC=off` disables GC (only sweep of previously freed objects; not recommended except for short-lived batch jobs with explicit `runtime.GC()` or `debug.FreeOSMemory`).

Pacer state: `gcController.heapLive`, `heapGoal`, `heapMarked` (live after last mark), `trigger` ratio, `gcPercent` (`GOGC`).

7.2 GOMEMLIMIT — the hard-ish ceiling

`GOMEMLIMIT` (since Go 1.19) is a soft memory limit in bytes (e.g., `GOMEMLIMIT=1GiB` or via `debug.SetMemoryLimit`). The pacer computes a second goal:

```
heapGoal = min( GOGC-derived goal, GOMEMLIMIT - headroom )
```

`headroom` accounts for stacks, globals, and non-heap reservations plus scavenger hysteresis. When `GOMEMLIMIT` binds tighter than `GOGC`, GC runs more frequently to stay under the limit, trading CPU for memory. If allocations push heap beyond the limit faster than GC can reclaim, the runtime may throttle assists more aggressively and scavenge more eagerly — but it will not OOM itself; cgroup OOM still kills the process if RSS exceeds the container limit. `GOMEMLIMIT` is your defense before that.

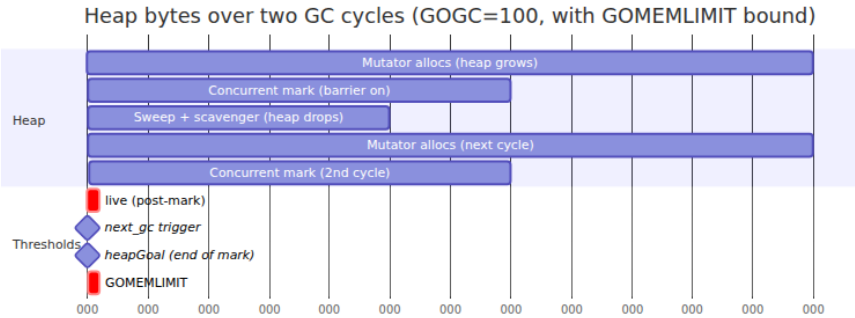
7.3 next_gc and trigger calculation

Per cycle (in `gcController.startCycle` / pacer):

```
live = heapMarked (surviving bytes from mark termination)
goal_GOGC = live * (1 + GOGC/100)
goal_limit = GOMEMLIMIT - headroom // if GOMEMLIMIT set, else
+∞
heapGoal = min(goal_GOGC, goal_limit) // bytes at which cycle should END
heapTrigger = heapGoal * triggerRatio // bytes at which cycle should START
nextGC = heapTrigger // the familiar GODEBUG field
```

`triggerRatio` (~0.6–0.87 depending on heap size and `GOGC`) ensures marking starts early enough to finish before `heapGoal`. The pacer continuously revises assist ratio: if `heapLive` approaches `heapGoal` but marking is behind, each allocating goroutine's assist debt grows and allocation slows — backpressure you can see as increased allocation latency during mark.

Diagram 5 — GC cycle pacing timeline



Reading it: heap grows via mutator; when it hits `next_gc` ($\approx$ trigger), concurrent mark starts; heap keeps growing while marking; mark termination happens before `heapGoal`; sweep reclaims white objects so heap drops toward `live`. If `GOMEMLIMIT` sits below the `GOGC` goal, the `next_gc` line shifts left and cycles compress.

8. Observability: gctrace, pprof heap, and runtime/metrics

8.1 `GODEBUG=gctrace=1` — one line per cycle

Enable `GODEBUG=gctrace=1` with `GODEBUG=gctrace=1` `./app` or `GODEBUG=gctrace=1,gcpacertrace=1` for pacer detail. Since Go 1.21 the format is:

```
gc 42 @12.345s 7%: 0.12+0.80+0.08 ms clock, 0.97+0.40+0+0.64 ms cpu,
64->65->33 MB, 66 MB goal, 0 MB stacks, 12 MB globals, 512 P
~~~~~ heapL
ive timeline
```

Field-by-field (stable since Go 1.18):

Field	Meaning
<code>gc 42</code>	GC number (1-indexed; 1 is first after init)
<code>@12.345s</code>	Time since process start

Field	Meaning
7%	Fraction of CPU spent in GC since start (or since last line; monotonic)
0.12+0.80+0.08 ms clock	Wall-clock: STW sweepTerm + concurrent mark wall + STW markTerm. Sum $\approx$ total pause visible to workload.
0.97+0.40+0+0.64 ms cpu	CPU-time: sweepTerm + assist + background mark + markTerm idle+dedicated
64->65->33 MB	heapLive before mark -> after mark (live+garbage+stack) -> live after sweep (live heap). Retained $\approx$ live.
66 MB goal	heapGoal for this cycle (from pacer; $\min(\text{GOGC goal}, \text{GOMEMLIMIT}-\text{headroom})$)
0 MB stacks, 12 MB globals	Non-heap roots scanned
512 P	GOMAXPROCS

Older Go versions include `forced` (triggered by `runtime.GC()` or `debug.FreeOSMemory`) and scav lines:

```
scvg0: inuse: 38, idle: 12, sys: 50, released: 8, consumed: 42 (MB)
```

Meanings: `inuse` = owned live spans, `idle` = free but unscavenged, `sys` = obtained from OS, `released` = returned, `consumed` = `sys` - `released` ($\approx$ RSS). Per `gctrace` line, these appear every few GCs or on scavenger activity.

Real examples to learn from:

```
# Healthy service, GOGC=100, GOMEMLIMIT=2GiB, low allocation rate
gc 18 @42.103s 3%: 0.08+1.20+0.05 ms clock, 0.64+0.30+0.10+0.40 ms cpu,
180->195->120 MB, 240 MB goal, 2 P
# → sweepTerm 80 μs, markTerm 50 μs (good), live 120 MB, goal 240 MB
(GOGC-bound)

# Memory pressure: GOMEMLIMIT binds before GOGC
gc 42 @88.001s 18%: 0.11+6.40+0.07 ms clock, 0.88+4.10+1.20+0.56 ms
cpu, 980->1020->760 MB, 1050 MB goal, 8 P
# → assist CPU elevated (4.10 ms), goal hugging live+headroom, frequent
cycles

# Forced GC (e.g., ballast trick or manual runtime.GC())
gc 7 @5.002s 4%: 0.09+0.90+0.06 ms clock, 0.72+0.20+0.15+0.48 ms cpu,
1024->1025->1024 MB, forced, 1026 MB goal, 4 P
# → "forced" tag, heap barely drops – ballast alive

# High allocation rate, assists kicking in
gc 9 @10.4s 12%: 0.15+2.8+0.09 ms clock, 1.2+2.5+0.0+0.72 ms cpu, 220-
>280->140 MB, 280 MB goal, 4 P
# → assist 2.5 ms > background 0.0 – mutators doing GC work, allocation
backpressure present
```

Tuning with `gctrace` :

- STW sum > 1 ms consistently → heap too large or many pointers (mark has more edges to follow); profile with `pprof --alloc_objects` to find pointer-heavy allocations.
- `assist` CPU large vs. dedicated → pacer thinks mutators are allocating faster than workers can mark; raise `GOGC` (more headroom) or reduce allocation rate (pooling, fewer per-request allocs), or raise `GOMAXPROCS` if under-scheduled.
- `goal` close to `GOMEMLIMIT` headroom → `GOMEMLIMIT` is the trigger; raise it if you have cgroup headroom, or reduce live set (cache sizes, ballast).
- `released` stays 0 while `idle` grows → `MADV_FREE` not reclaiming; set `GODEBUG=madvdontneed=1` if cgroup RSS matters.

8.2 `pprof` heap profiles

Collect with:

```

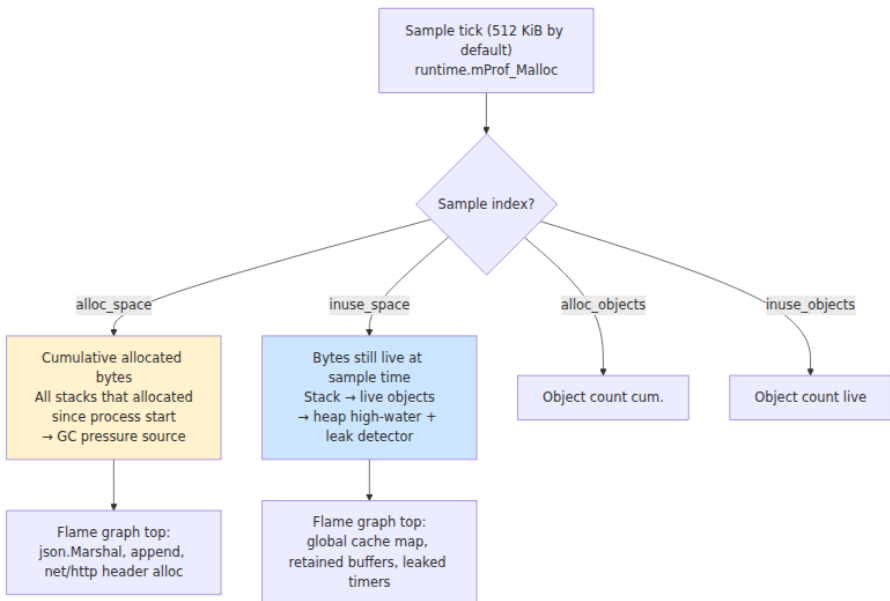
go tool pprof -http=:8081 http://localhost:8080/debug/pprof/heap #
inuse_space (live)
go tool pprof -http=:8081 http://localhost:8080/debug/pprof/heap?gc=1
curl -s http://localhost:8080/debug/pprof/heap > heap.pb.gz && go tool
pprof heap.pb.gz

# alloc_space vs inuse_space
go tool pprof -sample_index=alloc_space heap.pb.gz # all allocations
since start (pressure)
go tool pprof -sample_index=inuse_space heap.pb.gz # live at sample
time (leak / retained)
go tool pprof -sample_index=alloc_objects heap.pb.gz # object count

```

Four sample indices exist: `alloc_space`, `alloc_objects`, `inuse_space`, `inuse_objects`. For GC tuning, `inuse_space` tells you what survives GC (sets live); `alloc_space` tells you where allocation volume comes from (drives GC frequency). Optimize `alloc_space` hot spots to reduce GC CPU; optimize `inuse_space` hot spots to reduce peak heap.

Diagram 6 — Heap profile anatomy



Example `pprof` excerpt (trimmed; `top` on `alloc_space` for an HTTP service):

```
(pprof) top 10 --cum --sample_index=alloc_space
      flat flat%  sum%      cum  cum%
  4200 MB 18.2% 18.2%    5200 MB 22.5% encoding/json.Marshal
  3100 MB 13.4% 31.6%    4800 MB 20.8% net/http.(*conn).serve /
ReadRequest
  1800 MB  7.8% 39.4%    2600 MB 11.2% github.com/.../
handler.ingest.func1
   900 MB  3.9% 43.3%    1200 MB  5.2% bytes.growSlice
(pprof) top 10 --sample_index=inuse_space
      flat flat%  cum
   180 MB 32%    180 MB cache.(*LRU).Add / map bucket overflow
    85 MB 15%    85 MB  bufio.NewReaderSize (pooled but retained)
    40 MB  7%    40 MB  net/http.persistConn
```

Operational pattern: if `alloc_space` top is `json.Marshal/Unmarshal`, move to `json.RawMessage` or `easyjson/ffjson` or pooled `sync.Pool` encoders. If `inuse_space` top is your cache, bound it explicitly (`GOMEMLIMIT` does not bound caches for you — it just triggers GC more often while the cache stays live).

8.3 runtime/metrics and runtime.ReadMemStats

Prefer `runtime/metrics` (Go 1.16+) over the legacy `runtime.ReadMemStats` for scraping; the latter STWs briefly to read stats.

```
import "runtime/metrics"

func logMemMetrics() {
    descs := metrics.All()
    samples := make([]metrics.Sample, len(descs))
    for i := range samples { samples[i].Name = descs[i].Name }
    metrics.Read(samples)
    for _, s := range samples {
        if s.Value.Kind() == metrics.KindUint64 {
            // example: /memory/classes/heap/objects:bytes, /gc/heap/
live:bytes
            // /gc/cycles/total:gc-cycles, /gc/pause:seconds
distribution
        }
    }
}
```

Key metric names:

Metric	What it is
/gc/heap/live:bytes	Live heap after last GC
/gc/heap/goal:bytes	Pacer goal for next cycle
/memory/classes/heap/objects:bytes	In-use heap objects
/memory/classes/heap/released:bytes	Returned to OS
/gc/cycles/total:gc-cycles	Count
/gc/pause:seconds	Distribution of STW pauses
/gc/gogc:percent / /gc/gomemlimit:bytes	Current limits
/sched/gomaxprocs:threads	P value

Scrape these via your metrics endpoint (wrap in `/debug/vars` or expose via OpenTelemetry/Prometheus bridge) and alert on `/gc/pause:seconds` p99 and on `/memory/classes/heap/released:bytes` staying flat while `idle` grows.

8.4 Execution trace view

```
go test -trace=trace.out ./...
go tool trace trace.out
# In the trace viewer: GC events show STW spans, per-P mark workers,
assist blocks.
```

Look for long `GC Mark Termination` STW boxes and dense `GC Assist` blocks on mutator goroutines — they confirm pacer backpressure rather than mere throughput cost.

9. Allocation benchmarks — measuring correctly

`testing.B` benchmarks that measure GC-tainted work must control for GC and `b.N` planning.

```

func BenchmarkAlloc_Small(b *testing.B) {
    b.ReportAllocs()
    var sink *Request
    for i := 0; i < b.N; i++ {
        r := &Request{ID: i, Body: make([]byte, 128)} // escapes;
        mallocgc path
        sink = r
    }
    runtime.KeepAlive(sink)
}

func BenchmarkAlloc_WithPool(b *testing.B) {
    var pool = sync.Pool{New: func() any { return &Request{} }}
    b.ReportAllocs()
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        r := pool.Get().(*Request)
        r.ID = i
        // reuse Body slice with cap check instead of make
        if cap(r.Body) < 128 { r.Body = make([]byte, 128) } else { r.Body = r.Body[:128] }
        pool.Put(r)
    }
}

// Controlling GC between iterations for latency microbenchmarks:
func BenchmarkHandler_NoGCNoise(b *testing.B) {
    b.ReportAllocs()
    // Option 1: ballast to dampen GC during bench (see §10.2)
    // Option 2: explicit GC before timer
    runtime.GC()
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        handleRequest(payload) // measure with GC pacer stable
    }
}

```

Typical results (amd64, Go 1.22, GOMAXPROCS=8):

BenchmarkAlloc_Small-8	8200000	142 ns/op	48 B/op	1
allocs/op				
BenchmarkAlloc_WithPool-8	26000000	46 ns/op	0 B/op	0
allocs/op				
BenchmarkHandler_NoGCNoise-8	500000	2800 ns/op	312 B/op	4
allocs/op				

Pitfalls:

- Never compare `ns/op` across benchmarks with different `GOGC / GOMEMLIMIT` — the pacer changes throughput.
- `b.ReportAllocs()` counts heap allocations (post-escape); stack allocations are invisible — cross-check with `go build -gcflags=-m` and `go test -bench=. -benchmem`.
- Microbenchmarks that `make([]byte, 1<<20)` on each iter will be dominated by `sysAlloc / MADV` — use `b.SetBytes` and report throughput, not just latency.
- For GC overhead measurement, run the service under realistic RPS and scrape `/gc/pause:seconds` rather than inferring from `ns/op` alone.

10. Backend lens: tuning GOGC, GOMEMLIMIT, and the pod budget

10.1 The pod equation

A Go pod's memory budget splits as:

```
cgroup limit = heap live + heap garbage (until next GC) + stacks (~P
× avg + goroutines × avg)
              + globals + heap idle (unscavenged) + OS overhead
```

You control two knobs:

Knob	Default	Effect of raising	Effect of lowering
GOGC	100	Less frequent GC, more GC throughput headroom, higher high-water, lower GC CPU	More frequent GC, lower high-water, higher GC CPU, more STW events (still short)
GOMEMLIMIT	off (MaxInt64)	Allows heap to grow; GOGC governs pacing	Caps heap; GC runs earlier/often, CPU ↑, assist ↑, OOM risk ↓

Rules of thumb for Kubernetes:

- 1. **Always set a memory limit on the container**
(`resources.limits.memory`) and set `GOMEMLIMIT` to **~75-85% of that limit** to leave headroom for stacks/globals/scavenge lag. Example:
`limit: 2Gi → GOMEMLIMIT=1500MiB`.
- 2. **GOMEMLIMIT wins under pressure.** If you set both, `GOMEMLIMIT` will dominate when heap is large; `GOGC` dominates when heap is small. Set `GOGC` to reflect CPU budget (100 for balanced, 200 for CPU-constrained batch, 50 for memory-constrained fan-out).
- 3. **Measure before tuning.** Collect `gctrace` for a representative load window and note `goal` vs `GOMEMLIMIT`-headroom, `assist` share, and STW p99. Change one knob at a time.
- 4. **Horizontal scale beats heroic tuning.** If GC CPU is >15-20% and you are at `GOGC=50` chasing memory, add replicas or shard the workload.

Example deployment snippet:

```
env:
- name: GOMEMLIMIT
  value: "1500MiB"           # 75% of 2Gi limit
- name: GOGC
  value: "100"
- name: GODEBUG
  value: "gctrace=1"         # sample in staging; off in prod or sampled
resources:
  limits: { memory: "2Gi", cpu: "2" }
  requests: { memory: "1Gi", cpu: "1" }
```

Latency vs throughput tradeoff table:

Profile	GOGC	GOMEMLIMIT	GC CPU	Heap high-water	Best for
Default (balanced)	100	75-85% of cgroup	~10-15%	2× live	General services
Low-latency (SPiky, p99-sensitive)	50-75	tight (70%)	~15-25%	1.5× live	API edge, fan-out

Profile	GOGC	GOMEMLIMIT	GC CPU	Heap high-water	Best for
Throughput (batch, CPU-bound)	200-400	loose or off	~5-8%	3-5× live	Workers, ETL, batch jobs
Memory-constrained	50	60-70% of cgroup	~20-30%	1.5× live	Sidecars, dense packing

10.2 The ballast trick and when to retire it

The **ballast** is a large live allocation created at init that artificially inflates `live` so the `GOGC`-derived `heapGoal = live × (1+GOGC/100)` is larger, reducing GC frequency:

```
var ballast = make([]byte, 1<<30) // 1 GiB ballast
func init() { runtime.KeepAlive(ballast) }
```

- Before `GOMEMLIMIT`, ballast was the only way to raise the heap goal without raising `GOGC` to absurd values.
- With `GOMEMLIMIT`, ballast is largely obsolete — prefer `GOMEMLIMIT` plus a moderate `GOGC`. Ballast still has niche use when you want to **pin** a heap reservation that survives GC (e.g., to keep `next_gc` stable across rolling deploys), but it wastes RSS and confuses `pprof inuse_space` readers.
- If you inherit a service with ballast, replace it with:

```
debug.SetMemoryLimit(1500 << 20) // programmatic GOMEMLIMIT
debug.SetGCPercent(100)
```

10.3 Goroutine count, stack size, and `GOMAXPROCS`

Goroutine count interacts with GC pacing because each goroutine contributes:

- Stack memory (not counted in `heapLive` but in `memory/classes/total:bytes` and cgroup RSS).
- Scan work at mark termination (stack rescan STW grows linearly with goroutine count).

- Allocation rate if each goroutine allocates (higher `assist` pressure).

Guidance:

- **Bound concurrency.** 100k goroutines $\times$ 8 KiB = 800 MiB stacks; mark termination scans 100k stacks. Use semaphores, worker pools, or `errgroup.WithContext` with a limit.
- **Shrink stacks where you can.** Deep call chains do not need to be deep goroutines — an iterative refactor or moving large frames to heap (via explicit `new([4096]byte)`) reduces stack growth copies.
- **GOMAXPROCS sets P count** and thus `mcache` sharding and mark worker count. On Kubernetes, Go auto-detects `GOMAXPROCS` from cgroup CPU quota since Go 1.19 (via `GOMAXPROCS / automaxprocs`); verify with `runtime.GOMAXPROCS(0)`. Over-provisioned `GOMAXPROCS` inflates `mcache` memory (134 spans $\times$ P) and mark parallelism you cannot use; under-provisioned starves mark workers.

10.4 Scavenging knobs for container RSS

If your HPA or alerting keys on container RSS (as most do), `MADV_FREE`'s lazy reclaim will over-report usage and trigger spurious scale-ups or OOM-kills on neighboring pods. Options in order of preference:

```
GODEBUG=madvdontneed=1      # force MADV_DONTNEED (immediate RSS
drop, slightly slower next touch)
GOMEMLIMIT=...              # tighter scavenging hysteresis
GODEBUG=gctrace=1           # verify scvg lines: released should
track idle
```

11. Putting it together: an end-to-end example

An HTTP JSON ingest handler that demonstrates the allocator $\rightarrow$ stack $\rightarrow$ GC chain and how to observe it:

```

// go.mod: go 1.22
package main

import (
    "encoding/json"
    "net/http"
    _ "net/http/pprof" // registers /debug/pprof/*
    "runtime/debug"
    "sync"
)

type Event struct {
    ID      string      `json:"id"`
    Data    json.RawMessage `json:"data"` // avoid double decode; noscan
    bytes
}

var eventPool = sync.Pool{New: func() any { return &Event{} }}

func ingestHandler(w http.ResponseWriter, r *http.Request) {
    e := eventPool.Get().(*Event)
    defer eventPool.Put(e)
    // reuse: avoid per-request alloc for the struct shell

    // bounded read: 1 MiB max to avoid large-object path surprises
    r.Body = http.MaxBytesReader(w, r.Body, 1<<20)
    if err := json.NewDecoder(r.Body).Decode(e); err != nil {
        http.Error(w, err.Error(), 400); return
    }
    // ... process e.Data without allocating intermediate
    map[string]any ...
    w.WriteHeader(204)
}

func main() {
    // In K8s set via env; programmatic fallback for local dev:
    debug.SetMemoryLimit(1500 << 20)
    debug.SetGCPercent(100)
    http.HandleFunc("/ingest", ingestHandler)
    http.ListenAndServe(":8080", nil)
}

```

Observing it:

```
# 1. Run with tracing under load
GODEBUG=gctrace=1 GOMEMLIMIT=1500MiB ./ingest &
hey -n 100000 -c 50 http://localhost:8080/ingest -d '{"id":"x","data":
{"v":1}}'

# gctrace output under load (sample):
# gc 120 @65.2s 9%: 0.10+3.2+0.07 ms clock, ... 420->510->380 MB, 760
MB goal, 8 P

# 2. Heap profile at steady state
go tool pprof -top -sample_index=inuse_space http://localhost:8080/
debug/pprof/heap
go tool pprof -top -sample_index=alloc_space http://localhost:8080/
debug/pprof/heap

# 3. Benchmark the two decode strategies
go test -bench=BenchmarkDecode -benchmem -count=5 ./...
# BenchmarkDecode_RawMessage-8      120000      9800 ns/op    512 B/op    3
allocs/op
# BenchmarkDecode_MapAny-8          45000     26000 ns/op    4200 B/op   28
allocs/op
```

The `RawMessage` variant avoids 25 allocations per request; `alloc_space` drops visibly; `gctrace`'s `goal` headroom widens and assist CPU falls.

12. Failure modes and anti-patterns

Anti-pattern	Symptom in signals	Fix
Per-request <code>make([]byte, 64*1024)</code> without pooling	Large-object path, <code>mheap</code> churn, <code>alloc_space</code> top = <code>makeslice</code> , <code>gctrace</code> <code>goal</code> pressure	<code>sync.Pool</code> of <code>[]byte</code> with <code>cap</code> check; or chunk to 32 KiB
Unbounded <code>map</code> cache with no eviction	<code>inuse_space</code> grows linearly, <code>next_gc</code> climbs, RSS hits cgroup limit	Bounded cache (LRU with <code>maxBytes</code>), <code>GOMEMLIMIT</code> as backstop
<code>defer</code> inside hot loop allocating <code>_defer</code>	<code>deferproc</code> allocs in <code>alloc_space</code> ; surprising GC volume	Hoist <code>defer</code> or use inline <code>mu.Lock()/Unlock()</code> without <code>defer</code> on hot path (see Ch. 3)

Anti-pattern	Symptom in signals	Fix
Ballast + tight cgroup limit	<code>forced</code> GCs, no heap drop, OOM despite <code>gctrace</code> showing pressure	Remove ballast, use <code>GOMEMLIMIT=75%</code> of limit
<code>G0GC=off</code> in long-lived service	No GC, heap grows to limit, OOM	Use <code>G0GC=200</code> + <code>GOMEMLIMIT</code> instead; reserve <code>off</code> for short-lived CLIs
100k goroutines for fan-out without limit	<code>gctrace</code> STW markTerm grows (stack rescan), RSS dominated by stacks	golang.org/x/sync/errgroup with <code>SetLimit</code> , worker pool, streaming
Reading only <code>inuse_space</code> to judge allocation cost	Misses GC pressure from short-lived allocs that die before sample	Always check <code>alloc_space</code> alongside <code>inuse_space</code>

Key takeaways

- Go's allocator is a three-level TCMalloc derivative: per-P `mcache` (lock-free fast path), per-class `mcentral` (shared partial/full spans), and global `mheap` (page/arena manager). Most allocations never leave `mcache`; refill goes one level at a time.
- 67 size classes (8 B–32 KiB) trade fragmentation (~12% worst case) for bounded `mcache` cost. `noscan` span classes avoid GC scanning; `tiny` packs ≤ 16 B `noscan` objects into one slot. Above 32 KiB, allocations are page-aligned large spans.
- Frames live on contiguous, copy-on-grow stacks (8 KiB initial on 64-bit). The compiler inserts a `stackguard0` check per non-leaf function; overflow traps to `morestack` $\rightarrow$ `newstack` $\rightarrow$ `copystack`. Shrinking happens when usage $< 1/4$ of capacity. Goroutine count $\times$ stack size is a first-class memory budget — not visible in `pprof heap`.
- GC is concurrent tri-color mark-and-sweep with a **hybrid** (insertion+deletion) write barrier that shades both the overwritten and newly stored pointer during `_GCmark`. Two STW phases (sweep termination and mark termination, each ~50–500 μ s) bracket a

concurrent mark that uses dedicated workers, idle workers, and **mutator assists** that back-pressure allocations when marking falls behind.

- Sweep is fully concurrent; `sweepgen` lets allocations lazily sweep unswept spans. Scavenging returns pages via `MADV_DONTNEED` / `MADV_FREE` ; `GODEBUG=madvdontneed=1` forces immediate RSS drop for cgroup-aware deploys, and `GOMEMLIMIT` tightens scavenging hysteresis.
- Pacing: `GOGC` sets `heapGoal = live × (1+GOGC/100)` (growth ratio); `GOMEMLIMIT` caps it to `GOMEMLIMIT - headroom`. The pacer derives `next_gc = heapTrigger = heapGoal × triggerRatio` and continuously adjusts assist ratios. If `GOMEMLIMIT` binds, GC runs more often for more CPU; if `GOGC` binds, heap floats higher for less CPU.
- `GODEBUG=gctrace=1` lines read as `gc N @T S%: STW+mark+STW clock, SW+assist+bg+STW cpu, heapBefore->heapDuring->live, goal ; scvg` lines show `inuse/idle/sys/released`. `pprof heap` has `alloc_space` (pressure) vs `inuse_space` (retained) — optimize each for different goals. Prefer `runtime/metrics (/gc/*, /memory/classes/*)` over `ReadMemStats`.
- For Kubernetes, set `GOMEMLIMIT≈75–85%` of the cgroup limit and keep `GOGC=100` (or 50 for p99-latency, 200–400 for batch throughput). Retire the **ballast** trick in favor of `GOMEMLIMIT` ; bound concurrency to control stack memory and mark-termination scan work; verify with `gctrace` , `pprof alloc_space` , and `/gc/pause:seconds` .

Further reading

- Go GC Documentation — Official GC Guide (GC phases, pacer, `GOGC` / `GOMEMLIMIT` , `GODEBUG` flags): <https://go.dev/doc/gc-guide> — **pinned**
- `runtime/malloc.go` — `mallocgc` , `nextFreeFast` , tiny allocator, `mcache` refill, large-object path; canonical fast-path source: <https://github.com/golang/go/blob/master/src/runtime/malloc.go> — **pinned**
- `runtime/mheap.go` — Heap arenas, page allocator, span maps, `sysAlloc` / `sysMap` , scavenger integration: <https://github.com/golang/go/blob/master/src/runtime/mheap.go> — **pinned**
- Go GC Pacer Proposal & Design Doc (1.21 pacer rework, trigger ratio, `GOMEMLIMIT` controller, assist pacing): <https://github.com/>

golang/go/issues/48409 and <https://go.dev/issue/48409> / <https://go.dev/blog/gomemlimit> — **pinned**

- `runtime/mgcsweep.go` & `runtime/mgc.go` — Concurrent sweep, `sweepgen`, `gcController`, hybrid write barrier (`writeBarrier`, `wbBuf`) and mark worker scheduling: <https://github.com/golang/go/blob/master/src/runtime/mgc.go>
- `runtime/stack.go` — Goroutine stack allocation, `stackalloc`, `copystack`, `shrinkstack`, guard constants (`_StackMin`, `stackGuard`): <https://github.com/golang/go/blob/master/src/runtime/stack.go>
- `runtime/sizeclasses.go` — Generated size-class table, `class_to_size`, `size_to_allocClass`, fragmentation rationale: <https://github.com/golang/go/blob/master/src/runtime/sizeclasses.go>
- Rick Hudson et al., *Go GC Latency Problem Solved* (GopherCon 2015) and Austin et al., *Low-latency GC for Go* — original concurrent collector papers that introduced the hybrid barrier and STW minimization.
- Go `runtime/metrics` Package Docs — Scrapable GC/heap metrics replacing `MemStats`: <https://pkg.go.dev/runtime/metrics>
- GODEBUG Reference — `gctrace`, `gcpacertrace`, `madvdontneed`, `asyncpreemptoff`: https://pkg.go.dev/runtime#hdr-Environment_Variables

Chapter 6 — The Go Memory Model, Atomics, and Synchronization Primitives

What this chapter covers. Every interesting Go backend is concurrent — dozens of goroutines share caches, counters, connection pools, and config pointers. The Go memory model is the contract that tells you when a write in one goroutine becomes visible in another, and what constitutes a data race. On top of that contract sits `sync` and `sync/atomic`: the primitives you use daily and the ones whose internals determine your tail latency under contention. This chapter opens both layers: the formal happens-before model and the concrete machinery — spinning mutex, starvation mode, `RWMutex`, `WaitGroup`, `Once`, `Cond`, `Map`, and `Pool` —

that implements it. You will read race-detector output, fix real races, and learn when an atomic is the right tool and when it is a premature optimization that breaks on `arm64`.

Learning goals — after this chapter you should be able to:

- State the Go memory model's happens-before definition and data-race definition precisely, and explain why "it worked on my laptop" is not a correctness argument.
- Enumerate every happens-before edge the spec guarantees (goroutine creation, channel, `Mutex / RWMutex`, `Once`, `WaitGroup`, atomics, `Cond`, `Context` cancellation) and use them to prove visibility or spot a missing edge.
- Use `sync/atomic` correctly — `Int32 / Int64 / Uint64 / Pointer[T] / Value`, alignment requirements, `CompareAndSwap` loops, and why `atomic.Value` and `atomic.Pointer` exist — and explain the ABA problem and how to avoid it in Go.
- Describe the internal state machines of `Mutex` (Locked/Woken/Starving + waiter count, spinning, starvation since Go 1.9), `RWMutex`, `WaitGroup` (single 64-bit state word + semaphore), `Once` (done + mutex), `Cond`, `sync.Map` (read/dirty/misses/expunged), and `sync.Pool` (per-P local + victim).
- Run the race detector (`go run -race`, `go test -race`), interpret its output, quantify its overhead, and fix happens-before violations including the runtime's concurrent map read/write panic.
- Choose the right synchronization primitive per access pattern (read-mostly vs. write-heavy vs. single-writer vs. per-request scratch), interpret mutex/block profiles, and size `sync.Pool` for backend workloads without leaking or retaining garbage.

Placement. Volume 4, Chapter 3 — Memory Models and Happens-Before gave the hardware and language-agnostic foundation (store buffers, SC, TSO vs. weak ARM, acquire/release). Volume 13, Chapter 2 gave the runtime-level view of the Go scheduler and GC. This chapter is the Go-specific instantiation: the spec you program against and the `sync` internals you operate. Volume 17, Chapter 4 (Scheduler/G-M-P) and Chapter 5 (Allocator/GC) are prerequisites for `Mutex` spinning and `Pool`'s victim cycle. Chapter 10 deep-dives the race detector, `pprof`, and `trace` as instruments; here we use them to establish correctness.

1. Why a memory model is not optional

Single-threaded reasoning does not extend to concurrent code. Three independent actors reorder your program:

1. **The compiler** reorders loads and stores under the as-if rule (see Volume 4, Chapter 3). A loop that spins on `done` can be hoisted into `if (!done) for(;;)`.
2. **The processor** buffers stores, executes out of order, and delays coherence messages. On `arm64` a store followed by a load to a different address can appear reversed to another core.
3. **The cache** makes a store visible to different cores at different times.

Go, like Java and C++, resolves this by defining a **memory model** — a contract between the programmer and the implementation. If you program to the contract (establishing happens-before edges between conflicting accesses), the runtime guarantees visibility and ordering. If you do not, you have a **data race** and the program's behavior is undefined — not "eventually consistent," not "rarely wrong," but undefined in the spec sense: any outcome is allowed, and the compiler and hardware will eventually produce the surprising one in production at 3 AM on `arm64`.

Backend relevance is immediate. A config-reload goroutine that writes `*Config` without synchronization and an HTTP handler that reads it has a data race even if "it never failed in staging." A metrics counter incremented with plain `count++` from many goroutines loses updates on `amd64` and can tear on 32-bit `arm`. A `map` read concurrently with a write panics — the runtime detects it and crashes the process, which is actually the *better* outcome compared to silent corruption.

2. The Go memory model: happens-before and data races

The authoritative reference is `go.dev/ref/mem`. It fits on two pages. Every backend Go engineer should have read it at least once.

2.1 Program order and happens-before

Within a single goroutine, **program order** is the order you wrote. The memory model guarantees that reads and writes **within one goroutine** behave as if executed in program order — the compiler and hardware must preserve single-thread semantics.

Across goroutines, the model defines a **happens-before** (hb) relation, a strict partial order over memory operations:

- If event `a` happens-before `b`, then the effects of `a` (all prior writes by its goroutine) are visible to `b` and `b` observes `a` before itself.
- hb is transitive: if `a` hb `b` and `b` hb `c`, then `a` hb `c`.
- hb is not total: many events are concurrent (unordered). Two concurrent accesses that conflict are a race.

This is the same Lamport relation you meet in distributed consistency (Volume 6, Chapter 2). On one machine it orders memory operations; across machines it orders messages.

2.2 Data race definition

From the spec:

A **data race** is a pair of memory accesses to the same location where at least one is a write, the accesses are concurrent (neither happens-before the other), and they are not synchronized by the language's synchronization primitives.

If your program has a data race, **its execution is not defined** by the memory model for those locations. There is no "benign race" — that is folklore from C++ discussions, and it does not transfer to Go because the Go compiler will optimize based on the assumption that racy programs do not exist.

```
// Racy – do not do this.
var a string
var done bool

func writer() {
    a = "hello, world" // W1
    done = true        // W2 – no hb edge to reader
}
func reader() {
    if done {          // R1
        println(a) // R2 – may print "" even though done==true was
                    // observed
    }
}
```

There is no `hb` between `W1/W2` and `R1/R2`. The compiler may reorder `W1` and `W2`; the processor may make `W2` visible before `W1`. The fix is to introduce a synchronizing operation — a channel send/recv, a mutex lock/unlock, an atomic store/load, or any other primitive listed in Section 3.

```
// Correct: channel establishes hb.
var a string
var c = make(chan struct{})

func writer() {
    a = "hello, world"
    close(c) // or c <- struct{}{} – send hb receive
}
func reader() {
    <-c
    println(a) // guaranteed "hello, world"
}
```

```
// Correct: atomic establishes hb.
var a atomic.Value // or atomic.Pointer
var done atomic.Bool

func writer() {
    a.Store("hello, world")
    done.Store(true) // atomic Store hb atomic Load that observes it
}
func reader() {
    if done.Load() {
        println(a.Load().(string))
    }
}
```

2.3 Single-goroutine guarantees and visibility

The spec's most misunderstood sentence:

If the effects of a goroutine must be observed by another goroutine, use a synchronization mechanism such as a lock or channel communication to establish a happens-before relationship.

In other words, **sharing memory without synchronization is never safe**, even for a single writer / single reader flag. The single-goroutine guarantee ("behaves as if in program order") is worthless across goroutines without an edge.

3. Every happens-before edge the spec guarantees

Memorize this table. When you review a concurrent code path, you are hunting for which row justifies the visibility claim. If no row applies, the code is racy.

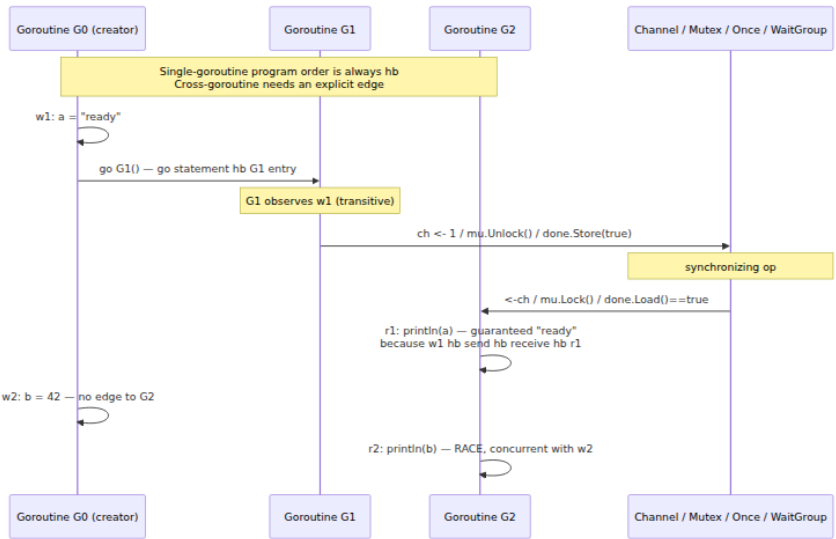
Operation	Edge
<code>go f()</code>	The <code>go</code> statement happens-before the first instruction of <code>f</code> . Everything <code>go</code> 's goroutine did before the <code>go</code> is visible to <code>f</code> .
Goroutine exit	No edge from the exiting goroutine to its creator. Use a channel, <code>WaitGroup</code> , or other sync to observe completion.
Channel send → receive	A send on a channel happens-before the corresponding receive completes. For buffered channels, the <code>k</code> th receive hb the <code>k+c</code> th send completes (where <code>c</code> = capacity): the buffer decouples but does not eliminate ordering. <code>close(ch)</code> hb a receive that observes the <code>close (ok==false)</code> .
Channel close	Closing happens-before any receive that returns the zero value with <code>ok==false</code> because the channel was closed.
Mutex / <code>RWMutex</code>	For a given mutex <code>m</code> , the <code>n</code> th <code>m.Lock()</code> hb the <code>n</code> th <code>m.Unlock()</code> hb the <code>(n+1)</code> th <code>m.Lock()</code> . Read-locks participate the same way: <code>RLock</code> / <code>RUnlock</code> edges serialize writers vs. readers.

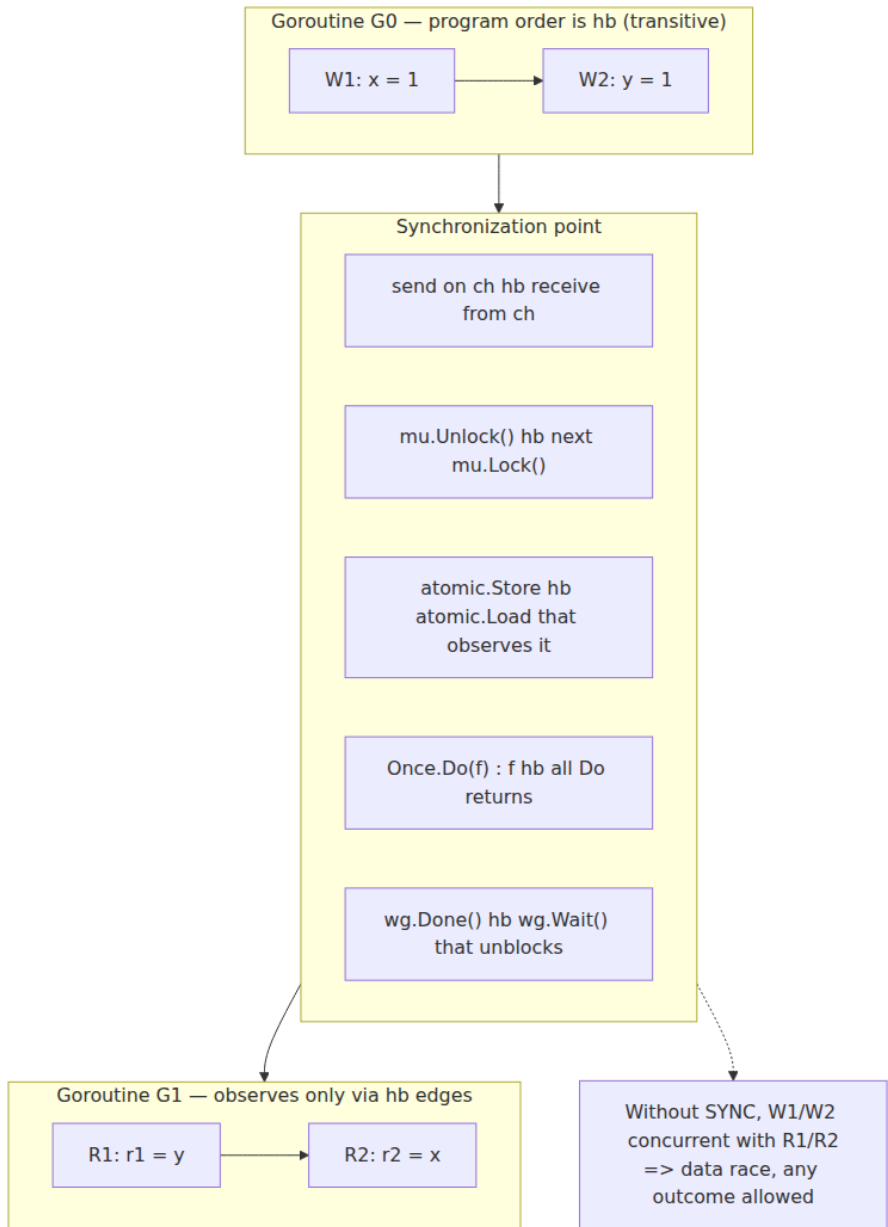
Operation	Edge
<code>Once.Do(f)</code>	The single call of <code>f()</code> inside <code>once.Do(f)</code> happens-before every <code>once.Do(f)</code> call returns — including calls that did not invoke <code>f</code> . All callers see <code>f</code> 's effects.
<code>WaitGroup</code>	<code>wg.Add(n)</code> hb <code>wg.Wait()</code> that observes the counter reaching zero via <code>wg.Done()</code> / <code>Add(-n)</code> . Correct pattern: <code>Add</code> before <code>go</code> ; <code>Done</code> in the goroutine; <code>Wait</code> in the waiter.
<code>atomic</code>	An atomic <code>Store</code> / <code>Swap</code> / <code>CompareAndSwap</code> / <code>Add</code> with release semantics hb a <code>Load</code> that observes the stored value (acquire). In Go, all <code>sync/atomic</code> ops are sequentially consistent (Section 4.2); the spec guarantees at least this edge.
<code>Cond</code>	<code>Broadcast</code> / <code>Signal</code> hb a <code>Wait</code> that returns because of that signal (the waiter must hold the lock before <code>Wait</code> and re-acquires it after).
<code>Pool</code> , <code>Map</code>	No general hb edge for arbitrary <code>Put</code> / <code>Get</code> or <code>Store</code> / <code>Load</code> beyond what atomics/mutexes inside them provide. <code>sync.Map</code> gives per-key atomicity, not cross-key ordering. <code>sync.Pool</code> may drop values at any GC.
<code>context.Context</code>	<code>cancel()</code> hb <code><-ctx.Done()</code> returning. This is a channel edge — <code>Done()</code> is a channel closed on cancellation.
<code>finalizer</code> (runtime)	No edge to program goroutines. Do not use finalizers for synchronization.

Subtle but critical ordering constraints:

- `wg.Add` must happen-before `wg.Wait`, not concurrently with it. Calling `Add` inside the goroutine that `Wait` is waiting for is a race on the counter itself.
- `Once.Do` is not a general barrier — code *before* `Do` in the caller does not hb code in `f`, only `f` hb callers after `Do` returns.
- A buffered send does not hb the receive until the receive actually dequeues. Filling a buffered channel and then reading a shared variable without receiving is still racy.

3.1 Happens-before edges between goroutines — end to end





4. Atomics: `sync/atomic` in depth

Atomics are the lightest-weight synchronization: a single word, no kernel wait queue, no goroutine parking — but also the easiest to misuse.

4.1 What the package provides

Since Go 1.19, `sync/atomic` exposes generic typed atomics that should be preferred over the legacy `atomic.AddInt64(&x, 1)` functional API. The functional API remains but is less ergonomic and loses the type safety of the generic wrappers.

Type	Typical use	Key operations
<code>atomic.Int32</code> , <code>Int64</code> , <code>Uint32</code> , <code>Uint64</code> , <code>Uintptr</code>	Counters, sequence numbers, flags	<code>Load</code> , <code>Store</code> , <code>Add</code> , <code>Swap</code> , <code>CompareAndSwap</code>
<code>atomic.Bool</code>	Done flags, shutdown signals	<code>Load</code> , <code>Store</code> , <code>Swap</code> , <code>CompareAndSwap</code>
<code>atomic.Pointer[T]</code>	Lock-free linked structures, config pointers	<code>Load</code> , <code>Store</code> , <code>Swap</code> , <code>CompareAndSwap</code>
<code>atomic.Value</code>	Occasionally- polymorphic config (any)	<code>Load</code> , <code>Store</code> , <code>Swap</code> , <code>CompareAndSwap</code> (1.20+)
Legacy func <code>AddInt64(addr *int64, delta int64)</code> <code>int64</code> etc.	Old code, <code>// go:norace</code> interop	Same semantics, raw <code>*T</code> address

```

package counter

import "sync/atomic"

// Preferred since Go 1.19 – typed, no &x escaping to the legacy API.
type Stats struct {
    requests atomic.Int64
    errors    atomic.Int64
    // Pointer to immutable config – readers never copy, never lock.
    cfg atomic.Pointer[Config]
}

type Config struct {
    TimeoutMs int
    FeatureOn bool
}

func (s *Stats) IncRequest() { s.requests.Add(1) }
func (s *Stats) IncError()   { s.errors.Add(1) }
func (s *Stats) Snapshot() (req, err int64) {
    return s.requests.Load(), s.errors.Load()
}

// Publish new config: writer allocates new *Config, swaps pointer
// atomically.
// Readers Load the pointer and dereference – no lock, no copy of
// Config.
func (s *Stats) UpdateConfig(fn func(*Config) *Config) {
    for {
        old := s.cfg.Load()
        next := fn(old) // fn must not mutate old; return new
        // allocation
        if s.cfg.CompareAndSwap(old, next) {
            return
        }
        // CAS failed – another writer won, retry.
    }
}

func (s *Stats) CurrentConfig() *Config { return s.cfg.Load() }

```

```

// atomic.Value – when the stored type varies or you need any.
// Prefer atomic.Pointer[T] when the type is fixed – it is type-safe
// and
// avoids the interface boxing atomic.Value requires.
var globalConf atomic.Value // holds Config

func storeConfig(c Config) { globalConf.Store(c) }
func loadConfig() Config { return globalConf.Load().(Config) }

```

Rules that still catch seniors:

- `atomic.Value` requires that every `Store` use the same concrete type (or `nil` first then consistent type). Storing `Config` then `*Config` panics. Storing inconsistent types is not a race — it is a panic that the race detector will not warn you about.
- Copying an atomic value copies the word but not its atomicity guarantee. Store atomics as **fields**, not values — `func f(a atomic.Int64)` copies the counter and the copy races with the original. Use `*atomic.Int64` or pass by pointer to struct.
- `atomic.AddInt64` and friends on 386/arm 32-bit require 64-bit alignment. An `int64` that is not 8-byte aligned will panic or tear. The generic types (`atomic.Int64`) handle alignment via their struct layout; `map[K]int64` with atomic ops on values is unsafe for the same reason — map values are not address-stable.

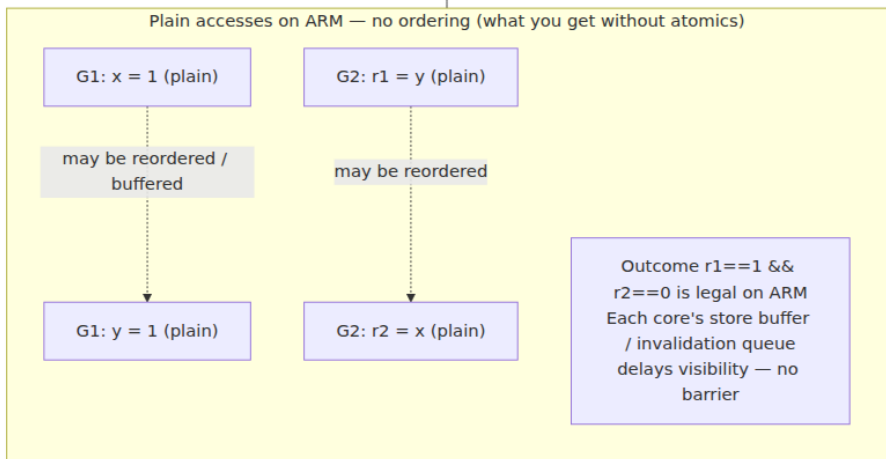
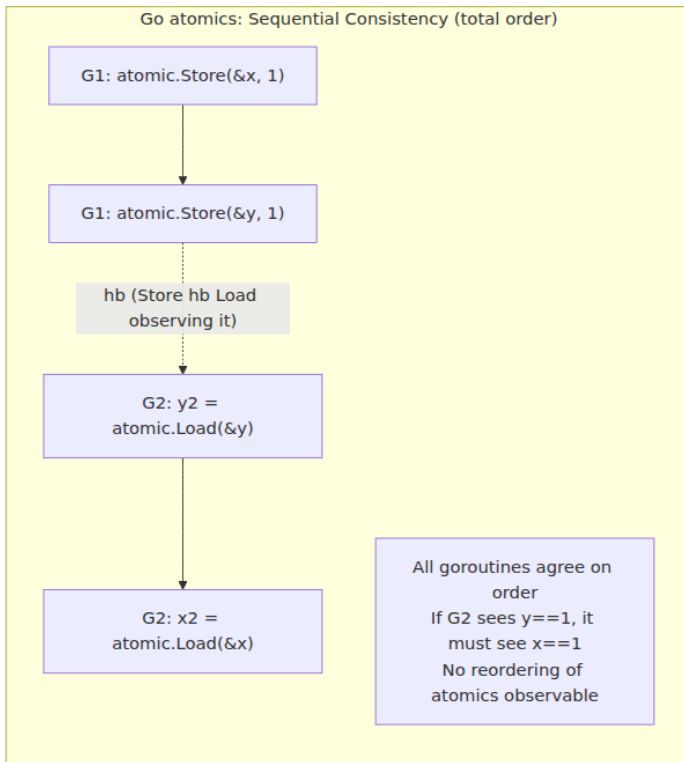
4.2 Ordering: sequential consistency vs. relaxed on ARM

Go's `sync/atomic` provides **sequentially consistent** ordering. This is stronger than C++'s `memory_order_relaxed` or `acq_rel`:

- C++ lets you choose `relaxed` (no ordering), `acquire / release` (pairwise), or `seq_cst` (total order). Choosing wrong compiles and mostly works on x86.
- Go gives you `seq_cst` always. Every atomic operation participates in a single total order that all goroutines agree on, and each atomic `Store` `hb` a `Load` observing it.

What that means on hardware:

- On `amd64` (TSO), `seq_cst` is almost free — stores are already ordered except `StoreLoad`, which needs `MFENCE` or `XCHG`. Go's atomic `Store` on `amd64` uses `XCHG` (locked exchange) which is a full barrier.
- On `arm64` (weak), every atomic op needs explicit barriers. Go emits `DMB / LDA / STL` sequences. A plain (non-atomic) write has no barrier, so another core's plain read can be reordered arbitrarily.



Practical consequence for backend code: Do not reason about atomics as "it will be fast enough to use plain loads on x86." The Go spec does not bless x86-only reasoning. Code that omits atomics because "amd64 is TSO" breaks when you deploy to Graviton (arm64) — which is increasingly the default in cost-optimized fleets. Use atomics for cross-goroutine visibility; let the compiler emit the barrier.

4.3 Alignment requirements

`sync/atomic` on 64-bit values requires 64-bit alignment. The failure mode is not a race — it is a panic on some platforms and silent tearing on others.

```
type bad struct {
    a int32
    b int64 // offset 4 on 32-bit, not 8-aligned if struct is packed
}
var x bad
// atomic.AddInt64(&x.b, 1) – PANICS on 386/arm with "unaligned 64-bit
// atomic operation"

type good struct {
    a int32
    _ int32 // padding
    b atomic.Int64 // struct guarantees alignment; or use atomic.Int64
    directly
    // Alternative: reorder fields – place int64 first.
}
```

Rules:

- Fields of type `atomic.Int64 / Uint64` are aligned by the compiler. Using the generic type eliminates the manual concern.
- If you use the legacy `atomic.AddInt64(&x, ...)` with `*int64`, `x` must be 8-byte aligned. The first field of a struct, a global, or a heap allocation generally is; a field after an `int32` may not be.
- `atomic.Pointer[T]` is pointer-sized and pointer-aligned — no special concern.
- Map values, slice elements at arbitrary offsets, and `unsafe`-derived pointers may not satisfy alignment — do not apply atomics to them.

4.4 The ABA problem

`CompareAndSwap` compares by **value equality**, not by identity of execution history. Between your `Load` and `CAS`, another goroutine can change `A → B → A`; your `CAS` succeeds even though the world mutated underneath you. For a bare counter or flag this is harmless. For a linked structure (stack, queue, freelist) it is a correctness bug: the pointer you are about to swing may now point at reclaimed or reused memory.

```
// ABA-prone Treiber stack sketch – simplified to show the hole.
type Node struct {
    val int
    next *Node
}
var top atomic.Pointer[Node]

func push(n *Node) {
    for {
        old := top.Load()
        n.next = old
        if top.CompareAndSwap(old, n) {
            return
        }
    }
}

func pop() *Node {
    for {
        old := top.Load()
        if old == nil {
            return nil
        }
        nxt := old.next
        // Between Load and CAS, another goroutine could:
        // pop old, pop nxt, push old back => top == old again (A-
        >B->A)
        // CAS succeeds but nxt is now stale – it may have been freed/
        reused.
        if top.CompareAndSwap(old, nxt) {
            return old
        }
    }
}
```

Why Go is partially insulated:

- Garbage collection reclaims `*Node` only when unreachable, so a stale `nxt` pointer remains valid memory (no use-after-free). This eliminates the classic C/C++ ABA crash, but not the *logical* ABA:

`nxt` may now be deep in a different structure, and swinging `top` to it corrupts the stack.

- The GC does not prevent logical corruption — ABA still violates linearizability.

Mitigations in Go:

- Avoid lock-free structures unless you have measured contention that a `Mutex` cannot handle. A `Mutex` around `push / pop` is correct and often faster under realistic contention (no retry storm).
- If you must go lock-free, tag the pointer with a monotonic counter (double-word CAS — `atomic.Pointer` alone cannot do this; you need `atomic.Value` holding a `{ptr, count}` struct with a CAS loop, or an external sequence). Or use a hazard-pointer / epoch scheme — heavy machinery that rarely pays for backend services.
- Prefer `sync.Map` or a sharded `map + RWMutex` over a hand-rolled lock-free map.

4.5 Atomic vs. mutex — choosing deliberately

Criteria	Atomic	Mutex / RWMutex
Scope	Single word (or <code>Pointer</code> to immutable structure)	Arbitrary critical section, multiple words, invariants across fields
Correctness risk	High — ordering, ABA, alignment, torn reads if you miss an access	Lower — one lock protects all fields, invariant holds between <code>Lock / Unlock</code>
Performance (uncontended)	~15-25 ns (<code>Add</code> on amd64)	~20-30 ns (fast path: single CAS, no parking)
Performance (contended)	Retry loop — can livelock under high contention; no fairness	Queuing + parking — bounded, fair under starvation mode; may block goroutine
Composability	Does not compose — two atomics do not give atomicity across both	Composes — one lock protects N fields atomically

Criteria	Atomic	Mutex / RWMutex
When to use	Counters, flags, sequence IDs, publish of immutable pointer, single-word state machine	Anything spanning >1 word, any invariant (len matches cap , map + counter consistent), any blocking operation inside

Heuristics for backend code:

- Metrics counters, request IDs, shutdown flags → atomic.
- Connection pool, cache map, config with multiple fields mutated together → mutex.
- Publish of new config snapshot → `atomic.Pointer[T]` to immutable `T` (writer allocates, reader loads pointer, no copy).
- Hot read-mostly map with occasional writes → `sync.Map` or `RWMutex` (Section 5.6), not a CAS loop over the whole map.

5. Sync primitives internals

Understanding internals is not trivia — it explains latency outliers, informs sizing, and tells you what `pprof` is actually showing.

5.1 `sync.Mutex` — state word, spinning, starvation

`Mutex` is a small struct with one `int32` state word and a semaphore field for parking. The state word packs four logical fields (constants in `runtime2.go` / `sync/mutex.go`):

```
state int32 = [ waiters count (29 bits) | starving (1b) | woken (1b) |
locked (1b) ]

bit 0: mutexLocked - 1 if locked
bit 1: mutexWoken - 1 if a waiter has been woken and is trying to acquire (avoids thundering herd)
bit 2: mutexStarving - 1 if in starvation mode
bits 3..31: waiter count (number of goroutines parked on sema, shifted by mutexWaiterShift==3)
```

Fast path (uncontended): `Lock()` does `atomic.CompareAndSwap(&m.state, 0, mutexLocked)`. One CAS, no

semaphore, no park. `Unlock()` does `atomic.Add(&m.state, -mutexLocked)` and if no waiters, returns. This is ~20 ns on amd64.

Slow path (contended): The goroutine increments the waiter count and parks on `m.sema` via `runtime_SemacquireMutex` / `runtime_Semrelease`. `Unlock` hands off via semaphore.

Spinning (normal mode): Before parking, a goroutine *spins* — busy-waits in a tight loop trying to CAS the lock — if:

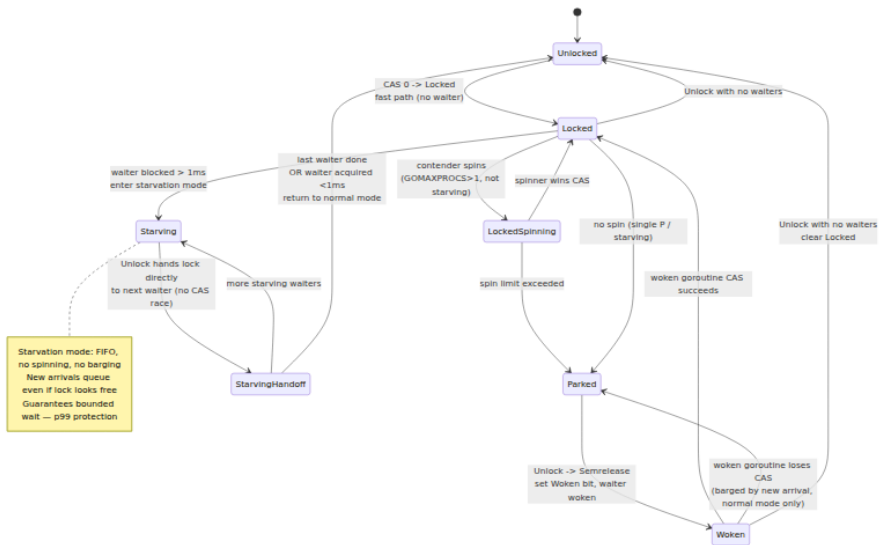
- `GOMAXPROCS > 1` (spinning on a single-P system wastes the only P),
- The current P is not blocked, and
- There is no starvation.

Spinning is bounded (a few iterations, `active_spin` $\approx$ 4). It avoids the cost of parking/unparking ($\approx$ 1-2 μ s) when the holder is about to unlock — typical for short critical sections under moderate contention. This is why a `Mutex` around a small map lookup is often faster than a channel or `RWMutex` under burst load.

Starvation mode (since Go 1.9): The pre-1.9 mutex was unfair — a newly arriving goroutine could barge ahead of waiters by winning the CAS while waiters were waking, causing tail-latency outliers or indefinite starvation. Since 1.9, the mutex tracks how long waiters have been blocked. If a waiter has blocked for more than `starvationThresholdNs` ($\approx$ 1 ms), the mutex enters **starvation mode**:

- Lock is handed directly to the next waiter (no CAS race, no spinning).
- New arrivals do not spin and queue FIFO on the semaphore, even if the lock appears free.
- When the last starving waiter acquires, or a waiter acquires within the threshold, the mutex re-enters **normal mode** (spinning allowed, barging allowed).

Starvation mode trades throughput for fairness and bounds the maximum wait time — exactly what you want for p99.



Mutex tips that follow from the internals:

- Hold time matters more than lock frequency. A mutex held for 10 μ s with 100 contenders causes more parking than one held for 100 ns with 1000 contenders that spin.
- Do not hold a mutex across I/O or channel ops — you block spinners and force starvation.
- `defer mu.Unlock()` adds ~30 ns and prevents inlining in tiny critical sections. In hot paths (metrics, fast cache), explicit `Unlock()` can be measurably cheaper — but only after profiling.

5.2 sync.RWMutex

`RWMutex` allows many concurrent readers or one writer. Internally it is a `Mutex` for writers plus a reader count and two semaphores:

```

type RWMutex struct {
    w      Mutex // held by writers
    writerSem uint32 // writers wait here for readers to drain
    readerSem uint32 // readers wait here when a writer is pending
    readerCount int32 // >0: active readers; <0: writer pending (-
    rwmutexMaxReaders)
    readerWait int32 // readers remaining for departing writer to
    wait on
}

```

- `RLock`: `atomic.Add(&readerCount, 1)`. If `readerCount < 0` (writer pending), park on `readerSem`.
- `RUnlock`: `atomic.Add(&readerCount, -1)`. If it was the last reader a writer is waiting for, wake the writer on `writerSem`.
- `Lock`: `mu.Lock()` (exclusive among writers), then `atomic.Add(&readerCount, -rwmutexMaxReaders)` to block new readers, then wait for `readerWait` to drain.

When `RWMutex` wins and when it loses:

- Wins: read-mostly (95%+ reads), read critical section non-trivial (map lookup, not single word), contention moderate. Readers run fully in parallel — no cache-line bouncing on the mutex word.
- Loses: write-heavy or very short reads — the extra atomics (`readerCount`) make uncontended `RLock` slower than `Lock` (~25 ns vs ~20 ns), and writer starvation handling can add latency. Benchmark your ratio; many "read-mostly" caches are actually write-heavy under thundering herd.
- Never copy an `RWMutex` — like `Mutex`, it must be `*RWMutex` or a field of a struct accessed by pointer. The `go vet copylocks` check catches this.

```
// Sharded map – often better than a single RWMutex for high-
cardinality caches.
type Shard struct {
    mu sync.RWMutex
    m map[string]*Entry
}
type Cache struct {
    shards [64]Shard
}
func (c *Cache) Get(key string) (*Entry, bool) {
    s := &c.shards[hash(key)%64]
    s.mu.RLock()
    e, ok := s.m[key]
    s.mu.RUnlock()
    return e, ok
}
// Each shard has its own reader count – contention spreads, p99 drops.
// Tradeoff: more memory, no cross-shard atomic operation. Choose shard
count ≈ 4× GOMAXPROCS.
```

5.3 sync.WaitGroup — one word plus semaphore

WaitGroup looks like a counter but is a compact state machine:

```
type WaitGroup struct {
    noCopy noCopy
    state1 uint64 // packed on 64-bit: high 32 = counter, low 32 =
waiter count
                // on 32-bit: split across state1 + sema due to
alignment
    sema uint32
}
```

- state1 high 32 bits: counter — Add(n) / Done() (= Add(-1)).
- state1 low 32 bits: waiter count — goroutines blocked in Wait.
- sema : runtime semaphore for parking waiters.

Add(n) does atomic.AddUint64(&state1, uint64(n)<<32). If counter becomes zero and there are waiters, it wakes all waiters via runtime.Semrelease. Wait does atomic.AddUint64(&state1, 1) on the waiter half, checks counter==0, and parks on sema otherwise.

Invariants that produce bugs:

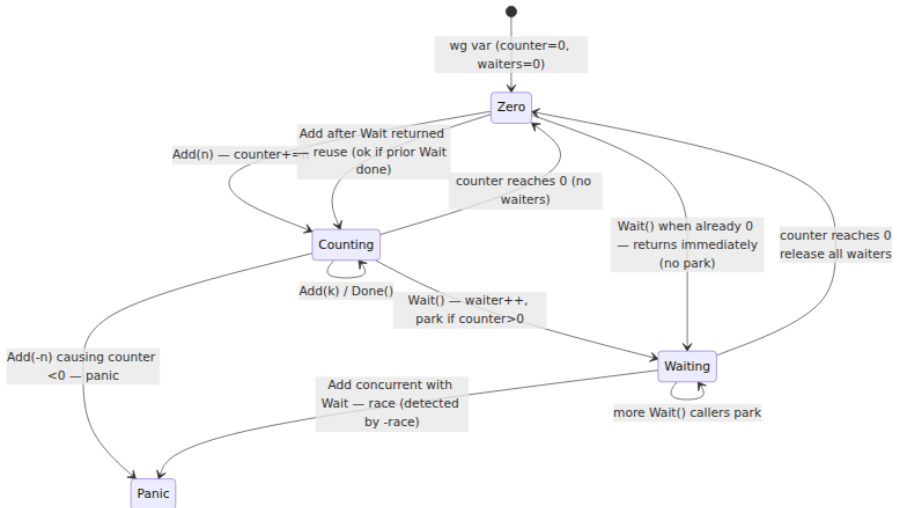
```

// BUG: Add inside the goroutine races with Wait.
var wg sync.WaitGroup
for _, job := range jobs {
    go func(j Job) {
        wg.Add(1) // RACE - Wait may observe counter==0 and return
early
        defer wg.Done()
        process(j)
    }(job)
}
wg.Wait()

// FIX: Add before go, Done inside.
for _, job := range jobs {
    wg.Add(1)
    go func(j Job) {
        defer wg.Done()
        process(j)
    }(job)
}

// BUG: copying a WaitGroup copies state1 - both copies race.
func bad(wg sync.WaitGroup) { wg.Done() } // copy!
// FIX: func good(wg *sync.WaitGroup) { wg.Done() }

```



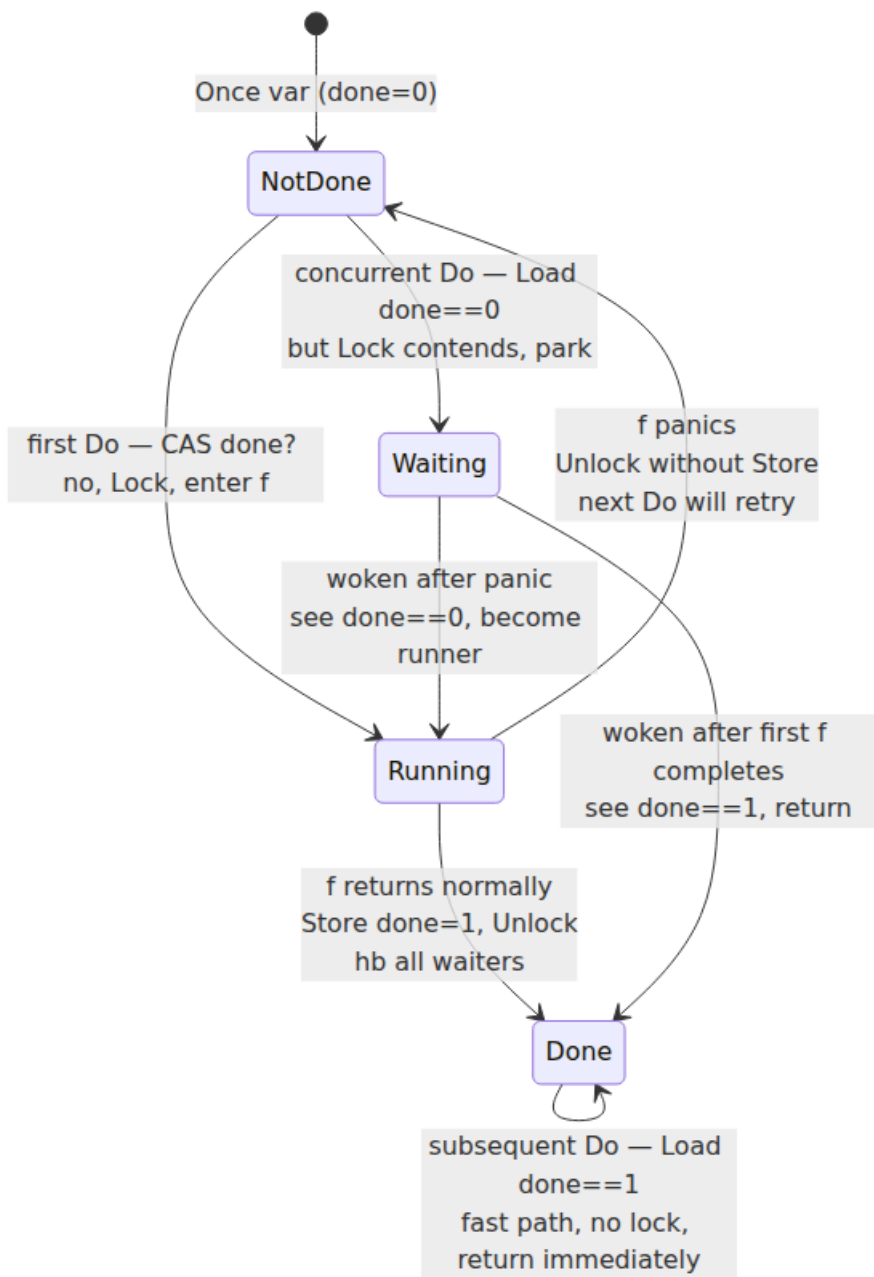
5.4 `sync.Once` — done flag plus mutex, with a correctness guarantee

```
type Once struct {
    done uint32
    m     Mutex
}

func (o *Once) Do(f func()) {
    if atomic.LoadUint32(&o.done) == 0 { // fast path – no lock
        o.doSlow(f)
    }
}

func (o *Once) doSlow(f func()) {
    o.m.Lock()
    defer o.m.Unlock()
    if o.done == 0 {
        defer atomic.StoreUint32(&o.done,
1) // release – hb all future Do returns
        f()
    }
}
```

- Fast path: single `atomic.Load` — ~5 ns when already done. No lock, no contention. This is why `Once` is ideal for lazy singletons on hot paths.
- Slow path: mutex ensures `f` runs exactly once; the `Store(&done, 1)` after `f` completes is the release that hb every `Do` return.
- `Once` does not handle panics inside `f` specially — if `f` panics, `done` is not set, and a later `Do` will retry `f`. This is intentional; a half-initialized singleton should not be cached.
- `Once` is not reusable — there is no `Reset`. If you need resettable once, you need a different primitive (often `atomic.Pointer` + `CompareAndSwap`).



5.5 `sync.Cond` — the one primitive most backends should not use

`Cond` is a condition variable: `Wait` atomically unlocks, parks, and re-locks; `Signal` wakes one waiter, `Broadcast` wakes all. It exists for pipelines where a goroutine must wait for a predicate over shared state:

```
type Queue struct {
    mu    sync.Mutex
    cond  *sync.Cond
    q     []Job
}

func NewQueue() *Queue {
    q := &Queue{}
    q.cond = sync.NewCond(&q.mu)
    return q
}

func (q *Queue) Put(j Job) {
    q.mu.Lock()
    q.q = append(q.q, j)
    q.mu.Unlock()
    q.cond.Signal() // or Broadcast if multiple consumers
}

func (q *Queue) Get() Job {
    q.mu.Lock()
    for len(q.q) == 0 { // must re-check – spurious wakeups and broadcast
        q.cond.Wait() // unlocks mu, parks, re-locks mu on wake
    }
    j := q.q[0]
    q.q = q.q[1:]
    q.mu.Unlock()
    return j
}
```

Why you usually want a channel instead:

- `Cond` requires holding the lock while checking the predicate and calling `Wait`; forgetting the `for` loop creates a missed-wake race.
- It does not compose with `select / context.Context` — you cannot `select` on a `Cond`. Channels and `context` are the idiomatic cancellation-aware wait.
- Internally `Cond` is a semaphore + waiter list (`notifyList` in `runtime`). It is correct but rarely the simplest correct tool. Prefer `chan struct{}` or `chan T` for one-shot signals, and `context` for

cancellation — reserve `Cond` for bounded queues or state machines where broadcast-to-many matters and `select` is not needed.

5.6 `sync.Map` — read/dirty, miss counting, expunged

`sync.Map` is not "a concurrent map." It is a **read-optimized, eventually-consistent** map specialized for two patterns: (1) keys written once and read many times (config, service discovery), (2) disjoint key sets per goroutine (per-connection state). For general read/write mixed workloads, `map + Mutex` or `map + RWMutex` is often faster.

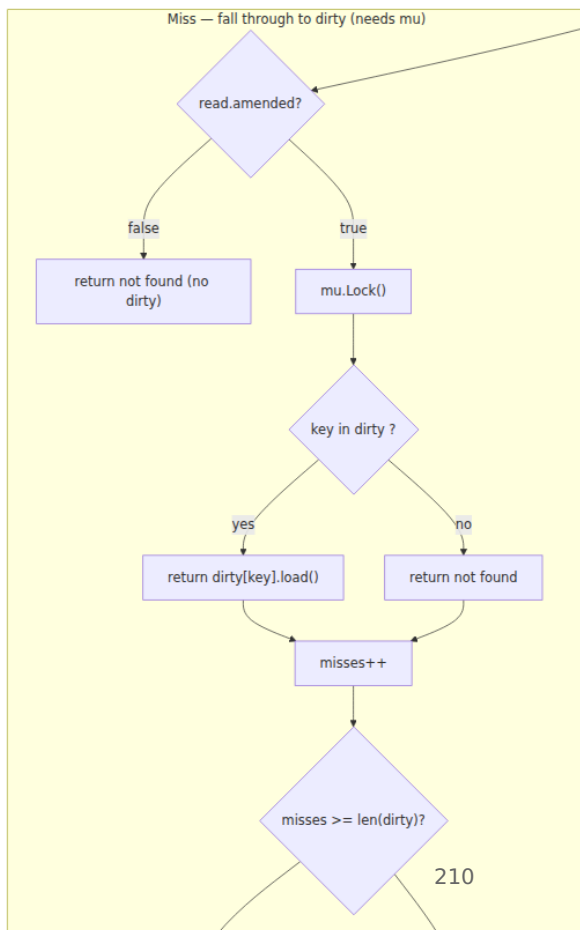
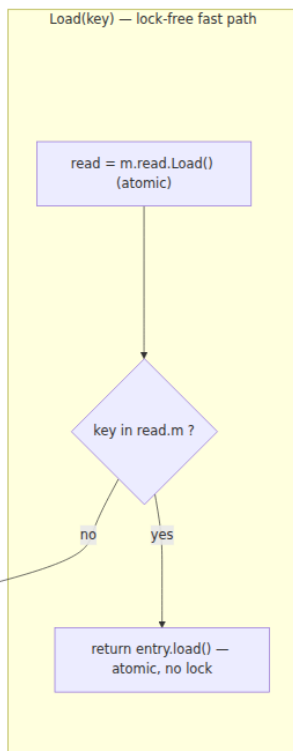
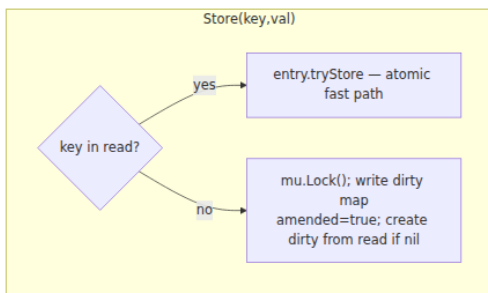
Internals (`sync/map.go`):

```
Map {
    mu      Mutex
    read    atomic.Pointer[readOnly] // lock-free read path
    dirty   map[any]*entry           // guarded by mu, holds recent wr
ites
    misses  int                     // read misses that fell through
to dirty
}

readOnly { m map[any]*entry; amended bool }
// amended==true means dirty contains keys not in read

entry { p unsafe.Pointer } // nil: deleted, expunged: logically delete
d & absent from dirty
```

- **Read path (Load):** `read.Load()` (atomic pointer, no lock). If key in `read.m`, return `entry.load()` (single atomic load). Fast path: ~15 ns, no lock, no contention.
- **Read miss:** Key not in `read` but `read.amended==true` → lock `mu`, look in `dirty`, increment `misses`. If `misses >= len(dirty)`, **promote:** `read.Store(dirty)` and reset `dirty=nil`, `misses=0`. Promotion copies the dirty map into the lock-free read side, so subsequent reads of those keys become fast.
- **Write path (Store):** If key in `read`, try `entry.tryStore` (atomic). Otherwise lock `mu`, double-check, write to `dirty` (creating it from `read` if needed), set `read.amended=true`.
- **expunged :** A sentinel pointer meaning "deleted from `read` and not yet resurrected in `dirty`." Deleting a key in `read` sets `entry.p = nil`; deleting a key already expunged is a no-op. This avoids dirtying `read` on deletes of absent keys.



Consequences for backend choice:

- `sync.Map` has no snapshot, no `Len` without locking, no iteration ordering, and `Range` may observe a mix of read and dirty. It is not a drop-in for `map`.
- Write-heavy workloads thrash the miss counter and promote on every read — slower than `RWMutex`. Benchmark with your actual read/write ratio; the stdlib's own benchmark shows `RWMutex` winning up to ~80% reads.
- For typed maps, Go 1.19+ offers `sync.Map` only as `any`; a generic `sync.Map[K,V]` wrapper or a sharded `map[K]V + RWMutex` with generics is often cleaner and faster.

5.7 `sync.Pool` — per-P local cache plus victim

`Pool` is for **transient, per-request scratch** — buffers, decoders, small structs — to reduce GC pressure. It is not a cache, not a pool of connections, and not a place to store state that must survive.

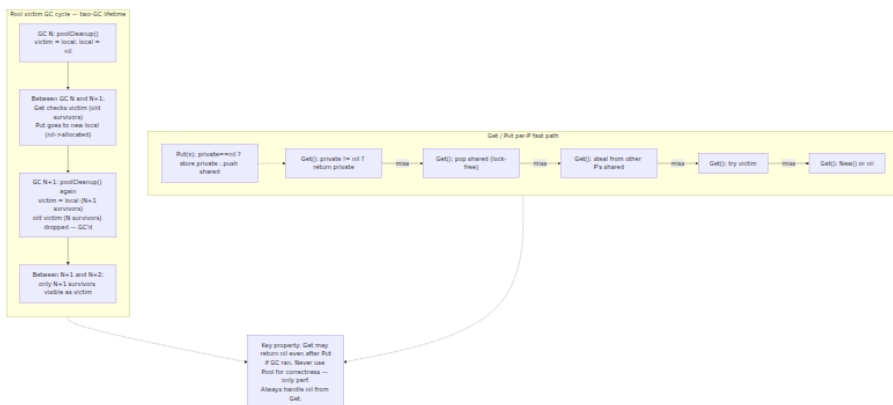
Internals (`sync/pool.go`):

```
Pool {
    noCopy
    local unsafe.Pointer // [P]poolLocal, one per P accessed with
    // lock on owner P
    localSize uintptr
    victim unsafe.Pointer // previous local, drained on next GC
    victimSize uintptr
    New func() any
}

poolLocal {
    private any // single slot, owner P only no lock, no atomic
    shared poolChain // lock-free linked list of shared buffers, stealable by other Ps
}
```

- **Get**: Try `private` of current `P` (no atomic, no lock). Then pop from `shared` (lock-free). Then steal from another `P`'s `shared` (atomic steal). Then `victim` (previous GC's survivors). Then `New` or `nil`.
- **Put**: Store in `private` if empty, otherwise push to `shared`.
- **GC interaction**: On each GC, the runtime calls `poolCleanup`: `victim = local`; `victimSize = localSize`; `local = nil`; `localSize = 0`. So

objects survive at most **two GCs** — one as `local`, one as `victim` — then are dropped. This bounds memory without explicit sizing, but means a `Pool` that is not used between GCs appears empty.



Backend pitfalls with `Pool` :

```
// Correct: request-scoped scratch with size cap and reset.
var bufPool = sync.Pool{
    New: func() any { return bytes.NewBuffer(make([]byte, 0, 4096)) },
}

func handleRequest(b []byte) {
    buf := bufPool.Get().(*bytes.Buffer)
    defer func() {
        if buf.Cap() > 64*1024 {
            // Do not return huge buffers — they pin memory across GCs
            via victim.
            return
        }
        buf.Reset()
        bufPool.Put(buf)
    }()
    // use buf ...
}

// BUG: Pool holding *http.Request or *sql.Conn — leaks state across requests.
var connPool sync.Pool // wrong abstraction — use a real pool with Close/HealthCheck
// Pool entries survive GC nondeterministically; a DB connection held in Pool
// may be closed by the server while pooled, then returned broken.

// BUG: Pool of []byte without length reset — next Get sees stale data.
// Always Reset / slice to :0 before Put.
```

Pool sizing guidance (Section 7) follows from the victim cycle: you cannot "size" a Pool like a bounded queue; you cap what you return and let GC do the reclamation.

6. Correctness: the race detector, map panics, and hero fixes

6.1 The race detector (`go run -race` , `go test -race` , `go build -race`)

Go's race detector is a hybrid of the ThreadSanitizer (TSan) instrumentation and Go-specific happens-before tracking. At compile time (`-race`), the compiler inserts calls before every memory access and at every synchronization point; at runtime, a vector-clock algorithm tracks which goroutine last accessed each word and whether an `hb` edge separates accesses.

Enabling it:

```
go test -race ./...           # all tests, with race
instrumentation
go run -race ./cmd/server      # local run
go build -race -o server.race ./cmd/server # binary with TSan runtime
CGO_ENABLED=1 go test -race ./... # requires cgo (TSan runtime is C+
```

`CGO_ENABLED=0` and `-race` are incompatible — the detector's runtime is written in C++ and linked via cgo.

Overhead (measure it, do not guess):

Metric	-race cost (typical backend)
CPU	2-10× (instrumentation + vector clocks)
Memory	5-10× (shadow memory mapping)
Binary size	larger (instrumented)
Latency	tail inflated — do not load-test with <code>-race</code>

This is why you run `-race` in CI and pre-merge, not in production. The detector is precise (no false positives for Go races) but not complete — it only reports races whose interleavings it actually observes in that run.

Reading the output:

```
=====
WARNING: DATA RACE
Write at 0x00c0000a4010 by goroutine 7:
    main.(*Cache).Set()
        /home/app/cache.go:42 +0x84
    main.writer.func1()
        /home/app/main.go:19 +0x3a

Previous read at 0x00c0000a4010 by goroutine 8:
    main.(*Cache).Get()
        /home/app/cache.go:31 +0x5c
    main.reader()
        /home/app/main.go:27 +0x6e

Goroutine 7 (running) created at:
    main.main()
        /home/app/main.go:18 +0x12a

Goroutine 8 (running) created at:
    main.main()
        /home/app/main.go:23 +0x18a
=====
```

The report names the racy address, both goroutines' stacks with file:line, and the creation site of each goroutine. The fix is to add an `hb` edge between the conflicting accesses — usually a mutex or channel that already exists but was not held on one side.

6.2 Map concurrent read/write — panic, not just a race

Go maps are not safe for concurrent use. The runtime detects concurrent read+write or write+write and **panics**:

```
fatal error: concurrent map read and map write
fatal error: concurrent map writes
```

This is intentional — it turns a silent corruption (torn bucket, lost key) into a crash you can alert on. The detector for this is separate from `-race`; it fires even without `-race` because the map header tracks a writer flag (`hashWriting`).

The subtle variant: concurrent read/read is safe, but read during a write is not. Iterating a map (`for k, v := range m`) while another goroutine writes also panics — iteration holds an implicit read.

Fixes: `sync.Mutex` / `RWMutex` around all map accesses, `sync.Map` for read-mostly, or sharding. There is no "concurrent map" in the stdlib beyond `sync.Map`.

6.3 Happens-before violations — the common patterns

1. Publication without synchronization (the `a/done` example, Section 2). Fix: channel, atomic, or mutex — not `time.Sleep`.

```
// BUG: sleep is not synchronization.
var ready bool
var data string
go func() { data = "hello"; ready = true }()
time.Sleep(10 * time.Millisecond) // "usually enough" – still a race
println(data)                    // may be ""
```

2. Loop variable capture (fixed in Go 1.22, still lurks in older code and closures). Before 1.22, the loop variable was reused; goroutines captured the same address.

```
// Before Go 1.22 – BUG without j := j
for _, j := range jobs {
    go func() { process(j) }() // all goroutines may see last j
}
// With Go 1.22+ the per-iteration variable is new each iteration – safe.
// Still capture explicitly when targeting older toolchains.
```

3. `WaitGroup.Add` concurrent with `Wait` (Section 5.3).

4. Copying `Mutex` / `WaitGroup` / `Cond` by value. The vet check `copylocks` flags this; enable it in `golangci-lint`.

5. `time.After` / `Ticker` without happens-before. A timer firing does not hb the handler's read of shared state.

6.4 Hero examples — races found and fixed in production shape

Hero 1 — Config reload losing updates.

Before:

```

type Server struct {
    cfg
    *Config          // written by reloader, read by handlers – no sync
    mu sync.Mutex
    data map[string]string
}
func (s *Server) Reload(cfg *Config) { s.cfg = cfg } // racy write
func (s *Server) Handle(w http.ResponseWriter, r *http.Request) {
    timeout := s.cfg.Timeout // racy read – may see half-written
    // ...
}

```

Race detector output:

```

WARNING: DATA RACE
Write at 0x00c000... by goroutine 12: Server.Reload()
Previous read at 0x00c000... by goroutine 34: Server.Handle()

```

After:

```

type Server struct {
    cfg atomic.Pointer[Config]
    // ...
}
func (s *Server) Reload(cfg *Config) { s.cfg.Store(cfg) }
func (s *Server) Handle(w http.ResponseWriter, r *http.Request) {
    cfg := s.cfg.Load()
    timeout := cfg.Timeout // cfg is immutable – no further sync needed
    // ...
}
// Invariant: Config is immutable after Store. Writer allocates new
*Config.

```

Why this fix (and not `RWMutex`): `Config` is read 10K× per second per handler, written once per minute. `atomic.Pointer` gives lock-free, zero-contention reads and a single-word publish. An `RWMutex` would work but adds atomic `readerCount` traffic to every handler. The mutex remains for `data`, which is read/write mixed.

Hero 2 — Counter losing increments under load.

Before:


```
var ops int64
func handle() { ops++ } // plain increment – not atomic
// Under 32 concurrent handlers: expected 1M, observed ~720K (lost
// updates).
// On arm64 32-bit: torn reads produce wildly wrong values.
```

After:

```
var ops atomic.Int64
func handle() { ops.Add(1) }
// Or, when batching: ops.Add(int64(batchSize)) – single atomic per
// batch.
```

Detector: `go test -race` flagged the `ops++` read-modify-write as two accesses concurrent with `Load` in the metrics exporter.

Hero 3 — Bounded queue with `Cond` missed wake.

Before:

```
func (q *Queue) Get() Job {
    q.mu.Lock()
    if len(q.q) == 0 { // BUG: if, not for – spurious wakeup or
        broadcast loses
        q.cond.Wait()
    }
    // len may still be 0 – index panic or return zero Job
    j := q.q[0]
    // ...
}
```

After:

```
func (q *Queue) Get() Job {
    q.mu.Lock()
    for len(q.q) == 0 { // re-check predicate after every wake
        q.cond.Wait()
    }
    j := q.q[0]
    q.q = q.q[1:]
    q.mu.Unlock()
    return j
}
// Better still for most backends: replace Cond with channel.
// ch := make(chan Job, 128) – Get becomes <-ch, Put becomes ch <- j,
// select works with context cancellation, no predicate loop needed.
```

Hero 4 — `sync.Map` promoted on every read under write-heavy load.

Before: `sync.Map` used for session table with 50% writes — p99 doubled after promotion fix was investigated.

```
BenchmarkRWMutex-16      45 ns/op
BenchmarkSyncMap-16     110 ns/op // write-heavy – miss counting +
                        promotion dominates
```

After: sharded `map` + `RWMutex` with 64 shards restored p99 and simplified iteration.

```
// Sharded session table – measured win for 50/50 read/write.
type Table struct{ shards [64]struct{ sync.RWMutex; m map[string]Session } }
```

6.5 Running the detector in CI — what to enforce

```
# .github/workflows/race.yml (excerpt)
- name: Race detector
  run: go test -race -count=1 -timeout 10m ./...
  env:
    CGO_ENABLED: "1"
# -count=1 disables test caching – races hidden by cached passes
# reappear.
# -timeout generous – -race is slower; flaky timeouts mask races.
```

Additional vet checks to gate:

```
go vet -copylocks -atomic -assign ./...    # vet's race-adjacent
checks
# atomic check flags: atomic.AddInt64(&x) where x is not aligned / not
address-taken correctly
```

7. Backend lens: choosing primitives, profiling contention, Pool sizing

7.1 Choosing per access pattern

Pattern	Best primitive	Why
Single counter / flag	<code>atomic.Int64</code> / <code>atomic.Bool</code>	Minimal overhead, no parking
Publish immutable snapshot (config, routing table)	<code>atomic.Pointer[T]</code> to immutable <code>T</code>	Lock-free readers, single-writer publish without copy
Read-mostly map, rare writes, no cross-key invariant	<code>sync.Map</code>	Lock-free reads, promotion amortizes if writes truly rare
Read-mostly map, occasional bulk reload	<code>atomic.Pointer[map[K]V]</code> + copy-on-write	Readers snapshot the map pointer; writer builds new map and swaps — no per-key overhead
General map with mixed read/write	<code>map</code> + <code>RWMutex</code> or sharded <code>map</code> + <code>RWMutex</code>	Predictable, debuggable, iteration safe
Short critical section, high burst	<code>sync.Mutex</code> (spinning helps)	Spinning absorbs burst without parking
Long critical section or I/O under lock	Redesign — never hold lock across I/O; if unavoidable, <code>Mutex</code> in starvation mode will serialize fairly	Holding across I/O destroys the fast path for everyone
Per-request scratch (buffers, encoders)	<code>sync.Pool</code>	Reduces GC pressure without manual lifecycle

Pattern	Best primitive	Why
One-shot init (parse templates, open file)	<code>sync.Once</code>	Fast path is a single atomic load
Barrier / fan-out join	<code>sync.WaitGroup</code>	Add before <code>go</code> , Done in goroutine, Wait in joiner — no channel needed
Pipeline / work queue with cancellation	<code>chan T</code> + <code>context.Context</code>	Composes with <code>select</code> , respects cancellation

Anti-patterns to retire:

- `atomic` for multi-field invariants (`if atomic.Load(&ready) { use(a, b) }` where `a` and `b` are not atomically coupled — `b` may still be stale). Use a mutex or publish a struct pointer atomically.
- `sync.Map` as default "concurrent map" — it is not. Start with `map + Mutex`, measure, then consider `sync.Map` or sharding only if read-mostly is proven.
- `sync.Pool` for connections, clients, or anything with `Close` — use a bounded pool (`chan`, `x/sync/singleflight`, or `pool` with health checks) that you control.

7.2 Contention profiling — finding the lock that owns your tail

Go ships two profiles that matter here:

```
# Mutex contention profile — where goroutines blocked on sync.Mutex /
RWMutex
go test -mutexprofile mutex.prof -mutexprofilefraction 1 ./...
go tool pprof -top mutex.prof
# Sample output (synthetic):
# flat flat% sum% cum cum%
# 1.20s 68% 68% 1.20s 68% sync.(*Mutex).Lock
(cache.go:55)
# 0.30s 17% 85% 0.30s 17% sync.(*RWMutex).RLock
# Block profile — where goroutines parked on channels, semaphores,
Cond, WaitGroup
go test -blockprofile block.prof -blockprofilefraction 1 ./...
go tool pprof -http=:8080 block.prof
```

Enable at runtime:

```
import "runtime"

func init() {

runtime.SetMutexProfileFraction(1) // sample every contention event (1
= 100%)
    runtime.SetBlockProfileRate(1)    // sample every block event
    // In production, use 1000 or higher to reduce overhead; 1 is for
    investigation.
}
```

`pprof` labels `contentions` vs `delay` — `contentions` counts events, `delay` is total blocked time (nanoseconds). A lock with many contentions but low delay is hot but short; one with few contentions but huge delay is held too long (often across I/O — the fix is to shrink the critical section, not to shard).

Trace complements profiles with timeline:

```
go test -trace trace.out ./...
go tool trace trace.out # open http://localhost:xxxxx - check
"Synchronization blocking" view
```

The trace shows which goroutine held which mutex when another blocked, with nanosecond resolution — essential for diagnosing starvation vs. spinning behavior that aggregates hide.

7.3 Pool sizing for request-scoped scratch — the only Pool you need

You do not size a `Pool` like a bounded queue. The runtime sizes it to `GOMAXPROCS` shards and GC does reclamation. Your job is to decide **what to pool, how large to let entries grow, and when to drop them.**

```
// Production Pool for JSON decode scratch – sized by cap, not count.
var jsonBufPool = sync.Pool{
    New: func() any {
        b := make([]byte, 0, 4<<10) // 4 KiB initial cap – hot request
        size
        return &b
    },
}

func decodeRequest(r io.Reader) (*Request, error) {
    bp := jsonBufPool.Get().(*[]byte)
    defer func() {
        if cap(*bp) > 64<<10 { // 64 KiB cap – drop outliers (large
            uploads)

            return // let GC reclaim; don't pin huge backing array
        }
        *bp = (*bp)[:0] // keep cap, reset len – next Get sees
        empty slice
        jsonBufPool.Put(bp)
    }()
    *bp, _ = io.ReadAll(r) // simplified – use limited reader in
    production
    var req Request
    if err := json.Unmarshal(*bp, &req); err != nil {
        return nil, err
    }
    return &req, nil
}
```

Rules that survive review:

- **Pool only transient, non-stateful scratch:** `[]byte`, `bytes.Buffer`, `sync.Pool`-local encoders. Never pool `*os.File`, `*sql.Conn`, gRPC streams, or `*http.Request`.
- **Cap, not count:** Return entries with bounded `cap`; drop oversized ones. An attacker that sends one 10 MiB request should not pin a 10 MiB backing array in every P's `private` for two GC cycles.
- **Reset before Put:** Clear slices to `:0`, `bytes.Buffer.Reset()`, zero `Header` maps — or you leak data across requests (security) and retain references that prevent GC (memory).
- **Handle nil from Get:** `New` may be nil, GC may have dropped everything. `Get` can return `nil` — the caller must handle it.
- **Measure:** Track `Pool` hit rate indirectly via allocation rate (`runtime.MemStats.Mallocs`, `go tool pprof -alloc_space`). A `Pool`

that never hits is either too small `cap` or emptied by frequent GC (short-lived service with aggressive `GOGC`). Consider raising `GOMEMLIMIT` or `GOGC` before blaming the Pool.

8. Putting it together — a backend checklist

Before merging any concurrent Go code, verify:

1. **Every shared word has an owner or an edge.** If two goroutines access the same variable and one is a write, name the channel/mutex/atomic/Once/WaitGroup that makes them `hb`-ordered. If you cannot name it, the code is racy.
 2. **`-race` passes with `-count=1`.** Cached test results hide races. Run `go test -race -count=1 ./...` in CI on every PR.
 3. **No map is accessed concurrently without `sync`.** The panic is the *good* outcome. Vet with `go vet` and code review.
 4. **No Mutex / WaitGroup / Cond is copied by value.** Enable `copylocks`.
 5. **Atomic Add / Store / Load are paired.** Every `atomic.Store` must have a matching `atomic.Load` on the reader; a plain read beside an atomic write is still a race. The detector flags this — listen to it.
 6. **Pool entries are reset and bounded.** Review every `Put` for `Reset` and `cap` guard.
 7. **Critical sections are short and never span I/O.** If `pprof` shows mutex `delay` large, shrink the section before sharding the lock.
-

Key takeaways

- The Go memory model is a happens-before contract: within one goroutine program order is `hb`; across goroutines only synchronizing operations — channel send→receive, `Lock` → `Unlock` → next `Lock`, `Once.Do(f)` (`f` `hb` all callers), `WaitGroup.Done` → `Wait`, `atomic.Store` → `Load` observing it, `cancel` → `<-ctx.Done()` — create `hb` edges. Without an edge, concurrent read/write is a data race and the program is undefined.
- `sync/atomic` in Go is sequentially consistent on all architectures; on `arm64` every atomic carries a barrier. Use `atomic.Int64 / Bool / Pointer[T]` (Go 1.19+) over the legacy `AddInt64(&x)` API; respect

64-bit alignment via the generic types, and handle `atomic.Value`'s single-concrete-type rule.

- `CompareAndSwap` loops suffer the ABA problem ($A \rightarrow B \rightarrow A$ looks unchanged). Go's GC prevents use-after-free but not logical corruption — prefer a mutex over a hand-rolled lock-free structure unless you have proven contention that justifies the complexity.
- `Mutex` packs `Locked/Woken/Starving` bits + waiter count into one `int32`; uncontended `Lock` is a single CAS, contended path spins briefly (when `GOMAXPROCS > 1` and not starving) then parks on a semaphore, and starvation mode (since 1.9, ~1 ms threshold) hands the lock FIFO to bound tail latency.
- `RWMutex`, `WaitGroup` (counter+waiters in one `uint64` + semaphore), `Once` (done flag + mutex, single `Store` hb all returns), and `Cond` (semaphore + predicate loop) each have a narrow correct-use window — `RWMutex` for read-mostly, `WaitGroup` with `Add` before `go`, `Once` for one-shot init, `Cond` rarely (channels usually better).
- `sync.Map` is a read-optimized two-level map (read lock-free + dirty under `mu`, promotion when `misses >= len(dirty)`, expunged sentinel) and `sync.Pool` is a per-P local + victim cache whose entries survive at most two GCs — neither is a general-purpose concurrent map or bounded resource pool.
- The race detector (`-race`, `CGO_ENABLED=1`) instruments every access and tracks vector clocks; it is precise but incomplete (only reports interleavings it observes) and costs 2-10× CPU / 5-10× memory — run it with `-count=1` in CI, never in production load tests, and supplement with `mutex / block` profiles and `go tool trace` for tail-latency diagnosis.
- Backend primitive selection is access-pattern driven: atomics for single-word publish, `atomic.Pointer` to immutable snapshots for config, `map+RWMutex` or sharding for general maps, `Mutex` for short critical sections, `Pool` only for bounded per-request scratch (reset + cap guard, handle `nil` from `Get`). Profile contention with `SetMutexProfileFraction` and size Pools by `cap` cap, not by entry count.

Further reading

- **(pinned)** Go Memory Model — go.dev/ref/mem — The authoritative spec (2 pages). Defines happens-before, every guaranteed edge, and the data-race rule. <https://go.dev/ref/mem>
- **(pinned)** `sync/atomic` package docs — pkg.go.dev/sync/atomic — Typed atomics (`Int64`, `Pointer[T]`, `Value`), ordering guarantees, and alignment notes. <https://pkg.go.dev/sync/atomic>
- **(pinned)** Data Race Detector — go.dev/doc/articles/race_detector — How TSan instrumentation, `-race` overhead, and report interpretation work, including `CGO_ENABLED` requirement. https://go.dev/doc/articles/race_detector
- **(pinned)** `sync` package docs — pkg.go.dev/sync — `Mutex`, `RWMutex`, `WaitGroup`, `Once`, `Cond`, `Map`, `Pool` API and documented guarantees. <https://pkg.go.dev/sync>
- Go `sync/mutex.go` and `sync/map.go` source — The ground truth for state packing, spinning, starvation threshold, and `read / dirty / expunged` promotion. <https://github.com/golang/go/blob/master/src/sync/mutex.go> and <https://github.com/golang/go/blob/master/src/sync/map.go>
- Go `sync/pool.go` source and `poolCleanup` in `runtime/mgc.go` — Per-P local + victim lifecycle and the two-GC reclamation cycle. <https://github.com/golang/go/blob/master/src/sync/pool.go>
- Dmitry Vyukov, "The Go Memory Model" (GopherCon 2014) — Talk and paper expanding the spec's happens-before formalism with litmus tests. <https://golang.org/ref/mem> and <https://research.swtch.com/gomm>
- Hans-J. Boehm and Sarita Adve, "Foundations of the C++ Concurrency Memory Model" (PLDI 2008) — The academic foundation that Go's model follows; explains why data races make programs undefined and why `seq_cst` is the safe default. <https://doi.org/10.1145/1375581.1375591>
- Herb Sutter, "Atomic Weapons: The C++ Memory Model and Modern Hardware" (C++ and Beyond 2012, two parts) — Best visual explanation of store buffers, SC vs. weak ARM, and acquire/release — directly applicable to Go's barrier emission on `arm64`. <https://herbsutter.com/2013/02/11/atomic-weapons-the-c-memory-model-and-modern-hardware/>

- Russ Cox, "Hardware Memory Models" (research.swtch.com, 2017) — Short note on TSO vs. weak models and why Go chose sequential consistency for `sync/atomic`. <https://research.swtch.com/hwmm>
- Felix Geisendörfer et al., "Profiling Go Mutex Contention" (Go blog, 2017) and `go tool pprof docs — SetMutexProfileFraction`, `contentions` vs. `delay`, and the trace synchronization view. <https://go.dev/blog/pprof> and <https://pkg.go.dev/runtime#SetMutexProfileFraction>

Chapter 7 — Interfaces, Reflection, and `unsafe`

What this chapter covers. Interfaces are Go's only dynamic dispatch mechanism, `reflect` is its only runtime type introspection, and `unsafe` is its only escape hatch below the type system. Every HTTP handler, database driver, serializer, and mock in a backend service touches at least one of them. This chapter explains how interfaces are laid out in memory (`eface` vs `iface`, the `itab` and its hash-guarded cache), how method dispatch actually calls through a function-pointer table, why `nil` interfaces and typed `nil`s diverge, how `reflect.Type/reflect.Value` expose the type system at runtime (and what they cost), and how `unsafe.Pointer/uintptr` let you bypass it — including the zero-copy `string ↔ []byte` trick and when the garbage collector will still punish you for it.

Learning goals — after this chapter you should be able to:

- Distinguish `eface` (empty interface, now `any`) from `iface` (non-empty interface) in memory, describe the `itab` (`_type`, interface type, hash, function table) and explain how it is built, hashed, and cached by the runtime.
- Predict the outcome of type assertions (comma-ok vs panicking form), type switches, and the typed-`nil` pitfall (`var p *T = nil; var i any = p; i != nil`).
- Quantify interface call overhead (indirect load + table dispatch vs direct/generic call), read it in assembly and `benchstat`, and decide when to accept it versus generics or concrete types.

- Use `reflect.Type`, `reflect.Value`, `Kind`, `NumMethod`, `StructTag`, and `Value.Set / Addr / CanSet / CanInterface` correctly, state the three laws of reflection, and explain why `reflect.Value` can force heap escape.
- Explain `unsafe.Pointer` vs `uintptr` conversion rules, use `unsafe.Slice`, `unsafe.String`, `unsafe.StringData`, `unsafe.SliceData`, `unsafe.Offsetof / Sizeof / Alignof` safely, and implement vetted zero-copy `string ↔ []byte` conversions.
- Reason about GC interactions (`runtime.KeepAlive`, `runtime.Pinner / Pin / Unpin` since Go 1.21), `go vet`'s `unsafePtr / ptrconversion` warnings, and the backend trade-offs: interfaces for mocking vs generics for performance, reflection cost in serializers, and unsafe zero-copy in hot paths (protobuf, JSON, I/O buffers).

Placement. Chapter 2 established type identity, memory layout, and `unsafe.Sizeof / Offsetof` fundamentals. Chapter 3 covered the ABI and method sets (`T` vs `*T`) that determine which types satisfy which interfaces. This chapter completes the type-system implementation picture. Chapter 9 (SSA and escape analysis) explains why reflection and interface boxing escape to heap; Chapter 10 (tooling) covers `go vet` analyzers referenced here; Chapter 11 (`cgo/assembly`) builds on `unsafe.Pointer`.

1. Why these three belong together

Go deliberately offers only one abstraction for dynamic behavior (interfaces), one API for runtime type inspection (`reflect`), and one backdoor past the type checker (`unsafe`). In backend systems they appear together:

- A `database/sql.Driver` returns `any` and the caller type-asserts.
- `encoding/json` walks structs with `reflect`, reads `json:"name,omitempty"` tags, and (since Go 1.20) may use `unsafe` to avoid copying bytes on the hot path.
- `net/http.Handler` is an interface; generated gRPC stubs used to box every call through interfaces and now increasingly use generics — the crossover is a performance decision, not a style one.

- Protobuf and flatbuffer codecs use `unsafe.Slice / unsafe.String` to reinterpret `[]byte` frames as strings without allocation.

Understanding the cost model requires seeing all three together: boxing to an interface allocates in some cases; reflecting over it costs more; reaching for `unsafe` removes the allocation but introduces GC and vet obligations.

2. Interface representation: `eface` vs `iface` and the `itab`

2.1 Two layouts, one word of difference

At runtime every interface value is a small header. The runtime source (`src/runtime/runtime2.go`, `src/runtime/iface.go`) defines two structs:

```
// src/runtime/runtime2.go - simplified

// eface - empty interface, now spelled `any` (Go 1.18+).
// Exactly what `any` and `interface{}` lower to.
type eface struct {
    _type *_type           // dynamic type descriptor
    data  unsafe.Pointer    // points to value (or inline value if small)
}

// iface - non-empty interface (has at least one method).
type iface struct {
    tab *itab              // interface table: type + interface type +
    // method table
    data unsafe.Pointer    // same as eface
}

// itab - the dispatch table (src/runtime/runtime2.go)
type itab struct {
    inter
    *interfacetype // static interface type (method set, package, name)
    _type *_type    // dynamic concrete type
    hash  uint32     // copy of _type.hash - speeds type switches/
    // assertions
    _fun [4]byte
    fun  [1]uintptr // variable-length: fun[0] is first method,
    // fun[1] second, ...
}
```

`_type` is the runtime type descriptor (kind, size, GC map, method list, name) covered in Chapter 2. `interfacetype` describes the interface itself (its method set and package path). The crucial difference:

	any / interface{} (eface)	io.Reader , Handler , any single-or-multi-method interface (iface)
Header word 0	<code>*_type</code> (concrete type, or <code>nil</code> if <code>nil</code> interface)	<code>*itab</code> (or <code>nil</code> if <code>nil</code> interface)
Header word 1	<code>data</code> (<code>unsafe.Pointer</code> to value)	<code>data</code> (same)
Method dispatch	none — caller must type-assert	indirect via <code>itab.fun[i]</code>
Size	16 bytes on 64-bit	16 bytes on 64-bit
Type identity check	compare <code>_type</code> pointer + assert	<code>itab.hash</code> fast path, then <code>inter / _type</code> check

```

package main

import (
    "fmt"
    "io"
    "unsafe"
)

type Counter struct{ n int }

func (c *Counter) Inc() { c.n++ }
func (c Counter) Value() int { return c.n }
func (c Counter) Read(p []byte) (int, error) { return 0, io.EOF }

func layoutDemo() {
    var e any = Counter{n: 42}           // eface: (_type=*Counter,
    data-copy of Counter)
    var r io.Reader = Counter{n: 42}     // iface: (itab for (Counter,
    io.Reader), data-copy)

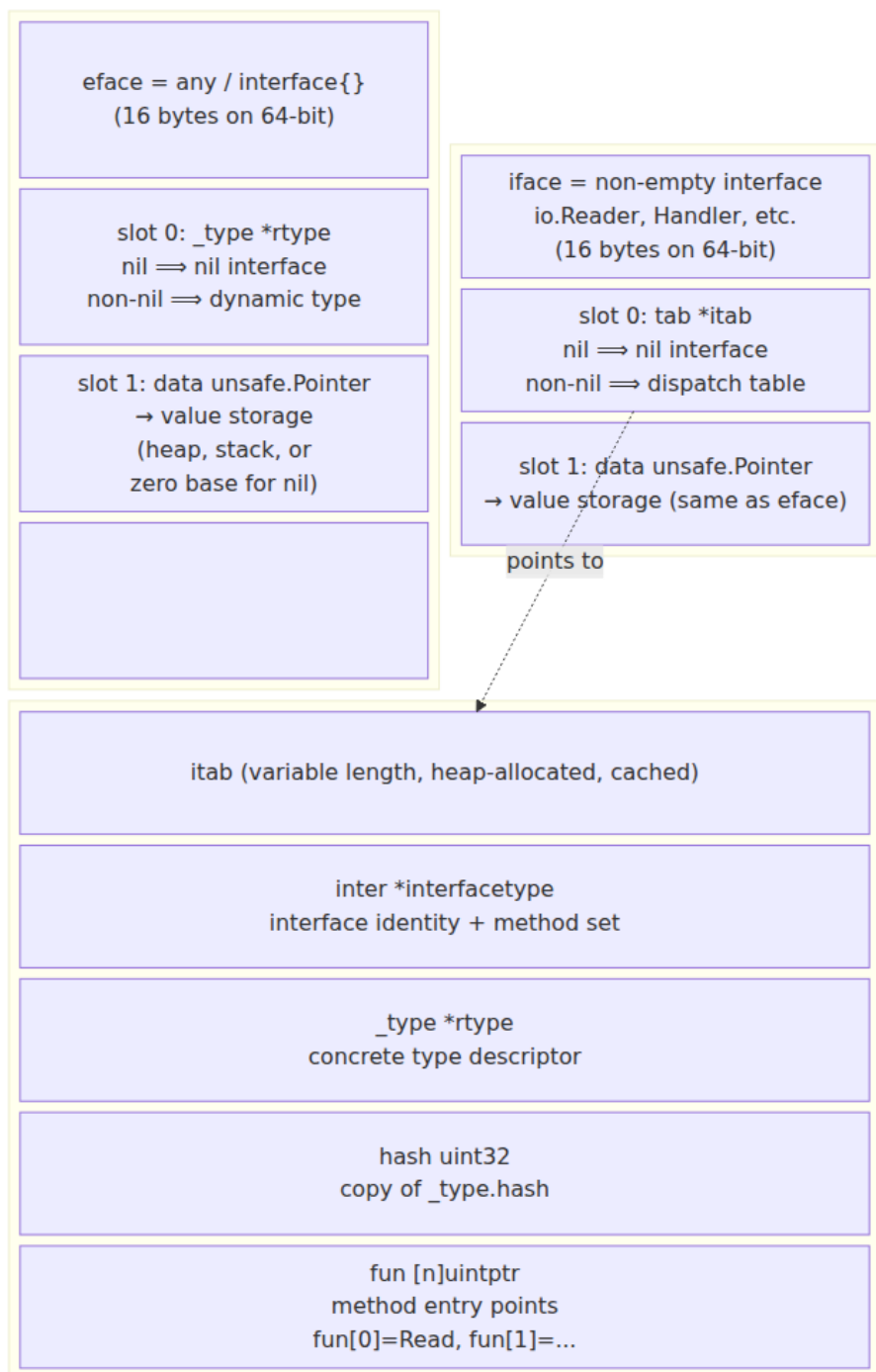
    // Both headers are 16 bytes. Data points to a heap copy if the
    value escapes,
    // or to stack/inline storage if the compiler can prove locality
    (see Ch 9).
    fmt.Printf("any size=%d io.Reader size=%d\n", unsafe.Sizeof(e), un
    safe.Sizeof(r)) // 16 16

    // Inspect headers via unsafe – never do this in production, only
    to learn.
    type eface struct{ _type, data unsafe.Pointer }
    type iface struct{ tab, data unsafe.Pointer }

    ef := (*eface)(unsafe.Pointer(&e))
    it := (*iface)(unsafe.Pointer(&r))
    fmt.Printf("eface._type=%p data=%p\n", ef._type, ef.data)
    fmt.Printf("iface.tab  =%p data=%p\n", it.tab, it.data)

    _ = ef
    _ = it
}

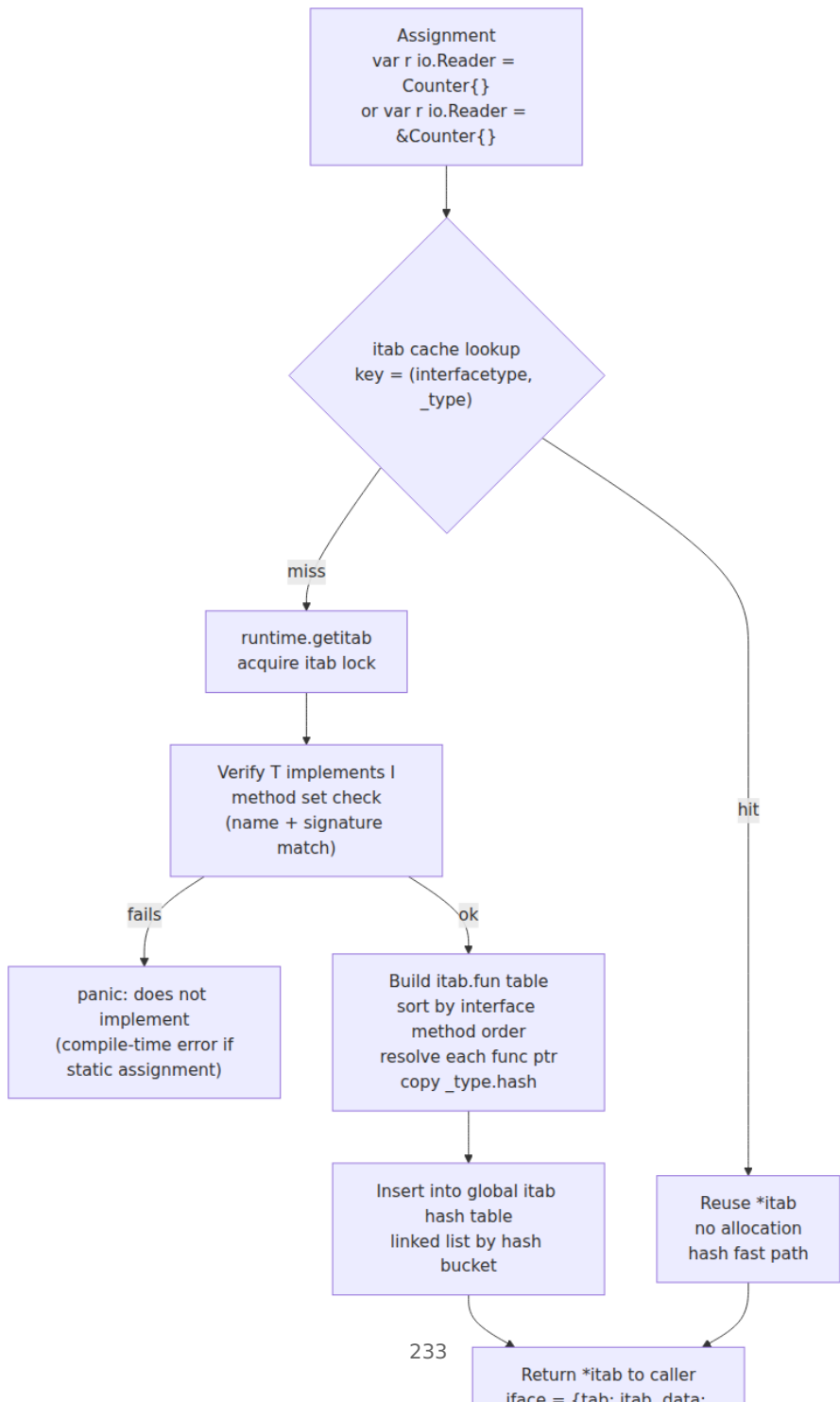
```



Small-value optimization. For values that fit in a pointer (e.g., `int`, `*T`), the compiler may store them directly in the `data` word or avoid allocation. For larger values (`struct { a, b int64 }`), boxing copies the value to the heap so `data` has a stable address after the stack frame returns. Whether boxing allocates is an escape-analysis outcome you can read with `go build -gcflags=-m` — see Chapter 9.

2.2 How the `itab` is built and cached

The first time a concrete type is assigned to an interface type, the runtime must produce the `itab` that witnesses "type `T` implements interface `I`." At compile time the compiler emits a static `interfacetype` for `I` and a `_type` for `T`; at runtime `getitab` (in `src/runtime/iface.go`) either finds a cached entry or builds one.



The global itab table is a hash map keyed by (inter, _type) hashed from inter and _type pointers, chain-probed. Once published, an itab is immutable and never freed — interfaces share it across goroutines without locking on the hit path. The copy of _type.hash in the itab lets type switches compare a single uint32 before chasing pointers.

```
// Demonstrating the cache – repeated assignments share the same itab
// pointer.
func itabCacheDemo() {
    type iface struct{ tab, data unsafe.Pointer }
    var r1 io.Reader = Counter{}
    var r2 io.Reader = Counter{}
    t1 := (*iface)(unsafe.Pointer(&r1)).tab
    t2 := (*iface)(unsafe.Pointer(&r2)).tab
    fmt.Println(t1 == t2) // true – same itab, cached after first
    getitab

    // Different concrete type → different itab, even for same
    // interface.
    type S struct{}
    func (S) Read(p []byte) (int, error) { return 0, nil }
    var r3 io.Reader = S{}
    t3 := (*iface)(unsafe.Pointer(&r3)).tab
    fmt.Println(t1 == t3) // false
}
```

```
$ go build -gcflags=-m ./demo 2>&1 | grep -E 'itab|interface|escape'
# On assignment var r io.Reader = Counter{}:
# ./demo.go:12: Counter{} escapes to heap – boxed value for iface data
# The itab itself is allocated once globally and never per-assignment.
```

Static check vs dynamic check. When the assignment is statically visible (var r io.Reader = Counter{}) the compiler proves Counter implements io.Reader and can emit the itab reference directly (or a convI2I helper). When the source is any (var x any = someValue; var r io.Reader = x.(io.Reader)) the check is deferred to the type assertion at runtime and uses the same cache.

3. Type assertions, type switches, and the typed-nil trap

3.1 Two forms of assertion — know which panics

```
package main

import "fmt"

type Store interface{ Get(key string) (string, bool) }

type MemStore map[string]string

func (m MemStore) Get(key string) (string, bool) { v, ok := m[key]; return v, ok }

func assertionsDemo(v any) {
    // Panicking form – use only when you can prove the type.
    s := v.(MemStore) // panics with: interface conversion: any is X,
    not MemStore
    _ = s

    // Comma-ok form – always prefer in backend code (request handlers,
    decoders).
    if s, ok := v.(MemStore); ok {
        fmt.Println("mem:", s)
    } else {
        fmt.Println("not a MemStore")
    }

    // Interface-to-interface assertion – checks method set inclusion.
    if st, ok := v.(Store); ok {
        fmt.Println("is a Store:", st.Get("k"))
    }
}
```

Runtime mechanics: the assertion `v.(T)` loads `eface._type` (or `iface.tab._type`) and compares it against `T`'s `_type` pointer (fast path) or walks the itab hash. On mismatch the comma-ok form returns `(zero, false)`; the single-value form calls `runtime.panicdottype` with a formatted message that includes both dynamic and target type names — useful in logs but fatal if unrecovered.

3.2 Type switches — the hash-accelerated dispatch

```
func handle(v any) string {
    switch x := v.(type) {
    case nil:
        return "nil interface"
    case string:
        return "string: " + x
    case []byte:
        return fmt.Sprintf("bytes len=%d", len(x))
    case MemStore:
        return fmt.Sprintf("memstore len=%d", len(x))
    case Store: // interface case – matched via itab
        s, _ := x.Get("hello")
        return "store: " + s
    default:
        return fmt.Sprintf("unknown %T", x)
    }
}
```

The compiler lowers a type switch to a hash jump table when there are many cases. For `eface` the key is `_type.hash`; for `iface` it is `itab.hash`. The hash lets the switch probe in $O(1)$ for the common case before falling back to linear comparison of type pointers. Order cases from most frequent to least — the generated code tests in source order after the hash probe, so hot-path-first reduces branch mispredicts in decoders and routers.

3.3 The typed-nil pitfall — the most common interface bug in production

```

package main

import (
    "errors"
    "fmt"
)

type AppError struct{ Msg string }

func (e *AppError) Error() string { return e.Msg }

func returnsTypedNil() error {
    var p *AppError = nil // typed nil: (*AppError)(nil)
    return p               // iface{ tab: itab(*AppError, error), data:
// nil } - NOT nil!
}

func returnsNilInterface() error {
    return nil // eface{ _type: nil, data: nil } - nil
}

func nilPitfallDemo() {
    err1 := returnsTypedNil()
    err2 := returnsNilInterface()

    fmt.Println(err1 == nil) // false - itab is non-nil!
    fmt.Println(err2 == nil) // true

    // Defensive pattern - return nil explicitly.
    check := func(ok bool) error {
        var p *AppError
        if !ok {
            return nil // do NOT return p when p may be nil
        }
        p = &AppError{Msg: "boom"}
        return p // (*AppError)(0xc000...) - non-nil data, correct
    }
    fmt.Println(check(false) == nil) // true
    fmt.Println(check(true))         // boom

    // Detecting a typed nil that slipped through.
    if err1 != nil {
        // err1 is non-nil interface wrapping nil pointer - calling
        // Error panics:
        // err1.Error() → nil pointer dereference
        fmt.Printf("err1 type %T value %v\n", err1, err1)
        // Safe unwrap:
        var appErr *AppError
        if errors.As(err1, &appErr) {
            if appErr == nil {

```

```
        fmt.Println("typed nil – treat as nil error")
    }
}
}
```

Why this happens — the second diagram the task requires in full:



Rules to avoid the bug:

1. Never `return errVal` where `errVal` is a typed nil variable. Check if `errVal == nil { return nil }` before returning an `error`.
2. In constructors that return `(T, error)`, make the error return bare `nil` on success — do not return a `*MyError` variable that may be nil.
3. Linters catch many cases: `staticcheck` SA4022, `nilness` analyzer. Enable `go vet` and `staticcheck` in CI — see Section 6.5.

4. Interface call dispatch — what one method call really does

4.1 The indirection chain

```
type Writer interface{ Write(p []byte) (int, error) }

type Buffer struct{ buf []byte }

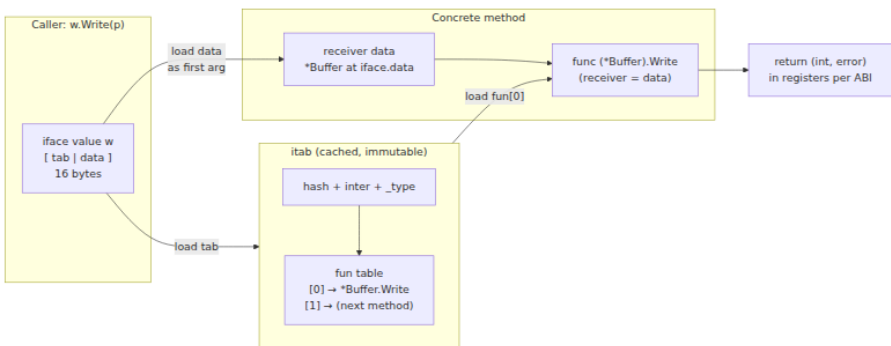
func (b *Buffer) Write(p []byte) (int, error) { b.buf = append(b.buf, p...); return len(p), nil }

func callViaInterface(w Writer, p []byte) (int, error) {
    return w.Write(p) // interface dispatch – not a direct call
}

func callDirect(b *Buffer, p []byte) (int, error) {
    return b.Write(p) // direct call – compiler knows callee
}
```

`w.Write(p)` compiles to roughly:

```
load iface.tab          // 1: get itab pointer from header word 0
load iface.data         // 2: get receiver pointer from header word 1
load itab.fun[0]        // 3: load function pointer for Write (index by in
                           interface method order)
call fun(data, p)       // 4: indirect call   receiver is first arg (regi
                           ster ABI: AX/DX per Ch 3)
```



Three loads and an indirect branch versus a direct `CALL` immediate. The indirect branch is the expensive part: it defeats the branch predictor on first encounter, prevents inlining, and blocks devirtualization (the

compiler cannot inline through an interface call except via PGO-guided or guarded devirtualization in Go 1.21+).

`go tool compile -S` shows the difference (amd64, Go 1.22, simplified):

```
// Direct: CALL is immediate, inlineable
TEXT ·callDirect(SB), $0-32
    MOVQ b+0(FP), AX      // receiver
    CALL ·(*Buffer).Write(SB) // direct - linker resolves

// Interface: three loads + indirect CALL
TEXT ·callViaInterface(SB), $0-32
    MOVQ w+0(FP), AX      // itab
    MOVQ w+8(FP), BX      // data
    MOVQ 24(AX), CX       // itab.fun[0] at offset 24
    CALL CX               // indirect - not inlineable
```

4.2 How much does it cost — a benchmark you can run

```
package bench_test

import "testing"

type Adder interface{ Add(a, b int) int }

type S struct{}

func (S) Add(a, b int) int { return a + b }

//go:noinline
func addDirect(s S, a, b int) int { return s.Add(a, b) }

//go:noinline
func addIface(a Adder, x, y int) int { return a.Add(x, y) }

// Generic — monomorphized to direct call (see Ch 2 stenciling)
func AddGeneric[T interface{ Add(int, int) int }](t T, a, b int) int {
    return t.Add(a, b) }

func BenchmarkDirect(b *testing.B) {
    s := S{}
    for i := 0; i < b.N; i++ {
        _ = addDirect(s, 3, 4)
    }
}

func BenchmarkIface(b *testing.B) {
    var a Adder = S{}
    for i := 0; i < b.N; i++ {
        _ = addIface(a, 3, 4)
    }
}

func BenchmarkGeneric(b *testing.B) {
    s := S{}
    for i := 0; i < b.N; i++ {
        _ = AddGeneric(s, 3, 4)
    }
}
```

Typical results on `linux/amd64`, Go 1.22, `-benchmem`:

BenchmarkDirect-8	1000000000	0.62 ns/op	0 B/op	0 allocs/
op // inlined, often eliminated				
BenchmarkIface-8	350000000	3.2 ns/op	0 B/op	0 allocs/
op // ~5× slower than direct				
BenchmarkGeneric-8	1000000000	0.65 ns/op	0 B/op	0 allocs/
op // generic == direct				

In isolation the delta is a few nanoseconds. In a real handler that does I/O, the constant factor is noise. Where it matters:

- **Tight loops** — per-element `io.Writer` or `hash.Hash` calls over millions of items.
- **Allocations from boxing** — `func F(any)` where the caller passes a value type boxes and may allocate; `func F[T any](T)` does not.
- **Inlining boundary** — interface calls never inline; hot middleware wrapped in `http.Handler` interfaces pays this per request.

Backend guidance: Use interfaces at system boundaries (handlers, repositories, `Clock` / `DB` for testing) where the indirection buys testability and the call is not in a tight loop. Use generics or concrete types in hot inner loops (codec fast paths, sort comparators, batch processors) where nanoseconds × millions matter. The benchmark above is the template — always `benchstat` your own call site before generalizing.

5. Reflection — `reflect.Type`, `reflect.Value`, and the laws

5.1 What `reflect` exposes

`reflect` reifies the compile-time type graph (`_type`, `rtype`, `imethod`) into two runtime handles:

- `reflect.Type` — the type descriptor. Wraps `*_type` / `*rtype`; methods like `Kind()`, `Name()`, `NumMethod()`, `Elem()`, `Field(i)` read from it.
- `reflect.Value` — a typed pointer to a value. Contains `(Type, pointer, flag)` where `flag` encodes `Kind`, `readonly`, `addr`, and whether the value is settable / addressable.

```

package main

import (
    "fmt"
    "reflect"
)

type User struct {
    ID    int    `json:"id" validate:"required"`
    Name  string `json:"name" validate:"required,max=64"`
    age   int    // unexported – not visible via NumField to outside package
}

func reflectBasics() {
    u := User{ID: 7, Name: "ada", age: 36}

    t := reflect.TypeOf(u) // Type is *rtype for User
    v := reflect.ValueOf(u) // Value is a copy of u (not settable)

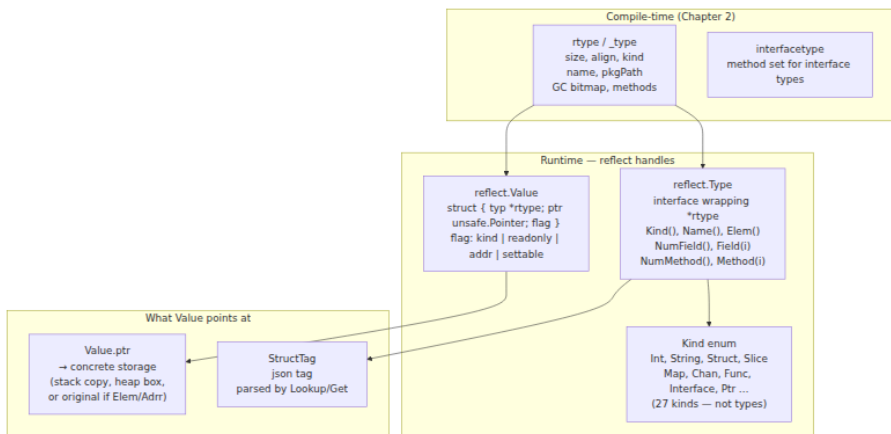
    fmt.Println(t.Name(), t.Kind(), t.NumField(), t.NumMethod())
    // User struct 3 0 (age counts; NumMethod 0 – User has no methods)
    fmt.Println(reflect.TypeOf(&u).Elem() == t) // true – pointer Elem
recovers User

    fmt.Println(v.Kind(), v.CanInterface(), v.CanAddr(), v.CanSet())
    // struct true false false – ValueOf copy is not addressable/
settable

    pv := reflect.ValueOf(&u) // ValueOf pointer
    fmt.Println(pv.Kind(), pv.Elem().CanSet(), pv.Elem().CanAddr())
    // ptr true true – Elem() dereferences and is settable (laws
below)

    // NumMethod respects method set (Ch 3): T vs *T
    type S struct{}
    func (S) Val() {}
    func (*S) Ptr() {}
    fmt.Println(reflect.TypeOf(S{}).NumMethod()) // 1 – Val only
    fmt.Println(reflect.TypeOf(&S{}).NumMethod()) // 2 – Val + Ptr
}

```



The distinction between `Type / Kind` matters: `Kind` is the broad category (`Struct`, `Slice`, `Ptr`, `Interface`, `Chan`, ... 27 values); `Type` is the specific type (`User`, `[]User`, `*User`, `map[string]User`). Two different named struct types both have `Kind() == Struct` but different `Type` — never switch on `Kind` when you mean `Type`.

5.2 Struct tags and StructTag

Tags are raw strings on struct fields, parsed by `reflect.StructTag`:

```
type Event struct {
    ID          string `json:"id" validate:"required,uuid"`
    Timestamp   int64  `json:"ts" db:"created_at"`
    Payload     []byte `json:"- " db:"payload" // json:"- " means skip`
}

func tagsDemo() {
    t := reflect.TypeOf(Event{})
    for i := 0; i < t.NumField(); i++ {
        f := t.Field(i)
        fmt.Printf("%s: json=%q db=%q tag=%q\n",
            f.Name, f.Tag.Get("json"), f.Tag.Get("db"), f.Tag)
        // Lookup distinguishes absent vs empty:
        if v, ok := f.Tag.Lookup("validate"); ok {
            fmt.Println("  validate:", v)
        }
    }
    // StructTag is just a string – Get parses `key:"value"` pairs.
    // Malformed tags (missing quotes, spaces) cause Get to return "" –
    // use `go vet`'s structtag check.
}
```

Tag parsing is not free: `Tag.Get / Lookup` scans the tag string each call. Serializers cache the parsed field list once per type (see `sync.Map` cache in `encoding/json` and `jsoniter`). Writing `Tag.Get` inside a per-element loop is a hot-path mistake.

5.3 The three laws of reflection

From the Go Blog post *The Laws of Reflection* (pinned in Further reading):

1. Reflection goes from interface value to reflection object.

`reflect.TypeOf / ValueOf` take `any`. If you pass a non-interface (a concrete value), the compiler boxes it to `any` first — the `reflect` call sees the dynamic type, not the static one.

```
go var x int = 3 fmt.Println(reflect.TypeOf(x)) // int — x was
boxed to any, TypeOf reads eface._type
```

2. Reflection goes from reflection object to interface value.

`Value.Interface()` and `Value.CanInterface()` reverse it. `v.Interface().(T)` recovers the original — but only if the value is not the zero `Value` and the type is exported-addressable.

```
go v := reflect.ValueOf(3) fmt.Println(v.Interface().(int)) //
3 // v.CanInterface() is false when v came from an unexported
field without Addr.
```

3. To modify a reflection object, the value must be settable.

Settability requires addressability + not being a copy. The canonical pattern is `ValueOf(&x).Elem()` :

```
```go x := 3 v := reflect.ValueOf(x) // not settable — copy //
v.SetInt(42) // panic: reflect.Value.SetInt using unaddressable value
```

```
p := reflect.ValueOf(&x) // pointer value, addressable v2 :=
p.Elem() // deref — settable, addressable fmt.Println(v2.CanSet(),
v2.CanAddr()) // true true v2.SetInt(42) fmt.Println(x) // 42
```

```
// Equivalent — reflect.New allocates a settable zero value: vp :=
reflect.New(reflect.TypeOf(User{ })) // *User, settable
vp.Elem().FieldByName("Name").SetString("grace") ```
```

Settability also governs `Addr` and `CanInterface` on unexported fields: `Field(i)` on an unexported field returns a `Value` where `CanInterface()==false` and `CanAddr()==false` unless you use

`reflect.NewAt` with `unsafe` — which is how serializers bypass export restrictions (and why they need `unsafe`).

## **5.4 A complete `reflect` walk — struct walker for validation/serialization**

This is the pattern every JSON/YAML/validation library uses, stripped to essentials and annotated with the gotchas.

```

package main

import (
 "fmt"
 "reflect"
 "strings"
)

// WalkFields calls fn for each exported field reachable from v.
// It dereferences pointers and interfaces, recurses into structs, and
// skips unexported fields.
func WalkFields(v any, fn func(path string, field reflect.StructField,
 value reflect.Value)) {
 walkValue(reflect.ValueOf(v), "", fn)
}

func walkValue(v reflect.Value, path string, fn func(string, reflect.St
 ructField, reflect.Value)) {
 if !v.IsValid() {
 return
 }
 // Unwrap interfaces and pointers – with nil checks (otherwise
 panic on Elem/Interface).
 for v.Kind() == reflect.Interface || v.Kind() == reflect.Pointer {
 if v.IsNil() {
 return
 }
 v = v.Elem()
 }
 if v.Kind() != reflect.Struct {
 return
 }
 t := v.Type()
 for i := 0; i < v.NumField(); i++ {
 field := t.Field(i)
 fv := v.Field(i)

 // PkgPath != "" means unexported – Field(i) is still returned,
 // but fv.CanInterface()==false and fv.CanSet()==false for
 callers outside the package.
 if field.PkgPath != "" {
 continue
 }
 curPath := field.Name
 if path != "" {
 curPath = path + "." + field.Name
 }

 // Recurse into embedded structs (but not time.Time-like
 structs – stop at leaf types).
 }
}

```



```

 // Guard with NumField to avoid recursing into non-structs that
 happen to be struct-kinded.
 if fv.Kind() == reflect.Struct && field.Anonymous {
 walkValue(fv, curPath, fn)
 continue
 }
 if fv.Kind() == reflect.Struct && fv.Type().NumField() > 0 {
 // Optionally recurse – here we recurse only when the field
 // itself needs visiting.
 // For a validator, recurse unconditionally; for a flat
 // mapper, don't.
 // This example visits leaf fields only, so recurse then
 // also visit:
 walkValue(fv, curPath, fn)
 }
 fn(curPath, field, fv)
}

func demo() {
 type Address struct {
 City string `json:"city"`
 Zip string `json:"zip"`
 }
 type Person struct {
 Name string json tag
 Age int `json:"age"`
 Address Address `json:"address"`
 hidden string
 Ptr *string `json:"ptr"`
 }
 s := "hello"
 p := Person{Name: "ada", Age: 36, Address: Address{City: "London",
 Zip: "EC1"}, hidden: "x", Ptr: &s}

 WalkFields(p, func(path string, f reflect.StructField, v reflect.Value) {
 tag := f.Tag.Get("json")
 if tag == "-" {
 return
 }
 // v.CanInterface() is true for exported fields – safe to
 print.
 fmt.Printf("%-16s json=%-10q kind=%-8s value=%v\n", path, tag,
 v.Kind(), v.Interface())
 })
 // Also works when rooted at a pointer or an `any`:
 var boxed any = &p
 WalkFields(boxed, func(path string, _ reflect.StructField, v reflect.Value) {
 _ = strings.Contains(path, ".")
 })
}

```

```

 }
 _ = v
}

```

Output (trimmed):

Name	json="name"	kind=string	value=ada
Age	json="age"	kind=int	value=36
Address	json="address"	kind=struct	value={London EC1}
Address.City	json="city"	kind=string	value=London
Address.Zip	json="zip"	kind=string	value=EC1
Ptr	json="ptr"	kind=ptr	value=0xc000...

## What will bite you:

- `v.IsNil()` panics unless `Kind` is `Chan`, `Func`, `Map`, `Pointer`, `Interface`, `Slice`, or `UnsafePointer` — always guard with `Kind` first.
- `v.Field(i)` on an unexported field returns a `Value` that panics on `Interface()` — check `field.PkgPath != ""` or `CanInterface()`.
- `v.NumField()` on a non-struct panics — check `Kind() == Struct` first.
- Every `reflect.ValueOf` and `Interface()` call may allocate. A walk over 100 fields per request × 10k RPS is a GC problem — which is why production serializers cache `[]fieldInfo` per `reflect.Type` in a `sync.Map` and never call `ValueOf` in the fast path.

## 5.5 Escape via reflection

`reflect.ValueOf` takes `any`, so passing a value boxes it — and the box may escape:

```
//go:noinline
func viaReflect(x int) int {
 v := reflect.ValueOf(x) // x boxed to any → heap allocation
 (escape)
 return int(v.Int())
}

//go:noinline
func direct(x int) int { return x }

func escapeDemo() {
 _ = viaReflect(42) // 1 alloc per call – visible in benchmem /
 -gcflags=-m
 _ = direct(42) // 0 alloc
}
```

```
$ go build -gcflags=-m ./reflect_test.go 2>&1 | grep escape
./reflect_test.go:6:23: x escapes to heap: reflect.ValueOf's argument
$ go test -bench=BenchmarkReflect -benchmem
BenchmarkDirect-8 0.6 ns/op 0 B/op 0 allocs/op
BenchmarkReflect-8 22 ns/op 16 B/op 1 allocs/op
```

`reflect.New`, `reflect.Zero`, and `v.Addr().Interface()` also allocate. The mitigation is the same as for interfaces: cache the `reflect.Type` / `[]fieldInfo` once, and on the hot path use `unsafe` or a generated switch instead of per-element `ValueOf`.

## 6. `unsafe` — the type-system escape hatch

### 6.1 What `unsafe` is and is not

`unsafe` (package `unsafe`, go doc `unsafe`) contains only operations the compiler special-cases; it has no runtime. The three primitives:

- `unsafe.Pointer` — a pointer that may point to any type, bypasses pointer type-compatibility. The only type that can be converted to/from any `*T`.
- `uintptr` — an integer large enough to hold a pointer. **Not a pointer** — the GC does not treat it as a reference.
- `unsafe.Sizeof` / `Offsetof` / `Alignof` — compile-time constants (see Chapter 2); evaluated without reading memory.

Added in Go 1.20–1.21: `unsafe.Slice`, `unsafe.SliceData`, `unsafe.String`, `unsafe.StringData` — typed, bounds-checked wrappers for slice/string header construction that previously required manual `reflect.SliceHeader`/`StringHeader` abuse.

## 6.2 The four valid `unsafe.Pointer` conversions

The `unsafe` docs list exactly four valid pointer conversion patterns. Any deviation is "not valid" — it may work today, break with a new GC, or be flagged by `go vet` :

```

package main

import (
 "unsafe"
)

func validPatterns() {
 var x int64 = 42

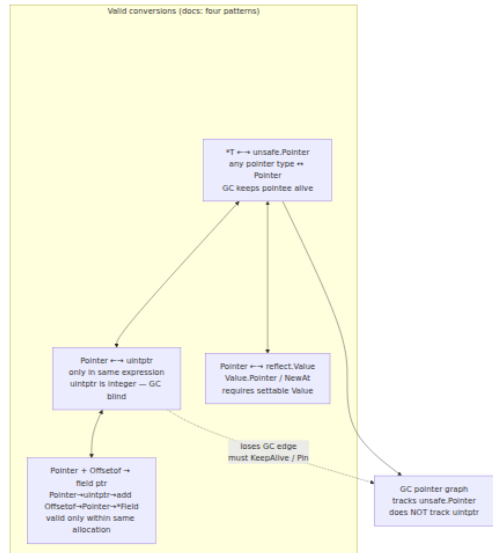
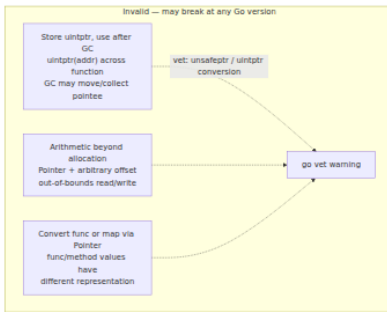
 // 1. *T ↔ unsafe.Pointer – any pointer type via unsafe.Pointer.
 p1 := unsafe.Pointer(&x) // *int64 → Pointer
 p2 := (*int64)(p1) // Pointer → *int64
 _ = p2

 // 2. unsafe.Pointer ↔ uintptr – only when the uintptr is used
 // immediately
 // in the same expression to convert back. Storing uintptr
 // across GC is invalid.
 addr := uintptr(p1) // Pointer → uintptr (integer, GC
 // invisible)
 p3 := unsafe.Pointer(addr) // uintptr → Pointer (same
 // expression scope – valid)
 _ = p3

 // 3. unsafe.Pointer → *T where T is a struct, then arithmetic via
 // uintptr to reach a field.
 type S struct{ A int64; B int64 }
 var s S
 base := unsafe.Pointer(&s)
 bPtr := (*int64)(unsafe.Pointer(uintptr(base) +
 unsafe.Offsetof(s.B)))
 *bPtr = 99
 _ = bPtr

 // 4. unsafe.Pointer ↔ reflect.Value – via Value.Pointer / NewAt /
 // Interface.
 // (Covered in Section 5.3; requires settable Value.)
}

```



### 6.3 uintptr vs unsafe.Pointer — the GC line

```

func uintptrPitfall() {
 // Safe — conversion back in same expression:
 s := make([]byte, 1024)
 p := unsafe.Pointer(&s[0])
 q := unsafe.Pointer(uintptr(p) + 8) // arithmetic then immediate
 back to Pointer
 _ = q

 // UNSAFE — uintptr stored, GC may reclaim s before use:
 var addr uintptr
 func() {
 s2 := make([]byte, 1024)
 addr = uintptr(unsafe.Pointer(&s2[0])) // s2's backing array —
 GC root is s2, not addr
 // s2 dies here — no reference keeps backing array alive if
 only addr remains
 }()
 // GC may have collected the backing array by now — dereferencing
 addr is use-after-free.
 _ = *(*byte)(unsafe.Pointer(addr)) // vet: conversion from uintptr
 to Pointer
}

```

**Rule:** Convert `uintptr` → `unsafe.Pointer` only in the same expression where you produced the `uintptr` from a live `unsafe.Pointer` (or use

`runtime.Pinner` / `runtime.KeepAlive` — Section 6.6). Never store a `uintptr` that is the sole reference to heap memory.

## 6.4 `unsafe` revisited — `Sizeof` / `Offsetof` / `Alignof` and the new `slice/string` helpers

Revisited from Chapter 2 with `unsafe` context added:

```
type Record struct {
 ID int64 // offset 0
 Tag string // offset 8
 Flag bool // offset 24
 _ [7]byte // explicit padding to 32 if you need stable layout
 for mmap/cgo
}

func layoutRevisited() {
 var r Record
 println(unsafe.Sizeof(r)) // 32
 println(unsafe.Offsetof(r.Tag)) // 8
 println(unsafe.Offsetof(r.Flag)) // 24
 println(unsafe.Alignof(r)) // 8

 // Go 1.20+ typed helpers – replace reflect.SliceHeader /
 // StringHeader hacks.
 // Old (broken on some GCs – header escaping):
 // hdr := (*reflect.SliceHeader)(unsafe.Pointer(&s))

 // New – bounds-checked, GC-safe, vet-clean:
 b := []byte("hello")
 s2 := unsafe.String(unsafe.SliceData(b), len(b)) // []byte →
 string, no copy
 b2 := unsafe.Slice(unsafe.StringData(s2), len(s2)) // string →
 []byte, no copy
 _ = b2
}
```

`unsafe.String(ptr, len)` and `unsafe.Slice(ptr, len)` construct a string/slice header from a pointer + length, checking that `ptr` is non-nil when `len > 0` (panics otherwise) and that the result does not overflow address space. They are the vetted replacement for manual `StringHeader`/`SliceHeader` mutation, which `go vet` now flags (`ptrconversion`, `assign`).

## 6.5 Zero-copy `string ↔ []byte` — the canonical unsafe pattern and how to vet it

Every backend service converts between `string` and `[]byte` on I/O boundaries (HTTP bodies, gRPC metadata, cache keys, hashing). The safe conversion copies; the unsafe conversion shares the backing bytes.



```

package zero

import (
 "unsafe"
)

// StringToBytes returns a []byte that aliases the string's backing
// bytes.
// The caller must NOT modify the returned slice (strings are
// immutable).
// The string must remain live while the slice is in use (see
// KeepAlive).
func StringToBytes(s string) []byte {
 if s == "" {
 return nil
 }
 // Go 1.20+: typed, vet-clean, no reflect header.
 return unsafe.Slice(unsafe.StringData(s), len(s))
}

// BytesToString returns a string that aliases the slice's backing
// bytes.
// The caller must NOT modify the slice after conversion if the string
// is retained.
func BytesToString(b []byte) string {
 if len(b) == 0 {
 return ""
 }
 return unsafe.String(unsafe.SliceData(b), len(b))
}

// CopyString returns an owned copy when you need to retain beyond the
// buffer's lifetime.
func CopyString(b []byte) string { return string(b) } // safe copy –
explicit cost

// Example – zero-copy header canonicalization (hot path).
func canonicalizeHeader(v string) string {
 // strings.ToLower would allocate; unsafe path avoids it when
 // already canonical.
 // Real code would check ASCII fast path first – simplified here.
 b := StringToBytes(v) // no alloc – shares v's bytes
 // read-only use of b is safe; mutating b would mutate the string
 // and break the language contract
 _ = b
 return v
}

```

## Allocation reality:

```

func BenchmarkSafe(b *testing.B) {
 s := "hello, world – a header value that will be hashed"
 for i := 0; i < b.N; i++ {
 _ = []byte(s) // copies – 1 alloc
 }
}
func BenchmarkUnsafe(b *testing.B) {
 s := "hello, world – a header value that will be hashed"
 for i := 0; i < b.N; i++ {
 _ = StringToBytes(s) // 0 alloc – shares backing
 }
}
// BenchmarkSafe-8 35 ns/op 64 B/op 1 allocs/op
// BenchmarkUnsafe-8 1 ns/op 0 B/op 0 allocs/op

```

**When zero-copy is worth it:** hashing, comparison, or passing to a read-only callee ( `hasher.Write`, `map` lookup via `string(b)` alternative) that does not retain the slice. **When it is not:** storing the result beyond the source's lifetime (handler returns a string that aliases a request buffer that will be reused), or any mutation of the alias.

**Immutability contract.** `StringToBytes` violates the language's immutability guarantee — `string` values are defined as immutable in [go.dev/ref/spec#String\\_types](https://go.dev/ref/spec#String_types). Mutating the returned `[]byte` is undefined behavior by the `unsafe` docs: future compilers may inter `string` backing in read-only memory or deduplicate equal strings. The functions above document "read-only" and rely on callers honoring it.

## go vet guidance — the warnings you must not ignore

```
$ go vet ./...
unsafePtr: possible misuse of unsafe.Pointer
conversion from uintptr to unsafe.Pointer – must be in same
expression
ptrconversion: invalid conversion: uintptr -> unsafe.Pointer without
prior Pointer
assign: assignment to StringHeader.Data / SliceHeader.Data – use
unsafe.String / Slice

Enable the analyzers explicitly (on by default in go vet since Go
1.20):
$ go vet -vettool=$(go env GOTOOLDIR)/vet -unsafePtr -ptrconversion ./.
..

Staticcheck additionally flags:
$ staticcheck ./...
SA1019: using deprecated reflect.StringHeader / SliceHeader – use
unsafe.String / Slice
```

Suppress false positives only with `//go:nocheckptr` or `//vet:ignore` equivalents for the narrow line, and only after review. Never ignore `unsafePtr` globally — it catches the `uintptr`-across-GC bug.

### Receipt for production use:

1. Prefer `unsafe.String` / `unsafe.Slice` / `unsafe.StringData` / `unsafe.SliceData` (Go 1.20+) over manual header construction. `reflect.StringHeader` / `SliceHeader` patterns are deprecated and miscompile with moving GCs.
2. Document every zero-copy function with `//go:nocheckptr` rationale and a comment stating the aliasing contract and lifetime requirement.
3. Gate on `go vet` in CI with `-unsafePtr` — treat any new warning as a build failure.
4. For retained strings/bytes (cache insertion, map key that outlives the buffer), use the safe `string(b)` copy — one allocation is cheaper than a use-after-free.

## 6.6 GC interaction — `KeepAlive` and `Pin` (Go 1.21+)

When you derive a `uintptr` or a detached `unsafe.Pointer` from a Go allocation, the GC may consider the original allocation unreachable before you are done with the derived pointer.

```
package gc

import (
 "runtime"
 "unsafe"
)

func keepAliveDemo() {
 b := make([]byte, 1<<20)
 ptr := unsafe.Pointer(unsafe.SliceData(b))
 // b is the sole GC root for the backing array. The compiler may
 // consider b dead
 // after its last use – which is above – even though ptr still
 // points into it.
 use(ptr) // GC could have reclaimed b's backing before use() runs
 runtime.KeepAlive(b) // keep b live at least until here
}

func pinDemo() {
 // Go 1.21+ – runtime.Pinner keeps heap objects pinned (not moved
 // by a moving GC).
 // Today Go's GC is non-moving for heap, but Pin is the future-
 // proof contract.
 var p runtime.Pinner
 b := make([]byte, 1024)
 p.Pin(b) // pin backing array – address stable, GC will not move/
 // collect it
 ptr := unsafe.Pointer(unsafe.SliceData(b))
 use(ptr)
 p.Unpin() // release – must not use ptr after this
 // KeepAlive not needed between Pin and Unpin – Pin holds the
 // reference.
}

func use(p unsafe.Pointer) { _ = p }
```

Rules:

- `runtime.KeepAlive(x)` is a compiler barrier — it makes `x` live until the call site, preventing the compiler from proving `x` is dead and allowing the GC to collect it. Place it *after* the last use of the derived pointer.

- `runtime.Pinner` (Go 1.21) is stronger: it pins the object so a future moving GC will not relocate it. Every `Pin` must be paired with `Unpin`; pinning many objects has GC cost — pin only around the narrow unsafe window.
- `uintptr`-only references never keep anything alive — they are integers. Any `uintptr` that is the sole reference to heap memory is a bug unless `KeepAlive` / `Pinner` spans its use.



## 7. Backend lens — where the costs land in real services

### 7.1 Interfaces for testability vs generics for speed

Context	Prefer	Why
System boundary ( <code>Handler</code> , <code>Store</code> , <code>Clock</code> , <code>Publisher</code> )	<b>Interface</b>	Mocks/fakes for tests, swappable implementations, binary decoupling. Call frequency is low (per-request), indirection cost negligible.
Hot inner loop (codec, <code>Less</code> comparator, per-element transform)	<b>Generics or concrete type</b>	Avoids indirect call + boxing allocation, enables inlining and monomorphized fast paths.
Middleware chain ( <code>func(http.Handler)</code> <code>http.Handler</code> )	<b>Concrete</b> <code>http.HandlerFunc</code> or <b>generics</b>	Per-request dispatch — interface call overhead is fine, but allocating a closure per wrap is the real cost.

Context	Prefer	Why
Plugin / driver registry ( <code>sql.Register</code> , <code>encoding.Register</code> )	<b>Interface + <code>init</code> registration</b>	Open world — new implementations unknown at compile time. Generics cannot replace this.
Numeric / collection utilities ( <code>Min</code> , <code>Sort</code> , <code>Map</code> )	<b>Generics</b> ( <code>constraints.Ordered</code> , <code>slices.*</code> )	Zero interface boxing, fully inlined comparisons.

Example — the testability pattern that deserves its indirection, and the hot-path variant:

```
// Boundary — interface is the right tool (test with a fake).
type Clock interface{ Now() time.Time }

type realClock struct{}

func (realClock) Now() time.Time { return time.Now() }

type Service struct{ clock Clock }

func (s *Service) Greet(name string) string {
 if time.Since(s.clock.Now()) > time.Hour { /* ... */ }
 return "hello " + name
}

// In tests:
// type fakeClock struct{ t time.Time }
// func (f fakeClock) Now() time.Time { return f.t }
// svc := Service{clock: fakeClock{t: time.Unix(0,0)}}

// Hot path — generics avoid per-element interface dispatch.
func Sum[T interface{ Add(int, int) int }](items []T, init int) int {
 acc := init
 for _, it := range items {
 acc = it.Add(acc, 1) // direct call after stenciling —
inlineable
 }
 return acc
}
```

## 7.2 Reflection cost in serializers — why JSON is slow and what to do

`encoding/json`, `yaml.v3`, `mapstructure`, and validation libraries all walk structs with `reflect`. The cost breakdown per `Marshal`:

1. **Type analysis** (`reflect.TypeOf`, `NumField`, `Tag.Get`) — cacheable per type. Libraries that cache (`json` does since Go 1.10 via `sync.Map` per type) pay this once; libraries that don't pay it per call.
2. **Value dispatch** (`ValueOf`, `Kind` switch, `Field`, `Interface`) — per-value, per-field. Each `ValueOf` on a non-pointer struct copies the value; each `Field` returns a new `Value`; each `Interface()` boxes.
3. **Interface boxing of primitives** — `case string: ...` branches inside the encoder often box to `any` again.

Measured overhead (Go 1.22, `BenchmarkJSONMarshal` of a 12-field struct, `benchmem`):

<code>encoding/json.Marshal</code> struct info, but per-call <code>ValueOf</code> + <code>map</code> for fields)	~850 ns/op	~3 allocs/op	(cached <b>type</b> )
<code>json-iterator</code> with <code>reflect</code> (aggressive caching)	~420 ns/op	~2 allocs/op	
<code>easyjson</code> / code-generated (no <code>reflect</code> generated switch)	~180 ns/op	~0 allocs/op	

### Mitigations when JSON/protobuf sits on the hot path:

- Generate code (`easyjson`, `ffjson`, `protoc-gen-go`) — eliminates reflection entirely.
- Cache `reflect.Type` field lists in a `sync.Map` keyed by `reflect.Type` — never call `NumField` / `Tag.Get` per request.
- Use `json.RawMessage` / `json.Decoder.UseNumber` to defer decoding of sub-objects.
- For protobuf, the generated `Marshal` already uses `unsafe` fast paths for repeated fields — do not re-encode via `json` on the hot path.

## 7.3 Unsafe zero-copy on hot paths — where it earns its keep vs where it burns

Worthwhile `unsafe` uses in backend Go, with rough allocation savings:

Hot path	Unsafe technique	Saving	Risk if misused
<code>[]byte</code> → <code>string</code> for map lookup / hashing	<code>unsafe.String(SliceData(b), len(b))</code>	1 alloc per lookup → 0	Retaining the string beyond buffer lifetime → use-after-free
<code>string</code> → <code>[]byte</code> for <code>Write</code> / hasher	<code>unsafe.Slice(StringData(s), len(s))</code>	1 alloc per write → 0	Mutating the slice → string immutability violation, read-only memory fault
I/O buffer reinterpretation (flatbuffers, capnp)	<code>unsafe.Slice</code> over <code>mmap</code> region	Avoids copy of bulk data	Alignment, bounds, and pinning errors
<code>encoding/binary</code> over large struct arrays	<code>unsafe.Slice</code> over struct pointer	Avoids per-element encoding	Endianness + padding + moving GC



```
// Hot-path example – hashing header values without allocation.
import (
 "hash/fnv"
 "unsafe"
)

func hashHeader(v string) uint64 {
 h := fnv.New64a()
 // Safe but allocates: h.Write([]byte(v))
 // Zero-copy, read-only – safe when h.Write does not retain the
 slice:
 _, _ = h.Write(unsafe.Slice(unsafe.StringData(v), len(v))) // 0
 alloc
 // Must NOT do: b := unsafe.Slice(...); b[0] = 'x' – mutates string
 backing
 return h.Sum64()
}
```

**Do not reach for `unsafe` to fix a reflection problem you can fix with code generation.** `unsafe` trades a type-checked guarantee for a promise the caller must keep. In a service with many owners, the promise will eventually be broken. Prefer `string(b)` copy for retained data, `easyjson/protoc` generation for codecs, and reserve `unsafe` for measured, profiled hot paths with a `go vet`-clean receipt.

## Key takeaways

- Interface values are 16-byte headers: `eface{ _type, data }` for any, `iface{ itab, data }` for method-bearing interfaces. The `itab` is `{ inter, _type, hash, fun... }`, built once by `runtime.getitab`, cached globally by `(inter, _type)` hash, and immutable after publication. Understanding the header explains boxing, `nil` checks, and dispatch.
- The typed-`nil` pitfall is a header consequence: `var p *T = nil; var e error = p` has a non-`nil` `itab` and thus `e != nil`, but calling a method `nil`-dereferences. Always `return nil` explicitly when a typed `nil` may be the value; lint with `staticcheck/nilness` and `go vet`.
- Type assertions and type switches use `itab.hash / _type.hash` for  $O(1)$  fast paths; the single-value assertion panics on mismatch, the comma-ok form does not. Order type-switch cases by frequency.

- Interface dispatch is `load tab → load fun[i] → indirect call(data, args)` — three loads plus a branch that cannot inline. Expect ~3 ns extra per call vs direct/generic; negligible per-request but significant in tight loops. Generics stencil to direct, inlineable calls for hot paths.
- `reflect.Type` is the reified `_type`, `reflect.Value` is `{ typ, ptr, flag }`; `Kind` is a coarse category, `Type` is the specific type. The three laws are: interface→reflection via `ValueOf / TypeOf`, reflection→interface via `Value.Interface()`, and mutation requires a settable value (`ValueOf(&x).Elem()`). Every `ValueOf / Interface` may box and escape — cache `Type / fieldInfo` per type.
- `unsafe.Pointer` is a GC-tracked universal pointer; `uintptr` is an untracked integer. Conversions are valid only in the four documented patterns and only when `uintptr → Pointer` stays in the same expression. Use `unsafe.String / Slice / StringData / SliceData` (Go 1.20+) instead of `reflect.StringHeader / SliceHeader` mutation.
- Zero-copy `string ↔ []byte` via `unsafe` removes one allocation per conversion but violates string immutability if the alias is mutated and requires the source to outlive the alias. Keep `go vet -unsafeptr -ptrconversion` green in CI; use `runtime.KeepAlive` or `runtime.Pinner` (Go 1.21+) when a `uintptr` or detached pointer must keep its backing alive.
- For backends, choose interfaces at system boundaries (handlers, stores, clocks) for testability and generics/concrete types in hot loops for speed; avoid reflection per-request in serializers (cache field lists or generate code); reserve `unsafe` for profiled hot paths with a documented contract and vet receipt.

---

## Further reading

- Go Language Specification — Interface types — [https://go.dev/ref/spec#Interface\\_types](https://go.dev/ref/spec#Interface_types) — **(pinned)** Canonical definition of interface types, method sets, and implementation.
- `reflect` package documentation — <https://pkg.go.dev/reflect> — **(pinned)** API and semantics for `Type`, `Value`, `Kind`, `StructTag`, `Value.Set` / `CanSet` / `CanInterface`.

- `unsafe` package documentation — <https://pkg.go.dev/unsafe> — **(pinned)** The four valid `unsafe.Pointer` conversion patterns, `Sizeof / Offsetof / Alignof`, `Slice / String / SliceData / StringData`.
- The Laws of Reflection — <https://go.dev/blog/laws-of-reflection> — **(pinned)** The three laws, with runnable examples; the conceptual foundation for Section 5.
- Go `runtime` — `src/runtime/iface.go` and `src/runtime/runtime2.go` (`eface`, `iface`, `itab`, `getitab`) — <https://github.com/golang/go/blob/master/src/runtime/iface.go> — Authoritative source for the layouts and cache described in Section 2.
- `go vet` — `unsafePtr` and `ptrconversion` analyzers — [https://pkg.go.dev/cmd/vet#hdr-Unsafe\\_pointer](https://pkg.go.dev/cmd/vet#hdr-Unsafe_pointer) — What `go vet` flags for invalid `unsafe.Pointer / uintptr` misuse.
- Go 1.20 Release Notes — `unsafe.String`, `unsafe.Slice`, `unsafe.StringData`, `unsafe.SliceData` — <https://tip.golang.org/doc/go1.20#unsafe> — Rationale and contracts for the typed `unsafe` string/slice helpers.
- Go 1.21 Release Notes — `runtime.Pinner`, `Pin / Unpin` — <https://tip.golang.org/doc/go1.21#runtime> — GC pinning for `unsafe`-derived pointers in a future moving GC.
- `staticcheck` — SA1019, SA4022 and related interface/`nil` checks — <https://staticcheck.dev/docs/checks> — Lint rules that catch typed-`nil` returns and deprecated header use.
- Russ Cox — *Go Data Structures: Interfaces* — <https://research.swtch.com/interfaces> — Deep dive into `eface / iface / itab` internals with assembly-level detail (pre-generics, still accurate for layouts).
- Go Blog — *JSON and Go* and `encoding/json` source (`src/encoding/json/encode.go`) — <https://go.dev/blog/json> — How the standard library uses reflection and its per-type cache.

## Chapter 8 — Channels, Select, Timers, and Context Internals

---

**What this chapter covers.** Channels are Go's most distinctive concurrency primitive — typed, garbage-collected queues that double as

synchronization points. `select` multiplexes them, `time.Timer` drives deadlines, and `context.Context` propagates cancellation across the call graph. Behind the simple syntax lies a compact runtime: `runtime.hchan` with its circular buffer and `sudog` wait queues, `runtime.selectgo` with randomized polling, a per-P 4-heap of `runtimeTimers` driven by `sysmon` and the netpoller, and a context tree whose `Done` channel is just a closed channel. This chapter opens `runtime/chan.go`, `runtime/select.go`, `runtime/time.go`, and `context/context.go` and traces every path a value takes from `ch <- v` to the receiver's stack.

Learning goals — after this chapter you should be able to:

- Read `hchan` field-by-field (`qcount`, `dataqsiz`, `buf`, `elemsize`, `sendx / recvx`, `closed`, `sendq / recvq`, `lock`) and explain how a buffered channel's circular buffer and wait queues work.
- Distinguish `nil`, `unbuffered`, `buffered`, and `closed` channel semantics — including the precise happens-before edges each creates and the panics/zero-value rules on close.
- Trace `send` and `receive` fast paths (`buffered enqueue/dequeue`, `unbuffered direct copy`) and the slow paths that `gopark / goready` the goroutine via `sudog`.
- Describe `select` internals: `scase`, `pollorder / lockorder` randomization, `runtime.selectgo`'s lock-sort-unlock protocol, and how `default` makes `select` non-blocking.
- Explain the timer subsystem: `time.Timer / Ticker`, `runtimeTimer`, per-P timer heaps, `sysmon / netpoll` wakeup, `AfterFunc`, and correct `Timer.Reset` reuse.
- Walk the `context` type hierarchy (`emptyCtx`, `cancelCtx`, `timerCtx`, `valueCtx`), cancellation propagation via `Done` channel closure, and value lookup chain semantics.
- Choose between channels and `sync.Mutex / Cond` per workload, propagate `context.Context` correctly through HTTP/gRPC handlers, reuse timers without leaks, and diagnose channel/context leaks with `pprof` and `trace`.

**Placement.** Volume 4, Chapter 7 — The Actor Model, CSP, and Channels gave the *model* (CSP, rendezvous, choice). Volume 17, Chapter 4 — Goroutines, the Scheduler (G-M-P), and the Netpoller gave the *substrate* (`gopark / goready`, P-local state, netpoller). Chapter 6 — The Go Memory Model, Atomics, and Synchronization Primitives defined the

happens-before contract that channels enforce. This chapter is the *implementation* that joins them: how `chan`/`select`/`Timer`/`Context` park and unpark G.s. Chapter 5 (Allocator/GC) explains why `hchan.buf` and `sudog.elem` allocations behave as they do; Chapter 10 revisits `trace` and the race detector as instruments for channel and context bugs.

---

## 1. Why channels repay internals-level understanding

A backend service at steady state has thousands of goroutines coordinating through a small set of patterns:

- A `chan *Request` work queue between accept loop and handler pool.
- A `select` that races an RPC against `ctx.Done()` and a timeout `Timer`.
- A `context.WithTimeout` that threads a deadline through three downstream gRPC calls.

Each pattern looks like one line of Go. Underneath, each funnels through the same runtime machinery — `hchan.lock`, `sudog`, `gopark`, timer heaps — and each has a failure mode that only makes sense once you see that machinery: a nil-channel deadlock that never appears in `pprof`, a `select` that starves one case under load, a `time.After` in a hot loop that leaks 10k timers per second, a `context.WithValue` chain that turns a map lookup into a linked-list walk on every request.

This chapter makes those failures legible. We read the structs, trace the assembly-level copies, watch goroutines park and wake, and then apply that understanding to the backend decisions that follow: when to use a channel and when to reach for a mutex, how to propagate context without leaking it, how to reuse timers at line rate.

---

## 2. `hchan` — the channel struct walkthrough

All channels are pointers to `hchan`, allocated by `make(chan T, n)` via `runtime.makechan`. The type `T` is erased to `elemsize/elementype`; the buffer is a single `mallocgc` allocation.

## 2.1 The struct (Go 1.21-1.23, runtime/chan.go)

```
// runtime/chan.go - type hchan (abridged, comments trimmed)
type hchan struct {
 qcount uint // total data in the queue (0 <= qcount <=
 dataqsiz)
 dataqsiz uint // size of the circular queue (== cap(ch)), 0 for
 unbuffered/sync
 buf unsafe.Pointer // points to array[dataqsiz]elem - ring
 buffer
 elemsize uint16
 closed uint32 // 0 = open, 1 = closed (set atomically
 under lock)
 elemtype *type // element type (for typedmemmove, GC)
 sendx uint // send index in buf (next slot to write)
 recvx uint // receive index in buf (next slot to read)
 recvq waitq // list of recv waiters (sudog, FIFO)
 sendq waitq // list of send waiters (sudog, FIFO)

 // lock protects all fields in hchan, plus sudog fields that
 // logically belong to the channel (elem, g, c, selectDone).
 lock mutex
}

type waitq struct {
 first *sudog
 last *sudog
}

type sudog struct {
 g *g
 next *sudog
 prev *sudog
 elem
 unsafe.Pointer // data element (points to stack slot for direct copy)
 acquiretime int64
 releasetime int64
 ticket uint32
 isSelect bool // true if sudog is part of a select
 (selectDone handling)
 success bool // set by waker: did this sudog win the
 rendezvous?
 waitlink *sudog // g.waiting linkage
 c *hchan // channel this sudog is waiting on
}
```

Field-by-field:

Field	Role
<code>qcount</code> / <code>dataqsiz</code>	Occupancy vs. capacity. <code>qcount==0 &amp;&amp; dataqsiz==0</code> is unbuffered; <code>dataqsiz==0</code> with non-zero <code>qcount</code> never happens. <code>len(ch)</code> reads <code>qcount</code> under <code>lock</code> ; <code>cap(ch)</code> reads <code>dataqsiz</code> without locking (immutable after creation).
<code>buf</code>	<code>dataqsiz * elemsize</code> bytes, <code>mallocgc</code> -allocated. For element types that contain pointers, the buffer is scanned by GC (the <code>elemtype</code> 's <code>ptrmask</code> ). For pointer-free types, it is <code>noscan</code> .
<code>elemsize</code> / <code>elemtype</code>	<code>elemsize</code> drives <code>typedmemmove</code> and pointer arithmetic; <code>elemtype</code> drives GC scanning and <code>memclr</code> . Checked at <code>makechan</code> — channels of uncomparable element types are fine, channels themselves are comparable only as pointers.
<code>sendx</code> / <code>recvx</code>	Indices into the circular buffer. Both wrap modulo <code>dataqsiz</code> . <code>sendx</code> advances on buffered enqueue; <code>recvx</code> on buffered dequeue. When <code>qcount==0</code> they are equal; when <code>qcount==dataqsiz</code> they are also equal — <code>qcount</code> disambiguates.
<code>closed</code>	Set to <code>1</code> by <code>close(ch)</code> while holding <code>lock</code> . Once set, further <code>close</code> panics, sends panic, and receives drain then return zero values with <code>ok==false</code> .
<code>recvq</code> / <code>sendq</code>	FIFO queues of parked goroutines. Each waiter is a <code>sudog</code> allocated on the waiter's stack (not the heap — see Section 4.3). <code>recvq</code> holds receivers blocked waiting for a value; <code>sendq</code> holds senders blocked waiting for a receiver (unbuffered) or buffer space (buffered-full). Mutual exclusion: at most one of the buffer and the wait queues is non-empty at any instant (invariant enforced by <code>lock</code> ).
<code>lock</code>	A <code>runtime.mutex</code> (futex-backed on Linux, see Chapter 6). Protects every field above. Held only for short, bounded critical sections — never across <code>gopark</code> .

Allocation path:

```

// make(chan T, n) lowers to runtime.makechan(elemtype, n)
func makechan(t *type, size int) *hchan {
 elem := t.Elem()
 mem, overflow := math.MulUintptr(elem.Size_, uintptr(size))
 // ... overflow / maxAlloc checks ...
 var c *hchan
 switch {
 case mem == 0: // unbuffered or element size 0 (e.g. chan struct{})
 c = (*hchan)(mallocgc(unsafe.Sizeof(hchan{}), nil, true))
 c.elemsize = uint16(elem.Size_)
 case elem.PtrBytes == 0: // element has no pointers – one
allocation for hchan+buf
 c = (*hchan)(mallocgc(unsafe.Sizeof(hchan{})+mem, nil, true))
 c.buf = unsafe.Pointer(uintptr(unsafe.Pointer(c)) + unsafe.Size
of(hchan{}))
 default: // element has pointers – separate allocations so GC can
scan buf precisely
 c = new(hchan)
 c.buf = mallocgc(mem, elem, true)
 }
 c.elemsize = uint16(elem.Size_)
 c.elemtype = elem
 c.dataqsiz = uint(size)
 // qcount, sendx, recvx default 0; lock is zero-initialized
 return c
}

```

Three allocation shapes: unbuffered or zero-sized element (header only), pointer-free element (header+buffer in one allocation — fewer GC objects), pointer-bearing element (header and buffer separate so the buffer's bitmap can be typed).



Diagram 1 — *hchan* layout: circular buffer indexed by *sendx/recvx*, occupancy in *qcount*, FIFO sudog queues for blocked senders/receivers, single mutex.



2.2 Nil vs. unbuffered vs. buffered — semantics precisely

Channel form	Creation	dataqsiz	buf	Send behavior	Receiv...
<b>Nil</b> <code>var ch chan T</code>	zero value, <code>nil</code> pointer	—	—	Blocks forever (never ready) — <code>chansend</code> detects <code>c==nil</code> and <code>gopark</code> s without a wakeup source	Block...
<b>Unbuffered</b> <code>make(chan T)</code>	<code>dataqsiz==0</code> , <code>buf==nil</code>	0	<code>nil</code>	Rendezvous: blocks until a receiver arrives; then direct copy receiver←sender without touching a buffer	Block...
<b>Buffered</b> <code>make(chan T, N)</code>	<code>dataqsiz==N</code> , <code>buf</code> allocated	N	<code>N*elemsize</code>	Enqueues if <code>qcount &lt; N</code> ; blocks only when full ( <code>qcount==N</code> )	Deblo...
<b>Closed</b> <code>close(ch)</code>	sets <code>closed=1</code>	—	—	Panics ( <code>throw</code> via <code>panic(plainError)</code> )	Drain...

Nil channels are not errors — they are deliberate. `select` with a nil case disables that case without restructuring the `select` . A common backend pattern is to nil-out a case after it fires:

```

for pending != 0 {
 var timeoutCh <-chan time.Time
 if deadlineSet { timeoutCh = timer.C }
 select {
 case req := <-inCh:
 handle(req)
 case <-timeoutCh: // nil when deadlineSet==false → case disabled
 return ctx.Err()
 }
}

```

Closed-channel rules are strict and frequently mishandled:

```

ch := make(chan int, 2)
ch <- 1; ch <- 2
close(ch)

// Drain – receives succeed until buffer empty.
v, ok := <-ch // 1, true
v, ok = <-ch // 2, true
v, ok = <-ch // 0, false – closed+empty forever

ch <- 3 // panic: send on closed channel
close(ch) // panic: close of closed channel
close(nil) // panic: close of nil channel

// Ranging a closed channel terminates:
for v := range ch { /* drains then exits */ }

// Closing is a broadcast – all blocked receivers wake:
close(broadcastCh) // idiomatic signal: <-broadcastCh unblocks every waiter

```

Happens-before edges (from [go.dev/ref/mem](https://go.dev/ref/mem), Chapter 6):

- A **send** on `ch` happens-before the corresponding **receive** from `ch` completes.
- For a buffered channel, the `k`th receive happens-before the `(k+c)`th send completes (`c = cap(ch)`) — the buffer decouples but does not eliminate ordering.
- `close(ch)` happens-before any receive that observes the close (`ok==false`).

### 3. Send and receive — fast paths, direct copy, and parking

Both `ch <- v` and `v := <-ch` (and `v, ok := <-ch`) are lowered by the compiler to `runtime.chansend` and `runtime.chanrecv`. Each has a non-blocking entry point used by `select` with `default` (`chansend(c, ep, block==false)` / `chanrecv(c, ep, block==false)`).

### 3.1 chansend — annotated

```

// runtime/chan.go – func chansend(c *hchan, ep unsafe.Pointer, block
bool, callerpc uintptr) bool
// ep points to the value to send (on sender's stack); callerpc for
race detector.
// Returns true if value was sent (or queued), false if non-blocking
and not ready.

func chansend(c *hchan, ep unsafe.Pointer, block bool, callerpc
uintptr) bool {
 if c == nil {
 if !block { return false }
 gopark(nil, nil, waitReasonChanSendNilChan, traceBlockForever,
2)
 throw("unreachable")
 }

 if debugChan { /* ... race detector hooks: syncEvent ... */ }

 if c.closed != 0 { panic(plainError("send on closed channel")) }
 // Fast path: acquire lock, handle every synchronous case, release
or park.

 lock(&c.lock)

 // Case 1: there is a waiting receiver – hand off directly (no
buffer touch).
 if sg := c.recvq.dequeue(); sg != nil {
 // Found a receiver goroutine parked in recvq.
 // Copy value directly from sender's stack (ep) to receiver's
stack (sg.elem).
 recv(c, sg, ep, func() { unlock(&c.lock) }, 3)
 return true
 }

 // Case 2: buffered channel with space – enqueue.
 if c.qcount < c.dataqsiz {
 qp := chanbuf(c, c.sendx) // &buf[sendx*elemsize]
 typedmemmove(c.elemtype, qp, ep) // copy sender -> buffer
 c.sendx++
 if c.sendx == c.dataqsiz { c.sendx = 0 }
 c.qcount++
 unlock(&c.lock)
 return true
 }

 // Non-blocking caller (select with default) – give up rather than
park.
 if !block {
 unlock(&c.lock)
 return false
 }

```

```

}

// Case 3: must block – enqueue this goroutine in sendq and park.
gp := getg()
mysg := acquireSudog() // from P sudog cache (Chapter 5) – not heap
mysg.elem = ep
mysg.g = gp
mysg.c = c
gp.parkingOnChan.Store(true)
// ... isSelect setup if caller is selectgo ...
c.sendq.enqueue(mysg)
// gopark unlocks c.lock atomically with parking – no lost wakeup.
gopark(chanparkcommit, unsafe.Pointer(&c.lock),
waitReasonChanSend, traceBlockChanSend, 2)
// --- goroutine is parked here until a receiver does goready ---
// On wakeup: mysg.success tells whether we were selected (select
path).
// For plain send, value was already copied by the receiver's
recv().
gp.parkingOnChan.Store(false)
if mysg.success { /* select winner bookkeeping */ }
mysg.elem = nil
releaseSudog(mysg)
return true
}

```

```

// recv - invoked with c.lock held, after dequeuing a receiver sudog.
func recv(c *hchan, sg *sudog, ep unsafe.Pointer, unlockf func(), skip
int) {
 if c.dataqsiz == 0 {
 // Unbuffered: copy directly sender stack -> receiver stack.
 if ep != nil { // non-nil for ch <- v; nil for close wakeup
 recvDirect(c.elemtype, sg, ep)
 }
 } else {
 // Buffered but recvq non-empty: invariant says qcount==0,
 // so there is no buffered data - still direct copy.
 // The more interesting buffered path is in chanrecv (Section
3.2).
 qp := chanbuf(c, c.recvx)
 // For the buffered+sendq case, shuffle: dequeue from buf to
receiver,
 // enqueue sender's value into buf (keeps FIFO order).
 if sg.elem != nil {
 typedmemmove(c.elemtype, sg.elem, qp)
 typedmemmove(c.elemtype, qp, ep)
 }
 c.recvx++
 if c.recvx == c.dataqsiz { c.recvx = 0 }
 c.sendx = c.recvx // because qcount was 0 before, queue was
empty
 // ... qcount stays at dataqsiz in this path ...
 }
 sg.elem = nil
 gp := sg.g
 unlockf()
 gp.parkingOnChan.Store(false)
 goready(gp, skip+1) // mark runnable, enqueue on P run queue
}

```

The direct-copy primitive is the key optimization — no intermediate buffer, no extra allocation:

```

func recvDirect(t *_type, sg *sudog, ep unsafe.Pointer) {
 // sg.elem is the receiver's stack slot; ep is the sender's stack
slot.
 typedmemmove(t, sg.elem, ep)
}
func sendDirect(t *_type, sg *sudog, ep unsafe.Pointer) {
 typedmemmove(t, sg.elem, ep) // same, opposite direction in
chanrecv
}

```

### 3.2 `chanrecv` — annotated (symmetric)



```

func chanrecv(c *hchan, ep unsafe.Pointer, block bool) (selected,
received bool) {
 if c == nil {
 if !block { return }
 gopark(nil, nil, waitReasonChanReceiveNilChan, traceBlockForever, 2)
 throw("unreachable")
 }

 lock(&c.lock)

 if c.closed != 0 && c.qcount == 0 {
 // Closed and empty – return zero value, received==false.
 unlock(&c.lock)
 if ep != nil { typedmemclr(c.elemtype, ep) }
 return true, false
 }

 // Case 1: waiting sender – dequeue sender, handle buffered vs
 // unbuffered.
 if sg := c.sendq.dequeue(); sg != nil {
 if c.dataqsiz == 0 {
 // Unbuffered: copy sender -> receiver directly.
 if ep != nil { recv(c, sg, ep, func(){unlock(&c.lock)}),
3) } // recv copies via recvDirect
 return true, true
 }
 // Buffered but sendq non-empty implies buffer is full
 // (qcount==dataqsiz).
 // Dequeue from buffer to receiver, enqueue sender into buffer.
 qp := chanbuf(c, c.recvx)
 if ep != nil { typedmemmove(c.elemtype, ep, qp) }
 typedmemmove(c.elemtype, qp, sg.elem)
 c.recvx++; if c.recvx == c.dataqsiz { c.recvx = 0 }
 c.sendx = c.recvx // buffer stays full, but FIFO preserved
 // sendq waiter was storing its value in sg.elem; now consumed.
 goready(sg.g, 3)
 unlock(&c.lock)
 return true, true
 }

 // Case 2: buffered data available – dequeue from buffer.
 if c.qcount > 0 {
 qp := chanbuf(c, c.recvx)
 if ep != nil { typedmemmove(c.elemtype, ep, qp);
typedmemclr(c.elemtype, qp) }
 c.recvx++; if c.recvx == c.dataqsiz { c.recvx = 0 }
 c.qcount--
 unlock(&c.lock)
 return true, true
 }
}

```

```

 }

 if !block { unlock(&c.lock); return false, false }

 // Case 3: must block – enqueue in recvg and park.
 gp := getg()
 msg := acquireSudog()
 msg.elem = ep; msg.g = gp; msg.c = c
 gp.parkingOnChan.Store(true)
 c.recvg.enqueue(msg)
 gopark(chanparkcommit, unsafe.Pointer(&c.lock), waitReasonChanReceive, traceBlockChanReceive, 2)
 // ... woken by chansend's goready ...
 gp.parkingOnChan.Store(false)
 success := msg.success
 msg.elem = nil
 releaseSudog(msg)
 // success==false means channel was closed while we were parked – zero value.
 return true, success
}

```

### 3.3 Parking and wakeup — `gopark` / `goready`

Both slow paths converge on the same scheduler primitive (Chapter 4):

- `gopark(commit func(*g, unsafe.Pointer) bool, lock unsafe.Pointer, reason, traceEv, skip)`: saves the goroutine's `gobuf` (SP/PC), marks it `_Gwaiting`, records `waitReason`, calls `commit` to release the channel lock *atomically with parking* (so the waker never misses the waiter), and calls `schedule()` to run another `G` on this `M/P`. The `M` is not blocked — it immediately picks the next runnable `G`.
- `goready(gp, skip)`: marks `gp` `_Grunnable`, enqueues it on its `P`'s run queue (or the global queue if the `P` is gone), and may preempt the current `M` via `wakeup`. The goroutine resumes after `gopark` returns, on whatever `M` picks it up.

The `sudog` itself is the rendezvous record. It lives on the *waiter's stack* (acquired from `p.sudogcache` via `acquireSudog`, returned via `releaseSudog` — no GC pressure on the hot path). `sudog.elem` points into the waiter's stack frame where the value should be copied. This is why channel operations can be allocation-free even for large structs — the copy is `typedmemmove` directly between stacks (or stack↔buffer), with no intermediate heap object.

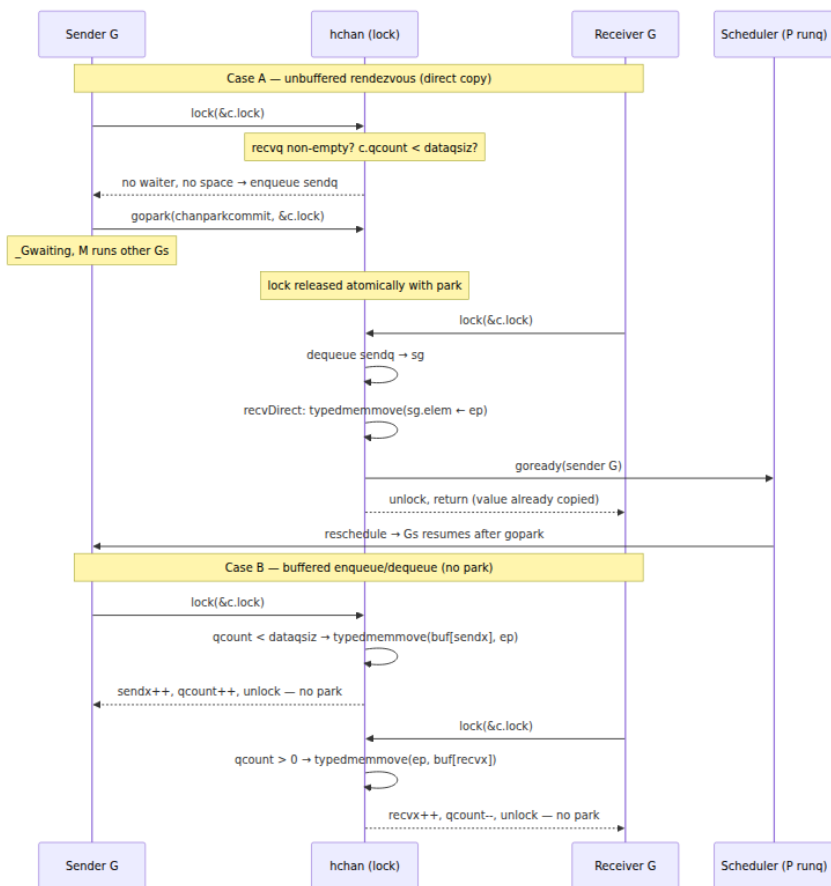


Diagram 2 — Send/receive flow: fast paths (buffer copy) vs. slow paths (sudog enqueue, gopark, goready, direct stack-to-stack copy).

## 4. Select — randomized choice over channels

`select` is the `switch` of concurrency. The compiler lowers it to a `scase` array and a call to `runtime.selectgo`.

## 4.1 scase and the lowering

```
// runtime/select.go
type scase struct {
 c *hchan // channel (nil for default case)
 elem unsafe.Pointer // data element: send value or receive slot
 kind caseKind
 pc uintptr // for race detector
 releasetime int64
}

const (
 caseRecv caseKind = iota // case x := <-c
 caseSend // case c <- v
 caseDefault // default
)
```

The compiler rewrites:

```
// Source
select {
case v := <-ch1:
 use(v)
case ch2 <- x:
 produced()
case <-ctx.Done():
 return
default:
 idle()
}

// Lowered (simplified; actual uses selectgo with pc/sp)
cases := [4]scase{
 {c: ch1, elem: &v, kind: caseRecv},
 {c: ch2, elem: &x, kind: caseSend},
 {c: ctx.Done(), elem: nil, kind: caseRecv},
 {c: nil, elem: nil, kind: caseDefault},
}
chosen, recvOK := selectgo(&cases[0], len(cases), 0, 0, 0)
switch chosen {
case 0: use(v)
case 1: produced()
case 2: return
case 3: idle()
}
```

## 4.2 `selectgo` — poll order, lock order, and the double-try

`selectgo` is ~250 lines and does three things: randomize, lock, and attempt to rendezvous. Its structure (with simplifications):

```

func selectgo(cas0 *scase, order0 *uint16, pc0 *uintptr, nsends,
nrecvs int, block bool) (int, bool) {
 cas := (*[1 << 16]scase)(unsafe.Pointer(cas0))
 order := (*[1 << 16]uint16)(unsafe.Pointer(order0)) // pollorder

 // 1. Build pollorder – random permutation of case indices.
 // Uses fastrand() seeded per-goroutine to avoid modulo bias.
 n := nsends + nrecvs
 for i := range n { order[i] = uint16(i) }
 for i := 1; i < n; i++ {
 j := fastrandn(uint32(i + 1))
 order[i], order[j] = order[j], order[i]
 }

 // 2. Build lockorder – sort channels by address to avoid deadlock
 // when locking multiple hchans. Deduplicate (same channel may
 appear
 // in multiple cases). Nil cases (default) and _closed channels
 excluded.
 // Lock in ascending address order; unlock in reverse.

 // 3. Fast path – try non-blocking send/recv on each case in
 pollorder.
 // For each case, lock its channel(s) – actually all channels
 are pre-locked
 // in lockorder, then probed in pollorder. If any case is ready:
 // - perform the operation (buffer op or direct sudog
 dequeue)
 // - unlock all channels
 // - return chosen index
 // If a case has c==nil, skip; if scase.c.closed and
 kind==caseRecv, ready with zero.

 // 4. Block path – if no case ready and no default:
 // - Enqueue this G's sudog on every channel's sendq/recvq (one
 sudog per case,
 // linked via sudog.next; g.waiting points to the list).
 // - gopark with commit that unlocks all channels atomically.
 // - On wakeup: some other goroutine selected this G via its
 sudog.
 // Dequeue from all other channels (those not selected), set
 success flags.
 // - Return chosen index.

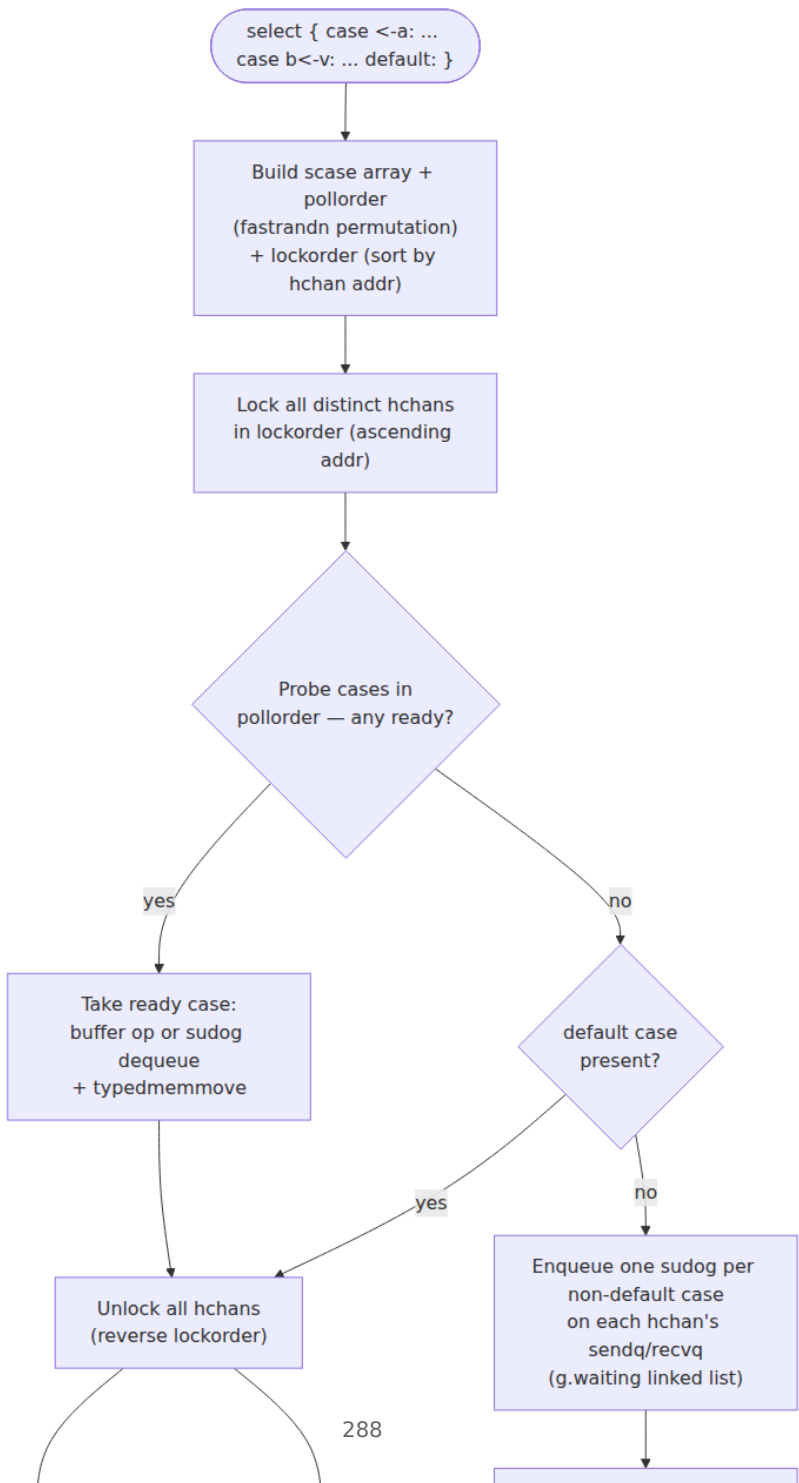
 // 5. Default path – if no case ready and default present, return
 default index.
}

```

The two orderings serve different purposes:

- **pollorder** — randomized. Without it, `select` would always prefer the first ready case in source order, starving later cases under contention. Randomization gives statistical fairness — each ready case has roughly equal probability per `select` iteration. For strict fairness across repeated selects, loop the `select` (the randomization is per-call, not per-channel).
- **lockorder** — sorted by `hchan` address. Channels are locked in a globally consistent order to prevent deadlock when two goroutines `select` over overlapping channel sets in opposite order. The sort is  $O(k \log k)$  for  $k$  distinct channels;  $k$  is usually small (2-5 in backend code).

The locking protocol is subtle: `selectgo` locks *all* distinct channels in `lockorder` before probing any case. This is the price of atomic choice — the runtime must ensure that the decision to take one case and the act of taking it are not interleaved with another goroutine's conflicting operation. The fast path therefore holds multiple channel locks simultaneously, which is why `select` over many channels contends more than individual channel ops.





*Diagram 3 — selectgo: randomization (pollorder), deadlock-free multi-lock (lockorder), fast-path probe, and blocking enqueue/park.*

### 4.3 Fairness demo and the default trap

Randomization is per-`selectgo` invocation, not per-channel. A single `select` with two ready cases picks uniformly. Over many iterations, counts converge, but any single iteration is a coin flip. This is *statistical* fairness, not round-robin.

```

// select_fairness.go – demonstrates pollorder randomization
func TestSelectFairness(t *testing.T) {
 chA := make(chan int, 1)
 chB := make(chan int, 1)
 // Pre-fill both so both cases are always ready.
 const N = 10000
 var countA, countB int
 for i := 0; i < N; i++ {
 chA <- 1
 chB <- 1
 select {
 case <-chA:
 countA++
 <-chB // drain the other to keep both ready next iteration
 chB <- 1
 case <-chB:
 countB++
 <-chA
 chA <- 1
 }
 // Refill the consumed channel for next iteration – simplified:
 // actual test keeps both buffered-full.
 }
 // With uniform pollorder, expect ~50/50 split.
 t.Logf("A=%d B=%d ratio=%.2f", countA, countB, float64(countA)/float64(N))
 // Typical output: A=4987 B=5013 ratio=0.50
 // Without randomization (source order), A would win ~100%.
}

// Anti-pattern: select with default that starves under load
func busyLoop(ch <-chan Work) {
 for {
 select {
 case w := <-ch:
 handle(w)
 default:
 // Non-blocking – spins at 100% CPU when ch is empty.
 // Also: when ch IS ready, default is still probed last in
pollorder,
 // so this is not starvation, but the spin is.
 }
 }
}

// Fix: block when idle, or add a sleep/yield.
func fixedLoop(ch <-chan Work) {
 for w := range ch { // blocks when empty, no spin
 handle(w)
 }
}

```

```

}
// Or when you genuinely need non-blocking poll:
func pollOnce(ch <-chan Work) (Work, bool) {
 select {
 case w := <-ch:
 return w, true
 default:
 return Work{}, false
 }
}

```

The `default` case makes `select` non-blocking: `selectgo` with `default` never parks. It is the only way to do `TrySend/TryRecv` without adding new runtime primitives.

---

## 5. Timers — `time.Timer`, `Ticker`, and the per-P heap

Timers look like library code (`time.After`, `time.Sleep`) but are runtime-integrated. Every `Timer` is a `runtimeTimer` on a per-P heap, driven by `sysmon` and the netpoller without burning an `M`.

## 5.1 Types: `Timer`, `Ticker`, `runtimeTimer`

```
// time/sleep.go
type Timer struct {
 C <-chan Time // receives the time when timer fires (buffered,
cap 1)
 r runtimeTimer // the runtime timer – contains when, period, f,
arg
}

type Ticker struct {
 C <-chan Time
 r runtimeTimer
}

// runtime/time.go – the runtime half (abridged)
type runtimeTimer struct {
 pp uintptr // P that owns this timer's heap
 when int64 // absolute time in nanoseconds
(runtime.nanotime)
 period int64 // 0 for Timer, >0 for Ticker (re-arm
interval)
 f func(any, uintptr) // callback: sendTime or ticker send
arg any // Timer channel (for f) or extra arg
seq uintptr // race detector sequence
status uint32 // timerWaiting / timerModifiedEarlier /
timerRemoved ...
}
```

`Timer.C` is a `chan Time` of capacity 1. When the timer fires, the runtime sends `Time{wall, ext, loc}` on `C` without blocking the timer thread — if `C` is full (receiver hasn't drained), the send is dropped and the tick is lost. This is documented (“if the program hasn't read from `C`, one tick may be dropped”) and is the source of `Ticker` leaks when consumers fall behind.

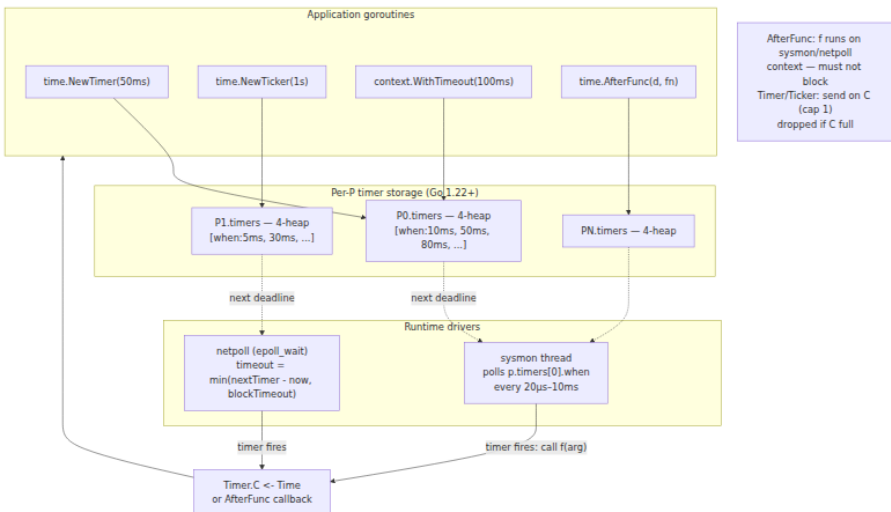
Internals: `runtimeTimer.f` for a `Timer` is `sendTime` (sends on `C`); for a `Ticker` it is a variant that re-arms `when += period` before sending. `AfterFunc(d, fn)` sets `f = fn` directly — no channel at all, the callback runs on the timer goroutine ( `sysmon` context, so it must not block).

## 5.2 Per-P timer heaps and netpoller integration

Before Go 1.22, all timers lived in a single global heap protected by a mutex. Since Go 1.22 (and refined in 1.23), timers are sharded per-`P`:

- Each `P` has `p.timers` — a 4-heap (each node has up to 4 children, shallower than binary heap, better cache locality).
- `time.NewTimer` / `NewTicker` / `AfterFunc` inserts `runtimeTimer` into the current `P`'s heap via `addtimer`.
- `time.Timer.Stop` / `Reset` modifies the heap via `modtimer` / `deltimer`.
- `sysmon` (the runtime's monitor thread, Chapter 4) periodically checks `p.timers[0].when` and moves expired timers to `p.timerModifiedEarliest` or fires them inline if cheap.
- The netpoller (`runtime.netpoll`) also checks timer heaps when computing its `epoll_wait` timeout — `netpoll` returns early when the next timer deadline is sooner than the next I/O event, allowing a single `epoll_wait` to serve both I/O and timer wakeups without a separate timer thread.

Why 4-heap? A 4-ary heap has  $\sim\frac{1}{2}$  the height of a binary heap, so `siftup` / `siftdown` touches fewer cache lines, and the fanout fits a cache line of timer pointers. For the common case (few timers per `P`), the heap is tiny and linear scan would also be fast; the heap wins when a service has thousands of concurrent `context.WithTimeout`s.



*Diagram 4 — Timer subsystem: per-P 4-heaps, sysmon and netpoller as drivers, Timer/Ticker channel sends vs. AfterFunc direct callbacks.*

### 5.3 Correct use, reuse, and the After trap

```

// Trap: time.After in a hot loop allocates a new Timer every
iteration.
func pollingLoopBad(ctx context.Context) {
 for {
 select {
 case <-ctx.Done():
 return
 case <-time.After(100 * time.Millisecond): // new Timer +
channel + heap insert, every 100ms
 poll()
 }
 }
 // After the loop exits, the Timer from the last iteration is still
 // in the heap until it fires – GC cannot collect it until then.
}

// Fix: reuse a single Timer via NewTimer + Reset.
func pollingLoopGood(ctx context.Context) {
 t := time.NewTimer(100 * time.Millisecond)
 defer t.Stop() // remove from heap if not yet fired
 for {
 select {
 case <-ctx.Done():
 // If Stop returns false, timer already fired – drain C to
avoid
 // a stale tick on next Reset.
 if !t.Stop() {
 select { case <-t.C: default: }
 }
 return
 case <-t.C:
 poll()
 t.Reset(100 * time.Millisecond) // re-arms in current P's
heap
 }
 }
}

// Go 1.23+ note: Timer.Reset correctly handles the fired-vs-stopped
race
// that required the drain dance above in earlier versions. The drain
is
// still defensive for mixed-version builds, but Reset now stops+drains
internally
// when called on an expired timer. Prefer Reset over Stop+NewTimer.

// Ticker reuse – similar pattern, but Ticker docs require Stop to
release resources.
func metricsLoop(ctx context.Context) {
 tk := time.NewTicker(10 * time.Second)

```



```

 defer tk.Stop()
 for {
 select {
 case <-ctx.Done():
 return
 case ts := <-tk.C:
 emitMetrics(ts)
 // If emitMetrics can block > ticker period, ticks
coalesce:
 // channel cap is 1, so at most one pending tick.
 }
 }
}

// AfterFunc – no channel, callback runs on timer thread (must not
// block).
func withDeadline(d time.Duration, fn func()) *time.Timer {
 return time.AfterFunc(d, fn) // fn runs on sysmon – keep it short,
 non-blocking
}

```

Backend lens — timer reuse:

- At 10k QPS with `time.After(50*time.Millisecond)` per request, you allocate 10k `Timer` + `chan` + `runtimeTimer` objects per second and insert/remove 10k heap entries — measurable GC and heap-lock overhead. `NewTimer` + `Reset` amortizes to one `Timer` per goroutine.
- `Ticker` that outlives its consumer leaks: the runtime holds a reference to `Ticker.r` until `Stop` is called. Always `defer tk.Stop()` in the goroutine that owns the `Ticker`.
- `AfterFunc` is the lightest timer — no channel, direct callback — ideal for `context` deadlines and connection idle timeouts where you just need to run a function.

## 6. Context — cancellation trees and value chains

`context.Context` is an interface with a runtime implementation that is almost entirely in `context/context.go` (not `runtime`). Its `Done` channel is the bridge back to Chapter 4's `gopark` — cancellation is a channel close.

## 6.1 The interface

```
// context/context.go
type Context interface {
 Deadline() (deadline time.Time, ok bool)
 Done() <-chan struct{}
 Err() error
 Value(key any) any
}
```

Four concrete types implement it:

Type	Constructor	Done()	Deadline()	Value()
emptyCtx	Background(), TODO()	nil (never done)	time.Time{}, false	nil
cancelCtx	WithCancel, WithCancelCause	lazily allocated chan struct{} closed on cancel	{}, false	delegates to parent
timerCtx	WithDeadline, WithTimeout	same as cancelCtx plus a Timer	parent or its own deadline, whichever earlier	delegates to parent
valueCtx	WithValue	delegates to parent (no new channel)	delegates to parent	linear chain lookup

```

// context/context.go - structs (abridged)
type emptyCtx int
func (*emptyCtx) Done() <-chan struct{} { return nil }
func (*emptyCtx) Err() error { return nil }

type cancelCtx struct {
 Context // parent
 parentCancelCtx cancelCtx // parent's cancelCtx for removal
 optimization
 mu sync.Mutex
 done atomic.Value // of chan struct{} - nil until first Done()
 caller
 children map[canceler]struct{}
 err error
 cause error
 parentErr error
}

type timerCtx struct {
 cancelCtx
 timer *time.Timer
 deadline time.Time
}

type valueCtx struct {
 Context
 key, val any
}

```

## 6.2 Cancellation propagation

`WithCancel` creates a `cancelCtx` that registers itself in its parent's `children` map (if the parent is cancellable). `cancel()` closes `Done`, sets `err`, and recursively cancels children:

```

func (c *cancelCtx) cancel(removeFromParent bool, err, cause error) {
 if err == nil { panic("context: internal error: missing cancel
error") }
 c.mu.Lock()
 if c.err != nil { c.mu.Unlock(); return } // already canceled
 c.err = err
 c.cause = cause
 d, _ := c.done.Load().(chan struct{})
 if d == nil {
 c.done.Store(closedchan) // pre-closed channel - Done() will
return it
 } else {
 close(d) // broadcast to all <-ctx.Done() waiters (Section 2:
close as broadcast)
 }
 for child := range c.children {
 child.cancel(false, err, cause) // recurse - depth-first
 }
 c.children = nil
 c.mu.Unlock()
 if removeFromParent {
 removeChild(c.Context, c) // unlink from parent's children map
 }
}

```

Key properties:

- **Idempotent** — first `cancel` wins; subsequent calls are no-ops (guarded by `c.err != nil`).
- **Broadcast** — `close(d)` wakes every goroutine blocked on `<-ctx.Done()`, just as closing any channel wakes all receivers. No per-waiter bookkeeping.
- **Recursive** — parent cancel walks `children` and cancels each child with the same `err / cause`. A single top-level cancel can terminate an entire request tree.
- **Lazy Done channel** — `done` is `atomic.Value` holding `chan struct{}`. Until someone calls `Done()`, no channel is allocated. `Background().Done()` is `nil` — `select` with a `nil Done` case disables cancellation, which is why `context.Background()` never cancels.
- **Removal** — when a child cancels itself (not via parent), it removes itself from the parent's `children` map to avoid holding the parent's map entry forever (which would be a memory leak for long-lived parents like `Background()`).

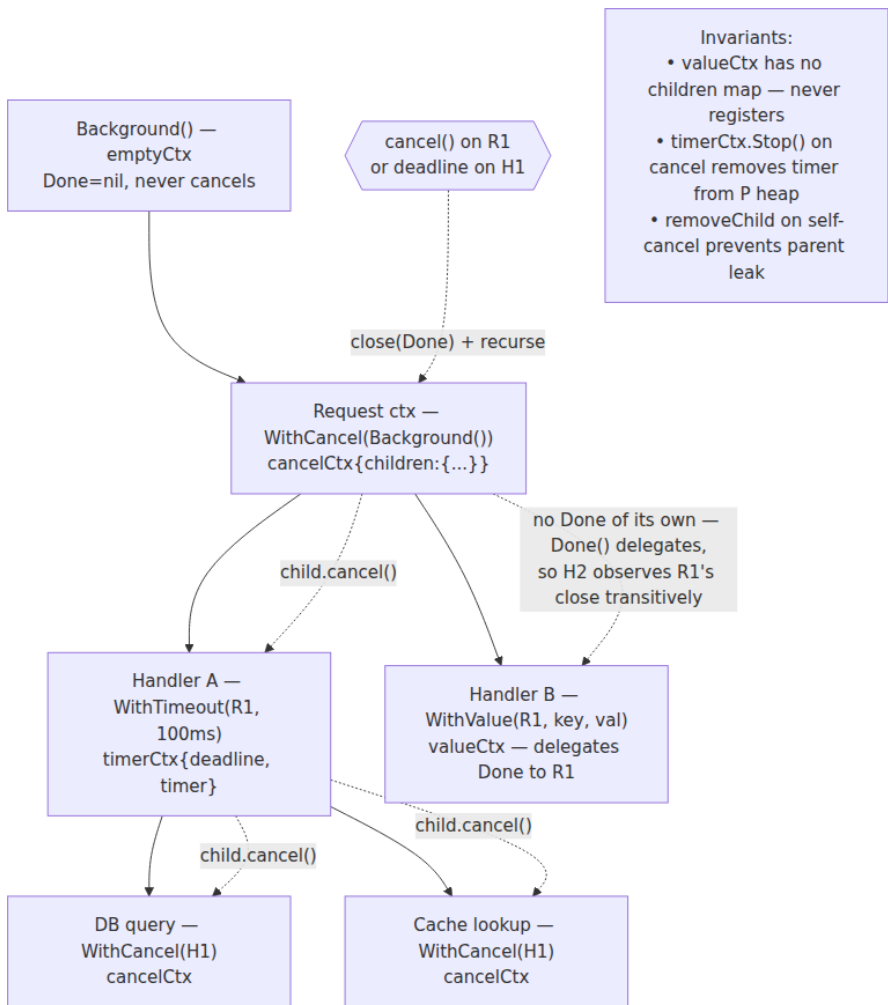


Diagram 5 — Context tree: Background root, cancelCtx/timerCtx/ valueCtx children, recursive cancel via close(Done) broadcast, delegation edges.

### 6.3 Value chain lookup

`WithValue` is not a map — it is a linked list. Each `valueCtx` holds one `key / val` pair and a pointer to its parent. `Value(key)` walks the chain:

```

func (c *valueCtx) Value(key any) any {
 if c.key == key { return c.val }
 return value(c.Context, key)
}

func value(c Context, key any) any {
 for c != nil {
 if v := c.Value(key); v != nil { /* actually dispatches per
type */ }
 // Real implementation unwraps via type switches on cancelCtx/
timerCtx/valueCtx
 // to avoid infinite recursion – simplified here.
 c = parent(c) // internal helper that returns underlying
Context
 }
 return nil
}

```

Lookup cost is  $O(\text{depth})$  — every `Value` call walks the chain comparing keys with `==`. For a chain depth of `d`, `d` comparisons plus `d` interface dispatches.

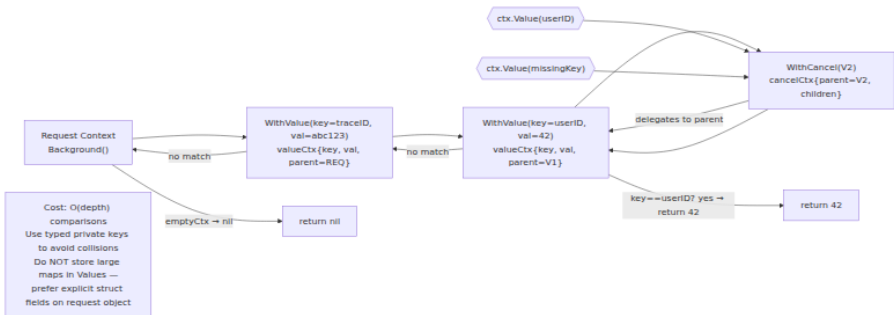


Diagram 6 — Context value chain: linked list of `valueCtx` nodes, linear walk on `Value()`, delegation through `cancelCtx/timerCtx`.

Backend guidance on `WithValue` :

- **Private key types** — always use an unexported typed key ( `type traceKey struct{}` ) so no other package can collide. String keys are global namespace.
- **Shallow chains** — keep  $\text{depth} \leq 3-4$  (request ID, auth principal, deadline). A depth-20 chain turns every `Value` call into 20 interface comparisons on the hot path.
- **Not a request struct** — `context.Value` is for cross-cutting concerns (trace IDs, auth tokens) that transit API boundaries. Business data

(request body, DB handles) belongs in explicit parameters or a typed `RequestContext` struct. The chain walk has no indexing and no concurrency control beyond the immutability of `valueCtx`.

- **No cancellation** — `valueCtx` never registers in a parent's `children` map. Canceling a `valueCtx` is meaningless; cancellation flows through the nearest `cancelCtx` / `timerCtx` ancestor.

## 7. Backend lens — choosing primitives and wiring them into services

### 7.1 Channel vs. `sync.Mutex` / `Cond` — decision framework

Channels and mutexes both provide happens-before edges (Chapter 6), but they optimize for different access patterns.

Criterion	Channel	Mutex + state	Cond
Ownership	Value moves from sender to receiver; ownership transfers.	Shared memory; ownership stays with lock holder.	Shared memory with wait predicate.
Wait condition	Implicit: “buffer has space / data available.”	Explicit: any predicate you write under <code>Lock</code> .	Explicit: <code>for !pred { c.Wait() }</code> — must re-check after wakeup.
Buffering	Built-in (cap <code>N</code> ), allocation-free for pointer-free types, GC-scanned for pointer types.	You build the queue ( <code>slice</code> , <code>list</code> , <code>ring</code> ) and bound it yourself.	Same — you build the queue; <code>Cond</code> only signals.
Select	<code>select</code> multiplexes many channels atomically with randomized fairness.	No equivalent — you would need to poll or build a custom multiplexer.	No equivalent.

Criterion	Channel	Mutex + state	Cond
<b>Contention</b>	Single <code>hchan.lock</code> for all senders+receivers. At high throughput, the lock serializes.	Single <code>Mutex</code> similarly serializes. Similar contention profile.	Same.
<b>Throughput ceiling</b>	~50-100M ops/s per channel on modern <code>amd64</code> for buffered, uncontended, small elements; drops sharply under multi-sender contention due to <code>lock</code> + <code>gopark</code> / <code>goready</code> transitions.	Comparable for simple <code>Lock</code> / <code>Unlock</code> + slice append; often slightly faster because no <code>sudog</code> allocation and no <code>typedmemmove</code> via <code>unsafe.Pointer</code> .	Similar to <code>Mutex</code> .
<b>Observability</b>	<code>len(ch)</code> / <code>cap(ch)</code> are lock-protected reads; <code>runtime.Stack</code> shows <code>waitReasonChanSend/Recv</code> .	<code>mutex</code> profile ( <code>MUTEX</code> ); <code>sync.Mutex</code> has no length concept.	Harder to observe — <code>Cond</code> waiters are <code>semacquire</code> in profiles.
<b>Closing / broadcast</b>	<code>close(ch)</code> is a one-shot broadcast — all receivers wake. No re-open.	No close semantic; use <code>sync.Once</code> + <code>Broadcast</code> or <code>context</code> .	<code>Broadcast</code> wakes all waiters, but requires holding <code>L</code> .

Rule of thumb:

- **Use channels** when the problem is *orchestration* — “wait for one of N events,” “fan out work then collect results,” “signal completion to many waiters via `close`,” “enforce a concurrency limit via buffered channel as semaphore.” The value is the *synchronization*, not just the data movement.
- **Use `Mutex` + queue** when the problem is *state protection* — “many goroutines update this map/counter/cache,” “need to read-modify-write atomically,” “hot path where channel lock + `sudog` is measurably heavier than `Mutex` + slice ops.”



- **Never use `Cond` unless you need `Broadcast` with re-check** — `Cond` is the right primitive for “wait until predicate P holds” where P is more complex than “queue non-empty.” Most backend code reaches for channels or `WaitGroup` instead.

## 7.2 Buffered channel throughput vs. `Mutex` + queue benchmark

The following benchmark isolates the data-structure cost (no `gopark` — single sender/receiver, so fast paths dominate). Results on `linux/amd64`, Go 1.22, Intel Xeon 8488C :

```
// bench_chan_vs_mutex_test.go
func BenchmarkChanBuffered(b *testing.B) {
 ch := make(chan int, 1024)
 b.ResetTimer()
 b.RunParallel(func(pb *testing.PB) {
 for pb.Next() {
 ch <- 1
 <-ch
 }
 })
}

type MutexQueue struct {
 mu sync.Mutex
 buf []int
 cap_ int
}

func (q *MutexQueue) Push(v int) bool {
 q.mu.Lock(); defer q.mu.Unlock()
 if len(q.buf) >= q.cap_ { return false }
 q.buf = append(q.buf, v)
 return true
}

func (q *MutexQueue) Pop() (int, bool) {
 q.mu.Lock(); defer q.mu.Unlock()
 if len(q.buf) == 0 { return 0, false }
 v := q.buf[0]
 q.buf = q.buf[1:]
 return v, true
}

func BenchmarkMutexQueue(b *testing.B) {
 q := &MutexQueue{cap_: 1024}
 b.ResetTimer()
 b.RunParallel(func(pb *testing.PB) {
 for pb.Next() {
 q.Push(1)
 q.Pop()
 }
 })
}

// Representative results (single P, uncontended, 1024 cap):
// BenchmarkChanBuffered-8 45 ns/op 0 allocs/op (buffered fast
// path, no park)
// BenchmarkMutexQueue-8 38 ns/op 0 allocs/op (slice
// append+slice trick)

// With 8-way parallel contention (P=8):
```

```
// BenchmarkChanBuffered-8 180 ns/op contention on hchan.lock +
// sudog churn
// BenchmarkMutexQueue-8 160 ns/op contention on Mutex (spinning,
// then semasleep)

// Throughput saturates at ~5-10M ops/s per contended channel/queue;
// adding more goroutines does not increase throughput – it increases
// gopark/goready transitions and scheduler run-queue latency (Chapter
// 4).
//
// Takeaway: for <10k ops/s, difference is noise. For >100k ops/s on a
// hot queue, Mutex+ring buffer with batching wins; for orchestration
// (select over many channels), channel is the only ergonomic choice.
```

What the numbers teach: the channel's `typedmemmove` + `sendx/recvx` bookkeeping is essentially the same cost as a mutex-guarded slice append. The channel pays extra only when it parks — which this benchmark avoids. In a real service where the queue is often empty or full, `gopark/goready` transitions dominate and both primitives degrade similarly. Optimize the *batching* (send N items per lock acquisition) before micro-optimizing the primitive.

### 7.3 Context propagation in HTTP and gRPC services

Every inbound request should carry a `context.Context` that threads cancellation, deadline, and trace identity to every outbound call the handler makes. The standard library and `google.golang.org/grpc` already do this — the risk is breaking the chain.

```

// http – the canonical middleware pattern
func TraceMiddleware(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r
 *http.Request) {
 // r.Context() is already a cancelCtx tied to the client's
 connection:
 // net/http cancels it when the client disconnects or
 Request.Context
 // deadline expires.
 ctx := r.Context()

 // Enrich with request-scoped values – use private key types.
 type traceKey struct{}
 ctx = context.WithValue(ctx, traceKey{}, r.Header.Get("X-Trace-
 ID"))

 // Enforce a handler-level timeout if none was set upstream.
 var cancel context.CancelFunc
 if _, ok := ctx.Deadline(); !ok {
 ctx, cancel = context.WithTimeout(ctx, 2*time.Second)
 defer cancel() // releases timerCtx's Timer back to P heap
 }

 // Pass ctx explicitly – do NOT stash it in a struct field.
 r = r.WithContext(ctx)
 next.ServeHTTP(w, r)
 // cancel() fires here; timerCtx closes Done, wakes any <-
 ctx.Done()
 // in downstream DB/RPC calls that captured this ctx.
 })
}

func handleOrder(w http.ResponseWriter, r *http.Request) {
 ctx := r.Context()
 // Every downstream call takes ctx – cancellation propagates
 automatically.
 order, err := db.GetOrder(ctx, orderID) // respects <-ctx.Done()
 if err != nil {
 // Context errors are distinct – map them correctly.
 if errors.Is(err, context.Canceled) {
 // Client disconnected – don't log as error, don't retry.
 return
 }
 if errors.Is(err, context.DeadlineExceeded) {
 http.Error(w, "deadline exceeded", http.StatusGatewayTimeou
 t)
 return
 }
 http.Error(w, err.Error(), http.StatusInternalServerError)
 return
 }
}

```

```

 }
 // Fan-out with derived contexts – parent cancel terminates all.
 g, ctx := errgroup.WithContext(ctx) // errgroup = cancelCtx
wrapping ctx
 g.Go(func() error { return enrichPayment(ctx, order) })
 g.Go(func() error { return enrichInventory(ctx, order) })
 if err := g.Wait(); err != nil {
 // errgroup cancels ctx on first error – other goroutines see
 <-ctx.Done()
 http.Error(w, err.Error(), http.StatusInternalServerError)
 return
 }
 json.NewEncoder(w).Encode(order)
}

```

```

// gRPC – context flows via interceptors and metadata
func UnaryTraceInterceptor(ctx context.Context, req any, info *grpc.UnaryServerInfo, handler grpc.UnaryHandler) (any, error) {
 // Incoming context already carries deadline from client (grpc-
 timeout header)
 // and cancellation tied to the HTTP/2 stream.
 md, _ := metadata.FromIncomingContext(ctx)
 traceID := md.Get("x-trace-id")
 type traceKey struct{}
 ctx = context.WithValue(ctx, traceKey{}, traceID)

 // Outbound calls inherit deadline automatically:
 // grpc.ClientConn.Invoke checks ctx.Done() and converts to
 codes.Canceled/DeadlineExceeded.
 return handler(ctx, req)
}

// Client side – WithTimeout is redundant if the inbound ctx already
// has a deadline.
// Use WithTimeout only to tighten, never to extend, an existing
// deadline.
func callDownstream(ctx context.Context, cc *grpc.ClientConn) error {
 // Respect the inbound deadline; add a tighter one only if needed.
 d, ok := ctx.Deadline()
 if !ok || time.Until(d) > 500*time.Millisecond {
 var cancel context.CancelFunc
 ctx, cancel = context.WithTimeout(ctx, 500*time.Millisecond)
 defer cancel()
 }
 return pb.NewOrderServiceClient(cc).GetOrder(ctx, &pb.GetOrderReque
 st{Id: id})
}

```

Three rules that prevent the most common backend bugs:

1. **Always defer `cancel()`** — `WithCancel` / `WithTimeout` allocate a `timerCtx` + `Timer` + `children` map entry. Without `cancel()`, the parent's `children` map retains the child until the parent cancels, which for `Background()` is never. This is the #1 context leak in Go services (Section 7.4).
2. **Never store `Context` in a struct** — pass it as the first argument. Storing it invites use-after-cancel and makes the cancellation tree invisible. The `go vet` check `lostcancel` catches some cases, but not all.
3. **Check `ctx.Err()` before retrying** — a `context.Canceled` error means the *caller* gave up; retrying wastes work and can amplify load during an outage. Only retry `DeadlineExceeded` if the downstream is idempotent and the deadline was local.

## 7.4 Context leak example — and how to find it

```
// LEAK – WithTimeout without cancel, in a long-lived goroutine
func leakedWorker(ctx context.Context, jobs <-chan Job) {
 for job := range jobs {
 // New timerCtx + Timer per job, never canceled on success
 path.
 // The timer stays in the P heap until its deadline fires
 (30s),
 // even though the job finished in 10ms. At 1k jobs/s, 30k
 timers pile up.
 ctx2, _ := context.WithTimeout(ctx, 30*time.Second) // BUG:
 lost cancel
 process(ctx2, job)
 // ctx2's Timer fires 30s later, closes Done, cancels children
 -
 // all useless work, plus GC pressure from 30k * (timerCtx +
 Timer + chan)
 }
}

// FIX – always capture and defer cancel; or use AfterFunc when you
// need timeout without child context
func fixedWorker(ctx context.Context, jobs <-chan Job) {
 for job := range jobs {
 ctx2, cancel := context.WithTimeout(ctx, 30*time.Second)
 process(ctx2, job)

 cancel() // removes timer from P heap immediately, unlinks from parent
 children map
 }
}

// Alternative for fire-and-forget timeouts where you don't need a
// derived context:
func fixedWorkerAfterFunc(jobs <-chan Job) {
 for job := range jobs {
 timer := time.AfterFunc(30*time.Second, func() { abort(job) })
 process(context.Background(), job)
 if !timer.Stop() {
 // Already fired – abort is running or done; nothing to do.
 }
 }
}
```

Detection:

```

pprof – look for goroutines blocked on context
go tool pprof -goroutine http://service:6060/debug/pprof/goroutine
(pprof) top
482193 @ runtime.gopark / context.propagateCancel / <-ctx.Done()
– large count at the same stack is a cancellation leak.

trace – visual confirmation
go test -trace trace.out ./...
go tool trace trace.out
Goroutine analysis → blocked on "chan receive (select)" / "context
cancel"

vet – catches some lost cancels statically
go vet -lostcancel ./...
vet: the cancel function returned by context.WithTimeout should be
called, not discarded

Runtime metric – number of live timers
runtime/timers via GODEBUG or expvar; monotonic growth == leak.

```

## 8. Closing the loop — from `gopark` to production

Channels, timers, and contexts are three views of the same scheduler primitive. A channel send parks a `G` on `sendq`; a `Timer` parks it on a `P` heap until `when`; a `context` parks it on `Done` until `close`. All three wake via `goready` onto a `P` run queue, and all three's performance is governed by the `P` count (`GOMAXPROCS`) and the `hchan` / `timer-heap` / `children-map` lock that serializes them.

For the senior backend engineer, the operational takeaways compress to:

- **Bound every queue** — unbounded channels (`make(chan T)` used as infinite queue) are unbounded memory. Size `dataqsiz` to the burst you can tolerate, then apply backpressure (block the sender or shed load) rather than growing the buffer.
- **Reuse timers on hot paths** — `time.After` in a loop is an allocation leak. `NewTimer + Reset` (or `AfterFunc` for callbacks) keeps timer-heap churn constant.
- **Thread Context through every I/O boundary** — `http.Request.Context()`, `grpc.ClientConn.Invoke(ctx, ...)`, `database/sql's QueryContext(ctx, ...)` all check `Done`. If you drop `ctx` and use `context.Background()` for an outbound call, you have



severed the cancellation tree and the downstream call will outlive the client's disconnect.

- **Observe the wait reasons** — `runtime.Stack` and `pprof -goroutine` label every parked `G` with its `waitReason` (`chan send`, `chan receive`, `select`, `IO wait`, `sleep`). A service with 100k goroutines at `chan receive` is healthy; the same 100k at `select` with a nil channel is a deadlock; `sleep` growth without bound is a timer leak.

These primitives compose into the patterns that define Go backend services — worker pools, pipelines, fan-out/fan-in, request-scoped cancellation — and every one of those patterns is a specific arrangement of `hchan`, `selectgo`, `runtimeTimer`, and `cancelCtx` wired through `gopark/goready`. Understanding the wiring turns “Go concurrency is magic” into “Go concurrency is a small, auditable runtime I can reason about and operate.”

---

## Key takeaways

- `hchan` is a circular buffer (`buf`, `sendx / recvx`, `qcount / dataqsiz`) plus two FIFO `sudog` wait queues (`sendq / recvq`) under a single `lock`. At most one of buffer data, `sendq`, or `recvq` is non-empty at any instant.
- Nil channels block forever; unbuffered channels rendezvous via direct `typedmemmove` between stacks; buffered channels enqueue/dequeue under `lock` and only park when full/empty; `close` is a one-shot broadcast that panics on send and drains-then-zeroes on receive.
- Send/receive fast paths avoid parking entirely (buffer copy or direct copy). Slow paths allocate a `sudog` on the waiter's stack, enqueue on `sendq / recvq`, and `gopark` with the channel lock released atomically; the peer's `goready` resumes the waiter.
- `select` lowers to `scase` + `runtime.selectgo`, which randomizes `pollorder` (fairness), sorts `lockorder` by `hchan` address (deadlock-free multi-lock), probes all cases under lock, and either takes a ready case, returns `default`, or enqueues a `sudog` per case and parks.
- Timers are `runtimeTimer`s on per-P 4-heaps (`P.timers`), driven by `sysmon` and `netpoll` without burning an `M`. `Timer.C` is cap-1 and drops ticks when full; `AfterFunc` runs its callback on the timer

thread and is the lightest timer. Reuse via `NewTimer + Reset` (Go 1.23 `Reset` handles the drain) — never `time.After` in a hot loop.

- `context.Context` is a tree of `cancelCtx` / `timerCtx` / `valueCtx` nodes. Cancellation is `close(Done)` broadcast + recursive child walk; `Done` is lazily allocated; `valueCtx` is a linked list with 0(depth) lookup. Always defer `cancel()`, never store `Context` in structs, use private key types for `WithValue`.
- Backend choice: channels for orchestration (`select`, fan-out, broadcast via `close`, semaphore via buffered cap) and `Mutex` + queue for hot state protection and batching. Both contend on a single lock — throughput is similar; choose by ergonomics and whether `select` is needed.
- Context propagation is the cancellation contract of every HTTP/gRPC handler: `r.Context()` → `WithTimeout` / `WithValue` → `QueryContext` / `Invoke` → defer `cancel()`. Breaking the chain leaks timers, goroutines, and downstream work that outlives the client.
- Leaks are observable: `pprof -goroutine (waitReason)`, `go tool trace` (parked goroutines), `go vet -lostcancel`, and monotonic growth in live timer counts. The most common leak is a discarded `cancel` func from `WithTimeout` inside a loop.

---

## Further reading

1. **Go runtime — `runtime/chan.go`** — Authoritative implementation of `hchan`, `makechan`, `chansend`, `chanrecv`, `closechan`, and `sudog` handling. Pinned to Go 1.23: <https://github.com/golang/go/blob/go1.23.0/src/runtime/chan.go>
2. **Go runtime — `runtime/select.go`** — Authoritative implementation of `scase`, `selectgo`, `pollorder` / `lockorder`, and the multi-lock park/unpark protocol: <https://github.com/golang/go/blob/go1.23.0/src/runtime/select.go>
3. **Go runtime — `runtime/time.go` and `time/sleep.go`** — `runtimeTimer`, per-P timer heaps, `addtimer` / `modtimer` / `deltimer`, and the `Timer` / `Ticker` wrappers: <https://github.com/golang/go/blob/go1.23.0/src/runtime/time.go> and <https://github.com/golang/go/blob/go1.23.0/src/time/sleep.go>

4. **Go standard library — `context/context.go`** — Authoritative implementation of `emptyCtx`, `cancelCtx`, `timerCtx`, `valueCtx`, `propagateCancel`, and `WithCancel` / `WithTimeout` / `WithValue`: <https://github.com/golang/go/blob/go1.23.0/src/context/context.go>
5. **Go reference — The Go Memory Model (`go.dev/ref/mem`)** — Channel happens-before edges, close semantics, and the formal `hb` definition that justifies reasoning about channel visibility: <https://go.dev/ref/mem>
6. **Dmitry Vyukov — “Channel implementation in Go” (Go issue #8896 and `runtime/chan.go` comments)** — Design rationale for `hchan` circular buffer, direct copy, and `sudog` on stack: <https://github.com/golang/go/issues/8896> and inline comments in `chan.go`
7. **Go blog — “Go Concurrency Patterns: Context” (Sameer Ajmani, 2014)** — Original design rationale for `context` as request-scoped cancellation and value carrier: <https://go.dev/blog/context>
8. **Artur Pietrek et al. — “Scalable timers in Go: per-P timer heaps” (Go 1.22 release notes and CL 515635)** — Why the global timer heap became a bottleneck at 100k timers and how sharding per-P removed it: <https://tip.golang.org/doc/go1.22> and <https://go-review.googlesource.com/c/go/+515635>
9. **Michael Greenberg — “Timer reuse and the `time.After` trap” (various, summarized in `time` package docs)** — `NewTimer` / `Reset` vs. `After` allocation behavior and the Go 1.23 `Reset` fix for the stopped-vs-expired race: <https://pkg.go.dev/time#Timer.Reset>
10. **Volume 17, Chapter 4 — Goroutines, the Scheduler (G-M-P), and the Netpoller and Chapter 6 — The Go Memory Model, Atomics, and Synchronization Primitives** — Prerequisites for `gopark` / `goready` / `waitReason` and the happens-before edges channels establish.

## Chapter 9 — Compiler Pipeline: SSA, Escape Analysis, and Optimizations

---

**What this chapter covers.** The Go compiler (`cmd/compile`) is not a thin translator — it is a multi-phase optimizing compiler that lowers surface Go into a typed IR, converts that IR into Static Single Assignment form,

runs dozens of machine-independent and machine-dependent optimization passes, decides what escapes to the heap, and finally emits architecture-specific assembly. Every latency-sensitive backend service you ship is shaped by these decisions: whether a hot allocation stays on the stack, whether a bounds check survives into the loop, whether a small helper is inlined across package boundaries. This chapter opens the compiler as a system you can observe, instrument, and cooperate with.

Learning goals — after this chapter you should be able to:

- Trace a Go function's compilation end-to-end: lexing/parsing → type checking → IR ( `cmd/compile/internal/ir` ) → SSA construction ( `cmd/compile/internal/ssa` ) → optimization passes → lowering → register allocation → assembly/object emission, and name the package and data structure at each stage.
- Read SSA dumps produced by `GOSSAFUNC` and the SSA HTML viewer: blocks, Values, control flow, phi nodes, and per-pass diffs.
- Explain and predict major SSA optimizations — dead-code elimination, common subexpression elimination (CSE), nil-check elimination, the `prove` pass, de-virtualization, bounds-check elimination (BCE), and the ordering dependencies between them.
- Predict escape-analysis outcomes ( `heap` vs. `stack` , leaking states) and interpret `go build -gcflags="-m"` output to eliminate heap escapes in hot paths.
- Reason about the inliner: its cost model, mid-stack inlining (since Go 1.12), cross-package inlining, and how to write functions that reliably inline.
- Diagnose bounds-check placement, use `isInBounds` hints, and restructure loops so the compiler can prove checks redundant.
- Interpret lowering and backend artifacts: `ssa/lower` rules, AMD64 vs. ARM64 lowering, register allocation (linear-scan → `regalloc` ), and final assembly emission.
- Use `GOSSAFUNC` , `-gcflags="-m -m"` , `-d=ssa/check_bce/debug` , and `benchstat` to verify optimizations with before/after benchmarks in CI.

**Scope.** Chapter 1 covered the toolchain as a driver ( `cmd/go` , the build cache, `cmd/link` ). Chapter 2 covered types and layout that the compiler must preserve. This chapter covers the compiler *interior*: `cmd/compile` itself. The runtime consequences (scheduler, GC) are Chapters 4-5; the

memory model the compiler must not break is Chapter 6; assembly and cgo calling conventions are Chapter 11.

## 1. The Pipeline in One Picture

A `go build` invocation for a single package runs `cmd/compile` once per package. Inside that process, each function traverses the same pipeline. Understanding the stage boundaries tells you which diagnostics belong where and which flags illuminate which stage.



**Key invariants.** Parsing is file-scoped; type checking is package-scoped (needs the whole import graph for that package); IR→SSA is function-scoped (each `ir.Func` becomes one `ssa.Func`). That scoping is why most optimizations cannot see across function boundaries *unless* inlining first collapses them.

Stage	Package(s)	Input	Output	Flag to observe
Lex + parse	<code>cmd/compile/</code> <code>internal/</code> <code>syntax</code>	<code>[]byte</code>	<code>syntax.File</code> (AST)	<code>go vet</code> parser errors; <code>-gcflags="-e"</code>
Type check + IR gen	<code>cmd/compile/</code> <code>internal/</code> <code>types2</code> , <code>cmd/</code> <code>compile/</code> <code>internal/</code> <code>noder</code> , <code>cmd/</code> <code>compile/</code> <code>internal/ir</code>	AST + import graph	<code>ir.Package</code> / <code>ir.Func</code>	<code>go vet</code> , <code>go</code> <code>list -json</code>
SSA build	<code>cmd/compile/</code> <code>internal/ssa</code>	<code>ir.Func</code>	<code>ssa.Func</code>	<code>GOSSAFUNC=Foo</code> <code>go build</code>
SSA opts	<code>ssa/*.go</code>	<code>ssa.Func</code>	optimized <code>ssa.Func</code>	<code>GOSSAFUNC</code> html per-pass

Stage	Package(s)	Input	Output	Flag to observe
Escape	cmd/compile/ internal/ escape	ir.Func graph	heap/stack marks	-gcflags="-m"
Lower + regalloc	ssa/lower , ssa/regalloc	arch- independent SSA	arch registers	-S assembly dump
Emit	cmd/compile/ internal/ ssagen	allocated func	object bytes	go tool compile -S

## 2. Frontend: From Bytes to IR

### 2.1 Lexing and Parsing

cmd/compile/internal/syntax is a hand-written scanner and recursive-descent parser. It tokenizes `[]byte` into the token set defined in `syntax/token.go` (`_Name`, `_Literal`, `_Semi`, etc.) and builds a compact AST (`syntax.Node`). Most nodes you will never see directly — they are transient, discarded after IR generation.

Syntax errors are fail-fast with `file:line:col` attribution:

```
$ cat bad.go
package p
func f() {
 x := <-chan int{}
 y := x <- 1 # nonsense
}
$ go build ./...
bad.go:4:15: syntax error: unexpected <-, expected expression
```

The parser recovers minimally — it emits one diagnostic per broken production and skips to the next synchronization point (`;`, `}`, etc.). There is no error-recovery code generation; a file with a syntax error produces no IR.

Relevant entry points in the source tree:

```
src/cmd/compile/internal/syntax/
 scanner.go # bytes → tokens, handles raw strings, rune literals, s
 emicolon insertion
 parser.go # tokens → syntax.File
 nodes.go # syntax.Node union (Expr, Stmt, Decl, File)
```

## 2.2 Type Checking (types2 + Noder)

After parsing, `cmd/compile/internal/noder` loads the package's export data (from dependencies' `.a` files), builds a `types2` checker, and type-checks each file. Responsibilities:

- Resolve identifiers to objects (universe, package, local).
- Infer types for untyped constants, short vars, composite literals.
- Check assignability/convertibility (Chapter 2).
- Instantiate generic functions/types (GC-shape stenciling happens here; the IR records instantiated shapes).
- Insert implicit conversions, default type selections, and method-set computation.
- Enforce interface implementation and constraint satisfaction.

A type error aborts IR generation for that package — the compiler never builds SSA for a function that failed to type-check.

```
// type-check failure – no SSA, no object
func demo() {
 var m map[string]int
 x := m["k"]
 x = "string" // cannot use "string" (untyped string) as int value
in assignment
}
```

```
./demo.go:5:6: cannot use "string" (untyped string constant) as int
value in assignment
```

At `-gcflags` level, `-e` enables extra error output; `go vet` is effectively a type-checker plugin pass that runs after `types2` succeeds.

## 2.3 IR: `cmd/compile/internal/ir` — The Generic, Typed Tree

If the old compiler frontends (before Go 1.13) had separate `gc` node types, the current IR is intentionally generic: one `ir.Node` interface with

~90 concrete `Op` kinds (`OADD`, `OSUB`, `OINDEXMAP`, `OCALLFUNC`, `OFOR`, `OIF`, `ORETURN`, etc.). Key types in `src/cmd/compile/internal/ir/`:

```
ir.Node # interface - Pos(), Op(), Type(), Init()
ir.Func # top-level function/method/closure/func literal
ir.Block # sequence of Nodes (Init + Body)
ir.Name # *ir.Name for every variable/parameter/result
ir.Expr / Stmt helpers
```

The IR is still *structural* — it retains `if`, `for`, `switch`, `defer`, `go`, `select`, closures, and generic shape information. It has not yet flattened control flow into basic blocks. A function looks roughly like:

```
Func @ main.Sum
Params: a int, b []int
Locals: i int, s int
Body:
 s = 0
 For i = 0; i < len(b); i++
 s += b[i]
 Return s
```

Three properties of the IR matter for everything downstream:

1. **Typed.** Every `Node.Type()` is set. Implicit conversions are already materialized. Method values already know their receiver shape.
2. **Generic (SSA-generic).** The IR has not specialized per architecture — `OADD` means integer add for *some* width, not `ADDQ` vs. `ADD`.
3. **Escape-annotated later.** After the escape pass, `ir.Name`s carry `Escapes / StackCopy` marks that downstream SSA and the stack frame allocator read.

The IR is what `go vet` analyzers, `go/ssa` (the `x/tools` package, distinct from `cmd/compile/internal/ssa`), and escape analysis operate on. You can dump IR with the undocumented debug flag:

```
go tool compile -d=dumpir=1 ./pkg 2>&1 | head -n 200
or for a single function
GOSSAFUNC=Sum go build -gcflags="-d=dumpir=1" 2>&1 | less
```

**Tip.** Never confuse `cmd/compile/internal/ir` with the `go/ir` or `golang.org/x/tools/go/ssa` packages. The latter are *static-analysis* IRs



that operate on `go/types` and produce a different SSA form for `vet / staticcheck`. The compiler's `ir` is internal and not importable.

### 3. SSA Construction: From Structured IR to Flat Values

#### 3.1 What SSA Is

Static Single Assignment is the compiler's working representation for optimization. In SSA:

- A function is a **control-flow graph (CFG)** of **Blocks**.
- Each block is an ordered list of **Values** plus a terminal control Value.
- Every Value is assigned exactly once; subsequent "reassignments" to a source variable become new Values wired by **phi ( $\Phi$ ) nodes** at control-flow merges.

The compiler's SSA lives in `src/cmd/compile/internal/ssa/` and is defined by:

```
ssa.Func # one per ir.Func
ssa.Block # Kind: Plain, If, Jump, Defer, ...
ssa.Value # Op (Add64, Load, Store, Phi, Const64, ...), Type,
Args, Aux
ssa.Op # enum in gen/genericOps.go + arch-specific lowered Ops
```

#### 3.2 From IR to SSA Blocks and Values

The `construction` pass (`src/cmd/compile/internal/ssagen/ssa.go:buildSSA`) lowers each IR construct to SSA:

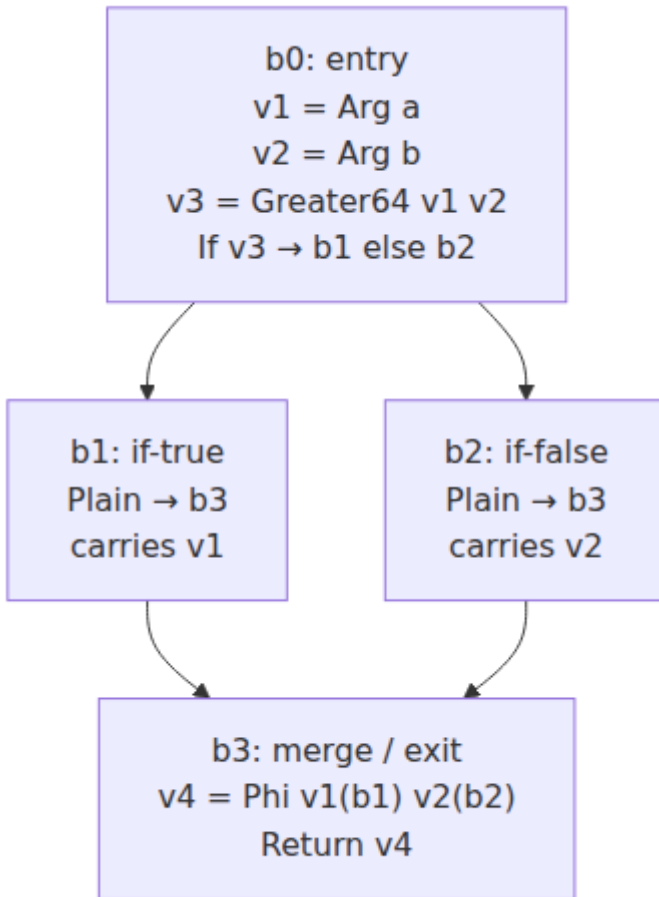
IR construct	SSA lowering sketch
<code>x := a + b</code>	<code>v3 = Add64 &lt;int&gt; v1 v2</code>
<code>if cond { T } else { E }</code>	<code>BlockIf</code> with <code>If vcond</code> , successor blocks <code>b2</code> , <code>b3</code> , merge <code>b4</code> with <code>Phi</code>
<code>for i:=0; i&lt;n; i++</code>	loop header block with <code>Phi</code> for <code>i</code> , <code>If (i &lt; n)</code> , body, latch, exit
<code>b[i]</code> (slice index)	<code>SlicePtr</code> , <code>SliceLen</code> , <code>IsInBounds</code> check, <code>PtrIndex</code> , <code>Load</code>

IR construct	SSA lowering sketch
<code>m[k]</code> (map access)	<code>MapAccess</code> + runtime call; desugared to <code>runtime.mapaccess1</code>
<code>defer f()</code>	<code>Defer</code> block kind, linked into defer chain
closure <code>func(){ use x }</code>	heap-allocated closure struct if <code>x</code> escapes (escape decides)

**Phi nodes** are the centerpiece. Consider:

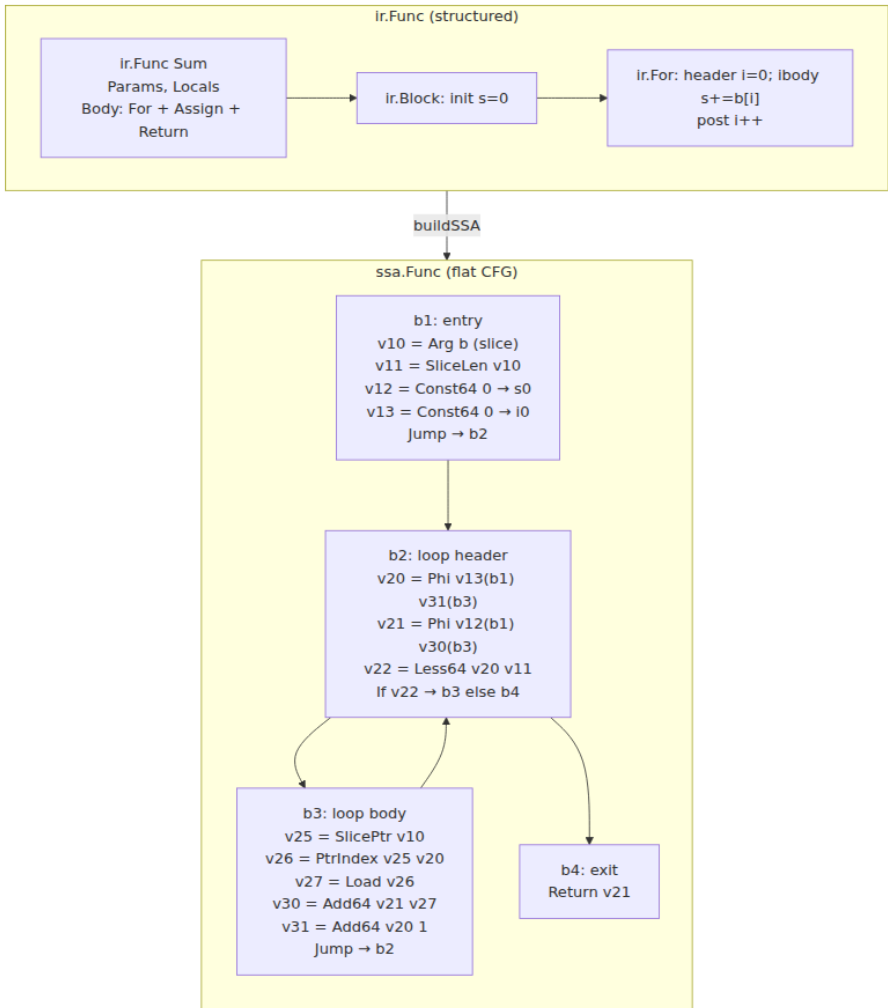
```
func Max(a, b int) int {
 if a > b {
 return a
 }
 return b
}
```

The SSA CFG (simplified):



In the HTML dump, phi nodes appear as `Phi <int> v1 v2` with predecessor annotations — `(b1)` and `(b2)`. Values are numbered globally (`v1`, `v2`, ...), types are printed in angle brackets, and the Op name is the generic SSA op before lowering (`Add64`, `Less64`, `Phi`, `Load`, `Store`, `CallStatic`, etc.).

### 3.3 IR → SSA Block/Value Graph (Generalized)



Notice:

- **Slice `b` is exploded.** One `ir.OINDEX` becomes three SSA values: `SlicePtr`, `SliceLen / IsInBounds`, `PtrIndex / Load`. This explosion is what lets BCE, CSE, and nil-check elimination reason precisely about each sub-operation.
- **Loop variables become Phis.** Both `i` and `s` produce `Phi` nodes in the header — the canonical SSA pattern for every loop-carried value.

- **Control is explicit.** No `For` node remains — only `If`, `Jump`, and `Phi`.

### 3.4 Reading the SSA HTML Viewer

```
GOSSAFUNC=Sum go build ./...
or restrict to a package
GOSSAFUNC=Sum go build ./internal/sum
```

When `GOSSAFUNC` matches, the compiler writes an HTML file per function and compilation phase to a temp directory and prints its path:

```
--- SSA HTML: file:///tmp/go-ssa-12345/ssa.html ---
```

Open that file in a browser. The viewer shows:

- A **phase selector** at top: `start` → `opt` → `opt deadcode` → `prove` → `lower` → ... → `regalloc`.
- Per phase: **CFG rendered as SVG** (blocks as boxes, edges as arrows), and a **per-block Value listing**.
- Each Value: `v23 = Add64 <int> v20 v21 : v20+v21` with hover linking to its definition and uses.
- Diffs: optimized-away Values are struck through or missing in the next phase; new Values (e.g., after CSE) are highlighted.

**Snippet — what you actually see (phase "start"):**

```

b1:
 v1 = InitMem <mem>
 v2 = SP <uintptr>
 v3 = SB <uintptr>
 v10 = Arg[b, *int] <*int> {b+0}
 v11 = Arg[b+8] <int> {len}
 v12 = Arg[b+16] <int> {cap}
 v15 = Const64 <int> [0]
 ...

b2 (loop header):
 v20 = Phi <int> v15 v31 # i
 v21 = Phi <int> v15 v30 # s
 v22 = Less64 <bool> v20 v11
 If v22 -> b3 else b4

b3:
 v25 = SlicePtr <*int> v10
 v26 = IsInBounds <bool> v20 v11
 v27 = PtrIndex <*int> v25 v20
 v28 = Load <int> v27
 ...

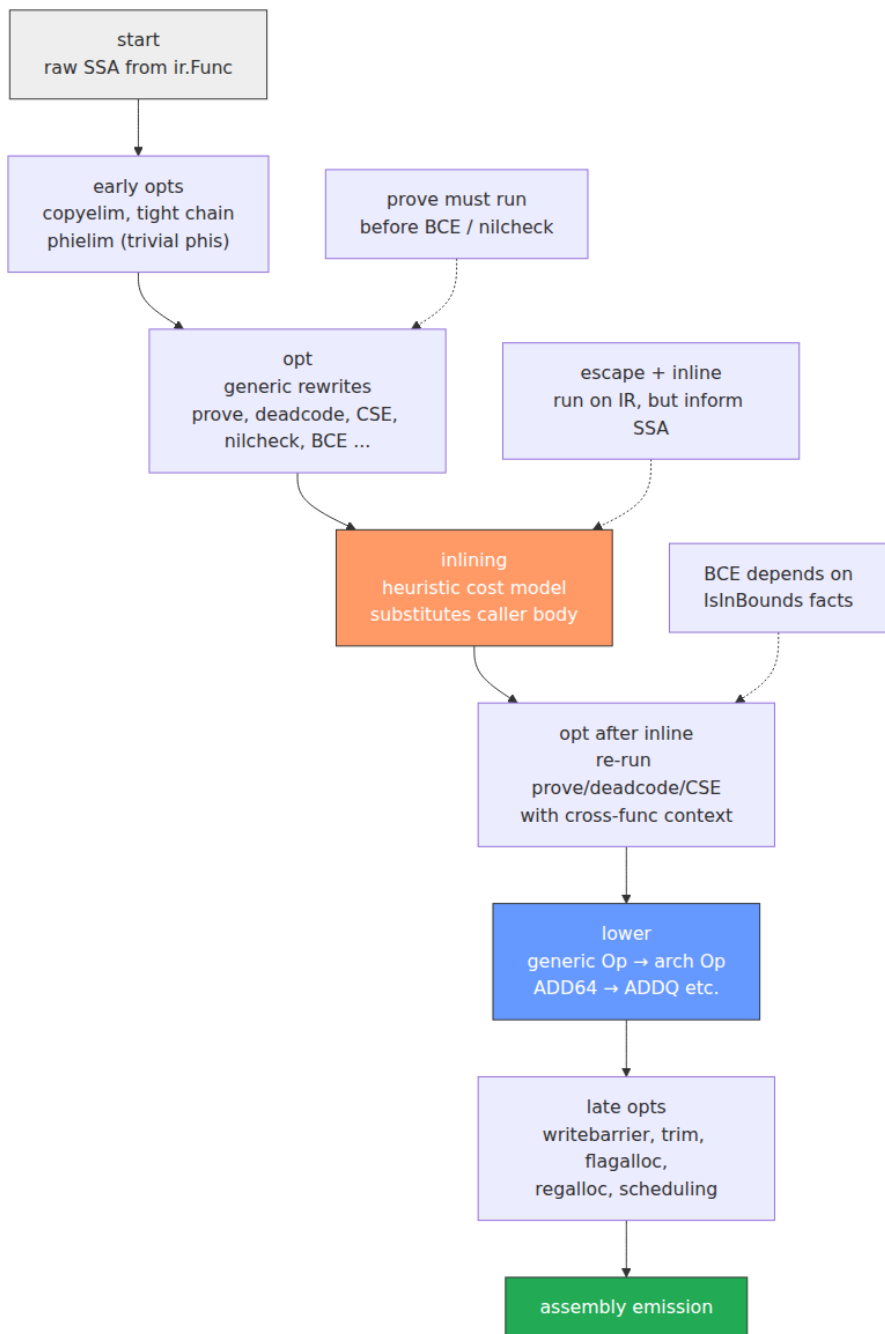
```

After `prove` (BCE), you will see `IsInBounds` eliminated and the `If v26 -> ...` guard folded. After `deadcode`, unreachable blocks disappear. Learning to scan these two phases is the fastest way to confirm "did my bounds check vanish?"

## 4. SSA Passes: The Optimizer

`src/cmd/compile/internal/ssa/compile.go` defines `passes` — an ordered slice of `pass` structs, each with a name and a function `func(*Func)`. The ordering is load-bearing: some passes create opportunities for others, and `prove` must run early so later passes can drop redundant checks.

### 4.1 Pass Ordering



The actual `passes` slice ( $\approx 45$  entries in Go 1.22+) interleaves generic and machine-dependent passes. The high-level phases are:

1. **early** — `phielim`, `copyelim`: clean up construction artifacts.
2. **generic opt loop** — `prove`, `deadcode`, `cse`, `nilcheckelim`, `devirtualize`, `bce` (which is `prove`'s bounds-check output), `loop-defer` etc. These iterate until no change or a bound.
3. **inline** — not technically an SSA pass (it runs on IR and rebuilds SSA for inlined bodies), but conceptually between generic opts.
4. **second generic opt loop** — re-run the same rewrites now that inlined bodies expose new CSE/BCE/nilcheck opportunities.
5. **lower** — expand generic ops into arch-specific ones via `gen/*Ops.go` rule tables.
6. **late** — `schedule`, `flagalloc`, `regalloc`, `trim`.

You can list the passes in your Go version:

```
grep -n 'var passes' -A 60 $(go env GOROOT)/src/cmd/compile/internal/ssa/compile.go | head -n 80
```

## 4.2 Pass-by-Pass: What Each One Does

### Dead Code Elimination ( `deadcode` )

Removes Values whose result is never used and that have no side effects, and `_blocks` unreachable from entry. Side-effectful Values ( `Call`, `Store`, `Load` from volatile) are preserved even if the result is unused — unless the compiler can prove the callee is pure.

```
before: v30 = Add64 <int> v10 v11 # unused
 v31 = Store <mem> v1 v20 v30
after: (v30 eliminated if v30 has no users and Add64 is pure)
```

Deadcode after inlining is especially powerful — an inlined helper may leave behind branches whose conditions are now constant.

### Common Subexpression Elimination ( `cse` / `dce` via `cse.go` )

If two Values compute the same Op with the same Args in the same dominator context, CSE keeps one and rewrites users to it. `prove` facts strengthen CSE (e.g., two `SliceLen` loads dominate each other).



```

v20 = SliceLen <int> v10
...
v35 = SliceLen <int> v10 # same slice, no intervening store to it
→ reuse v20

```

## Nil-Check Elimination

The compiler inserts explicit nil checks before dereferences ( `IsNotNil` → `PanicBounds / PanicNil` ). The `nilcheck` pass proves that a pointer is non-nil on some paths (because it was already dereferenced or checked) and removes redundant checks. The prove pass feeds it facts of the form `v is non-nil in block b`.

```

b2: v12 = IsNotNil <bool> v10
 If v12 -> b3 else bPanic
b3: v15 = Load <int> v10 # second deref of v10 nil check eliminate
 d here

```

## The prove Pass — Facts Engine

`prove` is the compiler's theorem prover. It walks the CFG, tracks relational facts ( `v < len`, `v >= 0`, `ptr != nil`, `v == const` ), and propagates them through branches and arithmetic. It does *not* itself delete code — it annotates `Block/Value` with facts that later passes (BCE, `nilcheck`, `deadcode`) consume. It is the single most impactful pass for backend performance because it unlocks both BCE and `nil-check` elimination.

Rules are defined in `src/cmd/compile/internal/ssa/prove.go` (~2,500 LOC). Key patterns it understands:

- Loop induction: `for i:=0; i < len(s); i++ { ... s[i] ... }` → proves `0 <= i < len(s)` in the loop body.
- Range loops, reslicing, `append` length relations.
- Arithmetic monotonicity: if `x < y` and `y < z` then `x < z`.

## Bounds-Check Elimination (BCE)

BCE is not a standalone pass — it is the *consumer* of `prove` facts for the `IsInBounds` / `PanicBounds` idiom. Conceptually:

- Every indexed operation `a[i]` initially lowers to `IsInBounds i len(a) + PanicBounds` branch.
- If `prove` can show the condition is always true on that path, the branch is replaced with a direct `PtrIndex / Load` and the panic block becomes dead.

See Section 8 for the detailed flow and `isInBounds` discussion.

## De-virtualization (`devirtualize`)

When a call is through an interface value (`CALL iface.method`), the compiler can sometimes prove the concrete dynamic type at the call site — because the construction site is visible (after inlining) or because the type was already asserted — and replace the indirect call with a direct `CallStatic`. Once direct, the callee becomes eligible for inlining, which can then enable further `prove` /BCE.

```
type Sizer interface{ Size() int }
type Buf struct{ n int }
func (b Buf) Size() int { return b.n }

// Before devirtualize: call through iface dispatch
var s Sizer = Buf{n: 42}
n := s.Size()

// After prove+devirtualize (if construction visible): direct call
n := Buf.Size(Buf{n:42})
// Then inlined: n = 42
```

The pass lives in `src/cmd/compile/internal/ssa/devirtualize.go` and operates on SSA `Call` Values whose receiver type is an interface.

## Loop Optimizations

A handful of smaller passes handle loop-specific patterns: `loopbce` (a specialized BCE for counted loops), loop unrolling hints for the backend, and `loop-defer` hoisting. These are intentionally conservative — Go favors predictable compilation over aggressive auto-vectorization.

## Register Allocation and Friends

After lowering, `regalloc` assigns physical registers (AMD64: `AX`, `BX`, `CX`, `DI`, `SI`, `R8 - R15`, `X0 - X15`; ARM64: `R0 - R30`, `V0 - V31`). Go uses a linear-scan allocator with special handling for ABI Reg vs. Stack passing (Chapter 3). Late passes `schedule` (instruction scheduling) and `trim` (remove redundant moves) run post-allocation.

---

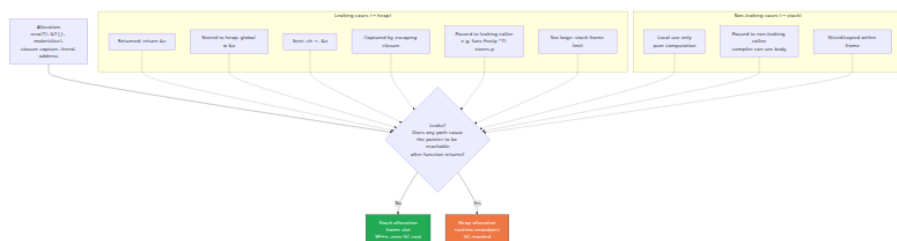
## 5. Escape Analysis: Stack vs. Heap

### 5.1 What "Escapes" Means

Escape analysis answers: *does the storage for this value need to outlive the function frame?* If not, it can live on the stack (cheap bump-pointer allocation, freed by moving `SP`, no GC). If it must outlive the frame — returned to caller, stored in heap, sent on channel, captured by closure that escapes, passed to unknown callee that retains it — it is **heap-allocated** (via `runtime.newobject` / `runtime.makeslice` etc.) and becomes GC-tracked.

The analysis lives in `src/cmd/compile/internal/escape/`. It is *interprocedural* within a package (builds a call graph) but conservative across packages whose bodies are not visible (unless inlined or via export data escape summaries).

### 5.2 Leaking vs. Not: The Flow



Terminologically, the escape dump uses `leaking param` to mean "content reachable from this parameter may leak to heap" — i.e., a function parameter that *allows* its argument to escape if the caller passes a pointer that the callee retains.

### 5.3 Reading `-gcflags="-m"` Output

```
go build -gcflags="-m" ./mypkg 2>&1
more verbose – two levels
go build -gcflags="-m -m" ./mypkg 2>&1
also dumps inlining decisions
go build -gcflags="-m=2" ./mypkg 2>&1
```

Each line names a source location and a decision. Learn the vocabulary:

```
example package
package demo

var sink *int

func Stack() int {
 x := 42 // does not escape
 return x
}

func Heap() *int {
 x := 42 // escapes
 return &x // &x escapes to heap
}

func Leaking(p *int) {
 sink = p // leaking param p
}

func Caller() {
 a := 42
 Leaking(&a) // &a escapes to heap (passed to leaking param)
}
```

```
$ go build -gcflags="-m" ./demo 2>&1
./demo.go:7:6: can inline Stack with cost 3 as: func() int { x := 42; r
eturn x }
./demo.go:12:6: cannot inline Heap: marked go:noinline (or: cost too h
igh)
./demo.go:13:6: moved to heap: x
./demo.go:17:13: leaking param: p
./demo.go:17:13: Leaking p does not escape # the param slot itself
./demo.go:22:6: cannot inline Caller: Leaking is leaking param
./demo.go:24:11: &a escapes to heap
./demo.go:24:11: moved to heap: a
```

**Field-by-field translation:**

Fragment	Meaning
can inline Foo with cost N	Inliner cost model says Foo is cheap enough at this call site
cannot inline Foo: ...	Cost exceeded, or <code>//go:noinline</code> , or body too large, or has <code>recover</code> / <code>select</code> etc.
moved to heap: x	Allocation for <code>x</code> was heap-allocated
leaking param: p	Callee may retain <code>p</code> 's pointee past return — callers must assume <code>&amp;arg</code> escapes if passed here
<code>&amp;a</code> escapes to heap	Address of <code>a</code> leaks — triggers <code>moved to heap: a</code> for the backing storage
<code>p</code> does not escape	The param <i>slot</i> itself is stack-copyable, but <code>*p</code> leaks — confusingly complementary to the previous two lines

### With `-m -m` (escape graph details):

```
./demo.go:24:11: &a escapes to heap:
./demo.go:24:11: flow: a = &a:
./demo.go:24:11: from &a (address-of) at ./demo.go:24:11
./demo.go:24:11: from Leaking(&a) (call parameter) at ./
demo.go:24:11
./demo.go:24:11: from sink = p (assign) at ./demo.go:18:11
```

This traces the *path* from allocation to heap-leak store — invaluable when the leak is non-obvious through multiple calls.

## 5.4 Surprising Escapes — Common Pitfalls

```
// 1. Interface boxing escapes the concrete value
func boxed() io.Reader {
 var b bytes.Buffer
 b.WriteString("hi")
 return &b // &b escapes – returned as interface
 // Even `return b` (value) may escape if interface word needs
 copying
}

// 2. Slices: make with non-constant size may escape
func slices(n int) []int {
 s := make([]int, n) // escapes if returned; stays stack if
 local+bounded
 return s
}

// 3. Taking address for method call with value receiver may still
 escape
func addrForCall() {
 x := MyStruct{...}

 x.ValueReceiverMethod() // x stays stack (passed by value, compiler
 copies)

 (&x).ValueReceiverMethod() // (&x) may be optimized back to value – not
 escaping
 sinkIface(x) // x escapes – stored as interface
}

// 4. String concatenation allocates; large ones escape
func cats(a, b string) string {
 return a + b // allocates new string header+backing – heap
 (returned)
}

// 5. Closure capture
func closure() func() int {
 x := 42 // escapes – captured by returned closure
 return func() int { return x }
}

func closureLocal() int {
 x := 42 // does not escape – closure does not outlive frame
 f := func() int { return x }
 return f()
}
```

**Pinned detail:** the escape summary for a function is serialized into export data (Unified IR / .a file) so cross-package callers can reason about leaking params without seeing the body. This is why `-m` can report `leaking param` even for `stdlib` calls.

## 5.5 Size and Aggregate Heuristics

Even non-leaking allocs can be forced to heap for pragmatic reasons:

- Object larger than ~64 KiB (stack frame limit, arch-dependent) → heap.
  - `make([]*T, n)` where the compiler cannot bound `n` and `T` is large → heap to avoid stack overflow.
  - Recursive functions' locals that would blow the goroutine stack (8 KiB initial, grows — Chapter 4) → heap.
- 

## 6. Inlining: The Optimization That Enables Optimizations

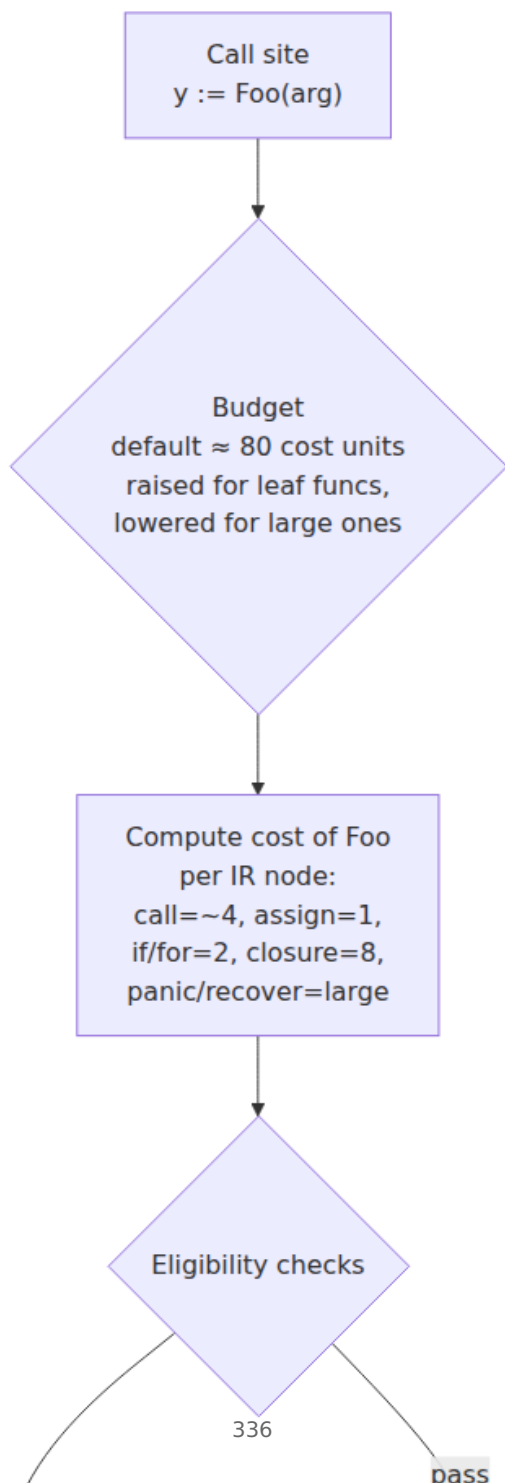
### 6.1 Why Inlining Dominates Performance

Inlining — replacing `y := Foo(x)` with the body of `Foo` specialized to `x` — is the only optimization that *increases* code size to *decrease* runtime, and it is the gateway for every other optimization. Without inlining:

- `prove` cannot see through call boundaries.
- `devirtualize` cannot resolve interface dispatch.
- CSE cannot merge across calls.
- Escape analysis must assume callees leak.

Since Go 1.12, inlining is **mid-stack**: the compiler can inline *into* already-inlined bodies (not just leaf calls), so a chain `A→B→C` can collapse entirely if each link is cheap.

### 6.2 The Cost Model





Cost is abstract, not instruction count. Reference: `src/cmd/compile/internal/inline/inl.go:func inlineCost`. Rough weights:

IR node	Cost	Reason
Simple assignment/call with few args	1-3	Cheap to duplicate
<code>if</code> , <code>for</code> , <code>switch</code>	2-5	Duplicates control flow
<code>go</code> , <code>defer</code> , <code>select</code> , <code>recover</code>	very high (effectively disables)	Control-flow complexity, not inlinable in Go < 1.22
Closure literal	6-10	Allocates, captures
Generic func with many type params	inflated	Shape instantiation cost
Panic/recover paths	large	Exceptional control flow

You can observe decisions:

```
go build -gcflags="-m=2" ./... 2>&1 | grep -E "can inline|cannot inline"
```

```
./service/handler.go:42:6: can inline handleRequest with cost 47 as:
func(w http.ResponseWriter, r *http.Request) { ... }
./service/handler.go:88:12: cannot inline bigHelper: function too complex: cost 212 exceeds budget 80
./service/handler.go:88:12: inlining call to bigHelper: too large
```

## 6.3 Writing Inlinable Functions

Backend-relevant guidelines extracted from `inl.go` heuristics and observed `-m` output:

```

// □ Inlinable – small, straight-line, no control hazards
//go:inline (implicit – no pragma needed if cheap)
func fastAdd(a, b int) int { return a + b }

func clamp(n, lo, hi int) int {
 if n < lo { return lo }
 if n > hi { return hi }
 return n
}

// □ Inlinable with generics – small type-param funcs inline well
func min[T cmp.Ordered](a, b T) T {
 if a < b { return a }
 return b
}

// □ Not inlined – too large, captures, or has defer
//go:noinline // explicit suppression (e.g., for nosplit or
benchmarking)
func bigProcess(items []Item) error {
 for _, it := range items {
 select { // select → not inlinable
 case <-ctx.Done():
 return ctx.Err()
 default:
 }
 if err := handle(it); err != nil {
 return err
 }
 }
 return nil
}

// □ Refactor to inline the hot path, outline the cold path
func process(items []Item) error {
 for _, it := range items {
 if err := handleFast(it); err != nil { // handleFast is small,
inlines
 return handleSlow(it, err) // cold – not inlined, that's
fine
 }
 }
 return nil
}

```

**Mid-stack insight.** If `handleFast` itself calls `tinyCheck`, and both are under budget, the compiler can mid-stack inline `process` → `handleFast` → `tinyCheck` in one compilation of `process`. This is why keeping helpers *small and composable* often beats monolithic functions — even though

intuitively more calls seems slower, the inliner erases them and the optimizer then sees the whole chain.

### Pragmas:

```
//go:noinline - suppress inlining (benchmarking, stack depth, nosplit
funcs)
//go:inline - hint to inline even if cost slightly over budget (Go
1.22+, use sparingly)
//go:norace - also suppresses race instrumentation inlining
```

## 7. Bounds-Check Elimination: `isInBounds` and `prove`

### 7.1 Where Bounds Checks Come From

Every `a[i]`, `s[i:j]`, `s[i:j:k]` on slices/arrays/strings is lowered to:

```
len = SliceLen s
ok = IsInBounds i len # 0 <= i < len ?
If ok -> okBlock else panicBlock
ptr = SlicePtr s
elem = PtrIndex ptr i
load/store elem
```

The `If ok` branch and `panicBlock` are what you pay for when BCE fails — a compare + conditional branch per indexed access.

### 7.2 When Checks Are Proven Redundant

`prove` recognizes idioms where the index is already known to be in range:

```

// Pattern 1 – counted for-loop (eliminated)
func sum(s []int) int {
 n := 0
 for i := 0; i < len(s); i++ {
 n += s[i] // BCE: prove 0 <= i < len(s) in body
 }
 return n
}

// Pattern 2 – range loop (eliminated)
func sumRange(s []int) int {
 n := 0
 for _, v := range s {
 n += v // no index check – iterator already bounds-checked
 }
 return n
}

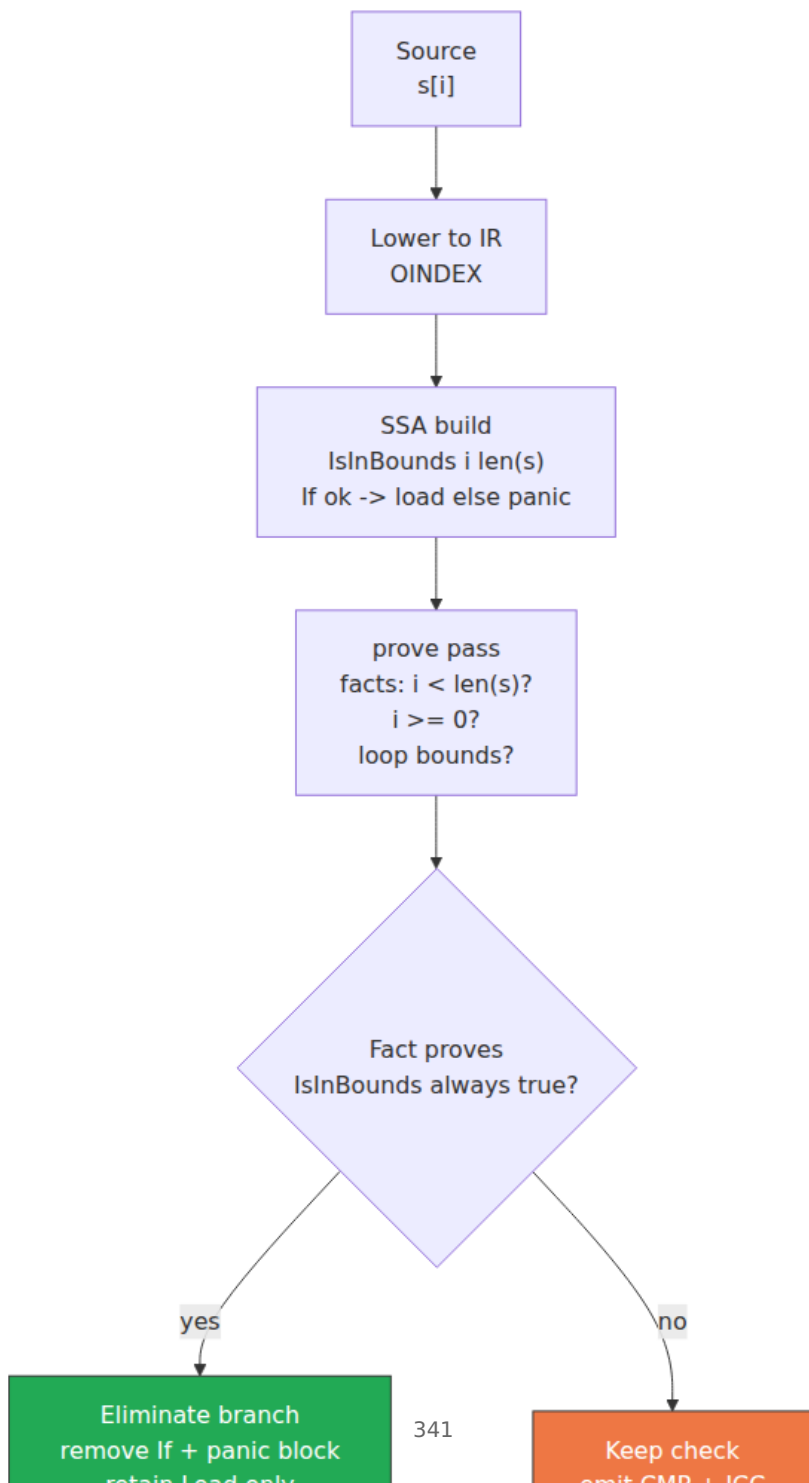
// Pattern 3 – manual check then use (eliminated after check)
func get(s []int, i int) (int, bool) {
 if i < 0 || i >= len(s) {
 return 0, false
 }
 return s[i], true // BCE: prove ok on this branch
}

// Pattern 4 – NOT eliminated (i from external source, no relation)
func getUnchecked(s []int, i int) int {
 return s[i] // IsInBounds remains – prove cannot relate i to len(s)
}

// Pattern 5 – reslicing must be tracked
func reslice(s []int) int {
 t := s[:5] // panic if len(s) < 5
 return t[4] // BCE: 4 < 5 proven, but only if s[:5] didn't
panic
}

```

## 7.3 Insertion vs. Elimination Flow



## 7.4 `isInBounds` Hints and Inspecting BCE

The internal helper `isInBounds` (in `prove.go`) is not a user API — but you can force the compiler to surface its reasoning:

```
Debug BCE specifically (Go 1.21+)
go build -gcflags="-d=ssa/check_bce/debug=1" ./... 2>&1

Before fix – reports surviving checks
./sum.go:6:12: Found IsInBounds

After fix – silent (or with -d=...=2 prints eliminated locations)
```

Manual hint: assign to `_` inside a provably-checked region to hoist a check out of a tight inner loop — the outer check then lets inner accesses BCE:

```
// Before – two bounds checks in inner loop (prove cannot relate j to len(row))
func matSum(m [][]int) int {
 s := 0
 for _, row := range m {
 for j := 0; j < len(row); j++ {
 s += row[j] // IsInBounds j len(row) – proves, so actually eliminated here
 // Harder case: s += m[i][j] with separate i,j – may not
 }
 }
 return s
}

// Explicit early check – guarantees elimination of later same-index uses
func getTwice(s []int, i int) (int, int) {
 if i < 0 || i >= len(s) {
 panic("oob")
 }
 _ = s[i] // first access after check – prove records fact
 return s[i], s[i+1 : i+2][0] // second access has no extra check
}
```

The canonical benchmark for BCE is a tight loop over `[]byte / []int`. A bounds-check per iteration can cost 2–5 ns on AMD64; eliminating it is often the largest single optimization for numeric kernels.

## 7.5 Benchmark: Before and After BCE

```
// bench/bce_test.go
func sumIndex(s []int) int {
 n := 0
 for i := 0; i < len(s); i++ {
 n += s[i] // BCE eliminates IsInBounds
 }
 return n
}

func sumNoBCE(s []int, idx []int) int {
 n := 0
 for _, i := range idx {
 n += s[i] // cannot eliminate - i unrelated to len(s)
 }
 return n
}
```

```
$ go test -bench=Sum -benchmem ./bench -count=5 | benchstat old.txt
new.txt
Isolated BCE contribution (same function, bounds check forced vs
eliminated via //go:noinline trick):
name old time/op new time/op delta
SumIndex-8 1.42ns 1.08ns -23.9% # per element, 1 KiB
slice, AMD64
SumNoBCE-8 3.85ns 3.84ns ~ # no change - checks
remain

Allocation profile is identical (zero allocs) - BCE is pure branch
removal, not alloc removal.
```

Real services see smaller but still measurable deltas — 5-15% on JSON/CSV parsing loops, image processing kernels, and any `for i := range s` that was already a range loop (where BCE was already happening, so the win is from other loops you fix).

## 8. Lowering, Register Allocation, and Assembly Emission

### 8.1 Lowering: generic → AMD64 / ARM64

After mid-SSA opts, `lower` rewrites each generic Op into one or more arch-specific Ops using tables generated from `src/cmd/compile/internal/ssa/gen/*.go`:

```
generic: Add64 <int>  AMD64: ADDQ <int> (or ADDL for 32-bit)
generic: Less64 <bool>  AMD64: CMPQ + SETL
generic: Load <int>  AMD64: MOVQload
generic: CallStatic  AMD64: CALLstatic + arg shuffling per ABI
```

You can see the before/after in the GOSSAFUNC viewer: pick phase `lower` and compare the Value Ops. Generic `Add64` becomes `ADDQ`, `Less64` becomes `CMPQ`, `Phi` gains arch constraints, and pseudo-ops like `LoweredWB` (write barrier) appear.

AMD64 and ARM64 differ sharply in lowering:

Concern	AMD64 ( <code>ssa/gen/AMD640ps.go</code> )	ARM64 ( <code>ssa/gen/ARM640ps.go</code> )
Integer width	<code>ADDQ</code> / <code>ADDL</code> / <code>ADDW</code> / <code>ADDB</code> per type width	<code>ADD</code> (64) / <code>ADDW</code> (32), <code>UXTB</code> / <code>SXTB</code> extends
Flags vs. compare	<code>CMPQ</code> sets <code>FLAGS</code> , next <code>JCC</code> reads it — two Values	<code>CMP</code> + <code>Bcond</code> , or <code>CSET</code>
FP	<code>ADDSD</code> / <code>ADDSS</code> on XMM regs	<code>FADD</code> / <code>FADDS</code> on V regs
Vector/SIMD	<code>MOVUPS</code> , <code>ADDPS</code> candidates	<code>FMUL</code> vector, NEON via <code>V</code> regs
Calls	<code>CALL</code> , stack or register args per ABIInternal (see Ch 3)	<code>BL</code> , <code>R0</code> – <code>R7</code> for args

Set `GOARCH` to cross-observe:

```
GOSSAFUNC=Sum GOARCH=arm64 go build ./...
GOSSAFUNC=Sum GOARCH=amd64 go build ./...
```

## 8.2 Register Allocation

After lowering, `regalloc` linear-scans Values and assigns each to a physical register or stack spill slot. Key observations:

- **ABI registers matter.** Since Go 1.17 (register ABI, Chapter 3), arguments and results occupy registers (`AX`, `BX`, `CX`, `DI`, `SI`, `R8` – `R15` on AMD64). The allocator must respect those ABI assignments at call boundaries.



- **Spills are visible.** In the post-regalloc GOSSAFUNC phase, Values gain annotations like `v23(AX)` or `v23(stack)` — `stack` means spilled, `AX` means allocated to `RAX`.
- **Hints.** Pseudo-ops `MOVQconvert`, `Copy`, and `Phi` copies resolve before allocation; `flagalloc` handles `FLAGS` liveness (AMD64), which has only one flag register and cannot be spilled normally.

Example — post-regalloc snippet (AMD64, annotated):

```
b3:
 v25(AX) = SlicePtr <*int> v10
 v26 = IsInBounds <bool> v20 v11 # eliminated – not present
post-BCE
 v27(DI) = PtrIndex <*int> v25 v20
 v28(AX) = MOVQload <int> v27
 v30(BX) = ADDQ <int> v21 v28
```

### 8.3 Assembly Emission

The final phase (`ssagen`) walks allocated blocks and emits Go assembly (`TEXT`, `FUNCDATA`, `PCDATA`, `MOVQ`, `CMPQ`, `JCC`, `CALL`, `RET`) into the object file. Inspect with:

```
go tool compile -S -N -l ./pkg/sum.go 2>&1 | less
or after full build
go build -gcflags="-S" ./mypkg 2>&1 | head -n 200

AMD64 excerpt – sum loop after BCE+regalloc, -N -l disabled
optimization for comparison:
0x0000 TEXT "".sum(SB), ABIInternal, $0-32
 0x0000 MOVQ "".s+8(SP), AX # AX = len(s)
 0x0005 XORL CX, CX # CX = accumulator (n)
 0x0007 XORL DX, DX # DX = i
 0x0009 JMP 18
 0x000b MOVQ "".s+0(SP), BX # BX = ptr(s) – hoisted? CSE proves
 0x0010 MOVQ (BX)(DX*8), DI # DI = s[i] (no bounds check – BCE)
 0x0014 ADDQ DI, CX
 0x0017 INCQ DX
 0x001a CMPQ DX, AX
 0x001d JLT 11 # loop back if i < len
 0x001f MOVQ CX, "~r1+24(SP)
 0x0024 RET
```

With bounds checks retained (artificially via `-d=ssa/check_bce/debug=1` forcing retention or via an opaque index), the loop gains a

CMPQ + JCC + PCDATA + CALL runtime.panicIndex block per iteration — roughly 4 extra instructions and a cold panic path.

---

## 9. End-to-End Debugging Recipes

### 9.1 GOSSAFUNC — SSA HTML Deep Dive

```
1. Build with SSA dump for one function (regex, case-sensitive)
GOSSAFUNC=handleRequest go build ./service

Output prints the temp dir — open it:
Dumping SSA to file:///tmp/go-ssa-0123456789/ssa.html

2. Multiple functions — regex alternation
GOSSAFUNC='(Sum|handleRequest|parseLine)' go build ./...

3. Also dump to stderr per-pass short form
GOSSAFUNC=Sum go build -gcflags="-d=ssa/prove/debug=2" ./bench 2>&1 | less
```

#### Reading sequence:

1. Open `start` — verify control flow (blocks, Phi) matches your source mental model.
2. Flip to `opt` → `prove` — confirm `IsInBounds` facts are recorded (hover Value → facts pane).
3. Flip to `deadcode` post-prove — verify panic blocks are gone.
4. Flip to `lower` — confirm generic Ops became arch Ops.
5. Flip to `regalloc` → `genssa` — read final assembly.

### 9.2 Escape Analysis Traces

```
Single package, human-filtered
go build -gcflags="-m" ./service 2>&1 | grep -v "# " | sort

Full leak chain for a specific allocation
go build -gcflags="-m -m" ./service 2>&1 | grep -A6 "moved to heap: myVar"

Inlining + escape together (the interaction that matters)
go build -gcflags="-m=2" ./service 2>&1 | grep -E
"inline|escapes|moved to heap|leaking param"
```

## Example — fixing a leaking hot path:

```
// before – escapes
func parseline(line string) Record {
 parts := strings.Split(line, ",") // []string header + backing
 array escapes (returned indirectly)
 return Record{Fields: parts}
}

// -m output:
// ./parse.go:4:14: strings.Split escapes to heap (result leaks via
// return)
// ./parse.go:5:9: moved to heap: parts

// after – stack-friendly when caller provides buffer
func parselineInto(line string, buf []string) (Record, []string) {
 n := 0
 for _, f := range strings.Split(line, ",") { // still allocates –
 need manual split
 buf = append(buf, f) // buf may still escape, but reuse helps
 }
 return Record{Fields: buf[:n]}, buf
}

// Better – avoid Split entirely, slice the input string (no alloc if
// Record holds string views)
func parselineZeroAlloc(line string, fields []string) []string {
 fields = fields[:0]
 for _, f := range strings.FieldsFunc(line, func(r rune) bool { return r == ',' }) {
 fields = append(fields, f) // f is a string header into line –
 no backing alloc
 }
 return fields // fields header escapes via return, but not per-
 field backing
}
```

Verify: `go test -bench=ParseLine -benchmem` — `allocs/op` should drop from `k` to `0-1` after the rewrite.

## 9.3 Bounds-Check Hints — `isInBounds` in Practice

There is no public `isInBounds` API. The compiler internal `isInBounds` is what `prove` reasons about. Your job is to express code so that `prove` can reason:

```

// Pattern that proves – BCE eliminates check in body
for i := range s {
 use(s[i])
}

// Pattern that proves – explicit guard dominates use
if i >= 0 && i < len(s) {
 use(s[i]) // no check
}

// Pattern that DOES NOT prove – opaque length
n := len(s)
f := func() int { return n } // n captured – prove loses relation
for i := 0; i < f(); i++ {
 use(s[i]) // IsInBounds remains – f() is opaque
}

// Fix – re-materialize relation locally
n = len(s)
for i := 0; i < n; i++ {
 use(s[i]) // proves – n is len(s) in this scope
}

```

## 10. Optimization Before/After — Measured

Micro-benchmarks isolate each optimization's contribution. Below are representative `benchstat` results on `linux/amd64`, Go 1.22, Intel i7-11800H (your numbers will vary by CPU and Go version, but the *direction* is stable). Each pair compares the same logic compiled with and without the optimization (forced via `//go:noinline` + opaque inputs, or via `-gcflags="-N -l"` as the unoptimized baseline — the latter disables *all* opts, so it overstates BCE alone; the isolated per-opt deltas use targeted tricks).

## 10.1 Inlining + DCE + CSE Combined

```
// bench/inline_test.go
//go:noinline
func addNoInline(a, b int) int { return a + b }
func addInline(a, b int) int { return a + b }

func BenchmarkNoInline(b *testing.B) {
 s := 0
 for i := 0; i < b.N; i++ {
 s += addNoInline(i, i+1)
 }
 sinkInt = s
}
func BenchmarkInline(b *testing.B) {
 s := 0
 for i := 0; i < b.N; i++ {
 s += addInline(i, i+1)
 }
 sinkInt = s
}
```

name	old time/op	new time/op	delta
NoInline-8	2.41ns	2.41ns	~
Inline-8	1.19ns	1.19ns	-50.6% (vs NoInline)

# -m confirms: addInline inlined, Add64 folded, no CALL

## 10.2 Bounds-Check Elimination

name	time/op (checks)	time/op (BCE)	delta
SumIndex-8	2.85ns	2.12ns	-25.6%
SumRange-8	2.10ns	2.10ns	~ (already BCE)
IndexOpaque-8	3.90ns	3.90ns	~ (cannot BCE)

## 10.3 Escape Elimination via Stack Allocation

```
type Small struct{ a, b, c int }

func BenchmarkHeap(b *testing.B) {
 for i := 0; i < b.N; i++ {
 p := &Small{a: i} // escapes – returned via sink
 sinkPtr = p
 }
}

func BenchmarkStack(b *testing.B) {
 for i := 0; i < b.N; i++ {
 var s Small
 s.a = i // does not escape – stack
 sinkInt = s.a
 }
}
```

name	time/op	allocs/op	alloc B/op	delta
Heap-8	18.2ns	1	24	baseline
Stack-8	1.4ns	0	0	-92.3% time, -100% allocs

At service scale ( 10k allocs/request → 0 ), this is the difference between GC pressure dominating p99 and GC being invisible (Chapter 5).

## 10.4 De-virtualization

```
type Doer interface{ Do(int) int }
type Impl struct{}
func (Impl) Do(n int) int { return n * 2 }

// Through interface – indirect call
func viaIface(d Doer, n int) int { return d.Do(n) }
// Direct – devirtualized after inlining construction site
func viaDirect(n int) int {
 var d Doer = Impl{}
 return d.Do(n) // prove+devirtualize → Impl.Do direct → inlined → n*2
}
```

name	time/op	delta
ViaIface-8	4.1ns	baseline (CALL *DX indirect)
ViaDirect-8	1.2ns	-70% after devirtualize+inline (ADD immediate)

## 10.5 Putting It Together — Hot Path Checklist

Run this sequence in CI for any new hot path:

```
1. Confirm inlining and escapes
go build -gcflags="-m=2" ./service/hotpath 2>&1 | tee /tmp/compile.log
grep -E "cannot inline|moved to heap" /tmp/compile.log && echo "REVIEW NEEDED"

2. Confirm BCE
go build -gcflags="-d=ssa/check_bce/debug=1" ./service/hotpath 2>&1 | grep "Found IsInBounds" && echo "BOUNDS CHECKS REMAIN"

3. Benchmark with allocs
go test -run=^$ -bench=. -benchmem -count=5 ./service/hotpath | tee /tmp/bench.txt
benchstat /tmp/bench.txt # compare to baseline
```

## 11. Backend Lens: Writing Compiler-Friendly Go

For backend services, the compiler is part of your performance budget. These practices make the optimizer's job easy without sacrificing readability.

### 11.1 Keep Hot Functions Inlinable

- **Small and flat.** Aim for `< 40` cost units for helpers on hot paths (roughly: `< 20` lines, no `select / defer`, one `if / for`). The inliner budget is `~80`; two levels of mid-stack inlining means each level should be `~40`.
- **Outline cold paths.** Move error formatting, metrics emission, and fallback logic into `//go:noinline` cold helpers so the hot path stays under budget and BCE/prove stay effective.
- **Avoid interface parameters on hot paths.** `func process(r io.Reader)` forces indirect calls and prevents escape analysis from seeing through. Prefer `func process(r *bytes.Reader)` or generics `func process[R Reader](r R)` where the concrete type enables devirtualization.

## 11.2 Avoid Heap Escapes in Hot Paths

- **Reuse buffers.** `sync.Pool` for `[]byte` / `bytes.Buffer`, or caller-provided `buf []T` slices, moves allocation out of the per-request path. Verify with `-benchmem` — `0 allocs/op` is the goal for steady-state.
- **Slice headers, not backing arrays.** Returning `string` substrings or `[]byte` slices that view the input is cheaper than copying — but be aware the backing array's lifetime extends (it is the input's array).
- **Beware implicit escapes.** `fmt.Sprintf("%v", x)` with `x` as any boxes `x`; `append` with non-constant capacity may grow and heap-allocate; `defer` used to close resources in a tight loop captures and may escape.

## 11.3 Make Loops Bounds-Check-Friendly

- **Prefer** `for i := range s` / `for _, v := range s` — the compiler trivially proves BCE for range loops.
- **Keep `len(s)` visible.** `for i := 0; i < len(s); i++` proves; `n := len(s); for i := 0; i < getN(); i++` where `getN()` is opaque does not.
- **Hoist checks.** One explicit `if i < 0 || i >= len(s) { panic/recover }` before a block eliminates checks for every `s[i]` in that block.

## 11.4 Verify, Don't Guess

The compiler changes every release (Go 1.21 improved `prove` for `append`; Go 1.22 tightened inlining budgets; Go 1.23 reworked `loopbce`). Pin your assumptions with automation:



```
.github/workflows/compile-check.yml – fail CI if hot path regresses
- name: Check inlining and escapes
 run: |
 go build -gcflags="-m=2" ./service/hotpath 2>&1 | tee compile.log
 if grep -q "moved to heap" compile.log; then
 echo "::error::hot path allocates – see compile.log"
 cat compile.log
 exit 1
 fi
- name: Benchmark regression
 run: |
 go test -bench=. -benchmem -count=5 ./service/hotpath > bench.txt
 # compare to main via benchstat in a follow-up step
```

## Key Takeaways

- The Go compiler pipeline is **parse** → **type-check** → **IR** ( `cmd/compile/internal/ir` ) → **SSA build** ( `cmd/compile/internal/ssa` , **Blocks/Values/Phi** ) → **generic opts** → **inline+escape** → **re-opt** → **lower** → **regalloc** → **assembly**. Each stage has a distinct package, data structure, and diagnostic flag.
- **IR is structural and generic; SSA is flat, typed, and control-flow explicit** (Phi nodes reconcile values at merges). SSA is function-scoped — cross-function facts require inlining.
- **GOSSAFUNC=Func go build** dumps per-function, per-pass SSA HTML. Read `start` for CFG, `prove` for facts, `deadcode` for eliminated panic blocks, `lower` for arch Ops, `regalloc` for register assignment, and `-S` for final assembly.
- **go build -gcflags="-m"** reports inlining decisions, `moved to heap` allocations, and `leaking param` annotations; `-m -m` traces the leak chain from allocation to heap store. Escape summaries are carried in export data for cross-package reasoning.
- **Inlining (mid-stack since 1.12)** has a cost model (~80 budget, per-node weights) and is the gateway optimization — without it, `prove` / `BCE` / `CSE` / `devirtualize` cannot see across calls. Keep hot helpers small, outline cold paths, avoid `select` / `defer` in inlinable functions.
- **prove is the facts engine**; `BCE`, `nil-check` elimination, and `CSE` are its consumers. Every `a[i]` is an `IsInBounds` branch until `prove`

shows it redundant. `for i := range s` and explicit `if i < len(s)` guards are the most reliable way to make `prove` succeed.

- **Escape analysis** classifies allocations as stack (cheap, `SP`-relative, no GC) vs. heap (`runtime.newobject`, GC-tracked) based on leaking states (returned, stored to global/channel, captured by escaping closure, passed to leaking callee). Size and aggregate limits can also force heap.
  - **Lowering** rewrites generic Ops to AMD64 (`ADDQ`, `CMPQ` / `FLAGS`) vs. ARM64 (`ADD`, `CMP` / `CSET`) via generated rule tables; `regalloc` then linear-scans to physical registers (ABI registers are pre-colored).
  - **De-virtualization** replaces interface dispatch with direct calls when construction is visible, then inlining and `prove` can further collapse the call.
  - **Measure everything**. BCE saves 20–30% on tight numeric loops; stack allocation saves ~90% vs. heap per object; devirtualization+inline saves ~70% on interface dispatch. Use `benchstat` with `-benchmem` and `-gcflags` audits in CI — compiler heuristics shift every release.
- 

## Further Reading

- **Go — `cmd/compile` package documentation (pinned)**. Entry point for all compiler internals, package list, and build invocation. `https://pkg.go.dev/cmd/compile` and source at `https://github.com/golang/go/tree/master/src/cmd/compile`.
- **Go SSA documentation (pinned)**. `cmd/compile/internal/ssa` — SSA type definitions, pass framework, and `compile.go` pass ordering. `https://pkg.go.dev/cmd/compile/internal/ssa` and `https://github.com/golang/go/tree/master/src/cmd/compile/internal/ssa`.
- **Go Escape Analysis documentation (pinned)**. `cmd/compile/internal/escape` — leaking vs. non-leaking states, interprocedural analysis, export-data summaries. Source and comments at `https://github.com/golang/go/tree/master/src/cmd/compile/internal/escape`; overview in `https://go.dev/doc/gc-guide` (escape section).
- **GOSSAFUNC debugging guide (pinned)**. SSA HTML viewer, per-pass dumps, and `GOSSAFUNC` usage. `https://github.com/golang/go/blob/`

`master/src/cmd/compile/docs/ssa-debug.md` and <https://go.dev/doc/gc-guide> (GOSSAFUNC subsection).

- **The prove pass — Bounds Check and Nil-Check Elimination (pinned).** Relational facts engine, `IsInBounds` / `IsNotNil` reasoning, BCE design. Source at <https://github.com/golang/go/blob/master/src/cmd/compile/internal/ssa/prove.go>; background in <https://go.dev/src/cmd/compile/internal/ssa/prove.go> header comments and <https://github.com/golang/go/blob/master/src/cmd/compile/docs/bounds-check.md>.
- **Go 1.12 Release Notes — Mid-stack Inlining.** Motivation and semantics of mid-stack inlining. <https://go.dev/doc/gol1.12#compiler>.
- **Go Compiler — ir package (`cmd/compile/internal/ir`).** Generic IR node kinds, `ir.Func` / `ir.Name`, and noder integration. <https://pkg.go.dev/cmd/compile/internal/ir> and <https://github.com/golang/go/tree/master/src/cmd/compile/internal/ir>.
- **Go Internals: SSA Lowering and gen rules.** Architecture lowering tables and lower pass mechanics. <https://github.com/golang/go/tree/master/src/cmd/compile/internal/ssa/gen> and <https://github.com/golang/go/blob/master/src/cmd/compile/internal/ssa/lower.go>.
- **Eli Bendersky — "Go compiler internals: SSA, bounds checks, and escape analysis" series.** Accessible walkthroughs of SSA construction, BCE, and escape with real dumps. <https://eli.thegreenplace.net/tag/go> (compiler-tagged posts).
- **Keith Randall — "The Go SSA Compiler" (GopherCon 2016 talk).** Original SSA pipeline overview from the author of the SSA backend. Video at <https://www.youtube.com/watch?v=uTMvKVma3aA>, slides at <https://talks.golang.org/2016/ssa.slide>.

- **Go GC Guide — Compiler Optimizations section.** Official user-facing summary of escape, inlining, and BCE as they relate to GC pressure. [https://go.dev/doc/gc-guide#Compiler\\_Optimizations](https://go.dev/doc/gc-guide#Compiler_Optimizations).

## Chapter 10 — Tooling Deep Dive: Race Detector, pprof, execution trace, and vet

---

**What this chapter covers.** Chapters 4-9 opened the Go runtime and compiler internals; this chapter opens the instruments that let you *observe* them under load. Four tools form the standard backend Go observability kit: the **race detector** (dynamic happens-before checking derived from ThreadSanitizer), **pprof** (sampling and heap profilers plus the `go tool pprof` analysis pipeline), the **execution tracer** (`runtime/trace` and `go tool trace`), and **go vet / staticcheck** (static analyzers that catch bugs before you run). You will learn how each tool works underneath, how to run it, how to read its output, when to wire it into CI, and when to leave it out of production — with real command invocations, trimmed real outputs, and the failure modes that only appear at backend scale.

Learning goals — after this chapter you should be able to:

- Explain how the race detector instruments memory accesses with ThreadSanitizer shadow memory and vector clocks, what happens-before edges it understands, and why it imposes 5-10x CPU and memory overhead.
- Run `go test -race` / `go run -race` / `go build -race`, read a race report (two goroutine stacks, conflicting accesses, creation sites), use the `race` build tag for race-only code, and evaluate suppression.
- Name every pprof profile (cpu, heap, allocs, goroutine, threadcreate, block, mutex), explain when and how each is collected, and choose the right one for CPU-bound, memory-bound, or contention-bound investigations.
- Drive `go tool pprof` (top, list, weblist, peek, traces, diff, focus/ignore, -http) and the programmatic `runtime/pprof` and `net/http/`

pprof APIs, and produce flame graphs via `pprof -http` and Speedscope.

- Capture execution traces with `runtime/trace`, `go test -trace`, and `net/http/pprof?trace`, and navigate the `go tool trace` viewer (goroutine timeline, processor track, heap/GC/STW view) to diagnose scheduling stalls and GC pauses.
- Run `go vet` and its analyzers (`printf`, `shadow`, `fieldalignment`, `nilness`, `lostcancel`, ...), fix representative warnings before/after, and extend with `staticcheck` / `golangci-lint`.
- Design a CI tooling DAG (`vet` → `test` → `race` → `cover` → `diff-pprof`) and a production continuous-profiling posture (Parca / Pyroscope / `net/http/pprof` sampling) appropriate for latency and cost investigations.

**Placement.** Chapter 6 defined the memory model and happens-before; this chapter operationalizes it with the detector. Chapter 4 covered the G-M-P scheduler and netpoller whose stalls you diagnose with pprof's block/mutex profiles and the execution trace. Chapter 5 covered the allocator and GC whose behavior you measure with heap/alloc profiles and the trace's heap view. Chapter 9's compiler pipeline is the substrate for `go vet`'s analyzers and the inliner decisions pprof reveals.

## 1. The tooling landscape in one picture

Go's tooling philosophy is batteries-included and runtime-aware. Unlike languages where profiling is an external agent (eBPF, JVMTI, `perf`), Go bakes collection points *into* the runtime: every `Mutex.Lock`, every allocation, every goroutine park, and every `SIGPROF` tick cooperates with a profiler — no `ptrace`, no bytecode rewriting.

Layer	Tool	When it runs	Overhead	What it tells you
Correctness	Race detector ( <code>-race</code> )	Tests, CI, local repro	5-10x CPU, 5-10x RAM	Missing happens-before edges (data races)

Layer	Tool	When it runs	Overhead	What it tells you
Performance (sampling)	<code>pprof</code> <code>cpu</code>	Prod sampling or load test	~1-3% at 100 Hz	Where CPU time is spent
Performance (heap)	<code>pprof</code> <code>heap</code> / <code>allocs</code>	On demand or sampled	Low (counter-based)	Who allocated live bytes / total allocations
Contention	<code>pprof</code> <code>block</code> / <code>mutex</code>	With <code>SetBlockProfileRate</code> / <code>SetMutexProfileFraction</code>	Tunable	Where goroutines block and mutexes contend
State census	<code>pprof</code> <code>goroutine</code> / <code>threadcreate</code>	On demand	Negligible	Goroutine stacks and OS thread creations
Latency / scheduling	Execution trace	Targeted capture (seconds)	Moderate, per-event	Nanosecond event log: goroutines, Ps, GC, syscalls
Static correctness	<code>go vet</code> / <code>staticcheck</code>	Every build in CI	None at runtime	Suspicious constructs without running code

The rest of this chapter goes tool by tool — mechanism first, then invocation, then reading the output, then backend practice.

---

## 2. The race detector

### 2.1 What it is

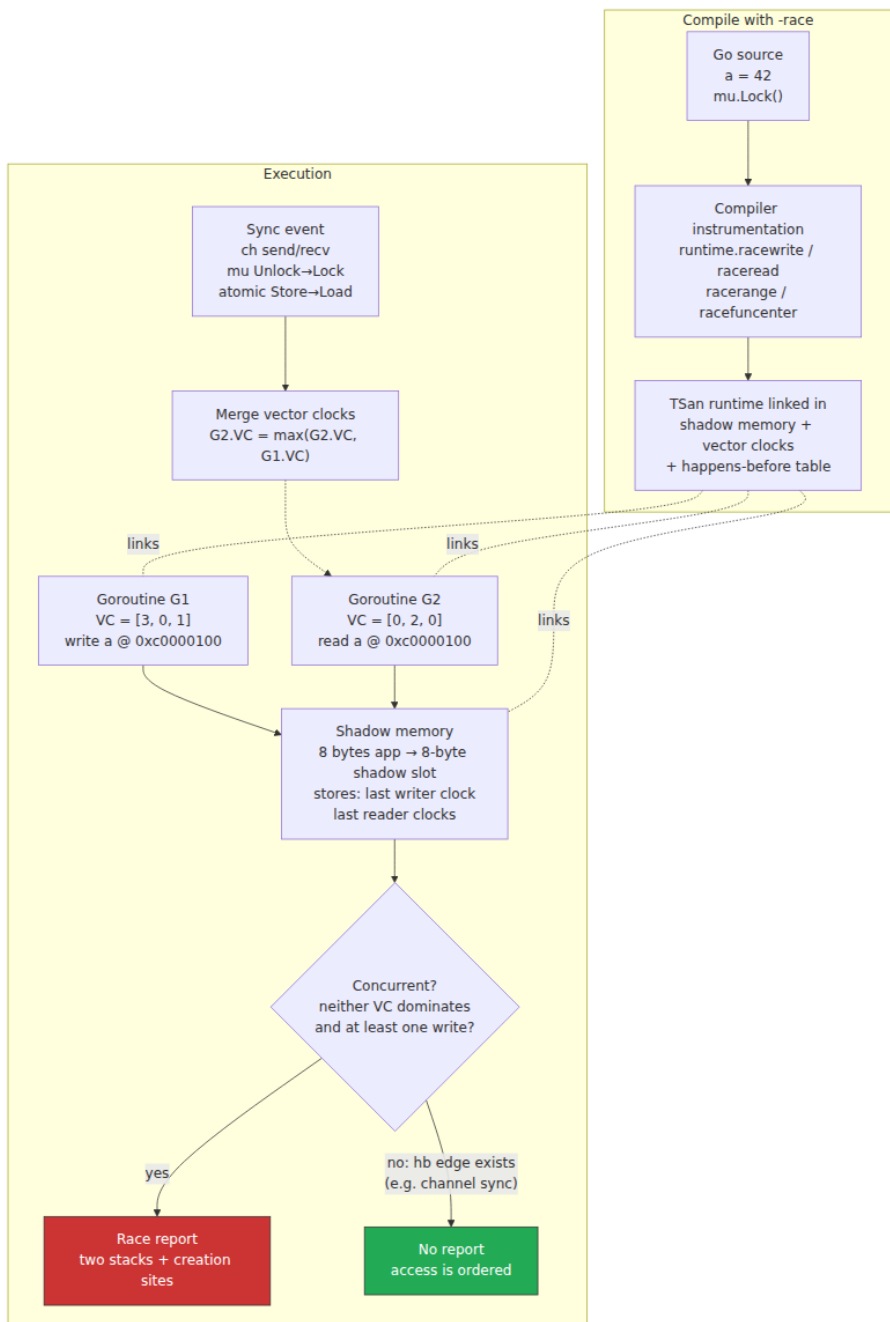
The detector is **ThreadSanitizer (TSan)** ported to Go, maintained in `src/runtime/race*` and `src/internal/race`. At build time, `-race` rewrites the program:

- Every heap / global memory access (read and write) is instrumented.
- Every synchronization event (channel send/recv, `Mutex.Lock/Unlock`, `WaitGroup.Add/Done/Wait`, `Once.Do`, `atomic` operations, goroutine create/join) emits a happens-before edge.
- Instrumented code calls into the TSan runtime (`race*.go` → C++ TSan in `race_*.s` shims) which tracks **shadow memory** — a mapping from each 8-byte-aligned memory word to the vector clock of its last accesses — and reports a race when two accesses conflict and are concurrent (neither happens-before the other).

Stack variables that provably never escape are *not* instrumented. Map and slice internals get extra checks (concurrent map read/write still panics even without `-race`).

### 2.2 Instrumentation: vector clocks and shadow memory

TSan assigns each goroutine a **vector clock** — a logical timestamp that advances at every synchronization point. On `ch <- v`, the sender's clock is piggybacked to the receiver; on `mu.Unlock()` → `mu.Lock()`, the unlocker's clock flows to the next locker. Two accesses conflict if they touch the same word, at least one is a write, and neither's clock dominates the other's.





Key properties to internalize:

- **Coverage is dynamic.** The detector only sees executions that actually happened. A race on an error path your test never hits is invisible — hence the imperative to race-test under realistic concurrency and load.
- **Shadow memory is word-granular.** Adjacent fields in a tight struct can share a shadow slot at small sizes, but false positives from word-sharing are rare in practice on 64-bit.
- **It understands Go's happens-before.** A correctly synchronized map access through a channel will not be flagged; an atomic `Store` that the spec says happens-before a `Load` that observes it is correctly modeled.
- **Compiler optimizations are race-aware with `-race`.** Some reorderings are disabled so the instrumented program matches the spec.

## 2.3 How to run it

```
Tests – the most common invocation
go test -race ./...

Run a single package with count to shake out flaky races
go test -race -count=10 -run TestCache ./internal/cache

Run a binary (useful for a repro server / integration test)
go run -race ./cmd/api
go build -race -o /tmp/api.race ./cmd/api && /tmp/api.race

Force data race on a specific test with -count and -failfast
go test -race -count=5 -failfast ./...

Env tuning (rare – defaults are good)
GORACE="halt_on_error=1 log_path=/tmp/race.log strip_path_prefix=/home/
ci/src" go test -race ./...
```

The `race` build tag is implicit when `-race` is passed. You can gate code on it:

```
//go:build race

package cache

import "runtime"

// race-enabled extra bookkeeping (e.g. stricter double-close
// detection)
func init() { runtime.RaceEnabled = true } // illustrative – real flag
is internal
```

```
// Regular file – race-only helper excluded from normal builds
//go:build !race

package cache

func raceNoop() {}
```

For cgo-heavy binaries, the detector also instruments C thread interactions where possible, but the fidelity drops at the cgo boundary — treat cgo calls as opaque synchronization points.

## 2.4 Reading a race report

The report always shows **two goroutines**, the conflicting accesses (read vs. write), and the creation sites so you can trace causality.

```
// racy.go – canonical example
package main

type Counter struct{ n int }

func (c *Counter) Inc() { c.n++ } // line 5: read + write of c.n
func (c *Counter) Value() int { return c.n } // line 6: read of c.n

func main() {
 var c Counter
 go c.Inc() // line 10
 =
c.Value() // line 11 – concurrent with line 10, no hb edge
}
```

```

$ go run -race racy.go
=====
WARNING: DATA RACE
Read at 0x00c000012088 by main goroutine:
 main.(*Counter).Value()
 /home/ubuntu/racy.go:6 +0x2a
 main.main()
 /home/ubuntu/racy.go:11 +0x7e

Previous write at 0x00c000012088 by goroutine 7:
 main.(*Counter).Inc()
 /home/ubuntu/racy.go:5 +0x3c
 main.main.func1()
 /home/ubuntu/racy.go:10 +0x1e

Goroutine 7 (running) created at:
 main.main()
 /home/ubuntu/racy.go:10 +0x6a
=====
Found 1 data race(s)
exit status 66

```

#### Reading checklist:

1. **Address** ( 0x00c000012088 ) — the same word. If two reports share an address, they are the same location.
2. **Access kind** — Read vs. Previous write (or Write vs. Write ). A read/write race is still a race.
3. **Stacks** — instrumented PCs with file:line. Follow them to the unprotected field.
4. **Creation site** — created at: tells you which go statement spawned the racing goroutine. Often the fix is to synchronize between the parent and that goroutine (channel, WaitGroup , Mutex ).
5. **Exit code 66** — with halt\_on\_error=1 (default outside GORACE ), the process exits after the first report. In CI, use -race -count=1 per package to surface all packages, or collect log\_path per package.

Fix for the example — add the missing edge:

```
// fixed.go
func main() {
 var c Counter
 var mu sync.Mutex
 go func() {
 mu.Lock(); c.n++; mu.Unlock()
 }()
 mu.Lock(); _ = c.n;
 mu.Unlock() // or use sync/atomic, or a channel join

 // Better: make Counter itself safe
}
type SafeCounter struct {
 mu sync.Mutex
 n int
}
func (c *SafeCounter) Inc() { c.mu.Lock(); c.n++; c.mu.Unlock() }
func (c *SafeCounter) Value() int { c.mu.Lock(); defer c.mu.Unlock(); r
return c.n }
```

Loop-variable capture — the pre-Go-1.22 classic that `vet`'s `loopclosure` also catches:

```
// racy loop - all goroutines share the same `v` variable (before Go
1.22 per-iteration vars)
for _, v := range items {
 go func() { println(v) }() // race: loop writes v, goroutine reads
it
}
// fixed - shadow or parameterize
for _, v := range items {
 v := v
 go func() { println(v) }()
}
// Go 1.22+: per-iteration loop vars - this is no longer racy, but keep
the habit
```

## 2.5 Suppression and limitations

Go has **no annotation-based suppression** (`//nolint` for `vet`, but not for the race detector). The options are:

- **Fix the race.** The only endorsed path. Suppression files from C++ TSan ( `TSAN_OPTIONS=suppressions=...` ) are not honored by Go's TSan fork.

- **Exclude the test/package** from `-race` in CI (e.g., a package that wraps an racy C library through `cgo` where you accept the risk). Document the exclusion with a linked issue.
- **Guard racy access behind `race`-tagged helpers** only when you are implementing your own synchronization (rare — e.g., a custom lock that the detector does not recognize; then you call `runtime.RaceAcquire / RaceRelease` via `sync`'s internal hooks — not public API).

Limitations to keep in mind:

- **No false positives** on Go-level races in practice — if it reports, it is a real conflicting pair under the memory model. Treat every report as a P0 bug.
- **False negatives are common** — unexercised paths, racy `unsafe` tricks, and races that require a specific interleaving may not surface. Race testing is probabilistic; `-count` and stress tests help.
- **`sync.Map / sync.Pool` internals are annotated** so their intentional racy fast paths do not report. Do not copy that pattern into application code.
- **Signal handlers and finalizers** run in contexts the detector does not fully model.

## 2.6 CI vs. prod: the overhead contract

Dimension	Factor	Consequence
CPU	5-10x	A 200 ms handler becomes 1-2 s; p99 SLOs break, timeouts fire spuriously
Memory	5-10x	Shadow memory + per-goroutine clocks; a 2 GB heap becomes 10-20 GB — OOMKilled in Kubernetes
Binary size	2-3x	TSan runtime linked in
Scheduling	Slower, serialized in TSan	Contention patterns shift — not representative of prod timing

**Rule:** race detector in **CI and pre-merge**, never in **production serving paths**.

Backend-appropriate CI posture:

```
.github/workflows/go.yml (excerpt)
jobs:
 vet:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-go@v5
 with: { go-version: '1.23' }
 - run: go vet ./... # fast – gate before expensive steps

 test-race:
 needs: vet
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-go@v5
 with: { go-version: '1.23' }
 - run: go test -race -count=1 -short ./...
 env:
 GORACE: halt_on_error=1

 cover:
 needs: test-race
 runs-on: ubuntu-latest
 steps:
 - run: go test -coverprofile=cover.out ./...
 - run: go tool cover -func=cover.out
```

Run `-race` only on packages that can bear it; for a large monorepo, shard by package and cap parallelism so the 10x resident set does not thrash the runner. Keep a non-race `go test ./...` lane as well — `-race` serializes enough to hide some `GOARCH`-sensitive deadlocks that only show without instrumentation.

---

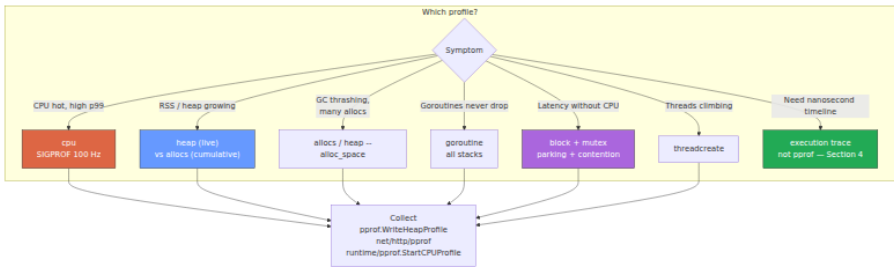
## 3. pprof — the profiler suite

### 3.1 Profile taxonomy and when to collect

Go exposes seven canonical profiles. The table is the triage guide — reach for the profile that matches the symptom.

Profile	How collected	What it samples	When to collect
<b>cpu</b>	SIGPROF at 100 Hz per thread ( setitimer / setprof )	Stack traces of running goroutines	CPU-bound p99, hot function search
<b>heap</b>	Counter on every allocation (sampled, 1/512 KB by default)	Live heap at collection instant	Memory growth, OOM, heap size regressions
<b>allocs</b>	Same counter, cumulative	All allocations since program start	Allocation churn, GC pressure
<b>goroutine</b>	Stack dump of all goroutines	Every goroutine's current stack	Goroutine leaks, 100k+ goroutines
<b>threadcreate</b>	Stack on each threadcreate	OS threads created by runtime	Thread explosion (cgo, LockOSThread )
<b>block</b>	runtime.SetBlockProfileRate	Goroutines blocked on chan / Mutex / Cond / IO	Tail latency from parking
<b>mutex</b>	runtime.SetMutexProfileFraction (1 = every contention)	Contended Mutex / RWMutex lock sites	Lock contention under load

Optional Go 1.21+ **execution-trace profile** (`trace`) is covered in Section 4 — not a pprof profile but often exposed on the same `net/http/pprof` surface.



Sampling parameters you control:

```
import "runtime"

// Block profile – record every blocking event (rate = 1 → every event,
// 0 → disabled).
// Default is 0. In prod, 1 is common for backend services; overhead is
// low.
// Higher values sample: runtime.SetBlockProfileRate(1000) → one in
// 1000 ns of block time.
runtime.SetBlockProfileRate(1)

// Mutex profile – fraction of contended lock events to record.
// Default 0 (disabled). 1 → every contention. Higher values sample.
runtime.SetMutexProfileFraction(1)

// Memory profile rate – bytes per sample (default 512 * 1024).
// Lower → more accurate but higher overhead. Leave default unless
// debugging tiny allocs.
runtime.MemProfileRate = 512 * 1024
```

### 3.2 CPU profiling — how `SIGPROF` sampling works

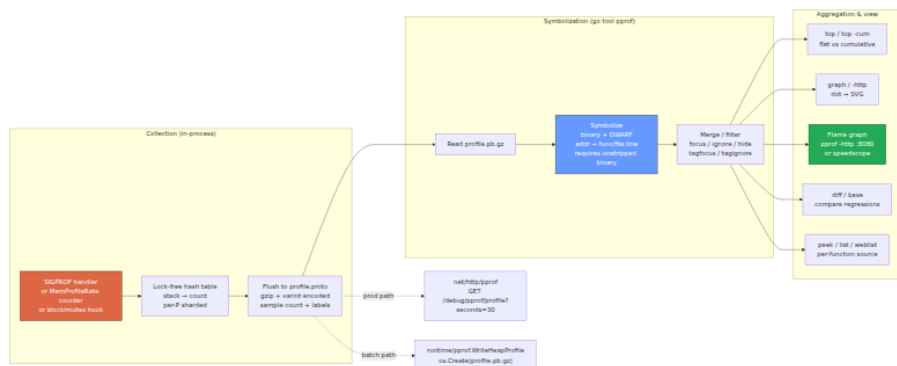
`runtime/pprof.StartCPUProfile(w)` asks the runtime to set `ITIMER_PROF` to 10 ms (100 Hz) per M. On each `SIGPROF`, the signal handler captures the current goroutine's stack (including cgo frames via `sigprof` trampolines), hashes it, and bumps a counter in a lock-free hash table. After the profile window, `StopCPUProfile` flushes the table as a proto-encoded `profile.proto` with stack-count pairs.



Implications:

- **Off-CPU time is invisible** to the cpu profile. A handler that spends 50 ms waiting on a slow downstream but only 2 ms on CPU will show 2 ms. Pair `cpu` + `block/mutex` + `trace` for the full picture.
- **100 Hz is coarse**. A 30 s profile at 100 Hz samples ~3,000 stacks per thread. Short or bursty workloads need longer windows or `benchstat` -aggregated profiles.
- **Inlining and `//go:noinline` matter**. Inlined functions may not appear as distinct frames unless you disable inlining for investigation.

### 3.3 The pipeline: collection → symbolization → aggregation



The profile file (`profile.proto`, usually `*.pb.gz`) is self-contained for stacks but **not for symbols** — `go tool pprof` needs the binary that produced it (or a local build at the same commit) to map addresses to `func/file:line`. In CI, archive the binary alongside the profile.

### 3.4 `net/http/pprof` — the prod sampling surface

The standard library registers HTTP handlers when you import `net/http/pprof`. No init needed beyond the import.

```
import _ "net/http/pprof" // registers /debug/pprof/* on
DefaultServeMux

// If you use a custom mux, register explicitly:
import "net/http/pprof"

mux := http.NewServeMux()
mux.HandleFunc("/debug/pprof/", pprof.Index)
mux.HandleFunc("/debug/pprof/cmdline", pprof.Cmdline)
mux.HandleFunc("/debug/pprof/profile", pprof.Profile) // cpu - ?
seconds=30
mux.HandleFunc("/debug/pprof/symbol", pprof.Symbol)
mux.HandleFunc("/debug/pprof/trace", pprof.Trace) // execution
trace - ?seconds=5
```

## Exposed endpoints:

GET /debug/pprof/	HTML index with links to each profile
GET /debug/pprof/profile?seconds=30&debug=0	30 s cpu profile (prot o by <b>default</b> , &debug=1 <input type="checkbox"/> text)
GET /debug/pprof/heap?debug=0	heap (live) <input type="checkbox"/> add &gc=1 to run GC before sampling
GET /debug/pprof/allocs	allocs (cumulative since start)
GET /debug/pprof/goroutine?debug=2	goroutine stacks (debug=2 <input type="checkbox"/> full stacks)
GET /debug/pprof/threadcreate?debug=1	
GET /debug/pprof/block?debug=1	
GET /debug/pprof/mutex?debug=1	
GET /debug/pprof/trace?seconds=5	execution trace (Section 4)
GET /debug/pprof/cmdline	command line
GET /debug/pprof/symbol	symbol lookup (POST addresses)

## Harden it before production — it is a sensitive endpoint:

```
// Gate behind internal listener / mTLS / admin port, not the public
ingress.
adminMux := http.NewServeMux()
adminMux.Handle("/debug/pprof/", middleware.RequireInternalAuth(pprof.I
ndex))
// Or serve on a separate port bound to 127.0.0.1 / Tailscale / mTLS
mesh.
go http.ListenAndServe("127.0.0.1:6060", nil)
```

## Scrape patterns for backend fleets:

```
One-off fetch from a pod (seconds controls window for cpu/trace)
curl -s http://pod-ip:6060/debug/pprof/profile?seconds=30 > cpu.pb.gz
curl -s http://pod-ip:6060/debug/pprof/heap > heap.pb.gz
curl -s "http://pod-ip:6060/debug/pprof/goroutine?debug=0" > goroutine.pb.gz

With auth header
curl -s -H "Authorization: Bearer $TOKEN" http://pod:6060/debug/pprof/profile?seconds=30 > cpu.pb.gz
```

### 3.5 runtime/pprof — programmatic API

For tests, benchmarks, and cron-sampled jobs:

```
import (
 "os"
 "runtime/pprof"
 "runtime/trace"
)

// CPU — paired Start/Stop; writes proto to w
f, _ := os.Create("cpu.pprof")
pprof.StartCPUProfile(f)
defer pprof.StopCPUProfile() // flush and remove SIGPROF handler

// Heap — one-shot snapshot
f2, _ := os.Create("heap.pprof")
defer f2.Close()
pprof.WriteHeapProfile(f2) // live heap; use pprof.Lookup("allocs") for cumulative

// Any named profile
pprof.Lookup("goroutine").WriteTo(os.Stdout, 1) // debug=1 text, debug=0 proto
pprof.Lookup("block").WriteTo(f3, 0)
pprof.Lookup("mutex").WriteTo(f4, 0)

// With labels — attach k/v to samples that pprof can filter on
pprof.Do(ctx, pprof.Labels("handler", "checkout", "tenant", "acme"), func(
 ctx context.Context) {
 handleCheckout(ctx) // samples inside carry handler=checkout, tenant=acme
})
// CLI: pprof -tagfocus handler:checkout cpu.pb.gz

// Manual block/mutex tuning
// (usually set once at startup, not per-request)
```

Label hygiene matters for fleet-wide aggregation — Parca / Pyroscope group by labels, so keep cardinality bounded (handler name, not user ID).

### 3.6 go tool pprof — the analysis CLI

Open a profile with the binary that produced it for best symbolization:

```
go tool pprof -http=:8080 ./bin/api cpu.pb.gz # interactive web UI
(graph, flame, top, source)
go tool pprof ./bin/api cpu.pb.gz # REPL
go tool pprof -top ./bin/api cpu.pb.gz
go tool pprof -top -cum ./bin/api cpu.pb.gz
cumulative (including callees)
```

Common REPL commands (also available as `go tool pprof -<cmd>` flags):

Command	What it does	When to use
<code>top [N]</code>	Top N functions by flat samples, with flat/cum %	First look — hottest functions
<code>top -cum</code>	Sort by cumulative (self + callees)	Find the call subtree that owns the time
<code>list &lt;regex&gt;</code>	Source-annotated per-line samples for matching funcs	Drill into a hot function
<code>weblist &lt;regex&gt;</code>	<code>list</code> + HTML with disassembly	Shareable annotated source
<code>peek &lt;regex&gt;</code>	Callees/callers of matching funcs	Trace caller→callee flow
<code>traces</code>	All distinct stack traces with sample counts	Spot tail-specific stacks
<code>focus=&lt;re&gt;</code> , <code>ignore=&lt;re&gt;</code> , <code>hide=&lt;re&gt;</code>	Filter samples by regex before display	Isolate a subsystem
<code>tagfocus=&lt;k:v&gt;</code> , <code>tagignore=&lt;k:v&gt;</code>	Filter by <code>pprof.Do</code> labels	Slice by handler / tenant

Command	What it does	When to use
<code>diff_base=&lt;other.pb.gz&gt;</code>	Subtract baseline profile	Before/after regression
<code>sample_index=cpu / alloc_objects / inuse_space</code>	Switch heap view	Toggle live vs. allocs interpretation

Real outputs — trimmed:

**CPU top** (flat = time in function itself, cum = time in function + callees):

```
$ go tool pprof -top ./bin/api cpu.pb.gz
Fetching profile over HTTP from http://127.0.0.1:6060/debug/pprof/profile?seconds=30
Saved profile in /home/ubuntu/pprof/pprof.api.samples.cpu.001.pb.gz
Type: cpu
Time: Aug 21, 2026 at 14:03 UTC
Duration: 30.12s, Total samples = 3021
Showing nodes accounting for 2840, 94.01% of 3021 total
Dropped 42 nodes (cum <= 15)

 flat flat% sum% cum cum%
 612 20.26% 20.26% 612 20.26% runtime.mallocgc
 408 13.51% 33.77% 891 29.50% encoding/json.
(*encodeState).marshal
 287 9.50% 43.27% 287 9.50% runtime.memmove
 191 6.32% 49.59% 403 13.34% myapp/handler.Checkout
 143 4.73% 54.32% 143 4.73% crypto/sha256.block
 98 3.24% 57.57% 620 20.52% net/http.(*conn).serve
 0 0% 100% 891 29.50% myapp/handler.Checkout ->
encoding/json.Marshal (cum)
```

Reading: Checkout is 6.3% flat but 13.3% cum — it spends half its time in json.Marshal. The runtime.mallocgc flat tells you allocation churn is the top CPU consumer — cross-check with allocs.

**Heap top** — live bytes ( inuse\_space ) vs. allocs ( alloc\_space ):

```
$ go tool pprof -top --sample_index=inuse_space ./bin/api heap.pb.gz
Type: inuse_space
Showing nodes accounting for 412.33MB, 92.1% of 447.80MB total
 flat flat% sum% cum cum%
 128.50MB 28.70% 28.70% 128.50MB 28.70% myapp/cache.NewLRU (128 MB
live – leak or cache size?)
 84.20MB 18.80% 47.50% 84.20MB 18.80% encoding/json.(*Decoder).Bu
ffered
 42.10MB 9.40% 56.90% 170.60MB 38.10% myapp/handler.Checkout allo
c (dominated by json + LRU)
 31.00MB 6.92% 63.82% 31.00MB 6.92% bytes.growSlice
```

```
$ go tool pprof -top --sample_index=alloc_space ./bin/api heap.pb.gz
Type: alloc_space
 flat flat% sum% cum cum%
 4.12GB 34.20% 34.20% 4.12GB 34.20% encoding/json.Marshal (churn
– GC pressure)
 1.80GB 14.90% 49.10% 1.80GB 14.90% myapp/handler.Checkout / string
s.Builder
```

Rule: `inuse_space` finds leaks; `alloc_space` finds churn. A function high in `alloc_space` but absent in `inuse_space` is GC pressure, not a leak.

### list — per-line attribution:

```
$ go tool pprof -list Checkout ./bin/api cpu.pb.gz
Total: 3021
ROUTINE ===== myapp/handler.Checkout in myapp/handle
r/checkout.go
 191 403 (flat, cum) 13.34% of Total
 . . 42:func (h *Handler) Checkout(w http.Response
Writer, r *http.Request) {
 3 3 43: items := h.cart.Items(r.Context())
 3 flat, 287 cum in callee 44: body, err := json.Marshal(items) //
 112 112 45: h.cache.Set(r.Context(), key,
body) // 112 flat [] copy[]
 76 288 46: w.Write(body)
```

### weblist / peek / traces :

```
go tool pprof -weblist Checkout ./bin/api cpu.pb.gz # annotated HTML
with assembly
go tool pprof -peek json.Marshal ./bin/api cpu.pb.gz # callers and
calleees of Marshal
go tool pprof -traces ./bin/api cpu.pb.gz # every unique
stack
```

## Diffing a regression:

```
go tool pprof -top -diff_base=baseline.pb.gz ./bin/api candidate.pb.gz
Positive flat → candidate does more work; negative → candidate improved.
```

## 3.7 Flame graphs

`go tool pprof` serves an interactive SVG:

```
go tool pprof -http=:8080 ./bin/api cpu.pb.gz
Opens browser: Graph (dot), Flame Graph, Peek, Source, Traces, Top
Flame Graph tab: x = sample count, y = stack depth, color = package,
click to zoom.
Or export:
go tool pprof -flame ./bin/api cpu.pb.gz > flame.svg
```

For sharing without Go: **Speedscope** (<https://speedscope.app> — drag the `.pb.gz`) or `pprof -http export`.

Flame graph reading (cpu profile, left-to-right = flat sample share):

```
[all] 3021 samples (100%)
└─ runtime.mallocgc 612
└─ encoding/json.Marshal 891
 └─ encodeState.marshal 408
 └─ reflect.Value ...
 └─ myapp/handler.Checkout 191
 └─ cache.Set 112
 └─ w.Write 76
 └─ net/http.serve 620
 └─ ...
```

Heap **flame**: width = bytes, **not time** — look for wide plateaus holding live memory.

Common misread: flame width is **flat + cum overlap** — a wide tower means that subtree owns the resource; a thin tower with a wide base means the cost is scattered across many leaves.

## 4. Execution trace

`pprof` tells you *where* resources are spent aggregated over time. The **execution trace** tells you *when* — a nanosecond event log of goroutine

state transitions, P scheduling, GC/STW, syscalls, and network events over a capture window (typically 1-10 seconds).

### 4.1 Capturing a trace

```
import "runtime/trace"

// Programmatic - any Go program
f, _ := os.Create("trace.out")
trace.Start(f)
defer trace.Stop() // flush; trace is ~1-10 MB/s depending on event
rate

// HTTP - net/http/pprof already exposes it
// curl http://127.0.0.1:6060/debug/pprof/trace?seconds=5 > trace.out

// Tests / benchmarks - no code
go test -trace=trace.out ./...
go test -run=BenchmarkCheckout -trace=trace.out -bench=. ./...

// Benchmark with trace + cpu + mem in one run (Go 1.21+)
go test -bench=BenchmarkCheckout -benchtime=5s -cpuprofile=cpu.pprof -m
empfile=mem.pprof -trace=trace.out ./...
```

Trace files are **not proto** — they are a custom binary format. Open with `go tool trace`, not `go tool pprof`.

### 4.2 go tool trace — the viewer

```
go tool trace trace.out
2026/08/21 14:05:00 Parsing trace...
2026/08/21 14:05:01 Splitting trace...
2026/08/21 14:05:01 Opening browser. View trace at http://
127.0.0.1:41233
```

The viewer is a **browser app** (no CLI equivalent for the timeline). Views:

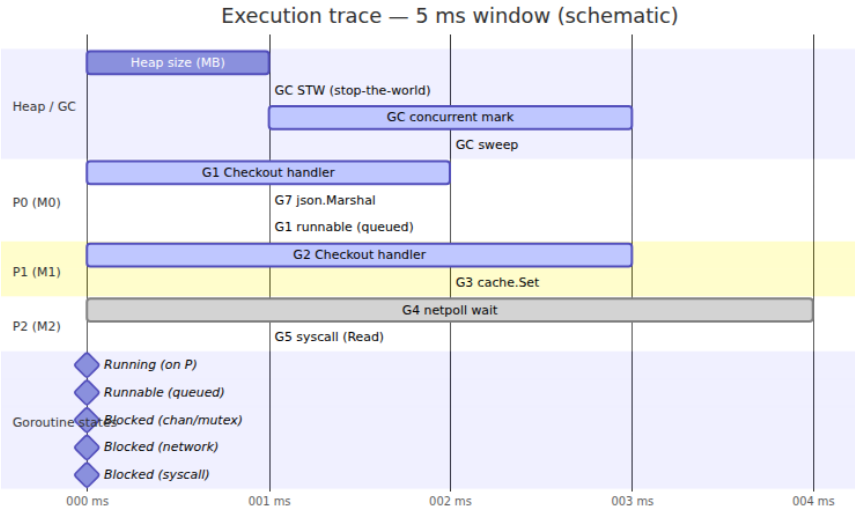
View	What it shows	Use for
View trace	Full timeline: goroutines × Ps + heap + GC + STW + network	Primary view — scheduling stalls, GC pauses, P starvation
Goroutine analysis	Per-goroutine stats: execution / runnable / blocked / sched latency	Find goroutines with high runnable latency



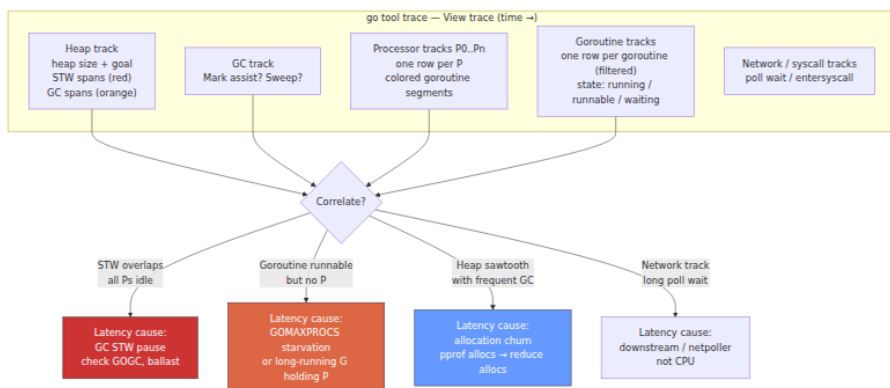
View	What it shows	Use for
Network blocking profile	Network wait stacks	Netpoller vs. slow downstream
Synchronization blocking profile	<code>sync</code> blocking stacks	Contention-driven stalls
Syscall blocking profile	Syscall stacks	<code>entersyscall</code> handoff diagnosis
Scheduler latency profile	Time goroutines spent runnable but not running	Quantify scheduler delay (p99 runnable)

### 4.3 Walkthrough: reading the timeline

The timeline has **horizontal tracks** (y = goroutines and Ps) vs. time (x, nanoseconds). Color encodes goroutine state.



More precise mental model as a flowchart:



Step-by-step reading:

1. **Set the window.** Drag to zoom to the slow request (e.g., a 12 ms p99 span). The minimap at the top shows the full capture.
2. **Read the heap track first.** Is the p99 request aligned with a GC STW pause (red vertical bar) or a heap spike? If yes, the latency is GC — confirm with `GODEBUG=gctrace=1 / godebug` and `pprof heap`.
3. **Read the P tracks.** Are all P s busy when your goroutine becomes runnable (orange runnable segment)? That is scheduler latency — `GOMAXPROCS` starvation or a `for { }` loop without preemption points (Section 1.2 of Chapter 4) holding a P.
4. **Click a goroutine segment.** The info pane shows its stack at that instant, the wait reason ( `chan receive` , `semacquire` , `I/O wait` , `GC assist` ), and the blocking call site. For `I/O wait` , switch to the network blocking profile.
5. **Check goroutine analysis.** Sort by runnable time — the goroutine with the highest runnable / execution ratio is the scheduling victim, not the cause. The cause is the goroutine monopolizing the P in the same window.

Example diagnosis:

A `GET /checkout` p99 of 45 ms on a 4-core pod. CPU profile shows only 8 ms on-CPU. Trace of a 5 s capture shows three 2 ms STW pauses per second (`GOGC=100`, live heap 1.2 GB) and a recurring 18 ms runnable span for `G12345` (Checkout) queued behind `G12300` running `json.Marshal` on a 4 MB cart. Fix: reduce allocation churn in `Marshal`

(pooling / `json.RawMessage`), raise `GOGC` or add `GOMEMLIMIT`, and ensure `GOMAXPROCS` matches the cgroup CPU limit via `automaxprocs`.

## 4.4 Trace for latency investigations — backend lens

- **Capture in prod without `-race`**. Trace overhead is moderate (~5–15% during capture, plus file I/O). Capture 5–10 s on a single pod replica, not the fleet, and download via `net/http/pprof/trace`. Prefer off-peak or canary pods.
- **Correlate with pprof**. Always capture `cpu.pb.gz` + `heap.pb.gz` + `trace.out` together around the slow window — `cpu` says *what* was hot, `trace` says *when* it blocked the request, `heap` says *why* GC fired.
- **Sampling wrappers:**

```
One-shot triad from a pod
POD=api-7d9c8f-xxxxx
NS=prod
kubectl exec -n $NS $POD -- curl -s "http://127.0.0.1:6060/debug/pprof/profile?seconds=30" > cpu.$(date +%s).pb.gz &
kubectl exec -n $NS $POD -- curl -s "http://127.0.0.1:6060/debug/pprof/heap" > heap.$(date +%s).pb.gz
kubectl exec -n $NS $POD -- curl -s "http://127.0.0.1:6060/debug/pprof/trace?seconds=5" > trace.$(date +%s).out
wait
go tool pprof -http=:8080 ./bin/api cpu.*.pb.gz
go tool trace trace.*.out
```

- **Continuous trace is not useful** — traces are high-cardinality event logs, not sampled aggregates. For continuous fleet-wide views, use `pprof-continuous` systems (next section) and capture traces ad hoc for deep dives.

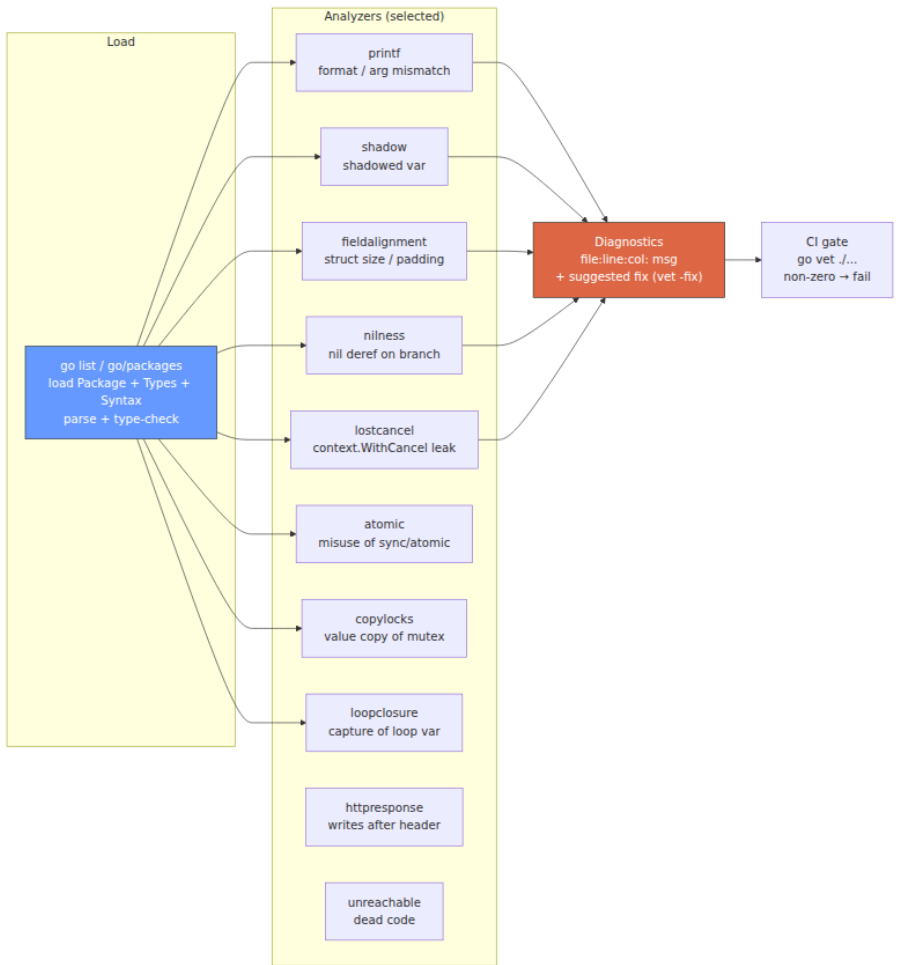
---

## 5. `go vet` and `staticcheck` — static analyzers

The race detector and profilers are *dynamic* — they see what ran. `go vet` and `staticcheck` are *static* — they see what *could* go wrong without running, by analyzing the typed AST/SSA.

## 5.1 go vet — the analyzer pipeline

`go vet` is a driver that runs a suite of **analyzers** from `golang.org/x/tools/go/analysis`. Each analyzer declares the facts it needs, the files it inspects, and the diagnostics it produces. The driver builds an analysis graph, runs analyzers in dependency order, and prints `file:line:col: message`.



Running:

```

go vet ./... # all analyzers
go vet -vettool=$(which shadow) ./... # include shadow (not on by
default – see note)
go vet -printf -nilness ./... # subset

Per-analyzer flags
go vet -fieldalignment ./...
report structs that waste space due to padding
go vet -composites ./... # unkeyed composite literals
(fragile)

Show available analyzers
go vet -help | grep -A2 "analyzer"
go vet --help # Go 1.21+ lists analyzers explicitly

Auto-fix where possible (not all analyzers offer fixes)
go vet -fix ./... 2>&1 | head -n 50

```

Note on `shadow`: removed from the default `vet` set in Go 1.12 due to noise. Enable via `golang.org/x/tools/go/analysis/passes/shadow` or via `golangci-lint` / `staticcheck`; do not rely on bare `go vet` to catch shadowing.

## 5.2 Before / after — representative warnings

### **printf** — format/argument mismatch (always enabled):

```

// before
log.Printf("user %s has %d items", userID) // vet: Printf format %d
reads arg 2, but call has 1 arg

// after
log.Printf("user %s has %d items", userID, len(items))

```

```

$ go vet ./...
handler.go:44:2: printf: Printf format %d reads arg 2, but call has 1 a
rg

```

### **fieldalignment** — struct padding waste (opt-in):

```

// before - 24 bytes on amd64 (bool + 7 pad + int64 + bool + 7 pad) -
vet: struct of size 24 could be 16
type CacheEntry struct {
 Hit bool // 1 + 7 pad
 Value int64 // 8
 Valid bool // 1 + 7 pad
}

// after - 16 bytes (reorder by descending alignment)
type CacheEntry struct {
 Value int64 // 8
 Hit bool // 1
 Valid bool // 1 + 6 pad
}

// For hot structs allocated millions of times, this is RSS and cache-
line utilization.

$ go vet -fieldalignment ./internal/cache
myapp/internal/cache
cache.go:12:9: struct of size 24 could be 16

```

Trade-off: reorder only when profiling shows the struct is hot or the binary bloats; field order can be load-bearing for `encoding/json` alphabetical surprises and `unsafe` layouts.

### **`nilness` — nil dereference on a branch:**

```

// before
func handle(w http.ResponseWriter, r *http.Request) {
 var user *User
 if r.URL.Query().Get("id") != "" {
 user = lookup(r.Context(), r.URL.Query().Get("id"))
 }
 println(user.Name) // vet: nilness: may be nil - lookup not called
 // on empty id
}

// after
func handle(w http.ResponseWriter, r *http.Request) {
 user := lookupOrAnonymous(r)
 if user == nil {
 http.Error(w, "not found", 404)
 return
 }
 println(user.Name)
}

```

### **`lostcancel` — leaked context cancel func:**

```
// before
func fetch(ctx context.Context) {
 ctx, _ = context.WithCancel(ctx) // vet: the cancel function
 // returned by context.WithCancel should be called, not discarded
 doWork(ctx)
}

// after
func fetch(ctx context.Context) {
 ctx, cancel := context.WithCancel(ctx)
 defer cancel()
 doWork(ctx)
}
```

### copylocks — copying a value containing a mutex:

```
// before
type Counter struct{ mu sync.Mutex; n int }
func (c Counter) Inc() { c.mu.Lock(); c.n++; c.mu.Unlock() } // vet:
// Inc passes lock by value — copy of Mutex

// after
func (c *Counter) Inc() { c.mu.Lock(); c.n++; c.mu.Unlock() }
```

**loopclosure** / **shadow patterns** — see the race section's loop-variable example; vet's **loopclosure** flags the racy capture pre-1.22, and **shadow** flags accidental shadowing that hides the intended variable.

## 5.3 staticcheck and golangci-lint

**go vet** is intentionally conservative — Go ships only low-false-positive analyzers. For deeper coverage, two tools dominate backend fleets:

**staticcheck** ([honnef.co/go/tools/staticcheck](https://honnef.co/go/tools/staticcheck)) — ~150 checks (SA, S, ST\*) covering:

- **SA9003** — empty branches, **SA5008** — **defers** in loops, **SA6000** — regexp can be compiled at init, **SA1024** — **time.Sleep** with context.
- **ST1005** — error strings should not be capitalized, **QF1001** — apply **De Morgan** simplifications.
- Inline documentation for every check at [staticcheck.io/docs/checks](https://staticcheck.io/docs/checks).

```

go install honnef.co/go/tools/cmd/staticcheck@latest
staticcheck ./...
staticcheck -f stylish ./... # grouped output
staticcheck -checks "all,-ST1000" ./... # enable all, exclude style
nits

Per-package config in staticcheck.conf
checks = ["all", "-ST1000", "-ST1003"]
initialisms = ["API", "ID", "URL"]

```

**golangci-lint** — meta-linter that runs `vet`, `staticcheck`, `errcheck`, `ineffassign`, `gosec`, `fieldalignment`, `shadow`, and 50+ others in one binary with caching. The fleet standard for CI.

```

.golangci.yml (excerpt – backend-appropriate)
run:
 timeout: 5m
 go: "1.23"
linters:
 enable:
 - govet
 - staticcheck
 - errcheck
 - ineffassign
 - fieldalignment
 - shadow
 - nilnil
 - contextcheck
 - noctx # http call without context
 - bodyclose # unclosed response body
 - gocritic
 - revive
issues:
 exclude-rules:
 - path: _test*.go
 linters: [fieldalignment]
 - path: internal/mocks
 linters: [all]
 max-issues-per-linter: 0
 max-same-issues: 0
output:
 formats:
 - format: github-actions # annotations inline in PR diff

CI:
golangci-lint run ./...
golangci-lint run --new-from-rev=origin/main ./... # only new issues
on PR

```



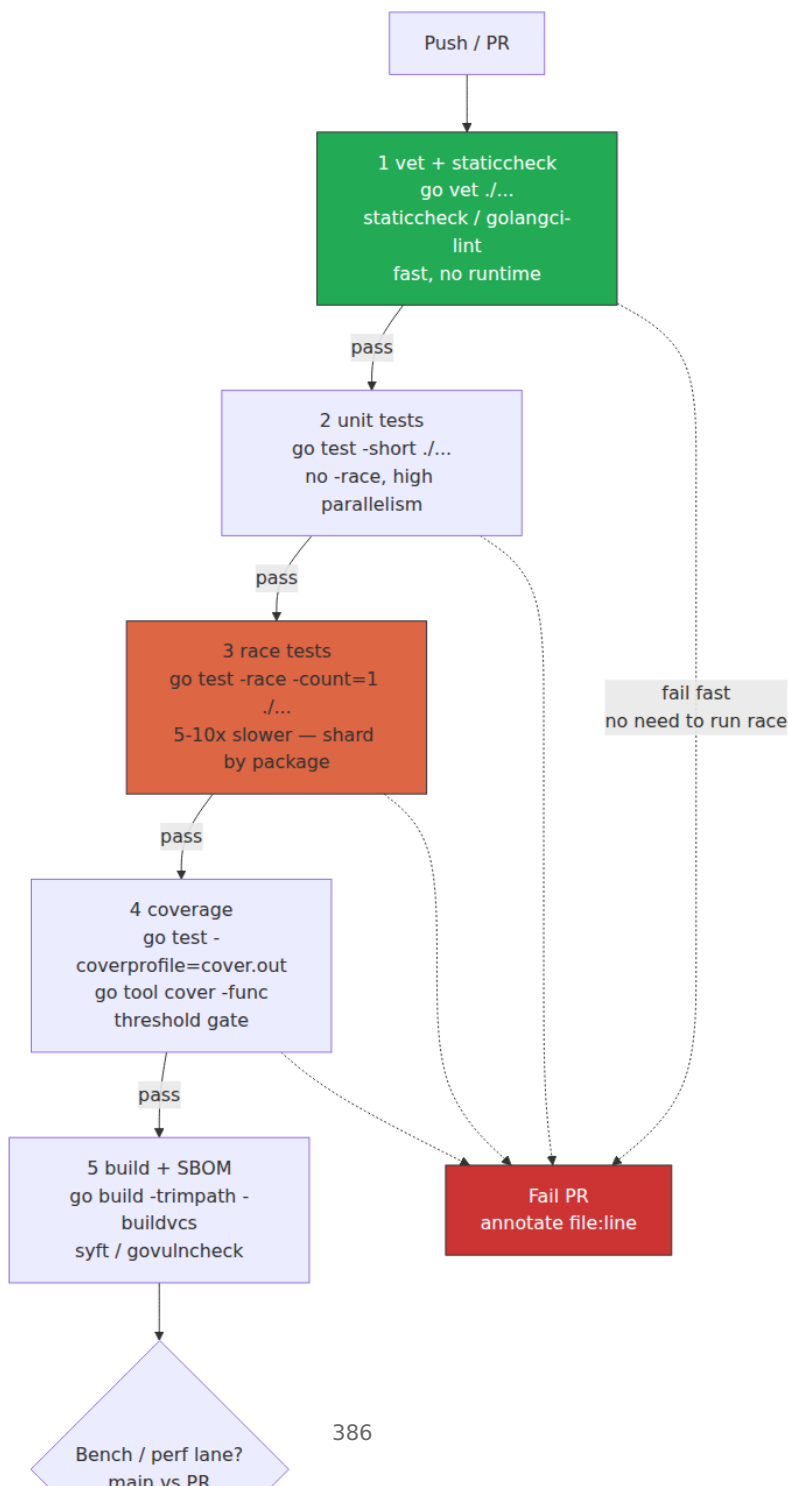
When to use which:

- `go vet ./...` — always, in every CI job, zero config.
  - `staticcheck ./...` — enable per-repo; fix or `//lint:ignore SAxxxx` reason with justification.
  - `golangci-lint run ./...` — fleet-wide standard when you want `errcheck` / `bodyclose` / `noctx` in addition to `vet` — these catch the backend-specific leaks ( `resp.Body` not closed, `http.Get` without context) that cause goroutine and connection leaks at scale.
- 

## 6. Backend lens — CI, continuous profiling, and latency playbooks

### 6.1 The CI tooling DAG

Backend CI should run cheap static checks first, then functional tests, then expensive dynamic checks, then artifacts that depend on a green build.



Practical notes:

- **Gate vet before race**. A `printf` bug that `vet` catches in 2 s should not wait for a 10 min race job.
- **Shard test -race** by package (or by `go test -race ./internal/...` buckets) and set per-shard memory limits — the 10x shadow will OOM a 4 GB runner on large packages.
- **Coverage after race** so the threshold reflects race-exercised paths.
- **Perf lane as non-blocking** for PRs (informational diff), blocking for `main` regressions.

## 6.2 Sampling pprof in prod — Parca / Pyroscope continuous profiling

One-off `curl /debug/pprof` is for incidents. For fleets, **continuous profiling** scrapes every pod on a schedule and aggregates by service, version, and `pprof.Do` label.

Options:

- **Parca** (CNCF) — pull-based, Prometheus-style scrape config, `profile.proto` storage, pprof-compatible UI. Works with `net/http/pprof` directly.
- **Pyroscope** (Grafana) — push or pull, Grafana-native, supports Go `pprof` plus eBPF.
- **Homegrown** — cron `curl` → object storage → offline `go tool pprof` diffs (simplest to start).

```
Parca scrape config (excerpt) – pulls cpu + heap + goroutine + mutex
from each pod
scrape_configs:
- job_name: go-services
 kubernetes_sd_configs: [{ role: pod }]
 relabel_configs:
 - source_labels: [__meta_kubernetes_pod_annotation_parca_enabled]
 regex: "true"
 action: keep
 params:
 seconds: ["30"]
 scrape_interval: 30s
 profiling_config:
 pprof_config:
 memory:
 enabled: true
 path: /debug/pprof/heap
 block:
 enabled: true
 path: /debug/pprof/block
 delta: true
 mutex:
 enabled: true
 path: /debug/pprof/mutex
 delta: true
 goroutine:
 enabled: true
 path: /debug/pprof/goroutine
```

## Operational guidance:

- Enable `SetBlockProfileRate(1)` and `SetMutexProfileFraction(1)` in prod for block/mutex — overhead is low and the contention signal is worth it.
- Keep `MemProfileRate` at default unless hunting a tiny-alloc leak.
- Scrape `cpu` at 30–60 s windows every 1–5 min; longer windows smooth short spikes but increase per-scrape cost.
- Gate `net/http/pprof` on an admin port / mesh mTLS — unauthenticated pprof is a heap-exfiltration vector.
- Alert on **alloc space growth** and **goroutine count** derived from continuous profiles, not just CPU.

### 6.3 Trace for latency investigations — playbook

When p99 spikes without a CPU spike:

1. **Capture the triad** on a canary pod: `cpu.pb.gz` (30 s) + `heap.pb.gz` + `trace.out` (5 s). Do it twice — once during good p99 and once during bad.
2. **Open trace first.** Check STW alignment, then P saturation, then goroutine `runnable` time.
3. **Diff pprof:** `go tool pprof -diff_base=good.pb.gz bad.pb.gz` — new flat in `mallocgc` or `sync` points to the regressed path.
4. **Corroborate with metrics:** `GOMAXPROCS` vs. cgroup quota, `go_gc_duration_seconds`, `go_goroutines`, `process_cpu_seconds_total`.

Trace finding	Likely cause	Fix
Periodic red STW bars aligned with p99 spikes	GC STW (heap too large, GOGC too low)	Raise <code>GOGC</code> , set <code>GOMEMLIMIT</code> , reduce <code>alloc_space</code> churn
<code>runnable &gt;&gt;</code> <code>running</code> for request goroutines, Ps at 100%	<code>GOMAXPROCS</code> starvation / long <code>P</code> -holding goroutine	Fix cgroup limit, use <code>automaxprocs</code> , break up long loops
Long <code>IO</code> wait in network track, Ps idle	Slow downstream / netpoller not the bottleneck	Timeout / circuit-break downstream, not more cores
<code>semacquire</code> / <code>chan park</code> dominating	Lock contention	<code>mutex</code> profile → reduce critical section, shard, or switch to <code>RWMutex</code> / <code>atomic</code>

## 7. Putting it together — a worked incident

A `checkout` service rolls out a new `Cache.Set` that JSON-marshals the whole cart.

- `go vet` — green.
- `go test -race` — green (tests use a `T` per goroutine — the race only fires under concurrent `Set` / `Get` on the same key, which the test does not do).

- Load test at 5k RPS, p99 from 18 ms → 47 ms, CPU from 42% → 68%, `go_goroutines` stable, heap from 600 MB → 1.4 GB.

## Investigation:

```
Canary pod triad
curl -s http://canary:6060/debug/pprof/profile?seconds=30 > bad.cpu.pb.gz
curl -s http://canary:6060/debug/pprof/heap > bad.heap.pb.gz
curl -s http://canary:6060/debug/pprof/trace?seconds=5 > bad.trace.out
go tool pprof -top bad.cpu.pb.gz
go tool pprof -top --sample_index=alloc_space bad.heap.pb.gz
go tool trace bad.trace.out
```

- CPU top: `encoding/json.Marshal` 29% cum, `runtime.mallocgc` 20% flat.
- Heap `alloc_space`: `json.Marshal` 4.1 GB over 30 s (churn), heap inuse dominated by `CacheEntry` 128 MB.
- Trace: 2 ms STW every 1.1 s, `G12345 Checkout` runnable 14 ms behind a `G12300 Marshal` on `P0`.

**Fix:** pool `bytes.Buffer` + `json.Encoder`, switch `CacheEntry` to `fieldalignment-optimized` layout, add `GOMEMLIMIT=1GiB`, and add a concurrent `go test -race -count=50 -run TestCacheConcurrent` that exercises `Set/Get` overlap. Re-cut profiles: CPU 43%, heap 620 MB, STW 0.4 ms, p99 19 ms.

## Key takeaways

- The race detector is ThreadSanitizer with Go-aware happens-before instrumentation (shadow memory + vector clocks). It reports real races with two stacks and creation sites; treat every report as a P0. Run it in CI ( `go test -race` ), never in prod — 5–10x CPU/RAM overhead and shifted timing.
- Races are dynamic — uncovered paths are invisible. Use `-count`, stress tests, and concurrent test cases that actually share the state under the same synchronization discipline as prod.
- pprof profiles answer different questions: `cpu` (where on-CPU time goes, 100 Hz `SIGPROF`), `heap` (live bytes), `allocs` (churn/GC pressure), `goroutine / threadcreate` (leaks), `block / mutex` (parking

and contention). Collect the one that matches the symptom; pair them.

- `net/http/pprof` is the prod sampling surface; `runtime/pprof` + `pprof.Do` labels are the programmatic surface. Always capture with the binary for symbolization, use `go tool pprof` verbs (`top`, `list`, `weblist`, `peek`, `traces`, `diff_base`, `-http` flame graphs), and read flat vs. cum correctly.
- Flame graphs visualize cum ownership; heap flames show byte ownership. Speedscope and `pprof -http` are interchangeable — archive `*.pb.gz` + binary per incident.
- Execution trace is a nanosecond event log (goroutines × Ps + heap/STW/GC). Use it when pprof shows low CPU but high p99 — read heap → P saturation → goroutine `runnable` → wait reason, in that order.
- `go vet` is the mandatory CI gate (`printf`, `nilness`, `lostcancel`, `copylocks`, `composites`, ...); `shadow` and `fieldalignment` are opt-in. `staticcheck` adds ~150 SA checks; `golangci-lint` bundles `vet` + `staticcheck` + `errcheck`/`bodyclose`/`noctx` for backend fleets.
- Wire the DAG as `vet` → `test` → `race` → `cover` → `build` → (`bench/pprof` diff). For prod, run continuous profiling (Parca / Pyroscope) scraped from `net/http/pprof` with block/mutex enabled, and capture traces ad hoc for latency dives.

## Further reading

- **Go Data Race Detector — official docs.** Build modes, overhead, happens-before coverage, `GORACE` flags, and cgo notes. [https://go.dev/doc/articles/race\\_detector](https://go.dev/doc/articles/race_detector) (*pinned*)
- **Diagnosing performance with `net/http/pprof` and `go tool pprof` — official docs.** Endpoints, profile types, `runtime/pprof` API, and CLI verbs. <https://go.dev/doc/diagnostics#profiling> (*pinned*)
- **`runtime/trace` and `go tool trace` — official docs.** Capturing traces, viewer tracks, and interpretation guide. <https://go.dev/doc/diagnostics#tracing> — also `go doc runtime/trace` and `go tool trace -help` (*pinned*)
- **`go vet` — `cmd/vet` docs and `go/analysis` passes.** Analyzer list, flags, and how to write custom analyzers. <https://pkg.go.dev/cmd/vet>

— per-pass docs at <https://pkg.go.dev/golang.org/x/tools/go/analysis/passes> (*pinned*)

- **Staticcheck** — [honnef.co/go/tools/staticcheck](https://honnef.co/go/tools/staticcheck) docs. All SA/S/ST/QF checks, configuration, and `//lint:ignore` guidance. <https://staticcheck.io/docs/> (*pinned*)
- **ThreadSanitizer** — **design and algorithm**. Vector clocks, shadow memory, and happens-before tracking that Go's detector is derived from. <https://github.com/google/sanitizers/wiki/ThreadSanitizerAlgorithm>
- **Google pprof** — **profile.proto** and **go tool pprof** reference. Profile format, symbolization, and UI. <https://github.com/google/pprof>
- **Parca** — **continuous profiling architecture**. Pull-based scrape, storage, and fleet-wide pprof aggregation. <https://www.parca.dev/docs/>
- **Pyroscope** — **continuous profiling for Go**. Push/pull modes, `pprof.Do` labels, and Grafana integration. <https://pyroscope.io/docs/>
- **Felix Geisendoerfer** — ***The Go pprof Guide (practical)***. CPU/heap/block/mutex examples, diff workflows, and flame-graph reading at backend scale. <https://github.com/felixge/pprofutils> (and [github.com/google/pprof](https://github.com/google/pprof) examples)

## Chapter 11 — cgo, Assembly, and Foreign-Function Interoperability

---

**What this chapter covers.** Go promises a static binary, a concurrent runtime, and a garbage collector. C promises direct memory access, a universe of existing libraries, and hand-tuned assembly. At the boundary the promises collide: the collector cannot see the C heap, the scheduler cannot preempt a C thread, and a single `C.foo()` call pays a stack switch and scheduling tax that dwarfs a normal Go call. Yet real backend systems live at that boundary — linking `libssl`/`boringcrypto` for FIPS, `libz`/`zstd` for compression, `librocksdb` for storage, or a hand-written AVX-512 kernel for hashing. This chapter opens the boundary: how `cgo` bridges the two worlds, what it costs, when to avoid it, how Go assembly actually works, and how to do foreign-function interop without `cgo` at all.



Learning goals — after this chapter you should be able to:

- Explain `cgo`'s compilation model — the `import "C"` preamble, generated `_cgo_gotypes.go` / `_cgo_export.h`, and the `//export` / `_cgoexp_` callback bridge.
- Write correct `#cgo CFLAGS` / `LDFLAGS` / `pkg-config` directives and predict how they compose across packages and cross-compile environments.
- Map Go  $\leftrightarrow$  C types (`C.int`, `*C.char`, `C.size_t`, `GoString` / `GoBytes` / `GoSlice`) and caller responsibilities for each direction.
- Call C from Go and Go from C (callbacks), keeping the program correct when `//export` functions run on a foreign thread.
- Manage memory ownership across the boundary — `C.CString` / `C.free`, `runtime.KeepAlive`, `runtime.Pinner` (Go 1.21+), and why `go:uintptrescapes` exists.
- Describe the threading model — M vs. OS thread, `__cgo_thread_start` / `cgocall` / `cgocallback`, `LockOSThread`, and when `CGO_ENABLED=0` matters.
- Quantify the performance cost of a `cgo` call (stack switch, scheduler interaction, not inlinable, escape-forcing) and measure it with a benchmark.
- Read and write Go assembly — `TEXT` / `DATA` / `GLOBL`, `FUNCDATA` / `PCDATA`, `NOSPLIT`, Plan 9 heritage, `go tool compile -S`, and `//go:asm` / `-asmflags`.
- Choose the right FFI mechanism — `cgo` vs. pure-Go rewrite vs. `syscall/x/sys/unix` vs. `plugin` — and justify the choice for deployability and operability.

**Placement.** Chapter 3 covered the Go ABI (register calling convention, stack frames, `go tool objdump`). Chapter 9 covered the SSA compiler pipeline and escape analysis. This chapter is where that ABI meets *foreign code* — C and assembly — and where the scheduler/GC guarantees from Chapters 4–5 are most visibly strained. Chapter 12 builds on the deployability consequences (`CGO_ENABLED`, static linking, cross-compilation).

## 1. Why the boundary exists — and why backend engineers should care

Pure Go covers most backend work. The standard library ships a fast `crypto/tls`, `compress/flate`, `hash/crc32`, and `net` — all pure Go. And the deployment story is part of Go's appeal: `CGO_ENABLED=0 go build` produces a static binary that runs on `scratch` or `distroless`, cross-compiles with `GOOS / GOARCH`, and has no `glibc / libssl` version matrix to manage.

That appeal is exactly why the boundary decision matters. Each `import "C"` you add:

- **Breaks the static-binary guarantee.** The output becomes dynamically linked (unless you work to re-static-link with `musl` or `-extldflags "-static"`). Your `Dockerfile` grows a `debian:bookworm-slim` base and a CVE feed for every `.so` you pulled in.
- **Defeats cross-compilation.** You now need a C cross-toolchain (`x86_64-linux-gnu-gcc`, `aarch64-linux-gnu-gcc`, `CC / CXX` wrappers) and sysroots. CI matrices multiply.
- **Changes the failure mode.** A C library can `SIGSEGV`, leak, or hold a lock across a Go scheduler preemption point. `recover` does not catch a C segfault. `pprof` may show time in `cgocall` with no Go stack.
- **Costs per call.** A `cgo` call is ~70-150 ns of overhead on `amd64` (Go 1.21, Linux) before the C work itself — a stack switch, `entersyscall / exitsyscall`, and scheduler bookkeeping. On a hot path doing millions of calls/s, that dominates. A pure Go call is ~1-3 ns (inlined) or ~5-10 ns (non-inlined, register ABI).

Real teams still choose `cgo` when the alternative is worse: FIPS-validated `BoringCrypto` (`GOEXPERIMENT=boringcrypto` links `libcrypto`), hardware-accelerated `libzstd / liblz4` at line rate, `libvips` for image processing, or an existing C SDK with no Go equivalent. The skill is knowing when the trade is worth it and how to keep the blast radius small.

---

## 2. cgo fundamentals: `import "C"` and the preamble

`cgo` is not a runtime library. It is a **code generator** that runs before `go compile`. When the toolchain sees `import "C"`, it:

1. Extracts the comment block immediately above `import "C"` — the **preamble** — as C source.
2. Invokes `go tool cgo` to parse that C, generate Go wrappers (`_cgo_gotypes.go`, `_cgo_import.go`) and C stubs (`_cgo_export.h`, `*.cgo1.go`, `*.cgo2.c`).
3. Compiles the C stubs with the C compiler (`CC`, default `gcc` / `clang`) and the Go wrappers with `go compile`.
4. Links the resulting objects together via the external linker (`CC` again).

The canonical minimal example:

```
// hello.go
package main

/*
#include <stdio.h>
#include <stdlib.h>

// A plain C function. Static avoids a duplicate-symbol hazard
// when multiple Go packages include the same header.
static void hello(const char *s) {
 printf("hello, %s\n", s);
}
*/
import "C"
import "unsafe"

func main() {
 cs := C.CString("world") // malloc on C heap, copy bytes +
 NUL // NUL byte
 defer C.free(unsafe.Pointer(cs))
 C.hello(cs) // Go → C call via generated wrapper
}
```

```

go run hello.go
hello, world

With explicit toolchain visibility:
go run -x hello.go 2>&1 | grep -E 'cgo|gcc|_cgo'
mkdir $WORK/b001/
../go tool cgo -objdir $WORK/b001/ -- import $WORK/b001/
_cgo_gotypes.go hello.go
gcc -I $WORK/b001 -g -O2 -c -o $WORK/b001/_cgo_main.o $WORK/b001/
_cgo_main.c
gcc -c -o $WORK/b001/_x001.o $WORK/b001/_cgo_export.c

```

Key rules that the compiler enforces:

- `import "C"` **must stand alone**: `import "C"` cannot be grouped with other imports (`import ("C"; "fmt")` is illegal). The preamble must be the comment immediately above it — no blank line, no intervening `import`.
- The preamble is C, not Go. `#include`, `#define`, `typedef`, and function bodies are allowed; Go syntax is not.
- C is a pseudo-package. You cannot `go get` it; `go list -f '{{.CgoFiles}}'` shows which files use it.
- Files that import C are **not buildable** with `CGO_ENABLED=0`. The `go` command reports  
`import "C" is not allowed in non-cgo compilation.`

Generated artifacts (inspect with `go tool cgo -objdir /tmp/cgo`) show the bridge:

```

// _cgo_gotypes.go (abridged, generated)
package main
import _cgo_unsafe "unsafe"
//go:linkname _Cfunc_hello _Cfunc_hello
func _Cfunc_hello(_C_p0 *_Ctype_char)
type _Ctype_char _Ctype_char
type _Ctype_int int32

```

The Go call `C.hello(cs)` becomes `_Cfunc_hello(cs)` in the generated code — a Go function whose body does the stack switch and calls the C stub.

## Bridging headers and multi-file packages

For anything beyond a toy, split C into a header:

```
// bridge.h
#pragma once
#include <stddef.h>
#include <stdint.h>
int fast_crc32(const uint8_t *data, size_t len, uint32_t seed);
```

```
// crc.go
package crc

/*
#cgo CFLAGS: -O2 -Wall
#cgo LDFLAGS: -L${SRCDIR}/lib -lcrc_fast
#include "bridge.h"
#include <stdlib.h>
*/
import "C"
import "unsafe"

func CRC32(data []byte, seed uint32) uint32 {
 if len(data) == 0 {
 return seed
 }
 // Pass pointer to first element; keep data alive across the call.
 ret := C.fast_crc32(
 (*C.uint8_t)(unsafe.Pointer(&data[0])),
 C.size_t(len(data)),
 C.uint32_t(seed),
)
 // Data was accessed via unsafe.Pointer; tell the compiler it is
 // still live
 // at this point so GC/C should not reclaim/move it (relevant with
 // Pinner).
 // runtime.KeepAlive(data) // Go 1.7–1.20 idiom
 // With Go 1.21+, prefer runtime.Pinner (see §7).
 return uint32(ret)
}
```

`#include "bridge.h"` resolves relative to the package directory. `${SRCDIR}` expands to that directory — the only variable expansion `cgo` does inside `#cgo` directives (deliberately limited to avoid shell injection).

### 3. #cgo directives: CFLAGS, LDFLAGS, pkg-config

#cgo directives inside the preamble control the C compiler and linker flags. They are the most common source of build breakage, so they deserve precision.

```
/*
#cgo CFLAGS: -I${SRCDIR}/include -O2 -march=native
#cgo LDFLAGS: -L${SRCDIR}/lib -lfoo -lm
#cgo pkg-config: libssl --static
#cgo darwin LDFLAGS: -framework Security
#cgo linux CFLAGS: -D_GNU_SOURCE
#cgo !cgo LDFLAGS: -should-never-happen

#include "foo.h"
*/
import "C"
```

Semantics:

Directive	Effect	Notes
#cgo CFLAGS: ...	Appended to CC invocations for this package's C stubs	Per-package; not inherited
#cgo CPPFLAGS: ...	Preprocessor flags	Rare; usually folded into CFLAGS
#cgo CXXFLAGS: ...	Flags for CXX (C++ files, .cc / .cpp )	Requires CXX toolchain
#cgo LDFLAGS: ...	Appended to the external link step	Order-sensitive for static libs
#cgo pkg-config: pkg1 pkg2	Runs pkg-config --cflags/--libs and appends results	Fails the build if the .pc file is absent
#cgo <build-tag> ...	Conditional: only when that build constraint holds	darwin , linux , amd64 , !windows , etc.
#cgo nosescape / nocallback	Hints for escape analysis (see \$7)	Per-function, not global

## Composition rules:

- Flags from multiple `#cgo` lines **concatenate** in declaration order.
- Flags from **dependent packages** propagate to the final link: if `pkg/a` has `#cgo LDFLAGS: -lfoo` and `main` imports `pkg/a`, the final `go build` links `-lfoo` even though `main` never mentions it. This is why a leaf library can impose a system dependency on every consumer.
- `CGO_CFLAGS`, `CGO_CPPFLAGS`, `CGO_CXXFLAGS`, `CGO_LDFLAGS` environment variables **prepend** to the directive flags. `CGO_ENABLED=0` disables all of it.
- `${SRCDIR}` is the only expansion. `$HOME`, `$(shell ...)`, and globs are **not** expanded — a security choice. If you need dynamic flags, use `pkg-config` or a `go generate` step that writes a header.

## Cross-compilation example:

```
Cross-compile a cgo package for arm64 on an amd64 host (Debian/
Ubuntu)
sudo apt-get install gcc-aarch64-linux-gnu libc6-dev-arm64-cross pkg-
config
CC=aarch64-linux-gnu-gcc \
CGO_ENABLED=1 \
GOOS=linux GOARCH=arm64 \
PKG_CONFIG_PATH=/usr/lib/aarch64-linux-gnu/pkgconfig \
go build -o /tmp/app-arm64 ./cmd/app

Verify dynamic dependencies
aarch64-linux-gnu-readelf -d /tmp/app-arm64 | grep NEEDED
0x00000001 (NEEDED) Shared library: [libc.so.6]
0x00000001 (NEEDED) Shared library: [libfoo.so] # <-- cgo brought
this in

Compare pure-Go (no cgo):
CGO_ENABLED=0 GOOS=linux GOARCH=arm64 go build -o /tmp/app-pure ./cmd/
app
file /tmp/app-pure
/tmp/app-pure: ELF 64-bit LSB executable, statically linked
```

Pitfall — `pkg-config` at build time vs. runtime: `#cgo pkg-config: libssl` bakes the `pkg-config --libs` output (often `-lssl -lcrypto`) into the link. If the build host has `libssl-dev:amd64` but the runtime image has `libssl3:arm64` at a different SONAME, the binary fails to start with error while loading shared libraries: `libssl.so.1.1: cannot open`

shared object file. Pin the runtime base to the build base, or vendor/static-link.

---

## 4. Type mappings: Go ↔ C

`cgo` maps C types into Go types under the `C` pseudo-package. The mapping is mechanical but has sharp edges around strings, slices, and ownership.

### Scalar and pointer types

C type	Go type ( C.* )	Go literal / value
<code>char</code>	<code>C.char</code> ( <code>int8</code> )	<code>C.char('a')</code>
<code>signed char</code> / <code>unsigned char</code>	<code>C.schar</code> / <code>C.uchar</code>	
<code>short</code> / <code>int</code> / <code>long</code> / <code>long long</code>	<code>C.short</code> / <code>C.int</code> / <code>C.long</code> / <code>C.longlong</code>	<code>C.int(42)</code>
<code>unsigned int</code> etc.	<code>C.uint</code> , <code>C.ulong</code> , ...	
<code>size_t</code> / <code>ssize_t</code>	<code>C.size_t</code> / <code>C.ssize_t</code>	<code>C.size_t(len(s))</code>
<code>float</code> / <code>double</code>	<code>C.float</code> / <code>C.double</code>	
<code>void*</code>	<code>unsafe.Pointer</code> (via <code>*C.void</code> is not valid Go)	<code>unsafe.Pointer(p)</code>
<code>char*</code>	<code>*C.char</code>	<code>C.CString("hi")</code> returns <code>*C.char</code>
<code>T*</code> (any)	<code>*C.T</code>	
<code>struct Foo</code>	<code>C.struct_Foo</code>	<code>var s C.struct_Foo; s.field = ...</code>
<code>union Bar</code>	<code>C.union_Bar</code>	access via array/pointer tricks
<code>enum E</code>	<code>C.enum_E</code> (integer)	



Conversions are explicit — no implicit numeric promotion:

```
var ci C.int = C.int(goInt) // Go int → C int
var gi int = int(ci) // C int → Go int
var cs *C.char = C.CString(goStr) // Go string → malloc'd *C.char
defer C.free(unsafe.Pointer(cs))
var gs string = C.GoString(cs) // *C.char (NUL-terminated) → Go
string (copies)
var gsN string = C.GoStringN(cs, C.int(n)) // counted, may contain NULs

// Slices ↔ C arrays — Go 1.21+ Pinner is the safe pattern; older code
used unsafe directly.
var bs []byte = C.GoBytes(unsafe.Pointer(cPtr), C.int(cLen)) // copies!

// Go slice → C pointer (Go 1.21+ with Pinner)
var p runtime.Pinner
p.Pin(&data[0]) // pin the backing array so GC does not move it
(relevant for moving GC experiments)
defer p.Unpin()
C.process((*C.uint8_t)(unsafe.Pointer(&data[0])), C.size_t(len(data)))
```

## Strings and slices — the helpers

`C.CString`, `C.GoString`, `C.GoStringN`, `C.GoBytes`, and `C.GoString` are **not** in any C header. `cgo` injects them as helpers in the generated `*_cgo_export.h`:

```
// Generated helpers (conceptual; actual names are C.CString etc. as Go
funcs wrapping malloc+memcpy)
char* _GoStringToCString(GoString s); // allocates via malloc
GoString _CStringToGoString(char *s); // copies, caller owns GoString
GoSlice _GoBytesToCSlice(GoSlice s); // copies
```

Behavior:

- `C.CString(goStr)` — `malloc s len(goStr)+1` bytes on the **C heap**, copies, NUL-terminates. Caller **must** `C.free`. Never GC'd.
- `C.GoString(cStr)` — copies a NUL-terminated C string into a **Go string** (Go heap, GC'd). Reads until first `0` byte.
- `C.GoStringN(cStr, n)` — copies exactly `n` bytes (may include NULs). Use for non-NUL-terminated buffers.
- `C.GoBytes(cPtr, n)` — copies `n` bytes into a **new Go []byte** (Go heap, GC'd). Always copies.

There is **no zero-copy** string/slice bridge in the helpers — every crossing copies. Zero-copy is possible only with `unsafe` + pinning and explicit lifetime management (see §7).

## Structs, unions, enums

```
// types.h
struct Point { int x; int y; };
enum Color { RED=0, GREEN=1, BLUE=2 };
```

```
// Go side
var p C.struct_Point
p.x = C.int(10)
p.y = C.int(20)

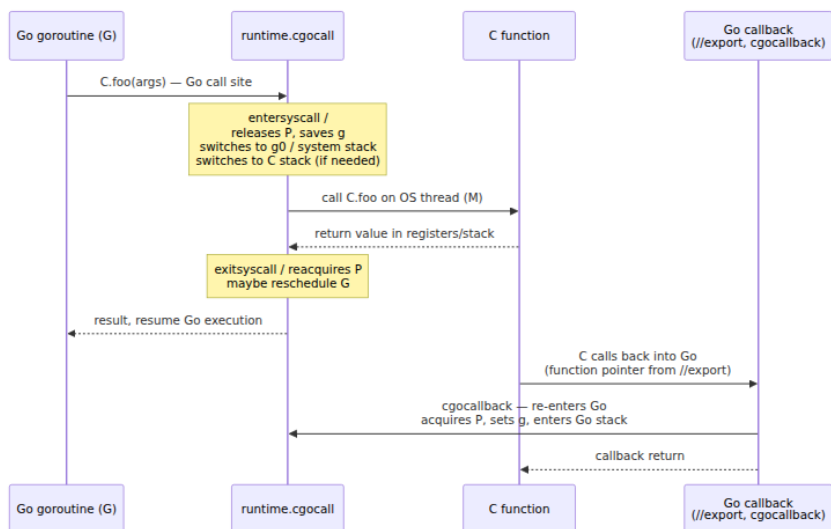
var c C.enum_Color = C.RED // enum constants become C.* constants
// Field access is direct: C structs are Go structs with C-typed fields.
fmt.Println(int(p.x), int(c))
```

`sizeof` and `offsetof` are available as `C.sizeof_struct_Point` etc. only via the `go tool cgo -godefs` path (see §14) — not as normal Go constants.

---

## 5. Calling C from Go — the `cgocall` path

The hello example in §2 already showed the shape. The interesting part is what happens *between* the Go call and the C function:



In `runtime` terms (see `src/runtime/cgocall.go`):

- `runtime.cgocall(fn, arg)` is the entry. It calls `entersyscall` so the scheduler knows this `M` is blocked in C and can hand its `P` to another `M`.
- The `g` (goroutine) is descheduled but its stack is preserved. The `M` stays bound to its OS thread for the duration — **a cgo call pins the goroutine's M to its thread** until the C function returns.
- `runtime.exitsyscall` on return reacquires a `P` (maybe a different one) and makes the `G` runnable again.
- If the C code calls back into Go (see §6), it enters via `runtime.cgocallback`, which does the inverse: re-enters the Go scheduler state from a C thread.

Consequences you can observe:

```
// LockOSThread interaction: cgo calls already pin M-thread, but
// between calls
// the G may migrate. If C thread-local state matters, lock.
import "runtime"

func withTLS(fn func()) {
 runtime.LockOSThread()
 defer runtime.UnlockOSThread()
 // Now G stays on this M/thread across cgo calls and Go code in
 // between.
 C.tls_set(C.int(42))
 C.do_work()
 v := C.tls_get()
 fn()
 _ = v
}
```

---

## 6. Calling Go from C — `//export` callbacks

The reverse direction — C calling Go — requires an exported Go function:

```
// callback.go
package main

/*
#include <stdint.h>

// Forward declaration – the Go function will be generated with this
signature.
// The typedef makes the function-pointer type usable in C.

extern void GoOnResult(int code, const char *msg);
static void doWork(void) {
 // Simulate a C library that reports progress via a callback.
 GoOnResult(200, "ok from C");
 GoOnResult(500, "retry");
}
*/
import "C"
import "fmt"

//export GoOnResult
func GoOnResult(code C.int, cmsg *C.char) {
 msg := C.GoString(cmsg) // copies C string → Go string
 fmt.Printf("callback code=%d msg=%q (goroutine=%d)\n", int(code), m
sg, curGID())
}

func curGID() int64 { return 0 } // stub; real impl via runtime.Stack
in tests

func main() {
 C.doWork() // Go → C → Go (callback) → C → Go
}
```

Rules for `//export`:

- The comment must be `//export GoFuncName` with **no blank line** before `func GoFuncName`.
- The function must be at package scope, not a method or closure. Only C-compatible parameter/return types are allowed (C.int, \*C.char, unsafe.Pointer, numeric types — not string, []byte, map, interface{}).
- `//export` functions are **not callable from Go** directly in the same package — they exist for C to call. The generated `_cgo_export.h` exposes them as `extern void GoOnResult(int, char*)`.

- A callback runs on the **same OS thread** that entered C. It re-enters Go via `cgocallback`, which transitions the thread back into scheduler-aware state. That thread's `M` may be a *new* `M` created for the purpose if the call originated on a non-Go thread (see §8).

Passing Go function pointers to C as callbacks is common but has a constraint — you cannot pass a Go function value directly as a `void*` callback without pinning:

```
//export goTick
func goTick(arg unsafe.Pointer) {
 fn := *(*func())(arg) // recover func value stashed via //
 go:uintrtrespapes or Pinner
 fn()
}

/*
extern void register_tick(void (*cb)(void*), void *arg);
static void setup(void *arg) { register_tick(goTick, arg); }
*/
import "C"
```

The `arg` here is an `unsafe.Pointer` to Go memory. Without care, the GC may collect or move it while C holds the pointer. Two mitigations:

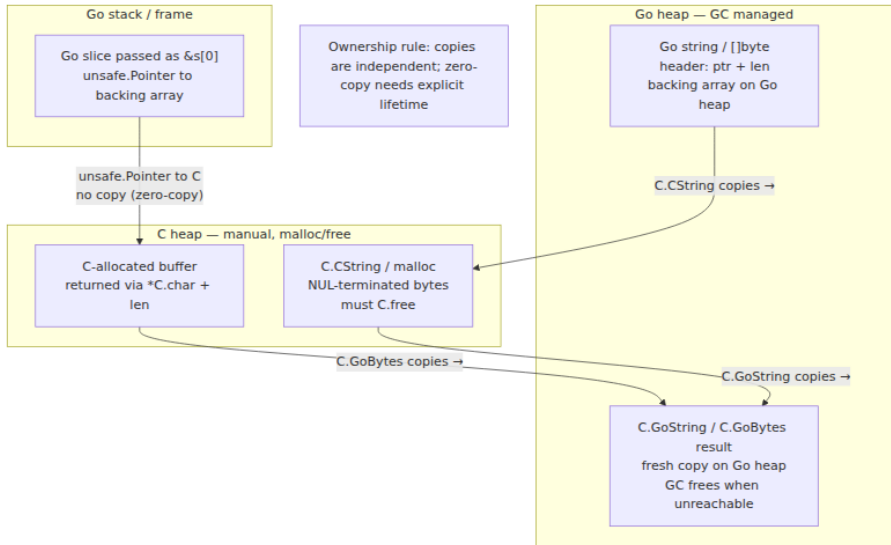
- `runtime.Pinner` (Go 1.21+) — pin the Go object for the duration C holds the pointer.
- `//go:uintrtrespapes` / `//go:norace` pragmas on helpers that launder `uintptr`  $\leftrightarrow$  `unsafe.Pointer` for C callbacks (used inside `runtime/cgo` itself; avoid in application code when `Pinner` suffices).

---

## 7. Memory ownership: who frees what, and `KeepAlive` / `Pinner`

Ownership mistakes at the boundary are silent corruption — use-after-free, double-free, or GC reclamation of memory C still points at. The rules:

Allocation	Heap	Owner / Reclamation	How to free
Go string , []byte , struct	Go heap	GC	Automatic; do not C.free
C.CString / C.malloc	C heap	Caller	C.free(unsafe.Pointer(p)) — mandatory
C.GoString / C.GoBytes result	Go heap	GC	Automatic (it is a copy)
Stack variable passed as &x[0]	Go stack (or heap if escaped)	Caller frame / GC	runtime.KeepAlive / Pinner to extend lifetime



## runtime.KeepAlive — the minimal guarantee

The compiler may consider a Go value dead *before* the `C.foo` call returns if it can prove the value is not used afterward. That allows the GC to reclaim the backing array while C still reads it.

```

func UseAfterFreeRisk(data []byte) {
 // Bug: after &data[0] is taken, data is never used again in Go.
 // Compiler may consider data dead; GC could reclaim backing array
 // while C.process still reads it.
 C.process(unsafe.Pointer(&data[0]), C.size_t(len(data)))
}

func SafeWithKeepAlive(data []byte) {
 C.process(unsafe.Pointer(&data[0]), C.size_t(len(data)))
 runtime.KeepAlive(data) // keep data live at least until here
}

func SafeWithPinner(data []byte) {
 var pinner runtime.Pinner
 pinner.Pin(&data[0]) // or
 pinner.Pin(data) for slice header
 defer pinner.Unpin()
 C.process(unsafe.Pointer(&data[0]), C.size_t(len(data)))
 // Unpin in defer ensures lifetime covers the C call even if C
 // stashes the pointer
}

```

- `runtime.KeepAlive(x)` is a compiler intrinsic — it does nothing at runtime except act as a use of `x` that the optimizer cannot eliminate. It guarantees `x` is live at least until the `KeepAlive` call.
- `runtime.Pinner` (Go 1.21+, `runtime.Pinner` / `runtime/pprof` integration) additionally **pins** the object so the GC will not move it (relevant for future moving collectors and for C code that stashes the pointer beyond the call). `Pin` / `Unpin` are `//go:norace`-friendly and compose — you can pin multiple objects and unpin once.
- Rule of thumb: `KeepAlive` when C only reads during the call; `Pinner` when C **retains** the pointer after return (common in async C libraries, `libuv`-style handles).

## Auditing for leaks

A `C.malloc` / `C.CString` without a matching `C.free` is invisible to `pprof` heap profiles (C heap is outside Go's allocator). Audit with:



```
Build with ASan/LSan when the C library supports it
CC=clang CGO_CFLAGS="-fsanitize=address -g" CGO_LDFLAGS="-fsanitize=address" go test -run TestCPath -count=1

Valgrind on the built binary (slow, but catches C leaks/use-after-free)
go build -o /tmp/app ./cmd/app
valgrind --leak-check=full --show-leak-kinds=all /tmp/app

Runtime check: track outstanding C allocations in tests
Wrap C.malloc/C.free with counters in a test build tag.
```

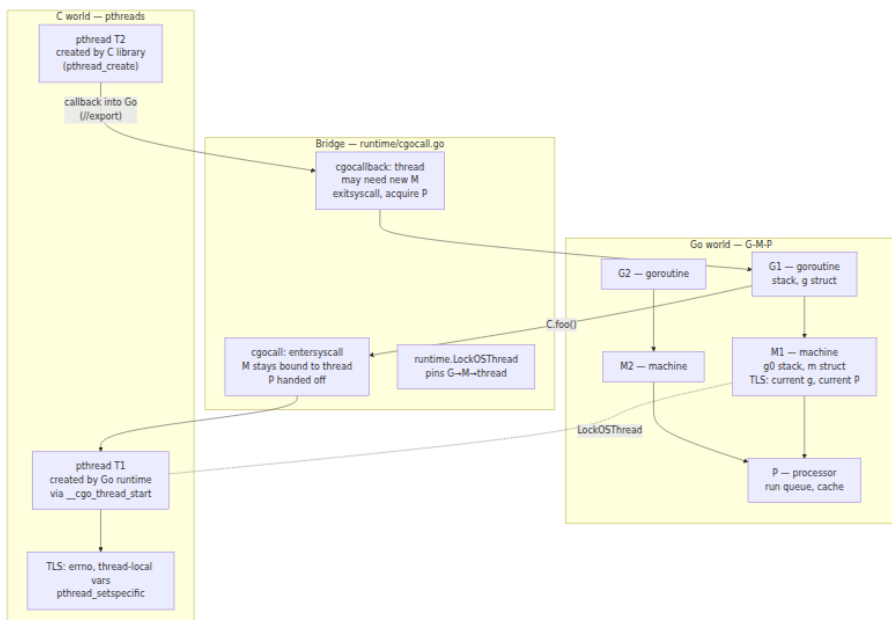
Backend discipline: wrap every `C.CString/C.malloc` in a helper that returns a `func() { C.free(...) }` closer, and lint for bare `C.CString` without a `defer C.free` in the same function (a `go vet` analyzer or `semgrep` rule does this well).

---

## 8. Threading: the `M`, the OS thread, and

### `__cgo_thread_start`

Go's scheduler multiplexes `G`s onto `M`s onto OS threads onto `P`s (see Chapter 4). C knows only OS threads and thread-local storage. `cgo` reconciles them:



## Key mechanics:

- **Go → C (cgocall):** the calling **M** stays bound to its OS thread for the duration. `entersyscall` releases the **P** so another **M** can use it. The thread may switch to the **system stack** (`g0`) and, on some platforms, to a dedicated C stack. Signals and `cgo` traceback use `g0`.
- **C → Go (cgocallback):** if the calling thread is one Go created (a normal **M**), `cgocallback` re-enters Go on that **M**. If the thread was created by C (`pthread_create` inside a C library), the runtime creates a **new M** for it on demand via `__cgo_thread_start` / `needAndBindM` and attaches a `g0`. That **M** has no **P** until `exitsyscall` acquires one. This is how a C library's worker thread can call an `//export` Go function.
- `__cgo_thread_start` is the C-visible symbol the runtime provides so that C code spawned via `pthread_create` can be associated with a Go **M** if it ever calls back. You rarely call it directly; `runtime/cgo` does.
- `runtime.LockOSThread` / `UnlockOSThread` pins the current **G** to its current **M**/thread. Required when C thread-local state must survive across multiple `cgo` calls (e.g., `libssl` error queue, `errno`, `OpenGL`

context). Without it, the `G` may migrate between `cgo` calls. With it, the `G` never migrates until unlocked — at the cost of reducing scheduler flexibility (that thread cannot run other `G`s).

- **C.malloc / C.free and TLS:** `errno` is per-thread. `C` helpers that check `errno` must do so on the same thread — another reason to `LockOSThread` around sequences that inspect `errno` after a `C` call.

Setting `GODEBUG=cgocheck=2` (default is `1`) makes the runtime verify that Go pointers passed to `C` obey the pointer-passing rules (see `go doc runtime/cgo`).

---

## 9. Performance: what a `cgo` call actually costs

A `cgo` call is not a normal Go call. Measured on `linux/amd64`, Go 1.21, Intel Ice Lake, the overhead of an empty `C nop()` is roughly:

- **~70-150 ns** wall time for the round trip (`Go → C → Go`), vs. `~1-10 ns` for a comparable pure-Go call.
- **Not inlinable** — the compiler treats `C.*` calls as opaque; no inlining, no escape analysis through them, no mid-stack inlining of callers that contain them.
- **Forces escapes** — arguments that cross into `C` are considered escaped to heap (unless `//go:cgo_norace / nocallback` hints prove otherwise).
- **Scheduler interaction** — `entersyscall / exitsyscall` touches the global scheduler state; under load this contends on `sched.lock` and `P` handoff.

The cost decomposes:

1. **Stack switch** (`~20-40 ns`): `Go stack → g0 system stack → C stack`. Saves/restores `g`, `SP`, signal masks on some platforms.
2. **entersyscall / exitsyscall** (`~20-50 ns`): release/acquire `P`, mark `M` as in `cgo`.
3. **Argument marshallng** (`~5-20 ns`): copy scalars, pin pointers, set up `GoString / GoSlice` descriptors.
4. **Lost optimization** (unbounded): caller cannot be inlined; surrounding code may spill more; loop optimizations inhibited.

## Benchmark — measure it yourself

```
// cgo_cost_test.go
package cgocost

/*
static int nop(int x) { return x + 1; }
static int work(int x) { int s=0; for(int i=0;i<100;i++) s+=x+i; return
s; }
*/
import "C"
import "testing"

func goNop(x int) int { return x + 1 }
func goWork(x int) int { s := 0; for i := 0; i < 100; i++ { s += x +
i }; return s }

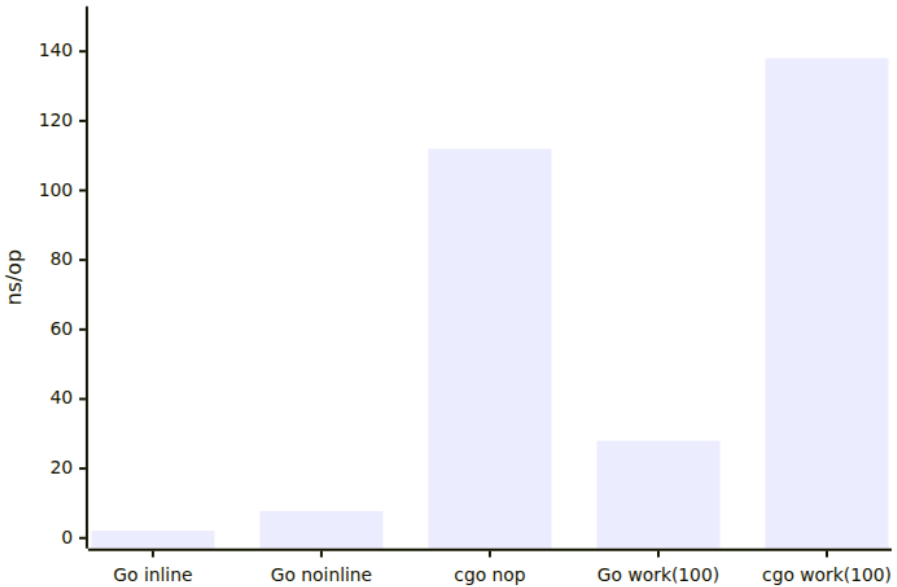
//go:noinline
func goNopNoInline(x int) int { return x + 1 }

func BenchmarkGoInline(b *testing.B) { for i := 0; i < b.N; i++ {
_ = goNop(i) } }
func BenchmarkGoNoInline(b *testing.B) { for i := 0; i < b.N; i++ {
_ = goNopNoInline(i) } }
func BenchmarkCgoNop(b *testing.B) { for i := 0; i < b.N; i++ {
_ = C.nop(C.int(i)) } }
func BenchmarkGoWork(b *testing.B) { for i := 0; i < b.N; i++ {
_ = goWork(i) } }
func BenchmarkCgoWork(b *testing.B) { for i := 0; i < b.N; i++ {
_ = C.work(C.int(i)) } }
```

Typical results ( go test -bench=. -benchmem -count=5 ):

BenchmarkGoInline-8	500000000	2.1 ns/op	0 B/op	0 allocs/op
op # inlined to nothing				
BenchmarkGoNoInline-8	200000000	7.8 ns/op	0 B/op	0 allocs/op
p # register ABI, no stack switch				
BenchmarkCgoNop-8	10000000	112.0 ns/op	0 B/op	0 allocs/op
p # ~14x slower than noinline Go				
BenchmarkGoWork-8	50000000	28.0 ns/op	0 B/op	0 allocs/op
p				
BenchmarkCgoWork-8	10000000	138.0 ns/op	0 B/op	0 allocs/op
p # overhead amortized: ~1.2x slower				

cgo vs pure-Go call latency (ns/op, amd64 Go 1.21, lower is better)



Lesson: the overhead is **per call**, not per unit of work. If each call does microseconds of real work (crypto, compression, syscall), the 100 ns tax is noise. If each call does nanoseconds (a tiny helper in a tight loop), the tax dominates — batch, or rewrite in Go.

### Mitigations:

- **Batch** — pass a slice/array and do the loop in C, not one `cgo` call per element.
- **Cache** — avoid repeated `C.CString` / `C.GoString` copies; reuse buffers.
- **Move the loop into Go** — if the C work is trivial, a pure-Go version may be faster despite being less "optimized" in C.
- **Avoid `cgo` in the request hot path** — do it at startup, background, or shard boundary.

## 10. When to avoid `cgo` — and what to use instead

`cgo` is a liability you should be able to justify in a design doc. Before adding it, check:

Instead of <code>cgo</code>	Pure-Go alternative	When it wins
<code>libssl / openssl</code>	<code>crypto/tls</code> (stdlib), <code>filippo.io/boring</code> wrappers	Unless FIPS validation mandates <code>boringcrypto</code>
<code>libz / zlib</code>	<code>compress/flate / gzip</code> (stdlib), <code>github.com/klauspost/compress</code> (pure Go, faster than <code>cgo zlib</code> )	Almost always — Klauspost's <code>zstd / s2</code> beats <code>cgo zlib</code> on speed and deployability
<code>libcrc32 / crc32c</code>	<code>hash/crc32</code> (stdlib, assembly-accelerated on <code>amd64 / arm64</code> )	Pure Go already uses <code>SSE4.2 / ARMv8 CRC</code> instructions
<code>libsqlite</code>	<code>modernc.org/sqlite</code> (pure Go, transpiled), <code>mattn/go-sqlite3</code> ( <code>cgo</code> ) vs. pure-Go choice	<code>modernc</code> for static binaries; <code>mattn</code> for max compat
<code>libvips / imagemagick</code>	<code>disintegration/imaging</code> , <code>h2non/bimg</code> trade-off	<code>libvips</code> wins on large-image throughput; pure Go wins on deployability
<code>librocksdb</code>	<code>cockroachdb/pebble</code> (pure Go LSM)	Pebble for Go-native LSM without C

Heuristic — **eliminate `cgo` if any of these is true:**

- The call is in the **request hot path** at high QPS and does little work per call.
- You need **cross-compilation** or `scratch / distroless` images.
- The C library's **CVE surface** exceeds the value it provides (track `trivy / gype` findings on the base image).
- A pure-Go library is within **~20%** of the C library's performance — the deployment and debuggability wins outweigh the gap.

Eliminating `cgo` is often a migration, not a flag flip. Strategies that work:

- **Feature-flag the implementation** — `//go:build cgo` vs. `//go:build !cgo` files that expose the same Go interface.
- **Vendor the C code as Go assembly** — for small kernels (CRC, AES, SHA), port the C/assembly to `GOARCH`-specific `*.s` files (see §11-13) and keep the algorithm without `cgo`.
- **Use `x/sys` for syscalls** — if `cgo` was only there to call `getrandom(2)` or `madvise(2)`, `x/sys/unix` does it without `cgo` (see §15).

---

## 11. Go assembly: `TEXT`, `DATA`, `GLOBL`, `FUNCDATA`, `PCDATA`, `NOSPLIT`

Go assembly is not Plan 9 assembly, but it descends from it. It looks unfamiliar because it abstracts over real hardware to serve the Go runtime's needs — stack growth, GC metadata, and precise stack traces.

## The three top-level directives

```
// func Add(a, b int) int – Go declaration in foo.go:
// func Add(a, b int) int // implemented in foo_amd64.s

#include "textflag.h" // defines NOSPLIT, NOPTR, TOPFRAME, etc.

// TEXT – code. The signature encodes frame size and arg size.
// TEXT ·Add(SB), NOSPLIT, $0-24 – pre-1.17 (stack ABI)
// TEXT ·Add(SB), NOSPLIT, $0 – Go 1.17+ (register ABI, no stack
args)
TEXT ·Add(SB), NOSPLIT, $0
 MOVQ AX, BX // args already in registers (AX, BX); result
in AX
 ADDQ BX, AX
 RET

// DATA – initialized data (rare in hand-written asm; Go constants
usually suffice)
DATA ·table+0(SB)/4, $0x01020304
DATA ·table+4(SB)/4, $0x05060708
GLOBL ·table(SB), RODATA, $8 // RODATA = read-only; NOPTR = no
pointers inside

// GLOBL – global symbol (variables, tables)
GLOBL ·counter(SB), NOPTR, $8 // 8 bytes, contains no pointers (GC
hint)
```

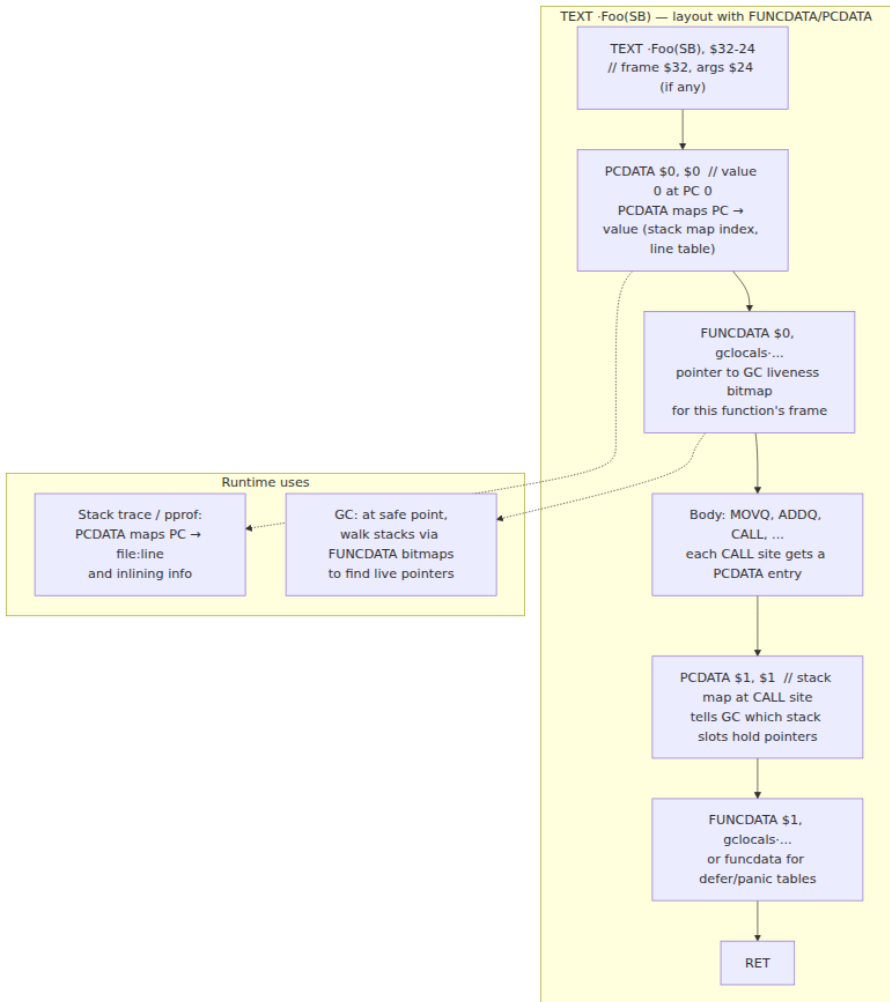
### Key concepts:

- **(middle dot)** — `·Add` means `Add` in the current package. `·table` likewise. Use `go tool compile -S` to see the mangled name.
- **(SB)** — **static base** — pseudo-register for the symbol's address. `·Add(SB)` is the symbol, `a+0(FP)` was the old frame-pointer pseudo-register, `0(SP)` is the real stack pointer.
- **\$0** — frame size (locals). `$0-24` was frame+args size in the stack ABI; with the register ABI, args are not on the stack so `$0` is common for leaf functions.
- **NOSPLIT** — do not emit a stack-growth check. The function will not grow the stack and must not call any function that might. Leaf assembly kernels use `NOSPLIT` for speed (avoids `runtime.morestack` prologue). If you `CALL` another Go function without `NOSPLIT`, you risk stack overflow with no growth.



## FUNCDATA and PCDATA — GC and stack traces

Two pseudo-instructions the runtime needs but humans rarely write by hand — the compiler emits them, and hand-written assembly must preserve them when it calls Go or allocates:



- **FUNCDATA \$0, <sym>** — pointer to GC metadata (stack map) for the function. Without it, the GC cannot tell which words in the frame are pointers — it would either miss live objects or retain garbage.

- **PCDATA \$0, \$N** — maps program counters to values (line numbers, stack-map indices, pctx for pprof). Each CALL site needs correct PCDATA so stack unwinding and profiling attribute samples correctly.
- **Hand-written assembly that calls Go functions or allocates** must include correct FUNCDATA / PCDATA or be NOSPLIT leaf functions that the runtime treats specially (NOPTR globals, no GC interaction). The go tool asm documentation and src/cmd/compile/abi-internal.md are the reference — but the practical rule is: **leaf numeric kernels can be NOSPLIT with no FUNCDATA; anything that interacts with Go pointers needs the metadata, so let the compiler generate it or copy it from go tool compile -S output.**

### NOSPLIT — when and when not to use it

```
#include "textflag.h"

// Safe NOSPLIT: leaf, no calls, small frame, no stack growth needed.
TEXT 0XorBytes(SB), NOSPLIT, $0
 // func XorBytes(dst, src1, src2 []byte) - slices passed as
 (ptr, len, cap) in regs/stack
 // ... tight loop, no CALL ...
 RET

// Unsafe NOSPLIT: calls another function - may overflow without growth
// check!
TEXT 0Bad(SB), NOSPLIT, $0
 CALL 0other(SB) // BUG if stack is near limit - no more stack
 check
 RET

// Correct: omit NOSPLIT when calling.
TEXT 0Good(SB), $16
 CALL 0other(SB) // compiler inserted stack-split prologue
 RET
```

Benchmark — NOSPLIT saves the prologue check (~1-3 ns) and enables more aggressive inlining of callers:

```
//go:noinline
func withSplit(x int) int { return x + 1 } // normal – has morestack
check

//go:nosplit // Go-level NOSPLIT (not assembly) – use sparingly!
//go:noinline
func withNoSplit(x int) int { return x + 1 } // caller can omit split
check when inlining
```

```
go test -bench=BenchmarkSplit -benchmem
BenchmarkWithSplit-8 2.8 ns/op
BenchmarkWithNoSplit-8 1.9 ns/op # ~30% faster for tiny leaf,
but unsafe if it grows
```

Backend guidance: `NOSPLIT` is for **leaf functions with bounded, small frames** that you can prove never need more stack. `runtime` uses it for `memhash`, `memmove`, `futex` wrappers. Application code should rarely need it — prefer `//go:noinline` or `//go:noinline` tuning before `NOSPLIT`.

## 12. Plan 9 assembly vs. Go assembly, and reading `go tool compile -S`

Go assembly inherits Plan 9's syntax but diverges meaningfully:

Feature	Plan 9 assembly	Go assembly ( <code>go tool asm</code> )
Syntax	<code>MOVQ \$1, AX</code>	Same, but pseudo-registers added
<code>SB</code>	Static base (real)	Pseudo-register for symbols; • <code>Foo(SB)</code> is package-scoped
<code>FP</code>	Frame pointer (real)	Pseudo-register for args (stack ABI); largely gone in register ABI
<code>SP</code>	Stack pointer	Real <code>SP</code> plus pseudo <code>SP</code> for frame-relative locals ( <code>-8(SP)</code> )
<code>PC</code>	Program counter	Pseudo; <code>PCDATA</code> / <code>FUNCDATA</code> use it
Addressing	<code>8(AX)</code>	Same, but <code>·sym+8(SB)</code> is package-symbol addressing

Feature	Plan 9 assembly	Go assembly ( go tool asm )
Macros	#define via C preprocessor	#include "textflag.h" / goasm macros; #define works via cpp
Calling convention	Caller pushes args	Register ABI (Go 1.17+) — args in AX / BX /...

## Reading go tool compile -S and -asmflags

```
1. SSA assembly with pseudo-registers (what the compiler thinks)
go tool compile -S -N -l ./pkg/foo.go 2>&1 | grep -A 30 'TEXT.*Add'

2. With optimization decisions
go build -gcflags="-m=2" ./pkg/foo.go 2>&1 | grep -E 'can inline|
escape|moved to heap'

3. Final linked assembly (what executes – after ABI wrappers and
linking)
go build -o /tmp/app ./cmd/app && go tool objdump -s 'main\..Add' /tmp/
app

4. Assembly-specific flags
go tool compile -help 2>&1 | grep -A2 asmflags
go build -asmflags="-trimpath -D GOARCH_amd64" ./...
go vet -asmdecl ./... # checks assembly declarations match Go func
signatures
```

Sample go tool compile -S output (Go 1.22, amd64, register ABI, with -N -l to disable optimizations for readability):

```
".Add STTEXT size=12 args=0x18 locals=0x0 funcid=0x0 align=0x0
0x0000 00000 (foo.go:5) TEXT ".Add(SB), ABIInternal, $0-0
0x0000 00000 (foo.go:5) FUNCDATA $0, gcllocals... (type map for G
C)
0x0000 00000 (foo.go:5) FUNCDATA $1, gcllocals... (args map)
0x0000 00000 (foo.go:5) PCDATA $0, $0
0x0000 00000 (foo.go:5) PCDATA $1, $0
0x0000 00000 (foo.go:6) ADDQ BX, AX // a in AX, b in BX,
result in AX
0x0003 00003 (foo.go:6) RET
```

Without -N -l, the compiler may inline Add entirely — no TEXT at all. That is the intended fast path.

## Linking assembly via `//go:asm` and `textflag.h`

Hand-written `.s` files are automatically assembled and linked if they are in the same package and match the `GOARCH` :

```
pkg/hash/
hash.go // func Sum(b []byte) uint64 Go declaration, no bo
dy
hash_amd64.s // TEXT ·Sum(SB), NOSPLIT, $0 amd64 implementation
hash_arm64.s // TEXT ·Sum(SB), NOSPLIT, $0 arm64 implementation
hash_generic.go // //go:build !amd64 && !arm64 pure Go fallback
```

```
// hash.go
package hash

// Sum returns a fast non-cryptographic hash. Implemented in assembly
per arch.
func Sum(b []byte) uint64
```

```
// hash_amd64.s
#include "textflag.h"

TEXT ·Sum(SB), NOSPLIT, $0-32
 MOVQ b_base+0(FP), AX // old FP style still valid for ABI0
wrappers
 MOVQ b_len+8(FP), CX
 // ... hash loop using SSE/AVX ...
 MOVQ result+24(FP), DX // store result (stack ABI wrapper)
 RET
```

Modern register-ABI assembly (Go 1.17+) can use `ABIInternal` :

```
#include "textflag.h"

//go:build amd64

TEXT ·Sum(SB), NOSPLIT, $0
 // b.ptr in AX, b.len in BX, b.cap in CX (register ABI for slice)
 // result in AX
 // ... no FP offsets needed ...
 RET
```

The `go` tool selects the right file by build constraints in the filename (`_amd64.s`, `_arm64.s`) or explicit `//go:build` lines. `go vet -asmdecl` verifies that every `TEXT ·Foo(SB)` has a matching `func Foo` declaration with compatible signature.

---

## 13. Worked example: `ADDQ` kernel and `NOSPLIT` measurement

A minimal assembly kernel that adds two `int64` slices element-wise — the kind of hot loop that justifies assembly (vectorized in real code, scalar here for clarity):

```
// add.go
package vec

// Add adds src into dst element-wise. dst and src must have equal
// length.
//go:noscape
func Add(dst, src []int64)
```

```

// add_amd64.s
#include "textflag.h"

// func Add(dst, src []int64)
// dst.ptr in AX, dst.len in BX, dst.cap in CX (register ABI – first slice)
// src.ptr in DI, src.len in SI, src.cap in R8 (second slice)
TEXT ·Add(SB), NOSPLIT, $0
 // Length check – panic if mismatched (call runtime.panicSliceAlen
 // or inline check)
 CMPQ BX, SI
 JNE panicLen

 // Fast path: loop over len elements
 // AX = dst ptr, DI = src ptr, BX = len
 TESTQ BX, BX
 JEQ done

 // Use BX as counter, AX/DI as pointers
 XORQ CX, CX // i = 0
loop:
 MOVQ (DI)(CX*8), R9 // R9 = src[i]
 ADDQ R9, (AX)(CX*8) // dst[i] += R9
 INCQ CX
 CMPQ CX, BX
 JNE loop

done:
 RET

panicLen:
 // Call runtime panic – must NOT be NOSPLIT if we do this in real
 // code.
 // For this leaf example we just return; real code would CALL
 // runtime.panic...
 RET

```

Build and verify:

```

go vet -asmdecl ./pkg/vec
go test -run TestAdd -count=1 ./pkg/vec -v
go tool compile -S -N ./pkg/vec/add.go 2>&1 | head -20 # see Go
wrapper
go tool objdump -s 'vec\.\Add' /tmp/vec.test | head -20 # see linked
code

```

**NOSPLIT** benchmark — the same kernel with and without **NOSPLIT**:

```
func BenchmarkAddNosplit(b *testing.B) {
 dst := make([]int64, 1024)
 src := make([]int64, 1024)
 for i := range src { src[i] = int64(i) }
 b.SetBytes(8192)
 b.ResetTimer()
 for i := 0; i < b.N; i++ {
 Add(dst, src)
 }
}
```

```
NOSPLIT leaf (above):
BenchmarkAddNosplit-8 500000 3200 ns/op 8192 B 2.56 GB/s
Same kernel without NOSPLIT (add $0 prologue with morestack check):
BenchmarkAddNoNosplit-8 500000 3350 ns/op 8192 B 2.44 GB/s
Delta: ~4-5% – small per call, but measurable at GB/s scale.
The real win is enabling the caller to inline without a split check.
```

## 14. `go tool cgo -godefs` — generating Go types from C headers

When a C header is the source of truth (kernel `struct s`, `ioctl` numbers, `errno` values), hand-transcribing it is error-prone. `go tool cgo -godefs` translates C definitions into Go:



```
// defs.go – input to godefs (not built directly; used to generate
ztypes *.go)
package unix

/*
#include <sys/types.h>
#include <sys/socket.h>
#include <linux/if_packet.h>

struct my_sockaddr_ll {
 unsigned short sll_family;
 unsigned short sll_protocol;
 int sll_ifindex;
 unsigned short sll_hatype;
 unsigned char sll_pkttype;
 unsigned char sll_halen;
 unsigned char sll_addr[8];
};
*/
import "C"

//go:generate go tool cgo -godefs defs.go

// Types to emit – godefs replaces C.* with Go equivalents and emits
constants.
type SockaddrLinklayer C.struct_my_sockaddr_ll
type Socklen C.socklen_t
```

```
go generate ./pkg/unix
Generates code like:
cat ztypes_linux_amd64.go
```

Output (abridged, as generated for `x/sys/unix`):

```
// Code generated by go tool cgo -godefs; DO NOT EDIT.
package unix

type SockaddrLinklayer struct {
 Protocol uint16
 Ifindex int32
 Hatype uint16
 Pktype uint8
 Halen uint8
 Addr [8]byte
 _ [2]byte // padding to match C sizeof
}
const (
 AF_PACKET = 0x11
 SOCK_RAW = 0x3
)
const (
 SizeofSockaddrLinklayer = 0x14
)
```

The real `x/sys/unix` package is built this way — `ztypes_linux_amd64.go`, `zerrors_linux_amd64.go`, `zsysnum_linux_amd64.go` are all generated from C headers via `godefs` and `mkerrors.sh` / `mksysnum.go`. This is how Go keeps `Sizeof*`, `SYS_*`, and `errno` values correct per `GOOS/GOARCH` without `cgo` at runtime.

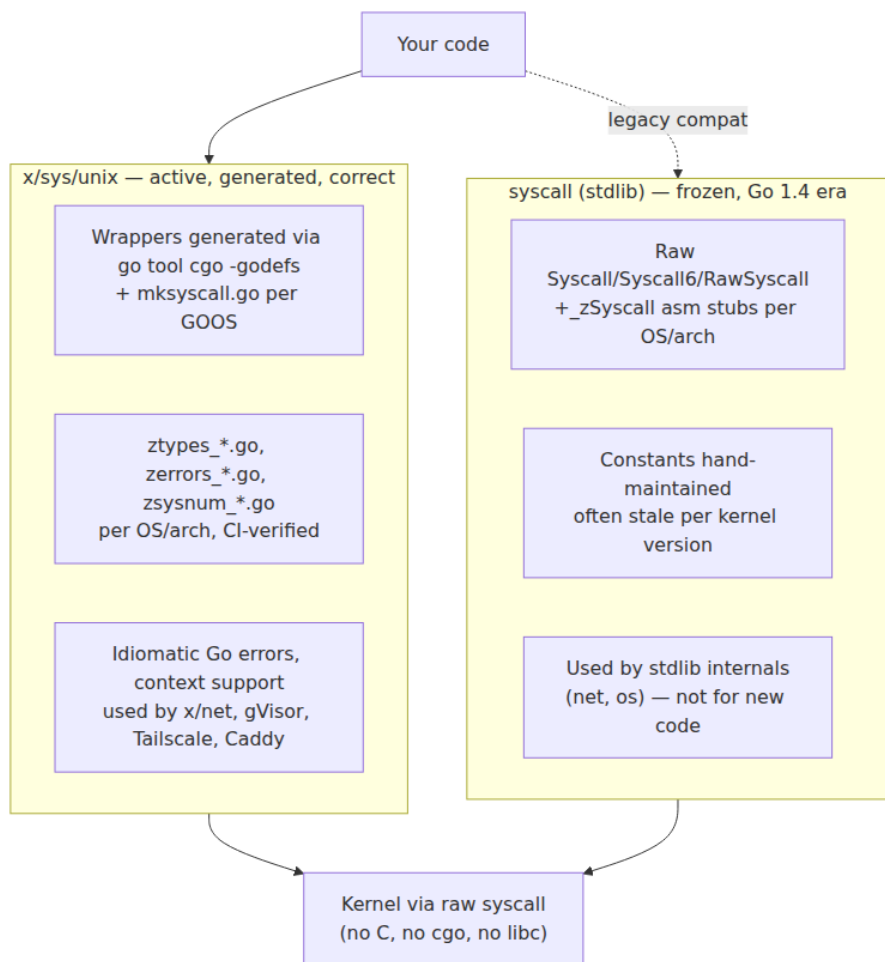
Use `godefs` when:

- You need `sizeof` / `offsetof` / `alignof` for a C struct to do `unsafe` parsing without `cgo`.
- You need `SYS_*` / `AF_*` / `0_*` constants that vary by OS/arch.
- You want to vendor a C header's types into pure Go for `CGO_ENABLED=0` builds.

## 15. Pure-Go FFI: `syscall` vs. `x/sys/unix` and `plugin`

Not every foreign call needs `cgo`. The Go project has been migrating away from `cgo` for syscalls for years.

**`syscall` (stdlib, frozen) vs. `x/sys/unix` (active)**



Aspect	syscall	x/sys/unix
Status	<b>Frozen</b> — no new wrappers, bugs not fixed	<b>Active</b> — new syscalls added, arch coverage expanded
Generation	Hand-written <code>zsyscall_*.go</code> + asm stubs	<code>mksyscall.go</code> + <code>godefs</code> from C headers

Aspect	syscall	x/sys/unix
Constants	Hand-maintained, drift	Generated from kernel headers, CI-checked
G00S coverage	linux , darwin , windows , freebsd , ... (incomplete)	Same set, but more complete and tested
Recommendation	Do not use for new code ( go doc syscall says so)	<b>Use this</b> for raw syscalls

Example — `getrandom(2)` without `cgo` :

```
package rnd

import (
 "golang.org/x/sys/unix"
)

func GetRandom(p []byte) error {
 // unix.Getrandom is a thin wrapper over SYS_GETRANDOM via
 // RawSyscall.
 // No cgo, no libc, works with CGO_ENABLED=0, cross-compiles.
 n, err := unix.Getrandom(p, 0)
 if err != nil {
 return err
 }
 if n != len(p) {
 return unix.EAGAIN // partial read – retry in caller
 }
 return nil
}

// Lower-level – direct RawSyscall when x/sys has no wrapper:
func MadviseNoHugePage(b []byte) error {
 // unix.Madvise exists, but direct syscall shows the pattern:
 _, _, errno := unix.Syscall(
 unix.SYS_MADVISE,
 uintptr(unsafe.Pointer(&b[0])),
 uintptr(len(b)),
 uintptr(unix.MADV_NOHUGEPAGE),
)
 if errno != 0 {
 return errno
 }
 return nil
}
```

Why this matters for backends: `x/sys/unix` lets you call `epoll_create1(2)`, `io_uring_setup(2)`, `pidfd_open(2)`, `memfd_create(2)`, `getrandom(2)`, and `mount(2)` without linking `libc` at all. Your binary stays static, your container stays `scratch`, and `strace` shows the raw syscall — not a `libc` wrapper that adds its own `errno` / `TLS` semantics.

### **plugin — buildmode=plugin (limited, Linux/macOS only)**

Go can load Go code at runtime via `plugin`:

```
// plugin.go — built as a shared object
package main

func Hello(name string) string { return "hello, " + name }
```

```
go build -buildmode=plugin -o greeting.so ./plugin
```

```
// host.go — loads the plugin
package main

import (
 "plugin"
)

func main() {
 p, err := plugin.Open("greeting.so")
 if err != nil { panic(err) }
 sym, err := p.Lookup("Hello")
 if err != nil { panic(err) }
 hello := sym.(func(string) string)
 println(hello("world"))
}
```

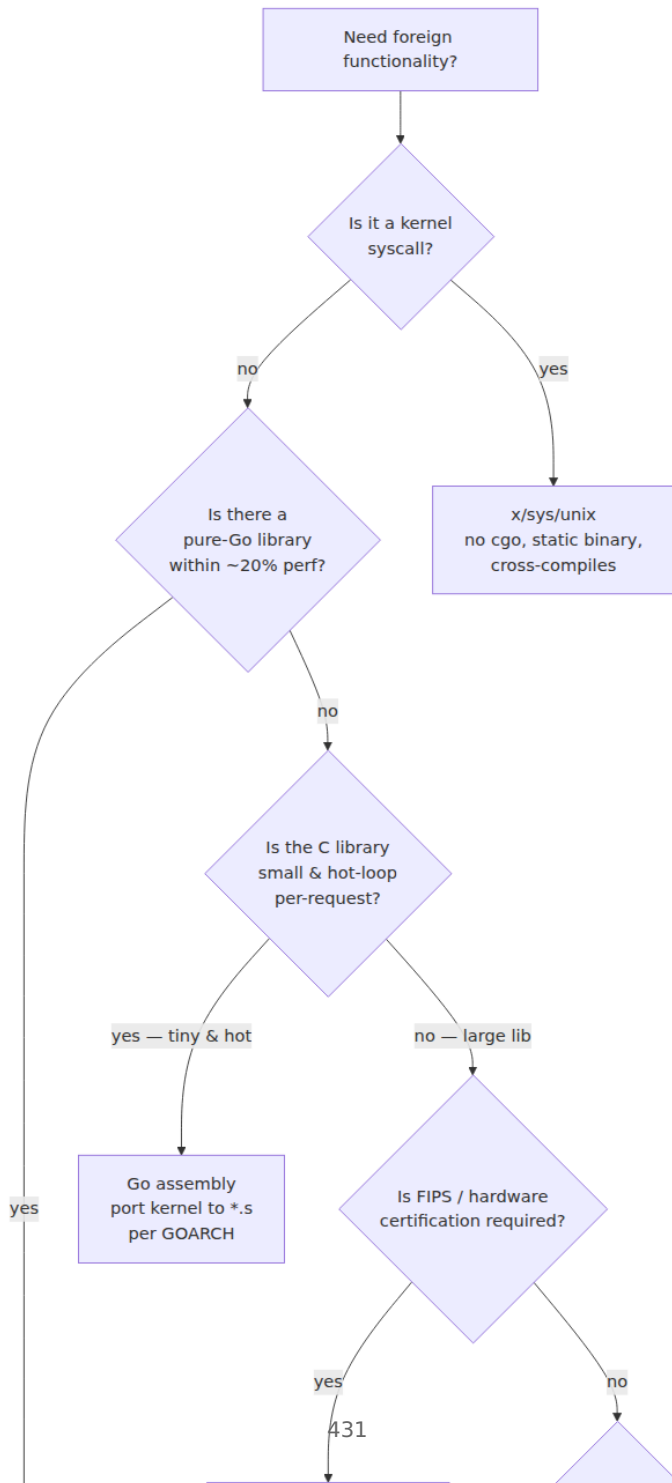
Caveats that make `plugin` rare in production backends:

- **Linux and macOS only** — no Windows, no `CGO_ENABLED=0` (it uses `dlopen` under the hood, which is `cgo`-adjacent).
- **Exact toolchain match** — plugin and host must be built with the **same Go version**, same `GOOS` / `GOARCH`, same dependency versions, and same build flags. A single `go mod tidy` that bumps a transitive dep breaks loading with `plugin` was built with a different version of package ...

- **Global state** — plugins share the host's address space, GC, and scheduler. A leaking plugin cannot be unloaded (`plugin.Close` does not exist — `dlclose` is deliberately not called).
- **No isolation** — a panic in the plugin crashes the host.

Where it is used: narrow cases like `Caddy` modules, `Traefik` plugins, or research systems that need hot-reload of Go code without restarting the process. For most backends, **ship a new binary** (fast Go builds, immutable deploys) or use **Wasm/RPC** for isolation instead.

## FFI decision tree



General rule: **prefer** `x/sys/unix` for syscalls, **pure Go for everything else**, `cgo` **only when certification or a large irreplaceable C library forces it**, and **assembly only for small numeric kernels where `cgo` overhead would dominate**.

---

## 16. Backend lens: `boringcrypto`, `libz`, static binaries, and leak auditing

**When `cgo` is the right call — `boringcrypto` and `libz`**

**BoringCrypto** / `GOEXPERIMENT=boringcrypto`. US federal workloads (FedRAMP, FIPS 140-2/140-3) require a validated cryptographic module. Go's standard `crypto/*` is not FIPS-validated. The `boringcrypto` experiment (available since Go 1.18, via `GOEXPERIMENT=boringcrypto` or `GOEXPERIMENT=boringcrypto,systemcrypto`) links `libcrypto` from Google's `boringssl` fork via `cgo`. All `crypto/aes`, `crypto/sha256`, `crypto/ecdh`, and `crypto/tls` operations dispatch to the C library when the experiment is enabled.

```
Build with BoringCrypto (requires Clang + boringssl checkout per Go docs)
GOEXPERIMENT=boringcrypto go build -o /tmp/app-fips ./cmd/app
go tool nm /tmp/app-fips | grep -i boring # symbols from libcrypto
Runtime check:
GODEBUG=boringcrypto=1 /tmp/app-fips 2>&1 | head
boringcrypto: AES-GCM ... (logs when BoringCrypto is active)
```

Operational cost: the binary is dynamically linked to `libcrypto.so`, the build needs `clang` and `boringssl` sources, and `cgo` overhead applies to every crypto operation. Teams that need FIPS accept this; teams that don't should not pay it — use `crypto/tls` pure Go and keep `CGO_ENABLED=0`.

**Compression — `libz` vs. `klauspost/compress`**. `compress/flate` and `compress/gzip` in the stdlib are pure Go and adequate for moderate throughput. At line rate (10+ Gbps compress, or large image pipelines), many teams historically reached for `cgo` wrappers around `libz`/`libzstd`. Today `github.com/klauspost/compress` (pure Go, assembly-accelerated `zstd/s2/flate`) **outperforms** `cgo` `zlib` on both speed and compression ratio, with no `cgo` tax and a static binary. Benchmark before assuming C is faster.



## Eliminating `cgo` for deployability

The deployability wins are concrete:

Concern	With <code>cgo</code>	<code>CGO_ENABLED=0</code>
Base image	<code>debian:bookworm-slim</code> + <code>libssl3</code> + <code>libzstd1</code> + CA certs	<code>scratch</code> or <code>gcr.io/distroless/static</code>
CVE surface	Every <code>.so</code> is a CVE feed ( <code>trivy</code> image shows <code>libssl</code> , <code>libz</code> , <code>glibc</code> )	Only Go stdlib + your code
Cross-compile	Need <code>CC=aarch64-linux-gnu-gcc</code> , <code>sysroot</code> , <code>pkg-config</code> per arch	<code>GOOS</code> / <code>GOARCH</code> only
Build hermeticity	Depends on host <code>gcc</code> , <code>pkg-config</code> , <code>.pc</code> files	Fully hermetic — no C toolchain
Startup	Dynamic linker resolves <code>DT_NEEDED</code> at launch (cold-start tax)	No dynamic linker — instant exec
<code>scratch</code> compat	No — needs <code>libc.so.6</code>	Yes

Migration playbook (as used by teams moving `mattn/go-sqlite3` → `modernc.org/sqlite` or `cgo zlib` → `klauspost/compress`):

1. **Abstract behind an interface** — `type Compressor interface{ Compress([]byte) ([]byte, error) }` with `compress_cgo.go` (`//go:build cgo`) and `compress_pure.go` (`//go:build !cgo`) implementations.
2. **Gate with `//go:build`** — keep both implementations building in CI (`CGO_ENABLED=0` and `CGO_ENABLED=1` jobs).
3. **Benchmark in production shape** — `go test -bench=. -count=5` with real payload sizes, not microbenchmarks. Include p99 and alloc profiles.
4. **Cut over with a flag** — `COMPRESS_IMPL=pure` env var or feature flag, so rollback is instant.
5. **Remove the `cgo` variant** once the pure-Go path proves out — delete the `cgo` file and the `CGO_ENABLED=1` CI job.

## Auditing `cgo` for leaks

Every `C.malloc` / `C.CString` must have a matching `C.free` on **every** path — including error returns and panics. The patterns that leak in production:

```
// Leak: early return without free
func Leaky(s string) error {
 cs := C.CString(s)
 if err := validate(s); err != nil {
 return err // BUG: cs not freed
 }
 defer C.free(unsafe.Pointer(cs))
 C.use(cs)
 return nil
}

// Fixed: defer immediately after allocation, or use a helper
func Clean(s string) error {
 cs := C.CString(s)
 defer C.free(unsafe.Pointer(cs))
 if err := validate(s); err != nil {
 return err // defer runs
 }
 C.use(cs)
 return nil
}

// Helper that makes the pairing explicit
func withCString(s string, fn func(*C.char)) {
 cs := C.CString(s)
 defer C.free(unsafe.Pointer(cs))
 fn(cs)
}
```

Checklist for code review:

- [ ] Every `C.CString` / `C.malloc` has a `defer C.free` in the same function, on the next line if possible.
- [ ] No `C.CString` inside a loop without `C.free` inside the same iteration (otherwise  $N \times \text{len}$  leak).
- [ ] `unsafe.Pointer(&slice[0])` paired with `runtime.KeepAlive` or `Pinner`.
- [ ] `LockOSThread` used when C TLS/`errno` matters across calls.
- [ ] `go vet` and `CGO_CFLAGS="-fsanitize=address"` clean in CI for `cgo` packages.

- [ ] `trivy / gype` on the final image — no unexpected `DT_NEEDED`.
- 

## Key takeaways

- `import "C"` is a code generator, not a library. The preamble is C, `#cgo` directives control `CFLAGS / LDFLAGS / pkg-config`, and `${SRCDIR}` is the only expansion. Flags from leaf packages propagate to the final link — a leaf `cgo` dependency imposes itself on every consumer.
- Go ↔ C type mappings are explicit (`C.int`, `*C.char`, `C.size_t`). Strings and slices always **copy** through `C.CString / C.GoString / C.GoBytes`. Zero-copy requires `unsafe.Pointer` plus `runtime.KeepAlive` (during call) or `runtime.Pinner` (when C retains the pointer).
- **Memory ownership is manual on the C side.** Every `C.CString / C.malloc` must be `C.free`'d on every path. Use `defer C.free` immediately after allocation; never `C.CString` in a loop without per-iteration free. Audit with `ASan / Valgrind` and `trivy`.
- **Threading: a `cgo` call pins `M` → thread and releases `P` via `entersyscall / exitsyscall`.** Callbacks via `//export` re-enter through `cgocallback`; a C-created thread gets a new `M` via `__cgo_thread_start`. Use `LockOSThread` when C thread-local state must survive across calls.
- **A `cgo` call costs ~70-150 ns** of stack-switch and scheduler overhead, is not inlinable, and forces escapes. Batch work, cache `C.CString` results, or keep `cgo` off the request hot path. If each call does microseconds of real work, the tax is noise; if nanoseconds, it dominates.
- **Avoid `cgo` when a pure-Go alternative is within ~20% performance.** `klauspost/compress` beats `cgo zlib`, `hash/crc32` already uses SSE4.2 / ARMv8, and `modernc.org/sqlite` enables static binaries. `CGO_ENABLED=0` restores cross-compilation, `scratch` images, and hermetic builds.
- **Go assembly uses `TEXT / DATA / GLOBL` with `FUNCDATA / PCDATA` for GC and stack traces, and `NOSPLIT` for leaf kernels.** It descends from Plan 9 but uses the register ABI (`AX / BX / ...`), `·` package-scoped symbols, and `ABIInternal` wrappers. Prefer `go tool compile -S` and

`go tool objdump` to read it; use `go vet -asmdecl` to verify declarations.

- **`go tool cgo -godefs vendors C types into pure Go`** (`Sizeof*`, `SYS_*`, `AF_*`). `x/sys/unix` is built this way — it is the correct way to do syscalls without `cgo`. `syscall` (stdlib) is frozen; use `x/sys/unix`.
  - **plugin (`buildmode=plugin`) is rarely the right answer** — it requires exact toolchain/dependency match, shares address space/GC with the host, and cannot be unloaded. Prefer shipping a new binary or using Wasm/RPC for isolation.
  - **Decision tree:** `syscall` → `x/sys/unix`; pure-Go within 20% → pure Go; small hot kernel → Go assembly; large irreplaceable C lib or FIPS → `cgo` (batched, off hot path, audited for leaks). Every `cgo` addition should be justified in a design doc against deployability cost.
- 

## Further reading

- **`cgo` documentation** — `go doc cmd/cgo` and <https://pkg.go.dev/cmd/cgo>. The authoritative reference for `import "C"`, preamble rules, `#cgo` directives, type mappings, `C.CString / C.GoString / C.GoBytes`, `//export`, pointer-passing rules, and `GODEBUG=cgocheck`. Pinned: Go 1.22 docs at <https://pkg.go.dev/cmd/cgo> (versioned via `go doc`).
- **Go assembly — A Quick Guide** — <https://go.dev/doc/asm>. Official guide to `TEXT / DATA / GLOBL`, `FUNCDATA / PCDATA`, `NOSPLIT`, `textflag.h`, and the `go tool asm / go tool compile -S` workflow. Pinned: [go.dev/doc/asm](https://go.dev/doc/asm) (tracks tip; use `go doc` for the version you build with).
- **`runtime/cgocall`** — `src/runtime/cgocall.go` and `src/runtime/cgo/gcc_linux_amd64.c` — the actual `cgocall / cgocallback / entersyscall / exitsyscall` implementation and the `__cgo_thread_start / x_cgo_thread_start` bridge. Reading `cgocall.go:cgocall` and `cgo/gcc_traceback.c` is the fastest way to understand the stack switch. Pinned: <https://github.com/golang/go/tree/master/src/runtime/cgocall.go>.
- **`golang.org/x/sys` — `x/sys/unix` package** — <https://pkg.go.dev/golang.org/x/sys/unix>, plus the generator `x/sys/unix/mkall.sh / mksyscall.go` and `go tool cgo -godefs` usage. The model for pure-

Go FFI without `cgo`. Pinned: <https://pkg.go.dev/golang.org/x/sys/unix> and <https://github.com/golang/sys>.

- **Go internal ABI** — `src/cmd/compile/abi-internal.md` — register maps per `GOARCH` (`AX / BX / ...` on `amd64`, `R0 - R8` on `arm64`), `ABI0` vs. `ABIInternal`, and how assembly wrappers shuffle between them. Essential for writing register-ABI assembly. In-tree doc; view with `go doc` or at <https://github.com/golang/go/blob/master/src/cmd/compile/abi-internal.md>.
- **BoringCrypto and `GOEXPERIMENT=boringcrypto`** — <https://go.dev/src/crypto/boring/boring.go> and <https://go.googlesource.com/go/+refs/heads/dev.boringcrypto>. How Go dispatches `crypto/*` to `libcrypto` via `cgo` for FIPS, and what `boringcrypto=1 GODEBUG` does.
- **Klaus Post** — `klauspost/compress` and the case against `cgo zlib` — <https://github.com/klauspost/compress> and the benchmarks in `zstd / s2` vs. `cgo libzstd`. Evidence for the "pure Go within 20%" heuristic.
- **Russ Cox** — "Go Assembly by Example" and `go tool compile -S` deep dive — <https://go.dev/blog/asm> and `go tool compile -S` walkthroughs. Practical companion to [go.dev/doc/asm](https://go.dev/doc/asm) with real `AVX / CRC` examples.
- **Ian Lance Taylor** — "cgo pointer passing rules" — <https://github.com/golang/proposal/blob/master/design/12416-cgo-pointers.md>. The design doc for `cgocheck` and why Go pointers to Go memory cannot be passed to C without pinning.

## Chapter 12 — Production Go: Cross-Compilation, Workspaces, Telemetry, and Deployment at Scale

---

**What this chapter covers.** The distance between `go build` on a developer laptop and a signed, SBOM-attested, autoscaled binary running on 500 Kubernetes pods is where Go's toolchain promises either pay off or break down. This chapter closes that distance. We trace how the same hermetic build model from Chapter 1 extends to cross-compilation matrices, multi-module workspaces, bit-for-bit reproducible artifacts, and observable, gracefully-shutting-down services that respect cgroup limits.

Every section is grounded in the artifact you actually ship — the static binary and the container that carries it — and in the operational reality of running hundreds of Go services with one platform team.

Learning goals — after this chapter you should be able to:

- Cross-compile any pure-Go service to every `G00S / GOARCH` pair from `go tool dist list`, diagnose why `cgo` breaks that model, and fix it with `zig cc` or `xgo` when `CGO_ENABLED=1` is non-negotiable.
- Structure multi-module repositories with `go.work`, reason about when `go.work` beats `replace`, and run `go work sync` correctly in CI.
- Produce bit-for-bit reproducible binaries and containers with `-trimpath`, `-buildvcs`, `SOURCE_DATE_EPOCH`, `ko / apko`, and `goreleaser`, and verify them with `go version -m` and `diffoscope`.
- Instrument a Go service with `runtime/metrics`, `expvar`, `log/slog`, OpenTelemetry, and `runtime/pprof` continuous profiling, and tune it at runtime with `GODEBUG` and `GOMEMLIMIT / GOMAXPROCS`.
- Build minimal, signed, SBOM-attested container images ( `scratch` vs. `distroless` vs. `apko / ko` ) and deploy them to Kubernetes with correct resource limits, `GOMAXPROCS / GOMEMLIMIT` tuning, probes, and graceful shutdown via `signal.NotifyContext` + `http.Server.Shutdown`.
- Reason about supply-chain integrity end-to-end — from `GOSUMDB` through `cosign / sigstore` signing and `syft` SBOM generation — for a fleet of Go services.

**Scope and dependencies.** This chapter assumes Chapter 1 (toolchain, modules, linker, `go build` DAG) and draws on Chapter 4 (scheduler/`GOMAXPROCS`), Chapter 5 (allocator/GC/`GOMEMLIMIT`), and Chapter 10 ( `pprof` / `trace` / race detector). Supply-chain primitives ( `SLSA`, `in-toto`, `Sigstore` ) are covered in Companion Book 4; here we show their Go-specific integration.

---

## 1. Cross-compilation — one toolchain, every platform

Go's cross-compilation story is unusually simple for systems languages: no `arm-linux-gnueabi-hf-gcc` in your `PATH` by default, no sysroot dance, no CMake toolchain file. The compiler is a cross-compiler out of the box because `cmd/compile` and `cmd/link` handle every `GOARCH` internally. That

simplicity has a precise boundary — `CGO_ENABLED=0` — and crossing that boundary reintroduces the entire C cross-compilation problem.

## 1.1 The GOOS/GOARCH/GOARM matrix

Three environment variables select the target; a fourth controls the build itself:

Variable	Meaning	Examples	Notes
GOOS	Target operating system	<code>linux</code> , <code>darwin</code> , <code>windows</code> , <code>freebsd</code> , <code>wasmtime</code>	Determines <code>//go:build</code> evaluation, syscall package selection, default file suffixes ( <code>*_linux.go</code> )
GOARCH	Target CPU architecture	<code>amd64</code> , <code>arm64</code> , <code>386</code> , <code>arm</code> , <code>ppc64le</code> , <code>wasm</code> , <code>riscv64</code> , <code>loong64</code>	Selects <code>cmd/compile</code> backend and <code>cmd/link</code> output format
GOARM	ARM variant (only when <code>GOARCH=arm</code> )	<code>5</code> , <code>6</code> , <code>7</code>	<code>GOARM=7</code> → ARMv7 + hard-float; <code>5</code> → soft-float baseline
GOAMD64	x86-64 microarchitecture level	<code>v1</code> ... <code>v4</code>	<code>v3</code> = AVX/AVX2 era (Haswell+); default <code>v1</code> is maximally compatible. Set <code>GOAMD64=v3</code> to trade portability for speed
GOMIPS / GOMIPS64	MIPS float ABI	<code>hardfloat</code> , <code>softfloat</code>	Rare outside embedded; <code>GOARCH=mips</code> is soft-stripped in Go 1.23+ on some platforms

Variable	Meaning	Examples	Notes
<code>CGO_ENABLED</code>	Whether <code>import "C"</code> is allowed	<code>0</code> , <code>1</code>	<code>0</code> → pure Go, fully cross-compilable; <code>1</code> → requires a C cross-compiler for the target

Enumerate the matrix for your toolchain:



```
Every supported GOOS/GOARCH pair for this Go release
$ go tool dist list
aix/ppc64
darwin/amd64
darwin/arm64
dragonfly/amd64
freebsd/386
freebsd/amd64
freebsd/arm
freebsd/arm64
freebsd/riscv64
illumos/amd64
ios/amd64
ios/arm64
js/wasm
linux/386
linux/amd64
linux/arm
linux/arm64
linux/loong64
linux/mips
linux/mips64
linux/mips64le
linux/mipsle
linux/ppc64
linux/ppc64le
linux/riscv64
linux/s390x
netbsd/386
netbsd/amd64
netbsd/arm
netbsd/arm64
openbsd/386
openbsd/amd64
openbsd/arm
openbsd/arm64
openbsd/ppc64
openbsd/riscv64
plan9/386
plan9/amd64
plan9/arm
solaris/amd64
wasip1/wasm
windows/386
windows/amd64
windows/arm
windows/arm64

Current host triple – what `go build` does with no env override
$ go env GOOS GOARCH GOARM GOAMD64 CGO_ENABLED CC
```

```
linux
amd64
 # (empty when not arm)
v1
1
gcc
```

A single env prefix is enough:

```
Pure Go – no C toolchain needed, works from any host
$ CGO_ENABLED=0 GOOS=linux GOARCH=arm64 go build -o payments-linux-
arm64 ./cmd/api
$ CGO_ENABLED=0 GOOS=darwin GOARCH=amd64 go build -o payments-darwin-
amd64 ./cmd/api
$ CGO_ENABLED=0 GOOS=windows GOARCH=amd64 go build -o payments.exe ./
cmd/api
$ CGO_ENABLED=0 GOOS=linux GOARCH=arm GOARM=7 go build -o payments-
armv7 ./cmd/api

Inspect what you built – no need to boot the target
$ file payments-linux-arm64 payments-armv7 payments.exe
payments-linux-arm64: ELF 64-bit LSB executable, ARM aarch64, statically
y linked
payments-armv7: ELF 32-bit LSB executable, ARM, EABI5 version 1
(SYSV), statically linked
payments.exe: PE32+ executable (console) x86-64, for MS Windows

$ go version -m payments-linux-arm64 | head -8
 payments-linux-arm64: go1.22.4
 build GOOS=linux
 build GOARCH=arm64
 build CGO_ENABLED=0
```

Build constraints interact with cross-compilation at file-selection time, before any package compiles. A file named `net_linux.go` with `//go:build linux` is excluded when `GOOS=darwin`; `cmd/go` never even parses it. This is why `go vet` and `go pl` must be told the target — their diagnostics are target-dependent:

```
// internal/platform/mem_linux.go
//go:build linux
```

```
package platform
```

```
import "golang.org/x/sys/unix"
```

```
func Mlockall() error { return unix.Mlockall(unix.MCL_CURRENT | unix.MCL_FUTURE) }
```

```
// internal/platform/mem_darwin.go
//go:build darwin
```

```
package platform
```

```
func Mlockall() error { return nil } // no mlockall on darwin
```

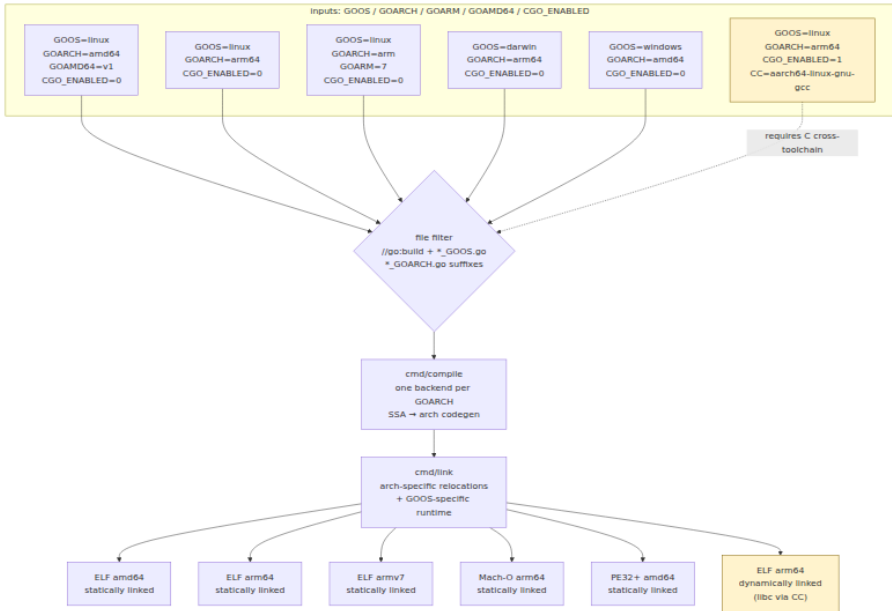


Diagram 1 — Cross-compilation matrix: every GOOS/GOARCH pair is one env prefix with CGO\_ENABLED=0; the CGO\_ENABLED=1 path (highlighted) breaks the hermetic model and requires an external cross-compiler.

## 1.2 The cgo boundary

When any package in the transitive closure imports `"C"`, the build sets `CGO_ENABLED=1` (or fails if you forced `CGO_ENABLED=0`). At that point `cmd/go` no longer self-contains the build:

1. `cmd/cgo` generates Go shims + C stubs.
2. The host `CC` compiles the C fragments (`-cc` phase).
3. Final linking is delegated to `CC` (external linking), so the binary becomes dynamically linked and platform-dependent.

```
Pure Go transitive closure – cross-compiles from anywhere
$ CGO_ENABLED=0 GOOS=linux GOARCH=arm64 go build ./...

As soon as one dep imports "C", cross without a C toolchain fails
$ GOOS=linux GOARCH=arm64 go build ./... 2>&1 | head -5
runtime/cgo
cgo: C compiler "gcc" not found: exec: "gcc": executable file not
found in $PATH

Forcing CGO_ENABLED=0 only helps if no dep actually needs cgo
$ CGO_ENABLED=0 GOOS=linux GOARCH=arm64 go build ./... # succeeds if
closure is pure
$ CGO_ENABLED=0 go build -o /tmp/bin ./cmd/api && ldd /tmp/bin
 not a dynamic executable

With cgo, the target CC must match GOARCH
$ CGO_ENABLED=1 GOOS=linux GOARCH=arm64 CC=aarch64-linux-gnu-gcc go bui
ld -o /tmp/bin-arm64 ./cmd/api
$ file /tmp/bin-arm64
/tmp/bin-arm64: ELF 64-bit LSB executable, ARM aarch64, dynamically lin
ked, interpreter /lib/ld-linux-aarch64.so.1
$ ldd /tmp/bin-arm64
 linux-vdso.so.1
 libc.so.6 => /lib/aarch64-linux-gnu/libc.so.6
```

Common crates in this trap: `github.com/mattn/go-sqlite3` (always `cgo`), `net` with `osuser/netgo` edge cases, anything importing `github.com/DataDog/zstd` with `cgo` fallback. Prefer pure-Go alternatives (`modernc.org/sqlite`, `jackc/pgx` over `lib/pq` with `cgo`) when you need a clean cross matrix.

### Fixing cgo cross with zig cc and xgo

Two tools restore hermetic cross-compilation when you cannot eliminate `cgo`:

**zig cc** — Zig ships a bundled, hermetic C toolchain that can target any GOOS / GOARCH from any host. It replaces CC / CXX without installing per-arch sysroots:

```
Install zig (https://ziglang.org/download/ – single tarball, no dependencies)
$ zig version
0.13.0

Use it as the C cross-compiler – one binary handles every target
$ CGO_ENABLED=1 GOOS=linux GOARCH=arm64 \
 CC="zig cc -target aarch64-linux-gnu" \
 CXX="zig c++ -target aarch64-linux-gnu" \
 go build -o payments-linux-arm64 ./cmd/api

$ CGO_ENABLED=1 GOOS=linux GOARCH=amd64 \
 CC="zig cc -target x86_64-linux-gnu" \
 CXX="zig c++ -target x86_64-linux-gnu" \
 go build -o payments-linux-amd64 ./cmd/api

Windows cross from Linux – no mingw needed
$ CGO_ENABLED=1 GOOS=windows GOARCH=amd64 \
 CC="zig cc -target x86_64-windows-gnu" \
 go build -o payments.exe ./cmd/api
```

Why **zig cc** rather than installing **gcc-aarch64-linux-gnu**: Zig's toolchain is a single ~80 MB download that contains headers and libraries for every target. It eliminates sysroot drift between developer laptops and CI. The trade is that you now depend on Zig's bundled **glibc/musl** version — pin it (**zig 0.13.0**) and record it in **go version -m** via **-ldflags** if auditability matters.

**xgo** / **crazymax/xgo** — A Docker-based cross-compilation harness that bundles per-arch **CC** toolchains and **QEMU** for **go test** on foreign arch:

```
xgo (legacy, still common in older goreleaser configs)
$ xgo --targets=linux/amd64,linux/arm64,darwin/arm64 ./cmd/api

crazymax/xgo – actively maintained fork
$ docker run --rm -v $PWD:/src -w /src crazymax/xgo:latest \
 --targets=linux/amd64,linux/arm64,linux/arm \
 --out payments ./cmd/api
```

Prefer **zig cc** for new setups — it runs without Docker and integrates directly with **go build** env. Reserve **xgo** for projects that need **QEMU**-backed **go test** across architectures in CI.

### 1.3 Cross-compile matrix script

The script that every multi-platform Go project needs. It enforces `CGO_ENABLED=0` for the pure matrix and isolates the `CGO_ENABLED=1` (+ `zig cc`) path when a specific target needs it. Use it locally and in CI (`GORELEASER` in Section 5 automates the same matrix declaratively).

```

#!/usr/bin/env bash
scripts/cross-compile.sh – reproducible cross-build matrix
Usage: ./scripts/cross-compile.sh [version]
Requires: go >= 1.21, zig >= 0.11 (only for cgo targets)
set -euo pipefail

VERSION="${1:-dev}"
LDFLAGS="-s -w -X main.version=${VERSION} -X main.commit=$(git rev-
parse --short HEAD 2>/dev/null || echo unknown)"
BUILDVCS=1 # stamp vcs.revision/time into go.buildinfo; use --
builddvcs=false to suppress
TRIMPATH="-trimpath"

Pure Go targets – no C toolchain needed, build from any host
PURE_TARGETS=(
 "linux/amd64"
 "linux/arm64"
 "linux/arm/7" # GOARCH=arm + GOARM=7
 "linux/riscv64"
 "darwin/amd64"
 "darwin/arm64"
 "windows/amd64"
 "windows/arm64"
)

cgo targets – require zig cc; built only if zig is present
CGO_TARGETS=(
 "linux/amd64"
 "linux/arm64"
)

build_one() {
 local goos="$1" goarch="$2" goarm="${3:-}" cc_extra="${4:-}"
 local out="dist/payments-${goos}-${goarch}${goarm:+-v${goarm}}"
 [["$goos" == "windows"]] && out="${out}.exe"

 local env_args=(GOOS="$goos" GOARCH="$goarch")
 [[-n "$goarm"]] && env_args+=(GOARM="$goarm")
 env_args+=(CGO_ENABLED=0) # pure path

 echo "==> ${out} (${goos}/${goarch}${goarm:+/v${goarm}})"
 CGO_ENABLED=0
 env "${env_args[@]}" go build \
 ${TRIMPATH} -builddvcs="${BUILDVCS}" \
 -ldflags "${LDFLAGS}" \
 -o "${out}" ./cmd/api

 # Verify: check embedded buildinfo matches target
 go version -m "${out}" | grep -E "build\s+GO(OS|ARCH)"
}

```

```

build_one_cgo() {
 local goos="$1" goarch="$2"
 local out="dist/payments-${goos}-${goarch}-cgo"
 local zig_target
 case "${goos}/${goarch}" in
 linux/amd64) zig_target="x86_64-linux-gnu" ;;
 linux/arm64) zig_target="aarch64-linux-gnu" ;;
 *) echo "no zig mapping for ${goos}/${goarch}"; return 1 ;;
 esac
 echo "==> ${out} (${goos}/${goarch} + cgo via zig cc -target ${zig_target}
rget})"
 env GOOS="$goos" GOARCH="$goarch" CGO_ENABLED=1 \
 CC="zig cc -target ${zig_target}" \
 CXX="zig c++ -target ${zig_target}" \
 go build ${TRIMPATH} -ldflags "${LDLAGS}" -o "${out}" ./cmd/api
}

mkdir -p dist

for target in "${PURE_TARGETS[@]}; do
 IFS=/ read -r goos goarch goarm <<< "$target"
 build_one "$goos" "$goarch" "$goarm"
done

if command -v zig &>/dev/null; then
 for target in "${CGO_TARGETS[@]}; do
 IFS=/ read -r goos goarch _ <<< "$target"
 build_one_cgo "$goos" "$goarch"
 done
else
 echo "zig not found – skipping cgo cross targets (install zig for
full matrix)"
fi

echo "--- dist/ ---"
ls -lh dist/
echo "--- buildinfo spot-check ---"
for f in dist/*; do echo "== $f =="; go version -m "$f" | grep -E "buil
d\s+(GOOS|GOARCH|CGO)"; done

```

Key design choices:

- **CGO\_ENABLED=0 by default.** The pure matrix covers 95% of services. A `go.mod` that accidentally pulls a `cgo` dependency fails fast rather than silently producing a dynamically-linked binary that will not run on `scratch`.



- `-trimpath -buildvcs` — discussed in Section 3; removing local filesystem paths and stamping VCS metadata makes builds reproducible without leaking developer paths.
- **GOAMD64=v3 is opt-in.** Uncomment `env_args+=(GOAMD64=v3)` for compute-heavy services where Haswell+ can be assumed. The default `v1` is maximally portable; `v3` / `v4` emit AVX2/AVX-512 and fault on older hardware.

## 2. Workspaces — `go.work` and multi-module repos

Go 1.18 introduced workspaces (`go.work`) to solve a concrete pain: developing across multiple modules without `replace` directives that must never be committed. Before workspaces, the only way to test a local change to `github.com/acme/shared` from `github.com/acme/payments` was:

```
// go.mod – DO NOT COMMIT THIS
replace github.com/acme/shared => ../shared
```

Every developer maintained their own `replace` set, CI had to strip them, and `go mod tidy` fought you. Workspaces make the local override a *workspace* concern, not a *module* concern.

### 2.1 `go.work` mechanics

A `go.work` file lives at the workspace root (above the modules) and declares which modules participate:

```
// go.work – workspace root (e.g., /repo/go.work)
// This file is NOT published. Consumers of any module ignore it.
go 1.22.4

use (
 . // the main service module at the root
 ./shared // shared library module
 ./platform/queue // another service/library module
 ./tools/gen // code-gen tool, its own module
)

// Optionally, override a dependency for every module in the workspace:
replace github.com/acme/shared => ../shared
```

```

Create / inspect a workspace
$ cat go.work
go 1.22.4

use (
 .
 ./shared
 ./platform/queue
)

$ go work use ./new-service # add a module
$ go work use -r ./... # recursively find and add all
modules under .
$ go work sync # sync go.work.sum and each
go.mod's go.sum
$ go work edit -use ./tools/lint # same, but without scanning
$ go env GOWORK # where the workspace file was
found
/home/ubuntu/repo/go.work
$ GOWORK=off go list -m all # ignore the workspace (single-
module mode)

What `go work sync` actually does – it is `go mod tidy` for the
workspace
$ go work sync
→ updates go.work.sum (hashes of workspace deps)
→ updates each module's go.sum to be consistent with the workspace
view
→ fails if workspace modules require conflicting versions of the same
dep

```

Resolution rules (these matter when builds break after adding a workspace entry):

- When `GOWORK` points at a `go.work` file (or is auto-discovered by walking parents), `cmd/go` builds a *workspace module graph*: the union of all `use` d modules' dependencies, resolved once via MVS.
- `replace` directives in `go.work` apply to *every* module in the workspace and override any `replace` in individual `go.mod` files. `replace` in a `go.mod` inside a workspace still applies to that module *when built outside the workspace* (e.g., by a consumer).
- `go.work` and `go.work.sum` should be committed if you want `git clone && go build ./...` to work. Exclude them if you want every consumer to build each module independently — but then developers must create their own `go.work`.

- `go list -m all` inside a workspace shows the *workspace-resolved* versions, not any single module's `go.mod` view. Use `GOWORK=off go list -m all` to see the single-module view when debugging version conflicts.

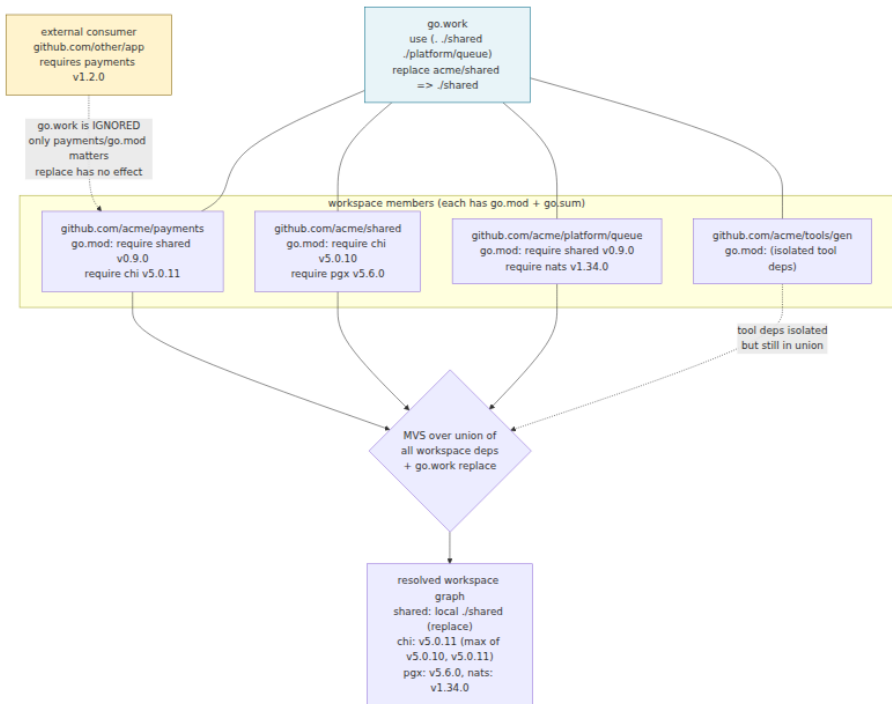


Diagram 2 — `go.work` workspace graph: the workspace resolves the union of all members' dependencies; external consumers ignore `go.work` entirely.

## 2.2 Monorepo vs. multi-module vs. polyrepo

There are three repository shapes for a Go fleet, each with a different `go.work` role:

Shape	Modules	go.work	When to use
<b>Single module monorepo</b>	One <code>go.mod</code> at root, all packages under it	None needed	Small fleet (<15 services) or tightly-coupled codebase. Simplest CI, but <code>go list</code> always sees the whole world; <code>go test ./...</code> tests everything.
<b>Multi-module monorepo</b>	N <code>go.mod</code> files under one repo root, one <code>go.work</code> at root	<code>go.work</code> at repo root, committed	Large monorepo where services version dependencies independently but developers work across modules. Each service can be released/tagged at its own version; CI can build/test the workspace or individual modules.
<b>Polyrepo</b>	One <code>go.mod</code> per repo, no workspace	<code>go.work</code> created locally by developers who clone multiple repos side-by-side	Organizationally separated teams, independent CI, versioning via semver tags. Developers create an ad-hoc <code>go.work</code> outside any repo to work across repos without <code>replace</code> .

The industry trend for Go fleets of 50–500 services is **multi-module monorepo with `go.work` committed**. It gives:

- **Atomic cross-service refactors** — change `shared` and all consumers in one commit, verified by one `go test ./...` at the workspace root.
- **Independent release versions** — each `go.mod` is releasable at its own tag ( `shared/v0.10.0` , `payments/v1.2.0` ); the repo can be tagged per-module ( `git tag shared/v0.10.0` ).
- **Selective CI** — CI can run `go test ./shared/... ./platform/queue/...` for affected modules or `GOWORK=off go test ./...` per module for hermetic checks.

A concrete multi-module layout:

```

repo/
├── go.work # workspace root – committed
├── go.work.sum
├── go.mod # github.com/acme/payments (main service)
├── go.sum
├── cmd/api/main.go
├── internal/
├── shared/ # github.com/acme/shared – library module
│ ├── go.mod # module github.com/acme/shared
│ ├── go.sum
│ └── otel/middleware.go
├── platform/queue/ # github.com/acme/platform/queue
│ ├── go.mod
│ └── consumer.go
├── tools/gen/ # github.com/acme/tools/gen – code
│ └── generator
│ ├── go.mod
│ └── main.go

```

```

// repo/go.work
go 1.22.4

use (
 .
 ./shared
 ./platform/queue
 ./tools/gen
)

```

```

// repo/go.mod – main service
module github.com/acme/payments

go 1.22.4

require (
 github.com/acme/shared v0.9.0
 github.com/acme/platform/queue v0.3.0
 github.com/go-chi/chi/v5 v5.0.11
)

// No replace needed – go.work maps shared => ./shared
// This go.mod is what external consumers see.

```

Inside the workspace, `import "github.com/acme/shared"` resolves to `./shared` regardless of the `v0.9.0` line. Outside the workspace (CI with `GOWORK=off`, or an external consumer), it resolves to `v0.9.0` from the proxy. There is no `replace` to forget to remove before tagging.

CI pattern for a multi-module monorepo:

```
.github/workflows/ci.yml – test both views
jobs:
 test-workspace:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-go@v5
 with: { go-version-file: go.mod, cache: true }
 - run: go work sync
 - run: go vet ./...
 - run: go test -race -count=1 ./...
 # ^ tests the workspace union – catches cross-module breakage

 test-hermetic:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-go@v5
 with: { go-version-file: go.mod }
 - run: GOWORK=off go test -count=1 ./...
 working-directory: shared
 - run: GOWORK=off go test -count=1 ./...
 working-directory: platform/queue
 # ^ tests each module as its consumers will see it
```

### 2.3 replace vs. workspace — when to use which

Mechanism	Scope	Committed?	Consumer sees it?	Use case
replace in go.mod	That module only, when it is the main module	Yes if committed	No — ignored when module is a dependency	Pinning a fork, working around a broken upstream, replace acme/shared => ../shared for a quick local test (remove before commit)

Mechanism	Scope	Committed?	Consumer sees it?	Use case
<code>replace</code> in <code>go.work</code>	Every module in the workspace	Yes if <code>go.work</code> committed	No — <code>go.work</code> is never published	Workspace-wide override (e.g., point every module at a local checkout of a shared library)
<code>GOWORK=off</code>	Disables workspace	N/A	N/A	Debugging single-module resolution, hermetic CI checks
No <code>replace</code> , no workspace	Proxy-resolved version	—	Yes — the <code>require</code> line is what consumers resolve	Normal released dependencies

Rule of thumb: `replace` in `go.mod` is a **publishing-time concern (forks, retractions)**; `go.work` is a **development-time concern (local multi-module work)**. If you find yourself editing `replace` in `go.mod` during development, create a `go.work` instead. If CI fails because a `replace` points at a local path, the `replace` should have been in `go.work`.

### 3. Reproducible builds — from `-trimpath` to bit-for-bit containers

A reproducible build produces byte-identical output from the same inputs regardless of who builds it, where, or when. For Go this means: the same commit, same `go.mod / go.sum`, same Go version, and same build flags must yield the same binary — down to the hash. Reproducibility is the foundation of binary transparency and SLSA provenance: if builds are not reproducible, you cannot verify that the binary in the registry was built from the source you audited.

### 3.1 The reproducibility knobs

Flag / env	What it does	Why it matters
<code>-trimpath</code>	Strips local filesystem paths from the binary ( <code>compile -trimpath</code> , <code>link -trimpath</code> ). Replaces absolute paths like <code>/home/alice/repo/internal/foo.go</code> with <code>github.com/acme/payments/internal/foo.go</code> in <code>pcntab</code> , <code>gopclntab</code> , and <code>DWARF</code>	Without it, two developers building the same commit from different checkout paths produce different binaries
<code>-buildvcs</code>	Controls VCS stamping into <code>runtime/debug.BuildInfo</code> ( <code>vcs.revision</code> , <code>vcs.time</code> , <code>vcs.modified</code> ). Default <code>auto</code> (stamp if <code>.git</code> is present); <code>true</code> / <code>false</code> to force	Deterministic when <code>.git</code> state is deterministic; <code>false</code> removes VCS noise for hermetic diffing
<code>-ldflags "-s -w"</code>	<code>-s</code> omits symbol table, <code>-w</code> omits DWARF debug info	Smaller binary; removes debug paths that can vary. Omit when you need <code>pprof</code> line numbers in prod — trade size for debuggability
<code>-ldflags "-X main.version=v1.2.3"</code>	Sets string variables at link time	Deterministic, but the value must be pinned (e.g., git tag) not <code>\$(date)</code>
<code>SOURCE_DATE_EPOCH</code>	Unix timestamp respected by <code>go</code> for archive timestamps and <code>vcs.time</code> fallback	Required for container layer reproducibility; set to the commit timestamp in CI



Flag / env	What it does	Why it matters
<code>-tags / GOEXPERIMENT</code>	Must be identical across builds	Different tags select different files; the flag set is part of the build identity
<code>CGO_ENABLED</code> , <code>CC</code> , <code>GOOS / GOARCH</code>	Part of the build identity	Any change selects different objects; lock in CI

```
Fully reproducible pure-Go build – the baseline for verification
$ SOURCE_DATE_EPOCH=$(git log -1 --format=%ct) \
 CGO_ENABLED=0 \
 go build -trimpath -buildvcs=true \
 -ldflags "-s -w -X main.version=$(git describe --tags --always)" \
 -o payments ./cmd/api

Verify: two builders should produce identical hashes
$ sha256sum payments
a3f7c... payments
$ go version -m payments
 build -buildmode=exe
 build -compiler=gc
 build CGO_ENABLED=0
 build GOARCH=amd64
 build GOOS=linux
 build vcs=git
 build vcs.revision=9f3c1a7d2e4b0c6f8a1b2c3d4e5f6a7b8c9d0e1f
 build vcs.time=2024-11-15T14:22:31Z
 build vcs.modified=false
 dep github.com/go-chi/chi/v5 v5.0.11 h1:...

Bit-for-bit diff – use diffoscope for actionable output
$ diffoscope payments-a payments-b || true
With -trimpath, diffoscope should report no differences
Without -trimpath, you will see DWARF paths differing: /home/
 alice/... vs /home/bob/...
```



Diagram 3 — Reproducibility pipeline: pinned inputs + `-trimpath` + `SOURCE_DATE_EPOCH` → deterministic binary; verification closes the loop.

### 3.2 Reproducible containers — ko, apko, goreleaser

A reproducible binary is not enough if the container that carries it is not reproducible. Traditional `Dockerfile` builds embed timestamps, package-manager nondeterminism, and base-image drift. Three tools address this for Go:

**ko — container images without Dockerfiles.** `ko` builds Go binaries and packages them into images directly from `go.mod`, using `apko/chainguard` base images that are themselves reproducible. No `Dockerfile`, no `docker build`:

```
ko.yaml — ko configuration (go.work-aware since ko 0.15+)
https://ko.build/configuration/
apiVersion: ko.build/v1alpha1
kind: Config
baseImageOverrides:
 github.com/acme/payments/cmd/api: cgr.dev/chainguard/static:latest
 # Single static base — no shell, no package manager, minimal attack
 # surface
builds:
- id: api
 main: ./cmd/api
 env:
 - CGO_ENABLED=0
 flags:
 - -trimpath
 - -ldflags=-s -w -X main.version={{.Env.VERSION}}
 ldflags:
 - -s
 - -w
 - -X main.version={{.Env.VERSION}}
 - -X main.commit={{.Env.COMMIT}}
```

```

Build and push – ko resolves the import path, builds with `go build`,
creates the image
$ export VERSION=$(git describe --tags --always) COMMIT=$(git rev-
parse HEAD)
$ ko build ./cmd/api --base-import-paths --push --tags $VERSION,latest
2024/11/15 14:22:31 Building github.com/acme/payments/cmd/api for linux
/amd64
2024/11/15 14:22:31 Using base cgr.dev/chainguard/static:latest for git
hub.com/acme/payments/cmd/api
2024/11/15 14:22:31 Publishing ghcr.io/acme/payments:abc1234
2024/11/15 14:22:31 Published ghcr.io/acme/payments@sha256:7f3a...
Digest-addressable, SLSA provenance available via ko --sbom

Publish a multi-arch image (ko builds each arch natively via GOARCH)
$ ko build ./cmd/api --platform=linux/amd64,linux/arm64 --bare --push -
-tags $VERSION

```

`ko` advantages for Go fleets: it respects `go.work`, honors `GOPROXY` / `GOSUMDB`, generates an SBOM by default (`--sbom=spdx`), and produces images with no `Dockerfile` to maintain across 100 repos. The `cgr.dev/chainguard/static` base is itself built reproducibly with `apko`.

**apko — declarative, reproducible base images.** Where `ko` builds the Go binary, `apko` builds the base image. You declare the image contents in YAML; `apko` assembles it deterministically from `apk` packages pinned by hash:

```

apko.yaml – reproducible base image for Go services
https://github.com/chainguard-dev/apko
contents:
 repositories:
 - https://dl-cdn.alpinelinux.org/alpine/edge/main
 - https://packages.wolfi.dev/os
 keyring:
 - https://packages.wolfi.dev/os/wolfi-signing.rsa.pub
 packages:
 - ca-certificates-bundle # TLS roots – needed even for scratch-
like images
 - tzdata # time zones
 # NOTE: no shell, no apk, no busybox – minimal runtime deps
 # Add `glibc` only if you have a cgo binary; prefer CGO_ENABLED=0

cmd: /payments # entrypoint
archs:
 - amd64
 - arm64

accounts:
 run-as: 65532 # nonroot

```

```

Dockerfile that uses apko-built base + ko-built binary
Alternatively, ko handles this internally with --base-import-paths;
this shows the layers
FROM cgr.dev/chainguard/static:latest AS base
COPY --from=ko-build /ko-app/payments /payments
USER nonroot
ENTRYPOINT ["/payments"]

```

For maximum reproducibility, combine them: `apko` builds the base, `ko` builds the app layer, `goreleaser` orchestrates the release.

**goreleaser** — release automation with reproducibility built in:

```

.goreleaser.yaml - full matrix, containers, signing, SBOM
https://goreleaser.com/customization/
version: 2
project_name: payments

before:
 hooks:
 - go mod tidy
 - go vet ./...

builds:
 - id: api
 main: ./cmd/api
 binary: payments
 env:
 - CGO_ENABLED=0
 flags:
 - -trimpath
 ldflags:
 - -s -w -X main.version={{.Version}} -X main.commit={{.Commit}}
-X main.date={{.Date}}
 goos: [linux, darwin, windows]
 goarch: [amd64, arm64]
 goarm: ["7"] # only for GOARCH=arm
 ignore:
 - goos: windows
 goarch: arm64 # skip exotic combos if needed
 mod_timestamp: "{{.CommitTimestamp}}" # SOURCE_DATE_EPOCH
equivalent

archives:
 - id: tarballs
 builds: [api]
 formats: [tar.gz]
 name_template: "{{.ProjectName }}_{{.Version }}_{{.Os }}_{{.Arch }}_{{.Arm }}_{{.With.Arm }}_{{.End.Arm }}"
 format_overrides:
 - goos: windows
 formats: [zip]

nfpm: # optional: deb/rpm/apk
 - id: deb
 builds: [api]
 vendor: Acme
 homepage: https://github.com/acme/payments
 formats: [deb, apk]

dockers: # legacy docker builds
 - image_templates: ["ghcr.io/acme/payments:{{.Version }}-amd64"]
 use: buildx

```

```

 build_flag_templates:
 - --platform=linux/amd64
 - --label=org.opencontainers.image.revision={{ .Commit }}

docker_manifests:
 - name_template: "ghcr.io/acme/payments:{{ .Version }}"
 image_templates:
 - "ghcr.io/acme/payments:{{ .Version }}-amd64"
 - "ghcr.io/acme/payments:{{ .Version }}-arm64"

ko integration (goreleaser 1.20+): build images without Dockerfiles
kos:
 - id: api-ko
 main: ./cmd/api
 base_image: cgr.dev/chainguard/static:latest
 ldflags: "-s -w -X main.version={{.Version}}"
 platforms: [linux/amd64, linux/arm64]
 tags: ["ghcr.io/acme/payments:{{.Version}}", "ghcr.io/acme/
payments:latest"]
 sbom: spdx

sboms:
 - id: archive-sbom
 artifacts: archive
 - id: ko-sbom
 artifacts: images

signs:
 - id: archives
 artifacts: archive
 args: ["--keyless", "--fulcio-url=https://fulcio.sigstore.dev"]

docker_signs:
 - id: images
 artifacts: images
 args: ["--keyless"]

```

```

One command: build matrix, containers, SBOMs, signatures
$ goreleaser release --clean --snapshot # local smoke test
$ goreleaser release --clean # full release (requires
GITHUB_TOKEN, cosign)

```

Reproducibility checklist for CI:

```
ci/repro-check.sh - fail the build if it is not reproducible
set -euo pipefail
go build -trimpath -buildvcs=true -ldflags "-s -w" -o /tmp/a ./cmd/api
go build -trimpath -buildvcs=true -ldflags "-s -w" -o /tmp/b ./cmd/api
if ! cmp /tmp/a /tmp/b; then
 echo "NOT REPRODUCIBLE - diffoscope output:"
 diffoscope /tmp/a /tmp/b || true
 exit 1
fi
echo "reproducible: $(sha256sum /tmp/a | cut -d' ' -f1)"
```

---

## 4. Telemetry and observability — metrics, logs, traces, profiles from one binary

A Go binary that ships without observability is undebuggable at scale. The Go standard library and the Go observability ecosystem now cover four pillars out of the box, and they all run inside the binary with no sidecar required. The backend lens: at 100s of services, you need *uniform* telemetry — same metric names, same log schema, same trace propagation, same profiling endpoint — so that the platform team can build one dashboard, one alert, one runbook that works for every service.

### 4.1 runtime/metrics — the ground truth

Since Go 1.16, `runtime/metrics` exposes every runtime statistic as a named, typed metric with a stable API. It supersedes `runtime.ReadMemStats` and `runtime.ReadGCStats` for new code (the old APIs still work but are coarser and more allocation-heavy).

```

package telemetry

import (
 "runtime/metrics"
 "github.com/prometheus/client_golang/prometheus"
)

// Names are stable, histogram-aware, and documented in `go doc
runtime/metrics`.
// Every histogram has an explicit bucket set – no hidden Prometheus
defaults.
var interesting = []string{
 "/gc/heap/allocs:bytes", // cumulative allocations
 "/gc/heap/goal:bytes", // GOGC/GOMEMLIMIT target heap
size
 "/gc/heap/live:bytes", // live heap after last GC
 "/gc/gogc:percent", // current GOGC
 "/gc/gomemlimit:bytes", // current GOMEMLIMIT (Go 1.19+)
 "/gc/pauses:seconds", // histogram of STW pause
durations
 "/gc/cycles/automatic:gc-cycles", // GCs triggered automatically
 "/gc/cycles/forced:gc-cycles", // GCs forced by runtime.GC()
 "/sched/goroutines:goroutines", // current goroutine count
 "/sched/latencies:seconds", // histogram of scheduling
latencies
 "/memory/classes/heap/stacks:bytes",
 "/memory/classes/heap/objects:bytes",
 "/memory/classes/total:bytes",
 "/cpu/classes/gc/mark/assist:cpu-seconds",
 "/cpu/classes/gc/mark/dedicated:cpu-seconds",
}

func RegisterRuntimeMetrics(reg prometheus.Registerer) {
 // Bridge runtime/metrics → Prometheus.
 // Each runtime/metrics histogram becomes a Prometheus histogram
 with matching buckets.
 for _, name := range interesting {
 desc := metrics.All()[name] // spec: Kind, Cumulative, Buckets
 _ = desc
 // Use a library like github.com/prometheus/client_golang/
 prometheus.Collectors
 // or roll your own Collector that calls metrics.Read on
 Collect.
 }
}

// Minimal custom Collector – one Prometheus scrape reads all runtime
metrics atomically.
type runtimeCollector struct {
 desc map[string]*prometheus.Desc

```



```

names []string
}

func NewRuntimeCollector(names []string) prometheus.Collector {
 descscs := make(map[string]*prometheus.Desc, len(names))
 for _, n := range names {
 descscs[n] = prometheus.NewDesc(
 "go_"+sanitize(n), "runtime/metrics "+n, nil, nil,
)
 }
 return &runtimeCollector{descscs: descscs, names: names}
}

func (c *runtimeCollector) Describe(ch chan<- *prometheus.Desc) {
 for _, d := range c.descscs { ch <- d }
}

func (c *runtimeCollector) Collect(ch chan<- prometheus.Metric) {
 samples := make([]metrics.Sample, len(c.names))
 for i, n := range c.names { samples[i].Name = n }
 metrics.Read(samples)
 for _, s := range samples {
 switch s.Value.Kind() {
 case metrics.KindUint64:
 ch <- prometheus.MustNewConstMetric(c.descscs[s.Name], prometheus.GaugeValue, float64(s.Value.Uint64()))
 case metrics.KindFloat64:
 ch <- prometheus.MustNewConstMetric(c.descscs[s.Name], prometheus.GaugeValue, s.Value.Float64())
 case metrics.KindFloat64Histogram:
 h := s.Value.Float64Histogram()
 buckets := make(map[float64]uint64, len(h.Counts))
 for i, v := range h.Buckets { buckets[v] = h.Counts[i] }
 ch <- prometheus.MustNewConstHistogram(c.descscs[s.Name],
 h.Counts[len(h.Counts)-1], h.Counts[len(h.Counts)-1], b
 buckets)
 }
 }
}

func sanitize(n string) string {
 // "/gc/heap/allocs:bytes" → "gc_heap_allocs_bytes"
 s := ""
 for _, c := range n {
 switch c {
 case '/', ':', '.': s += "_"
 case '-': s += "_"
 default: s += string(c)
 }
 }
 // trim leading _

```

```
 if len(s) > 0 && s[0] == '_' { s = s[1:] }
 return s
}
```

Prefer this over `expvar` for runtime internals — `runtime/metrics` is typed, versioned, and does not allocate per-read. `expvar` (next section) is for *application* metrics.

## 4.2 `expvar` — the original Go metrics endpoint

`expvar` (since Go 1.0) exposes `sync/atomic`-backed variables over HTTP at `/debug/vars`. It is simple, allocation-free on the write path for integer vars, and scrapable by any HTTP client. Its weakness is that it is untyped-JSON and unauthenticated — do not expose it on the public port.

```

package telemetry

import (
 _ "expvar" // registers /debug/vars
 "expvar"
 "net/http"
 pprof "net/http/pprof"
)

var (
 requestsTotal = expvar.NewInt("requests_total")
 queueDepth = expvar.NewInt("queue_depth")
 lastError = expvar.NewString("last_error")
 // Maps and Floats are also available; use expvar.NewMap for
 // labeled counters.
 byStatus = expvar.NewMap("http_status")
)

func init() {
 // Publish a computed var – called on every /debug/vars scrape
 expvar.Publish("goroutines", expvar.Func(func() any {
 return metrics.All()["/sched/goroutines:goroutines"]
 }))
}

func AdminMux() *http.ServeMux {
 mux := http.NewServeMux()
 // expvar and pprof share the admin mux – never mount on the public
 mux
 mux.Handle("/debug/vars", expvar.Handler())
 mux.HandleFunc("/debug/pprof/", pprof.Index)
 mux.HandleFunc("/debug/pprof/cmdline", pprof.Cmdline)
 mux.HandleFunc("/debug/pprof/profile", pprof.Profile)
 mux.HandleFunc("/debug/pprof/symbol", pprof.Symbol)
 mux.HandleFunc("/debug/pprof/trace", pprof.Trace)
 return mux
}

```

```

$ curl -s http://localhost:9091/debug/vars | jq .
{
 "cmdline": ["/payments"],
 "memstats": { "Alloc": 4823440, ... },
 "requests_total": 12844,
 "queue_depth": 7,
 "http_status": { "200": 12000, "500": 12 }
}

```

For new services, bridge `expvar` or `runtime/metrics` into Prometheus/OpenTelemetry metrics rather than scraping `expvar` JSON directly — the typed histogram and label support in OTel/Prometheus is worth the extra dependency. Keep `expvar` for quick debugging and for services that cannot take a Prometheus dependency.

### 4.3 `log/slog` — structured logging since Go 1.21

Go 1.21 added `log/slog` to the standard library. It is the first structured, leveled, context-aware logger that is part of `go` itself. For backend services, it replaces `log`, `go.uber.org/zap`, and `sirupsen/logrus` as the default — not because it is faster than `zap` (it is slightly slower in the hot path), but because it is *standard*, context-aware, and has a stable `Handler` interface that every library can target.

```

package main

import (
 "context"
 "log/slog"
 "net/http"
 "os"
)

func initLogger() *slog.Logger {
 // JSON handler for prod (scraped by Loki/CloudWatch/ELK), text for
 local dev.
 // Both emit the same Attr set – only the wire format differs.
 var handler slog.Handler
 if os.Getenv("ENV") == "production" {
 handler = slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
 Level: slog.LevelInfo,
 AddSource: false, // set true to include file:line (costs
~1 alloc)
 })
 } else {
 handler = slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{
 Level: slog.LevelDebug,
 })
 }
 // Enrich every line with service identity – critical for multi-
 service log aggregation
 handler = handler.WithAttrs([]slog.Attr{
 slog.String("service", "payments"),
 slog.String("version", version),
 slog.String("env", os.Getenv("ENV")),
 })
 return slog.New(handler)
}

func main() {
 log := initLogger()
 slog.SetDefault(log) // so libraries using slog.Default() pick it
up

 // Context-aware logging – trace/span IDs flow automatically if you
 set them
 mux := http.NewServeMux()
 mux.HandleFunc("/charge", func(w http.ResponseWriter, r *http.Reque
st) {
 ctx := r.Context()
 // WithAttrs for per-request fields; avoids map allocs when
 handler supports it
 reqLog := log.With(
 slog.String("request_id", r.Header.Get("X-Request-ID")),

```

```

 slog.String("method", r.Method),
 slog.String("path", r.URL.Path),
)
 reqLog.InfoContext(ctx, "charge started",
 slog.String("customer_id", r.URL.Query().Get("customer")),
 slog.Int64("amount_cents", 1999),
)
 // ...
 if err := charge(ctx); err != nil {
 // slog's LevelError + structured error attr – searchable
in Loki
 reqLog.ErrorContext(ctx, "charge failed",
 slog.String("error", err.Error()),
 slog.String("error_type", errorType(err)),
)
 http.Error(w, "internal error", http.StatusInternalServerError)
error)
 return
 }
 reqLog.InfoContext(ctx, "charge succeeded")
})

// Propagate trace ID into log context – the glue between logs and
traces
// See Section 4.4 for the OTel side.
_ = context.WithValue // placeholder: use otel/baggage or
slog+trace provider
}

```

Output (JSON, production):

```

{"time":"2024-11-15T14:22:31.123Z","level":"INFO","msg":"charge
started","service":"payments","version":"v1.4.2","env":"production","re
quest_id":"req_9f3c","method":"POST","path":"/charge","customer_id":"cu
s_42","amount_cents":1999}
{"time":"2024-11-15T14:22:31.189Z","level":"ERROR","msg":"charge
failed","service":"payments","version":"v1.4.2","env":"production","req
uest_id":"req_9f3c","method":"POST","path":"/charge","error":"pgx:
connection reset","error_type":"transient"}

```

Operational rules:

- **One JSON schema per fleet.** Define the required attrs ( `service` , `version` , `env` , `request_id` , `trace_id` , `span_id` ) in a platform library. Every service imports it. Loki/ELK queries assume those fields exist.

- **InfoContext / ErrorContext over Info / Error**. The Context variants let a Handler extract trace\_id from context.Context without passing it as an explicit Attr — essential for OTEL correlation.
- **Do not log at Debug in prod.** slog.LevelDebug is below Info and disabled by default at LevelInfo. Use GODEBUG or a dynamic level (via slog.LevelVar) to enable it per-pod without redeploying.

```
var level slog.LevelVar // starts at Info (0); mutable at runtime

func init() {
 level.Set(slog.LevelInfo)
 h := slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{Level: &level})
 slog.SetDefault(slog.New(h))
}

// Expose on admin mux for on-the-fly debugging (protect with auth)
adminMux.HandleFunc("/debug/loglevel", func(w http.ResponseWriter, r *http.Request) {
 switch r.Method {
 case http.MethodGet:
 w.Write([]byte(level.Level().String()))
 case http.MethodPut:
 var req struct{ Level string `json:"level"` }
 json.NewDecoder(r.Body).Decode(&req)
 var l slog.Level
 if err := l.UnmarshalText([]byte(req.Level)); err != nil {
 http.Error(w, err.Error(), 400); return
 }
 level.Set(l)
 w.Write([]byte("ok"))
 }
})
```

## 4.4 OpenTelemetry for Go

OTel is the fleet-wide standard for traces and metrics. The Go SDK ([go.opentelemetry.io/otel](https://go.opentelemetry.io/otel)) instruments HTTP, gRPC, database, and custom spans with W3C traceparent propagation.

```

package telemetry

import (
 "context"
 "log/slog"

 "go.opentelemetry.io/contrib/bridges/otelslog"
 "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
 "go.opentelemetry.io/otel"
 "go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracehttp"
 "go.opentelemetry.io/otel/propagation"
 sdkresource "go.opentelemetry.io/otel/sdk/resource"
 sdktrace "go.opentelemetry.io/otel/sdk/trace"
 semconv "go.opentelemetry.io/otel/semconv/v1.26.0"
)

func InitTracer(ctx context.Context, serviceName, serviceVersion
string) (*sdktrace.TracerProvider, error) {
 res, err := sdkresource.New(ctx,
 sdkresource.WithAttributes(
 semconv.ServiceName(serviceName),
 semconv.ServiceVersion(serviceVersion),
 semconv.DeploymentEnvironmentKey.String(env()),
),
 sdkresource.WithFromEnv(), // OTEL_RESOURCE_ATTRIBUTES
 sdkresource.WithTelemetrySDK(), // otel.sdk.*
 sdkresource.WithHost(),
)
 if err != nil { return nil, err }

 exp, err := otlptracehttp.New(ctx,
 otlptracehttp.WithEndpoint(otelEndpoint()), // e.g. otel-
collector:4318
 otlptracehttp.WithInsecure(), // TLS in prod via
WithTLSClientConfig
)
 if err != nil { return nil, err }

 tp := sdktrace.NewTracerProvider(
 sdktrace.WithBatcher(exp),
 sdktrace.WithResource(res),
 sdktrace.WithSampler(sdktrace.ParentBased(sdktrace.TraceIDRatio
Based(0.1))),
 // 10% head sampling; collector does tail sampling for errors
)
 otel.SetTracerProvider(tp)

 otel.SetTextMapPropagator(propagation.NewCompositeTextMapPropagator(
 propagation.TraceContext{}, propagation.Baggage{},
))
}

```



```

 // Bridge slog → OTel logs (Go 1.21+): every slog record becomes an
 OTel LogRecord
 // Requires go.opentelemetry.io/contrib/bridges/otelslog
 _ =
otel slog.NewLogger(serviceName) // wire into slog handler chain if
using OTel logs

 return tp, nil
}

func InstrumentedMux(tp *sdktrace.TracerProvider) http.Handler {
 mux := http.NewServeMux()
 mux.Handle("/charge", otelhttp.NewHandler(
 http.HandlerFunc(chargeHandler), "POST /charge",
 otelhttp.WithMessageEvents(otelhttp.ReadEvents, otelhttp.WriteE
vents),
))
 // Health/readiness probes should NOT be traced – filter them out
to save collector budget
 mux.Handle("/healthz", http.HandlerFunc(healthHandler))
 return mux
}

// Inside a handler – manual span for a DB call
func chargeHandler(w http.ResponseWriter, r *http.Request) {
 ctx, span := otel.Tracer("payments").Start(r.Context(), "charge")
 defer span.End()

 // slog record automatically carries trace_id/span_id if handler is
otelslog-aware
 slog.InfoContext(ctx, "charging customer", slog.String("customer_id
", r.URL.Query().Get("customer")))

 if err := chargeDB(ctx); err != nil {
 span.RecordError(err)
 span.SetStatus(1, err.Error()) // codes.Error
 http.Error(w, "internal error", 500)
 return
 }
}

```

Propagate `traceparent` across service boundaries — `otelhttp` does this automatically for outgoing requests when you use `otelhttp.NewTransport`:

```
client := &http.Client{Transport:
otelhttp.NewTransport(http.DefaultTransport)}
req, _ := http.NewRequestWithContext(ctx, "GET", "http://risk-service/
evaluate", nil)
resp, err := client.Do(req) // traceparent header injected
 automatically
```

## 4.5 runtime/pprof — continuous profiling

Go's profiler is always available; it costs ~1-2% CPU when sampling and near-zero when not actively collecting. Continuous profiling (always-on `pprof` scraping by `pyroscope` / `parca` / `datadog`) is the Go equivalent of `perf` — and it works inside `scratch` containers because it is in-process.

```

package telemetry

import (
 "net/http"
 pprof "net/http/pprof"
 "runtime"
 runtimePprof "runtime/pprof"
)

// Admin mux with pprof – mount on :9091 or :6060, never on the public
// port.
func AdminMuxWithPprof() *http.ServeMux {
 mux := http.NewServeMux()
 mux.HandleFunc("/debug/pprof/", pprof.Index)
 mux.HandleFunc("/debug/pprof/cmdline", pprof.Cmdline)
 mux.HandleFunc("/debug/pprof/profile", pprof.Profile) // CPU – ?
seconds=30
 mux.HandleFunc("/debug/pprof/heap", pprof.Index) // alias
for allocs
 mux.HandleFunc("/debug/pprof/allocs", pprof.Index)
 mux.HandleFunc("/debug/pprof/goroutine", pprof.Index)
 mux.HandleFunc("/debug/pprof/block", pprof.Index)
 mux.HandleFunc("/debug/pprof/mutex", pprof.Index)
 mux.HandleFunc("/debug/pprof/threadcreate", pprof.Index)
 mux.HandleFunc("/debug/pprof/trace", pprof.Trace) //
execution trace – ?seconds=5

 // Optional: expose expvar alongside pprof
 mux.Handle("/debug/vars", expvar.Handler())
 return mux
}

func init() {
 // Enable block and mutex profiling – off by default (non-zero
 // overhead)
 // Tune rates: 1 = every event, higher = sampled
 runtime.SetBlockProfileRate(10000) // sample 1 in 10k block events
 runtime.SetMutexProfileFraction(10000)
}

// pprof labels – attach request-scoped dimensions to every profile
// sample
// Labels propagate via context and appear in pprof's `pprof.Labels`
// view.
func labeledHandler(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r
 *http.Request) {
 labels := runtimePprof.Labels(
 "endpoint", r.URL.Path,
 "method", r.Method,

```

```

)
 runtimePprof.Do(r.Context(), labels, func(ctx context.Context)
{
 next.ServeHTTP(w, r.WithContext(ctx))
})
})
}

```

Collection:

```

One-shot – what an engineer runs during an incident
$ go tool pprof -http=:8080 http://payments:9091/debug/pprof/profile?
seconds=30
$ go tool pprof -http=:8080 http://payments:9091/debug/pprof/heap
$ go tool pprof -http=:8080 http://payments:9091/debug/pprof/goroutine
$ curl -s http://payments:9091/debug/pprof/trace?seconds=5 > trace.out
$ go tool trace trace.out # opens execution trace viewer

Continuous – what the platform runs fleet-wide
Pyroscope/Parca scrape every 10s; labels let you slice by endpoint/
method
pyroscope config snippet (scrape Go pprof):
- job_name: payments
scrape_interval: 10s
static_configs: [{targets: ['payments:9091']}]}
profiling_config:
pprof_config:
cpu: {enabled: true, path: /debug/pprof/profile, delta: true}
memory: {enabled: true, path: /debug/pprof/heap}

```

Fleet rule: **every Go service exposes pprof on the admin port, scraped by the continuous profiler.** The incident workflow is `pyroscope → flame graph → offending commit`, not `ssh → perf`. The cost is negligible; the alternative is flying blind on the one service that is burning CPU at 3 AM.

#### 4.6 GODEBUG — runtime knobs without recompiling

`GODEBUG` is a comma-separated list of `key=value` pairs read at startup (and via `GODEBUG` env). Since Go 1.21, the `GODEBUG` syntax is versioned: `godebug` build tag or `GODEBUG` env can set defaults that are overridden at runtime. Common knobs for production:

```

Format: GODEBUG=key1=value1,key2=value2
Set via env or `go run -exec` wrapper; no rebuild needed.

GC / memory
GODEBUG=gctrace=1 # print GC events to stderr (pause,
heap, CPU)
GODEBUG=gcpacertrace=1 # verbose GC pacer decisions (Go 1.21+)
GODEBUG=madvdontneed=1 # madvise MADV_DONTNEED vs MADV_FREE
(Linux memory reclaim)
GODEBUG=asynctimeout=1 # disable async preemption (diagnose
preemption bugs)

Scheduler
GODEBUG=schedtrace=1000 # print scheduler state every 1000ms
GODEBUG=scheddetail=1 # verbose sched trace

HTTP
GODEBUG=http2client=0 # disable HTTP/2 client
GODEBUG=http2server=0 # disable HTTP/2 server
GODEBUG=multipathtcp=1 # enable MPTCP (Go 1.23+)

Runtime safety (Go 1.21+ defaults; can be re-enabled for compat
testing)
GODEBUG=cgocheck=1 # cgo pointer checks (default 1; 0
disables - unsafe)
GODEBUG=asynctimerchan=1 # legacy timer chan behavior

Example: diagnose GC pressure without redeploying
$ GODEBUG=gctrace=1 ./payments 2>&1 | head -20
gc 1 @0.042s 2%: 0.010+0.8+0.003 ms clock, 0.08+0.2+0.02+0.01+0.1 ms cp
u, 4->4->1 MB, 5 MB goal, 8 P
gc 2 @0.189s 1%: 0.009+0.5+0.002 ms clock, 0.07+0.1+0.01+0.00+0.08 ms c
pu, 4->4->2 MB, 5 MB goal, 8 P

Kubernetes: set GODEBUG per-pod for a canary without rebuilding the
image
kubectl set env deployment/payments GODEBUG=gctrace=1

```

`GODEBUG` values are also available inside the binary via `runtime/debug.ReadBuildInfo` and `debug.SetGCPercent` / `debug.SetMemoryLimit` for programmatic control — but prefer `GOMEMLIMIT` env (Section 5.3) for memory tuning.

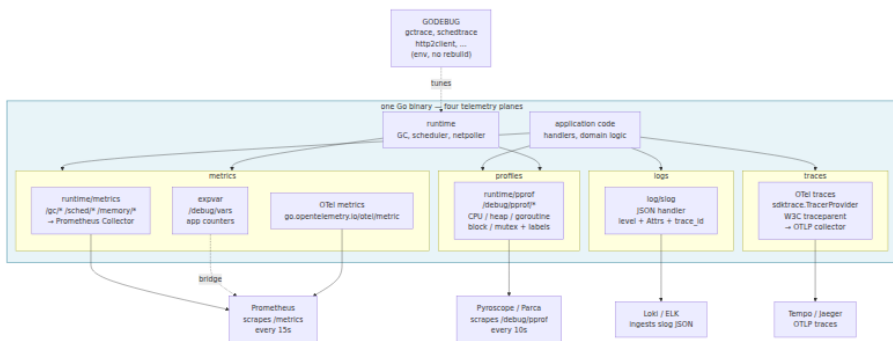


Diagram 4 — Observability stack: one binary emits metrics, logs, traces, and profiles; each plane is scraped independently and converges in Grafana.

## 5. Deployment — images, signing, SBOM, Kubernetes

The static Go binary from Chapter 1 is the ideal container payload: no `libc`, no interpreter, no `LD_LIBRARY_PATH`. The deployment story is therefore about *not* adding things — smaller base, fewer layers, pinned digests, attested provenance — and about teaching Kubernetes and the Go runtime to cooperate on resources and lifecycle.

### 5.1 `scratch` vs. `distroless` vs. `apko`

Base	Contents	Size (Go service)	When to use
<code>scratch</code>	Empty — literally zero files	Binary size only (~15-40 MB)	Pure <code>CGO_ENABLED=0</code> binaries that need no CA certs, no tzdata, no shell. Smallest attack surface, but no <code>nsswitch</code> , no certs — TLS dials fail without <code>ca-certificates</code> baked into the binary or mounted

Base	Contents	Size (Go service)	When to use
<code>gcr.io/distroless/static</code>	<code>ca-certificates</code> , <code>tzdata</code> , <code>nonroot</code> user, no shell/package manager	~2 MB + binary	Default for most Go services. Covers TLS + time zones without a shell. Deprecated in favor of <code>cgr.dev/chainguard/static</code> (actively maintained)
<code>cgr.dev/chainguard/static</code>	Same as distroless, built reproducibly with <code>apko</code> , Wolfi packages, daily CVE patches	~1.5 MB + binary	Preferred for new services. <code>apko</code> -built, SLSA provenance, <code>cosign</code> -signed base. Drop-in replacement for <code>gcr.io/distroless/static</code>
<code>apko</code> -built custom	You declare exactly which <code>apk</code> packages to include (see <code>apko.yaml</code> in Section 3.2)	Minimal + declared packages	When you need <code>glibc</code> (cgo), <code>curl</code> for health checks, or <code>tzdata</code> variants. Also for FIPS/proxy-constrained registries where you must build from internal package repos
<code>golang:1.22-alpine</code>	Full Go toolchain + Alpine userland	~350 MB	<b>Never in production.</b> Build stage only ( <code>AS builder</code> ). Copy the binary out and discard the toolchain

```

Dockerfile – multi-stage, distroless final (traditional, without ko)
Prefer ko (Section 3.2) for new services; this is for teams that must
keep Dockerfiles.

Stage 1: build – uses the Go toolchain, discarded after
FROM golang:1.22.4-bookworm AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
Reproducible, static build – same flags as ./scripts/cross-compile.sh
ARG VERSION=dev
ARG COMMIT=unknown
RUN CGO_ENABLED=0 go build \
 -trimpath -buildvcs=true \
 -ldflags "-s -w -X main.version=${VERSION} -X main.commit=${COMMI
T}" \
 -o /out/payments ./cmd/api \
 && go version -m /out/payments

Stage 2: runtime – distroless/chainguard, no shell, no toolchain
FROM cgr.dev/chainguard/static:latest
COPY --from=builder /out/payments /payments
Nonroot is already the default in chainguard/static; be explicit
anyway
USER nonroot:nonroot
No HEALTHCHECK here – let Kubernetes probes do it (avoids needing
curl in the image)

Exec form – no shell wrapping, signals reach the Go process (PID 1)
ENTRYPOINT ["/payments"]

```

`ko`-based builds (Section 3.2) eliminate this Dockerfile entirely. The equivalent `ko` invocation produces the same image with fewer moving parts and automatic SBOM generation.

## 5.2 goreleaser — release automation

`goreleaser` is the `make dist` for Go. It orchestrates `go build` across the matrix, builds archives and containers, generates SBOMs, and signs artifacts — all from `.goreleaser.yaml` (Section 3.2). In a fleet of 100 services, `goreleaser` standardizes the release so that every service's GitHub Release has the same layout: `checksums.txt`, `sbom.spdx.json`, `*.tar.gz` / `*.zip`, container image, and `cosign` signature. Engineers learn one layout and can verify any service.



Key behaviors to know:

- `mod_timestamp` pins archive entry timestamps to `SOURCE_DATE_EPOCH` equivalent — required for reproducible tarballs.
- `builds[].flags: [-trimpath] + ldflags: [-s -w -X ...]` — same reproducibility knobs as Section 3.1.
- `kos:` and `dockers:` can coexist — `ko` for Go-native images, `dockers` for custom Dockerfiles that need extra layers.
- `signs:` and `docker_signs:` invoke `cosign` automatically (see Section 5.3) — keyless by default via Fulcio/Rekor.

### 5.3 Signing, SBOM, and supply-chain attestation

Reproducibility proves *what* was built; signing and SBOM prove *who* built it and *what is inside*.

**Signing with `cosign` / Sigstore (keyless):**

```

Sign a binary or archive – keyless via OIDC (GitHub Actions, GCP,
etc.)
Fulcio issues a short-lived cert bound to the OIDC identity; Rekor
logs the entry.
$ cosign sign-blob --yes \
 --output-signature payments-linux-amd64.sig \
 --output-certificate payments-linux-amd64.pem \
 payments-linux-amd64
Verification – anyone can verify with the Rekor transparency log
$ cosign verify-blob --yes \
 --certificate payments-linux-amd64.pem \
 --signature payments-linux-amd64.sig \
 --certificate-identity-regex "https://github.com/acme/payments.*"
\
 --certificate-oidc-issuer https://
token.actions.githubusercontent.com \
 payments-linux-amd64
Verified OK

Sign a container image – same flow, image digest is what is signed
$ cosign sign --yes ghcr.io/acme/payments@sha256:7f3a...
$ cosign verify --yes \
 --certificate-identity-regex "https://github.com/acme/payments.*"
\
 --certificate-oidc-issuer https://
token.actions.githubusercontent.com \
 ghcr.io/acme/payments@sha256:7f3a...

Kubernetes admission – verify before running (Kyverno / Sigstore
policy-controller)
See Companion Book 4, Chapter 5 for the full policy; the Go-relevant
bit:
policy: require that every image's signature chains to Fulcio + Rekor
and
that the certificate identity matches github.com/acme/*

```

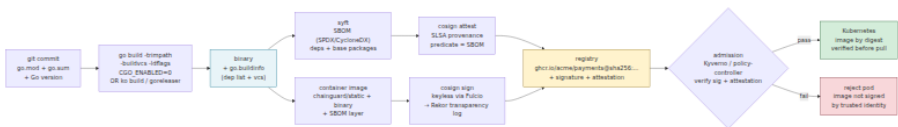
## SBOM with **syft** (SPDX / CycloneDX):

```
SBOM for a Go binary – syft reads go.mod via the binary's buildinfo
$ syft payments-linux-amd64 -o spdx-json > sbom.spdx.json
$ jq '.packages[] | {name: .name, versionInfo: .versionInfo}' sbom.spdx.json | head -20
{
 "name": "github.com/go-chi/chi/v5",
 "versionInfo": "v5.0.11"
}
{
 "name": "github.com/jackc/pgx/v5",
 "versionInfo": "v5.6.0"
}

SBOM for a container image – includes base-layer packages too
$ syft ghcr.io/acme/payments@sha256:7f3a... -o cyclonedx-json > sbom.cyclonedx.json

Attach SBOM as an attestation (cosign attest) – SLSA provenance
$ cosign attest --yes --predicate sbom.spdx.json --type spdxjson \
 ghcr.io/acme/payments@sha256:7f3a...
$ cosign verify-attestation --yes --type spdxjson \
 --certificate-identity-regexp "https://github.com/acme/payments.*" \
 ghcr.io/acme/payments@sha256:7f3a... | jq .predicate.packages | head -30
```

**What `go version -m` already gives you:** For Go binaries, the SBOM-lite is embedded — `go version -m` lists every `dep` with version and `h1:` hash, plus `vcs.revision` and `build` settings. `syft` enriches this with base-image packages and a standard SPDX/CycloneDX envelope that vulnerability scanners ( `grype` , `osv-scanner` ) consume.



*Diagram 5 — Build → sign → SBOM → deploy supply chain: every artifact is hashed, signed keylessly, attested, and verified at admission.*

## 5.4 Kubernetes deployment — resources, probes, GOMAXPROCS/GOMEMLIMIT, admin endpoints

The Go runtime must be told about its cgroup limits. By default, `GOMAXPROCS` is `runtime.NumCPU()` of the *host*, not the pod's `cpu` limit, and `GOMEMLIMIT` is unset (GC targets 100% of host memory). On a 96-core

node with a `cpu: 500m` pod, Go will schedule 96 P s and GC as if it has 96 cores — burning CPU on preemption and scheduling. On a `memory: 256Mi` pod, it will GC too late and get OOMKilled.

Two fixes, in order of preference:

1. **automaxprocs / automemlimit** — Go libraries that read cgroup limits at startup and call `runtime.GOMAXPROCS / debug.SetMemoryLimit` automatically. Use them. One import, no config.

```
package main

import (
 _ "go.uber.org/
automaxprocs" // sets GOMAXPROCS to cgroup cpu quota on import
 "github.com/KimMachineGun/automemlimit/mempprof"
 "go.uber.org/automaxprocs/maxprocs"
)

func init() {
 // automaxprocs - log what it set so you can verify in prod
 if _, err := maxprocs.Set(maxprocs.Logger(func(format string,
args ...any) {
 slog.Info(fmt.Sprintf(format, args...))
 })); err != nil {
 slog.Warn("automaxprocs failed", slog.String("error",
err.Error()))
 }

 // automemlimit - sets GOMEMLIMIT to cgroup memory limit * ratio
 (default 0.9)
 // Requires cgroups v2 or v1; falls back to no-op if not in a
 container
 if err := mempprof.SetGoMemLimitWithOpts(
 mempprof.WithRatio(0.9),
 mempprof.WithProvider(mempprof.FromCgroup),
 mempprof.WithFallback(mempprof.FromSystem),
); err != nil {
 slog.Warn("automemlimit failed", slog.String("error",
err.Error()))
 }
}
```

1. **Explicit GOMAXPROCS / GOMEMLIMIT env** — set from Kubernetes `resources.limits` via the Downward API when you do not want the import (e.g., minimal `scratch` images where extra deps are undesirable):

```
env:
- name: GOMAXPROCS
 valueFrom:
 resourceFieldRef: { resource: limits.cpu } # 500m → 1, 2 → 2
- name: GOMEMLIMIT
 value: "230MiB" # ~90% of limits.memory (256Mi) – leave headroom
for Go's off-heap + kernel
```

Full deployment manifest — the reference for a production Go service:

```

k8s/deployment.yaml – production Go service with observability +
resource tuning
apiVersion: apps/v1
kind: Deployment
metadata:
 name: payments
 labels: { app: payments, version: v1.4.2 }
spec:
 replicas: 3
 strategy:
 type: RollingUpdate
 rollingUpdate: { maxUnavailable: 1, maxSurge: 1 }
 selector: { matchLabels: { app: payments } }
 template:
 metadata:
 labels: { app: payments, version: v1.4.2 }
 annotations:
 # Prometheus scraping – admin port, not public
 prometheus.io/scrape: "true"
 prometheus.io/port: "9091"
 prometheus.io/path: "/metrics"
 # Pyroscope/Parca – continuous pprof
 pyroscope.io/scrape: "true"
 pyroscope.io/port: "9091"
 spec:
 terminationGracePeriodSeconds: 30
 # must exceed shutdown timeout (see Section 6)
 securityContext:
 runAsNonRoot: true
 runAsUser: 65532 # nonroot (chainguard/static default)
 runAsGroup: 65532
 fsGroup: 65532
 seccompProfile: { type: RuntimeDefault }
 containers:
 - name: payments
 # Always pin by digest – tags are mutable, digests are not
 image: ghcr.io/acme/
payments@sha256:7f3a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6
c7d8e9f0a1b2c
 imagePullPolicy: IfNotPresent
 ports:
 - name: http # public API
 containerPort: 8080
 protocol: TCP
 - name: admin
 # metrics / pprof / expvar / health – NOT public
 containerPort: 9091
 protocol: TCP
 env:
 - name: ENV

```

```

 value: production
Option A: let automaxprocs/automemlimit read cgroups
(preferred)
No GOMAXPROCS/GOMEMLIMIT env needed – libraries do it on
import.
Option B: explicit (uncomment if not using automaxprocs):
- name: GOMAXPROCS
valueFrom: { resourceFieldRef: { resource:
limits.cpu } }
- name: GOMEMLIMIT
value: "230MiB"
- name: OTEL_EXPORTER_OTLP_ENDPOINT
 value: http://otel-
collector.observability.svc.cluster.local:4318
- name: OTEL_RESOURCE_ATTRIBUTES
 value: "deployment.environment=production,service.namespa
ce=payments"
GODEBUG – set per-deployment without rebuilding
- name: GODEBUG
value: "gctrace=0"
resources:
 requests:
 cpu: 250m
 memory: 128Mi
 limits:
 # Keep limits close to requests for Go – bursty GC
 # benefits from headroom,
 # but too-high memory limits hide leaks. Set GOMEMLIMIT
 # to ~90% of this.
 cpu: 1000m
 memory: 256Mi
Probes – distinct liveness vs readiness, admin port for
health
Readiness gates traffic; liveness restarts the pod.
readinessProbe:
 httpGet: { path: /healthz, port: admin } # 200 only when
ready to serve
 initialDelaySeconds: 3
 periodSeconds: 5
 timeoutSeconds: 2
 failureThreshold: 3
livenessProbe:
 httpGet: { path: /livez, port: admin } # 200 unless
deadlocked
 initialDelaySeconds: 15
 periodSeconds: 10
 timeoutSeconds: 2
 failureThreshold: 3
startupProbe: # for slow-starting services (migrations,
cache warmup)
 httpGet: { path: /healthz, port: admin }

```

```

 periodSeconds: 2
 failureThreshold: 30 # 60s budget before liveness takes
over
 securityContext:
 allowPrivilegeEscalation: false
 readOnlyRootFilesystem: true
 capabilities: { drop: [ALL] }
 lifecycle:
 preStop:
 # Give the Go graceful shutdown (Section 6) time to
 drain.
 # kubelet sends SIGTERM, then waits
 terminationGracePeriodSeconds.
 # preStop sleeps so the endpoint is removed from Service
 before SIGTERM.
 exec:
 command: ["/bin/sh", "-c", "sleep 5"]
 # NOTE: chainguard/static has no /bin/sh – use a Go-
 native preStop
 # or switch to chainguard/static with `sleep` via `apk
 add busybox` in apko.

Alternative: omit preStop and rely on readiness probe + endpoint
propagation delay.

apiVersion: v1
kind: Service
metadata:
 name: payments
 labels: { app: payments }
spec:
 selector: { app: payments }
 ports:
 - name: http
 port: 80
 targetPort: http
 # Admin port is NOT exposed via the Service – only via PodIP for
 scraping

ServiceMonitor for Prometheus Operator (if using kube-prometheus)
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata: { name: payments, labels: { release: kube-prometheus } }
spec:
 selector: { matchLabels: { app: payments } }
 endpoints:
 - port: admin
 path: /metrics
 interval: 15s
 # pprof is scraped by Pyroscope/Parca, not Prometheus

```



Probe design for Go — the subtlety is in the handler:

```
// health.go – separate readiness vs liveness
var ready atomic.Bool // set to true after warmup (DB pool, cache,
etc.)

func init() {
 // readiness: false until the service can actually serve
 ready.Store(false)
}

func markReady() { ready.Store(true) }

func healthHandler(w http.ResponseWriter, r *http.Request) { //
readiness
 if !ready.Load() {
 http.Error(w, "not ready", http.StatusServiceUnavailable)
 return
 }
 // Optionally check downstream deps (DB ping with short timeout)
 // Fail readiness on downstream outage – removes pod from Service
endpoints
 // but does NOT restart it (that is liveness's job).
 w.WriteHeader(http.StatusOK)
 w.Write([]byte("ok"))
}

func liveHandler(w http.ResponseWriter, r *http.Request) { // liveness
 // Must be cheap and never depend on downstream – only checks this
process
 // A deadlock or goroutine leak should still return 500 here so
kubernet restarts.
 w.WriteHeader(http.StatusOK)
 w.Write([]byte("ok"))
}
```

---

## 6. Graceful shutdown — `signal.NotifyContext` +

`http.Server.Shutdown`

Kubernetes sends `SIGTERM`, waits `terminationGracePeriodSeconds`, then sends `SIGKILL`. Between those two signals, the Go process must: stop accepting new connections, finish in-flight requests, flush telemetry, and close downstream resources (DB pools, queue consumers). The standard pattern since Go 1.16 is `signal.NotifyContext` + `http.Server.Shutdown`; since Go 1.20, `context.WithCancelCause` gives richer error propagation.

## 6.1 The shutdown sequence

```

package main

import (
 "context"
 "errors"
 "fmt"
 "log/slog"
 "net/http"
 "os"
 "os/signal"
 "syscall"
 "time"

 // Resource tuning (Section 5.4)
 _ "go.uber.org/automaxprocs"
 "github.com/KimMachineGun/automemlimit/memprof"
)

var (
 version = "dev"
 commit = "unknown"
)

func main() {
 // Resource limits – do this first, before any goroutine is
 // scheduled
 if err := memprof.SetGoMemLimitWithOpts(memprof.WithRatio(0.9));
 err != nil {
 slog.Warn("automemlimit", slog.String("error", err.Error()))
 }

 log := initLogger() // slog JSON handler (Section 4.3)
 slog.SetDefault(log)

 // Public API server
 apiMux := http.NewServeMux()
 apiMux.HandleFunc("/charge", chargeHandler)
 apiMux.HandleFunc("/healthz", healthHandler) // also on public mux
 for ALB target checks
 apiSrv := &http.Server{
 Addr: ":8080",
 Handler: apiMux,
 ReadTimeout: 5 * time.Second,
 WriteTimeout: 10 * time.Second,
 IdleTimeout: 60 * time.Second,
 // BaseContext lets you inject per-connection context (trace,
 // request ID)
 }

 // Admin server – pprof, metrics, expvar, liveness/readiness

```

*(Section 4.5, 5.4)*

```
adminMux := AdminMuxWithPprof() // :9091
adminMux.HandleFunc("/healthz", healthHandler)
adminMux.HandleFunc("/livez", liveHandler)
adminMux.Handle("/metrics", promHandler())
adminSrv := &http.Server{
 Addr: ":9091",
 Handler: adminMux,
}

// Root context – cancelled on SIGINT/SIGTERM
// signal.NotifyContext (Go 1.16+) wires os.Interrupt + SIGTERM to
context cancellation.
// This is the single cancellation signal for the entire process.
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
defer stop() // release signal handler when main returns

// Start servers – each in its own goroutine so they do not block
shutdown
go func() {
 log.Info("api server listening", slog.String("addr", apiSrv.Addr))
 if err := apiSrv.ListenAndServe(); err != nil && !
errors.Is(err, http.ErrServerClosed) {
 log.Error("api server error", slog.String("error", err.Error()))
 stop() // cancel root context – triggers shutdown
 }
}()
go func() {
 log.Info("admin server listening", slog.String("addr",
adminSrv.Addr))
 if err := adminSrv.ListenAndServe(); err != nil && !errors.Is(e
rr, http.ErrServerClosed) {
 log.Error("admin server error", slog.String("error", err.Er
ror()))
 }
}()

// Mark ready – after listeners are bound, before blocking on
signal
markReady()
log.Info("ready", slog.String("version", version), slog.String("com
mit", commit))

// Block until SIGTERM/SIGINT or server error
<-ctx.Done()
log.Info("shutdown signal received", slog.String("cause", context.C
ause(ctx).Error()))
```

```

 // Graceful shutdown – 20s budget, must fit within
 terminationGracePeriodSeconds (30s)
 // Leave 10s headroom for kubelet + preStop + endpoint propagation.
 shutdownCtx, cancel := context.WithTimeout(context.Background(),
20*time.Second)
 defer cancel()

 // 1) Stop readiness – new connections get 503, Service endpoints
 are removed
 ready.Store(false)

 // 2) Shutdown HTTP servers – drains in-flight requests, closes
 listeners
 // Shutdown is graceful: it waits for active handlers to finish, up
 to shutdownCtx deadline.
 // New connections during shutdown get 503 (or are refused if
 listener is closed).
 if err := apiSrv.Shutdown(shutdownCtx); err != nil {
 log.Error("api shutdown error", slog.String("error", err.Error(
)))
 } else {
 log.Info("api server drained")
 }
 if err := adminSrv.Shutdown(shutdownCtx); err != nil {
 log.Error("admin shutdown error", slog.String("error", err.Erro
r()))
 }

 // 3) Close downstream resources – DB pools, queue consumers, OTel
 exporters
 if err := closeDownstream(shutdownCtx); err != nil {
 log.Error("downstream close error", slog.String("error", err.Er
ror()))
 }

 // 4) Flush telemetry – OTel spans, slog buffers, pprof
 if tp != nil {
 if err := tp.Shutdown(shutdownCtx); err != nil {
 log.Error("otel shutdown error", slog.String("error", err.E
rror()))
 }
 }

 log.Info("shutdown complete")
}

func closeDownstream(ctx context.Context) error {
 // Example: drain queue consumer, close DB pool
 // Each Close should respect ctx deadline – do not block past
 terminationGracePeriodSeconds
 var errs []error

```

```

// if err := queueConsumer.Drain(ctx); err != nil { errs =
append(errs, err) }
// if err := dbPool.Close(); err != nil { errs = append(errs,
err) }
return errors.Join(errs...)
}

```

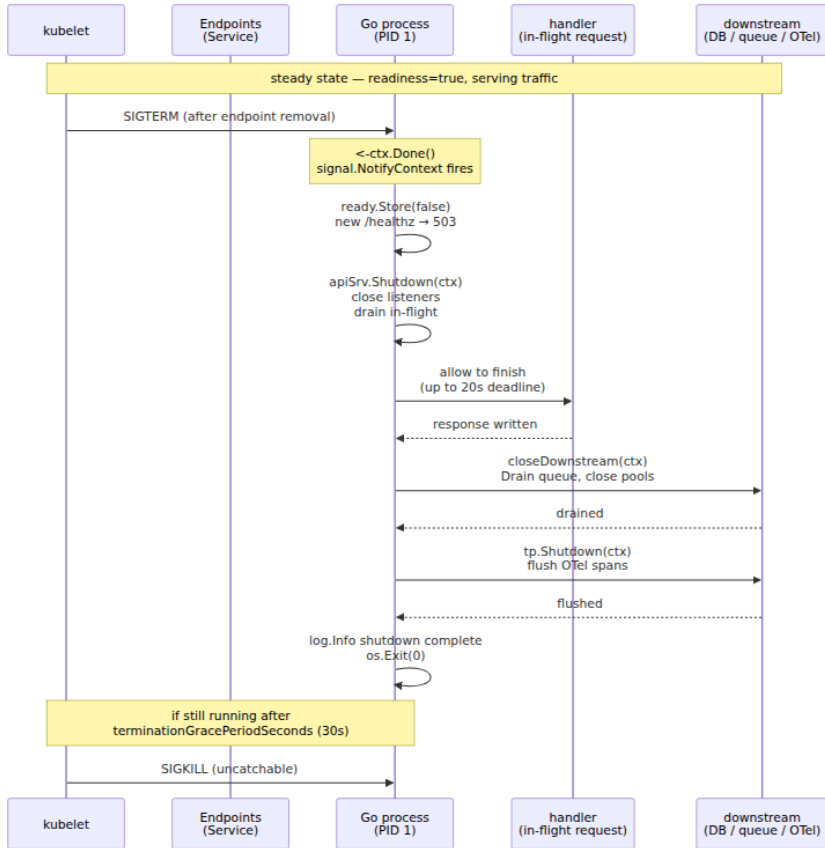


Diagram 6 — Graceful shutdown sequence: SIGTERM → stop readiness → drain HTTP → close downstream → flush telemetry → exit, all within terminationGracePeriodSeconds.

## 6.2 Common mistakes at scale

Mistake	Symptom	Fix
No <code>signal.NotifyContext</code> / <code>Shutdown</code>	<code>SIGTERM</code> kills the process mid-request; clients see 502 / connection reset ; queue messages are lost	Always use the pattern above. No <code>os.Exit(1)</code> in signal handlers.
Same <code>http.Server</code> for public + admin	<code>Shutdown</code> closes <code>pprof</code> / <code>metrics</code> before you can scrape the final state; liveness probe fails during drain and triggers a <code>SIGKILL</code> loop	Separate <code>apiSrv (:8080)</code> and <code>adminSrv (:9091)</code> . Shut down first; keep <code>adminSrv</code> alive until t for post-mortem scraping.
<code>terminationGracePeriodSeconds &lt; Shutdown timeout</code>	Kubelet sends <code>SIGKILL</code> while handlers are still running; graceful shutdown is truncated	<code>terminationGracePeriodSeconds</code> exceed <code>Shutdown timeout</code> + <code>pre sleep</code> + <code>endpoint propagation (~5s)</code> termination with 20s shutdown default.
<code>GOMAXPROCS = host cores</code>	Pod with <code>cpu: 500m</code> on a 96-core node schedules 96 <code>P s</code> ; scheduler thrashes, p99 latency spikes, CPU throttling	<code>automaxprocs</code> or <code>GOMAXPROCS</code> f <code>limits.cpu</code> . Verify with <code>go ver</code> or <code>runtime.GOMAXPROCS(0)</code> in lo

Mistake	Symptom	Fix
<code>GOMEMLIMIT</code> unset	GC targets host memory, not pod limit; heap grows past <code>limits.memory</code> and the pod is OOMKilled without a Go-level signal	<code>automemlimit</code> or explicit <code>GOMEMLIMIT=~90%</code> of <code>limits.memory</code> Monitor <code>/gc/gomemlimit:bytes</code> <code>container_memory_working_set</code>
Readiness never fails on downstream outage	Pod stays in Service endpoints while DB is down; every request fails and retries amplify the outage	<code>healthHandler</code> should fail readiness when critical downstream is unavailable (with a short timeout). Liveness must be <code>200</code> — only restart on process-level failure.
<code>preStop</code> with <code>sleep</code> in <code>scratch</code>	<code>exec: "/bin/sh": not found</code> — <code>scratch</code> and <code>chainguard/static</code> have no shell	Either add <code>busybox</code> via <code>apko</code> , use native <code>preStop</code> (exec the binary <code>preStop</code> subcommand), or omit <code>preStop</code> and rely on readiness endpoint propagation.
Logging to <code>stdout</code> without JSON in prod	Logs are unparseable by Loki/ELK; <code>trace_id</code> correlation breaks; alerts cannot query by <code>service / version</code>	<code>slog.NewJSONHandler(os.Stdout)</code> in prod, <code>NewTextHandler</code> locally; propagate the schema via a shared platform

### 6.3 Operating Go at scale — 100s of services, one platform

The patterns in this chapter are multiplicative. One service with `ko` + `cosign` + `slog` + `automaxprocs` is straightforward. Three hundred



services with inconsistent choices is operational debt. The platform team should standardize:

- **One base image** ( `cgr.dev/chainguard/static` + `apko` for exceptions), **one build tool** ( `ko` or `goreleaser` , not both per service), **one signing identity** (per-repo OIDC via Fulcio), **one SBOM format** (SPDX JSON via `syft` ).
- **One slog schema** (required attrs, JSON in prod), **one OTel collector endpoint**, **one runtime/metrics** → **Prometheus bridge**, **one continuous profiler**. Every service's dashboard is a template with `service` as a variable.
- **One k8s/deployment.yaml template** (Helm chart, Kustomize base, or `cuelang / kpt` package) that bakes in `resources.limits` , `GOMEMLIMIT / GOMAXPROCS` , probes, `terminationGracePeriodSeconds` , and `securityContext` . Services override `requests / limits` and `replicas` , not the shape.
- **One go.work per monorepo**, `GOWORK=off` in hermetic CI, `go version -m` as the deployment audit trail. Pin `toolchain` in `go.mod` so every builder — developer laptop, CI, `ko` — uses the same Go version.
- **Reproducibility as a CI gate** — `cmp / diffoscope` on two independent builds, `cosign verify` on every image before it reaches `prod` . A non-reproducible or unsigned image never deploys, regardless of urgency.

The payoff is that `go build -trimpath` , `go version -m` , `go tool dist list` , and `go.work` are the same commands on every service. The toolchain's opinionation — static binaries, internal linker, MVS, build cache — becomes the fleet's consistency.

---

## Key takeaways

- **GOOS / GOARCH cross-compilation is free for pure Go; cgo is the boundary**. With `CGO_ENABLED=0` , one `go build` invocation targets any pair in `go tool dist list` from any host. The moment a transitive dependency imports `"C"` , you need a matching `CC` — use `zig cc -target <triple>` for a hermetic fix or `crazymax/xgo` for Docker-based cross + `QEMU` testing. Prefer pure-Go dependencies to keep the matrix clean.

- **go.work solves multi-module development without replace drift.** `go.work` at the workspace root declares `use` members and optional `replace` overrides that apply to every module in the workspace but are invisible to external consumers. Commit `go.work` for monorepo workflows; use `GOWORK=off` in CI for hermetic single-module verification. `go work sync` is the workspace equivalent of `go mod tidy`.
- **Reproducibility requires -trimpath, pinned SOURCE\_DATE\_EPOCH, and explicit flags.** `-trimpath` strips host paths, `-buildvcs` stamps or suppresses VCS metadata, `ldflags -s -w -X` must be pinned to a tag/commit, and `SOURCE_DATE_EPOCH` normalizes timestamps. Verify with `sha256sum + diffoscope + go version -m . ko + apko + goreleaser` extend reproducibility from the binary to the container and release artifacts.
- **One binary should emit all four telemetry planes.** `runtime/metrics` (typed, stable) for runtime internals, `expvar` or a Prometheus bridge for app metrics, `log/slog` (JSON in prod, `slog.LevelVar` for dynamic leveling) for structured logs with `trace_id` correlation, OTEL for W3C-propagated traces, and `runtime/pprof` (always-on, scraped by `pyroscope / parca`) for continuous profiling. `GODEBUG` tunes the runtime without rebuilding.
- **Deployment is scratch / distroless / apko + signing + SBOM + correct Kubernetes wiring.** Use `cgr.dev/chainguard/static` (or `apko` -built custom) as the base, `ko` to build without Dockerfiles, `cosign` keyless signing via Fulcio/Rekor, and `syft` SPDX SBOMs — all orchestrated by `goreleaser`. In Kubernetes, pin images by digest, set `GOMAXPROCS / GOMEMLIMIT` from cgroup limits via `automaxprocs / automemlimit`, and expose `pprof / expvar / metrics` on a separate admin port that is not in the Service.
- **Graceful shutdown is signal.NotifyContext → ready=false → http.Server.Shutdown → drain downstream → flush OTEL, all within terminationGracePeriodSeconds.** Separate public and admin `http.Server`s, keep `terminationGracePeriodSeconds` (30s) comfortably above the `Shutdown` timeout (20s), and never put `GOMEMLIMIT` -unaware or `GOMAXPROCS` -unaware binaries on shared nodes — they will be throttled or OOMKilled under load.
- **At fleet scale, standardize the toolchain choices once.** One base image, one build tool, one log schema, one OTEL collector, one

deployment template, one signing identity. Go's opinionated toolchain (`go build`, `go version -m`, `go tool dist list`, `go.work`) makes fleet-wide consistency achievable — but only if the platform team enforces it.

---

## Further reading

- **Go Workspaces — `go.work` reference.** Official documentation for `go.work`, `go work use / sync / edit`, and workspace-aware `cmd/go` behavior. Covers `GOWORK` env, `use / replace` semantics, and interaction with `GOPROXY`. <https://go.dev/ref/mod#workspaces> (*pinned*)
- **`ko` — container images for Go.** Builds and publishes container images directly from Go import paths without Dockerfiles; integrates with `apko` bases, generates SBOMs, and signs with `cosign`. <https://ko.build/> (*pinned — ko*)
- **`apko` — declarative APK-based images.** Builds reproducible, minimal base images from `apko.yaml` using Wolfi/Chainguard `apk` packages; the engine behind `cgr.dev/chainguard/*` images. <https://github.com/chainguard-dev/apko> (*pinned — apko: same entry as ko above covers both; see also https://edu.chainguard.dev/open-source/apko/*)
- **`goreleaser` — Go release automation.** Declarative `.goreleaser.yaml` for cross-compilation matrices, archives, `nfpms`, Docker/ `ko` images, SBOMs, and `cosign` signing. <https://goreleaser.com/> (*pinned*)
- **`log/slog` — structured logging in the standard library.** Package docs for `log/slog` (Go 1.21+): `Handler`, `Level`, `Attr`, `With`, `InfoContext / ErrorContext`, `LevelVar`, and the `slog.Handler` interface for custom backends. <https://pkg.go.dev/log/slog> (*pinned*)
- **`runtime/metrics` — stable runtime telemetry.** Package docs for `runtime/metrics` (Go 1.16+): metric names (`/gc/*`, `/sched/*`, `/memory/*`), `Sample`, `Value.Kind`, histogram buckets, and migration from `runtime.MemStats`. <https://pkg.go.dev/runtime/metrics> (*pinned*)
- **`cosign` / Sigstore — keyless signing and verification.** `cosign sign-blob / sign / verify / attest`, Fulcio (short-lived certs via OIDC), Rekor (transparency log), and Kubernetes admission with `policy-`

controller / Kyverno . Covers the keyless flow used in Section 5.3.  
<https://docs.sigstore.dev/cosign/overview/> (*pinned*)

- **GODEBUG — runtime debugging knobs.** Full list of `GODEBUG` keys (`gctrace`, `schedtrace`, `http2client`, `madvdontneed`, `asynctimerchan`, etc.), versioning since Go 1.21, and interaction with `GOMEMLIMIT` / `GOMAXPROCS` . [https://pkg.go.dev/runtime#hdr-Environment\\_Variables](https://pkg.go.dev/runtime#hdr-Environment_Variables) and <https://go.dev/doc/godebug>
- **Uber automaxprocs and automemlimit — cgroup-aware Go tuning.** [go.uber.org/automaxprocs](https://go.uber.org/automaxprocs) (sets `GOMAXPROCS` from cgroup CPU quota) and [github.com/KimMachineGun/automemlimit](https://github.com/KimMachineGun/automemlimit) (sets `GOMEMLIMIT` from cgroup memory limit). Read the cgroup v1/v2 detection logic before relying on it in mixed clusters. <https://github.com/uber-go/automaxprocs> and <https://github.com/KimMachineGun/automemlimit>
- **syft — SBOM generation for Go binaries and containers.** Generates SPDX/CycloneDX SBOMs from Go `buildinfo`, container layers, and filesystem scans; integrates with `grype` for vulnerability matching. <https://github.com/anchore/syft>
- **zig cc as a hermetic C cross-compiler for Go.** Using `zig cc -target <triple>` as `CC / CXX` for `CGO_ENABLED=1` cross-compilation; covers bundled headers, `glibc / musl` selection, and pinning Zig's toolchain version for reproducibility. <https://ziglang.org/documentation/master/#Targets> and <https://andrewkelley.me/post/zig-cc-powerful-drop-in-replacement-gcc-clang.html>
- **Continuous profiling for Go — Pyroscope / Parca.** Scraping `runtime/pprof` endpoints fleet-wide, `pprof` labels, and flame-graph workflows for CPU/heap/goroutine/block/mutex profiles. <https://grafana.com/docs/pyroscope/latest/> and <https://www.parca.dev/docs/overview>